

数字系统设计

Digital System Design

-----时序逻辑电路设计

王维东

Weidong Wang

浙江大学信息与电子工程学院

College of Information Science & Electronic Engineering

信息与通信网络研究所

Zhejiang University

Spring 2026 – Lec. #7

任课教师

- 王维东
- 浙江大学信息与电子工程学院, 信电楼306
- 邮箱: wdwang@zju.edu.cn
- College of Information Science & Electronic Engineering
- Zhejiang University, Hangzhou, 310027
- Tel: 86-571-87953170 (O)
- Mobile: 13605812196

TA:

钟睿, 13687791509; 22331166@zju.edu.cn;

Office Hours: 玉泉信电楼 308室（可以钉钉、微信或邮件联系）.

Prerequisites预修课程

- 电子电路基础
- 电子线路
- C语言
- How to learn this Course?
 - Not only listening, thinking and waiting
 - But Exercise, Simulation, Practice!

课程简介

- 课程代码: **671C0050**
- 参考书
 - 阎石,《数字电子技术基础》,第6版,高等教育出版社,2016.
 - 王金明著,《数字系统设计与Verilog HDL》,电子工业出版社,第8版
 - 《数字系统设计-补充讲义》,浙江大学自编讲义
 - Stanford 大学 108A课程notes.
 - R.H.Katz, G.Borriello, *Contemporary Logic Design*, second edition,电子工业出版社,2005.
 - M.M.Mano, 数字设计(第四版),电子工业出版社,2010.
 - » <http://www.prenhall.com/mano>

Other Course Info

- **Website:**

- <http://mypage.zju.edu.cn/wdwd/教学工作/>
- **学在浙大: 数字系统设计**
- Check frequently

- **答疑**

- 玉泉信电楼308室/周一周四下午2:30-5:30
- 上课课间、课后均可
- Email, 钉钉/微信群/数字系统设计, 短信均可

Grading (考核)

Final grades will be computed approximately as follows:

- 平时（含课程作业、期中考试、小测验、Project、出勤等） **40%**

- 期中考试 **10%**
- Homework Sets **15% (12~20)%**
- Project **5% (5~7)%**
- Class Room Check+小测验等 **10% (7~10)%**
- 作业每周一上交, **截止期为课后一周内**有效。

- **Project**

- Project-1必做 (1 members)
- Project-2可选(成绩加分1%~5%, 但不超过平时成绩范围)

- Finial Exam期末**闭卷**考试 - **60%**

授课时间和地点:

- 2025年春夏学期,
 - 周一上午, 第1~2节(8:00-9:35)
 - 星期四上午, 第6~7节(13:25-13:00)
- 地点: 紫金港西2-415(多)
- <http://mypage.zju.edu.cn/wdwd/教学工作/>
- 学在浙大/数字系统设计
- 钉钉/微信/数字系统设计-答疑

课程结构

- 数字理论知识(必备)
 - 数字系统和编码、逻辑代数、门电路
- 数字电路分析与设计
 - 组合逻辑电路
 - 触发器、半导体存贮器、可编程器件
 - 时序逻辑电路
- 脉冲电路与接口
- 控制器与数字系统
 - 状态机
 - 控制器
 - 微码控制器
 - 测试和验证
- 微处理器简介
 - 指令集
 - **4位CPU**介绍

第六章 时序逻辑电路

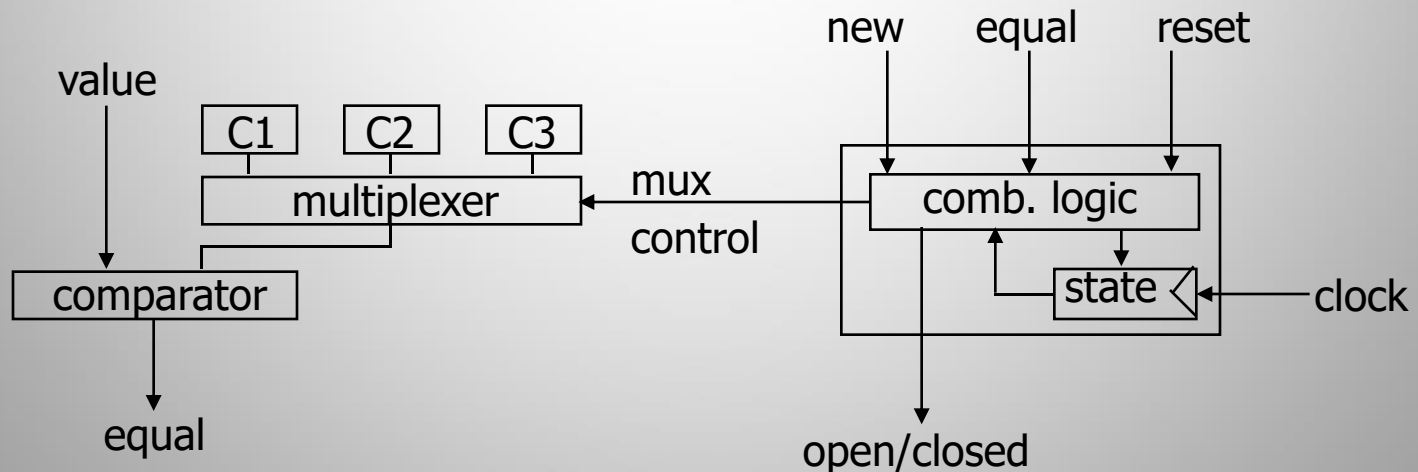
Sequential Logic时序逻辑介绍（序）

- **Sequential Circuits**时序
 - Simple circuits with feedback
 - Latches
 - Edge-triggered flip-flops 啪嗒作响
- **Timing Methodologies**定时
 - Cascading 级联 flip-flops for proper operation
 - Clock skew 时钟偏移

Sequential Circuits (复习)

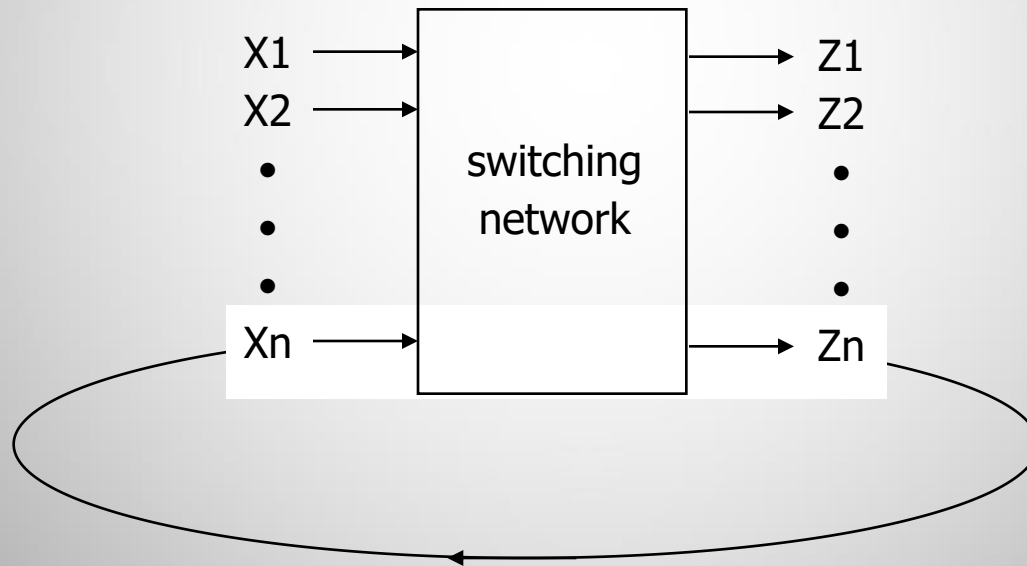
- **Circuits with Feedback**

- Outputs = $f(\text{inputs, past inputs, past outputs})$
- Basis for building "memory" into logic circuits
- Door combination lock is an example of a sequential circuit
 - » *State is memory*
 - » *State is an "output" and an "input" to combinational logic*
 - » *Combination storage elements are also memory*



Circuits with Feedback

- **How to control feedback?**
 - What stops values from cycling around endlessly

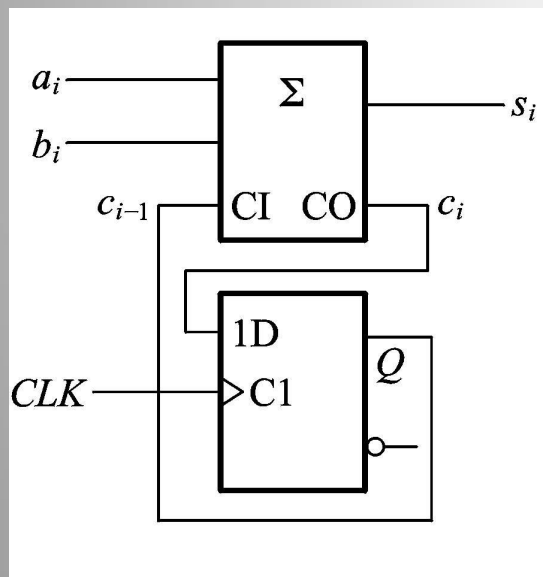


6.1 概述

一、时序逻辑电路的特点

1. 功能上：任一时刻的输出不仅取决于该时刻的输入，还与电路原来的状态有关。

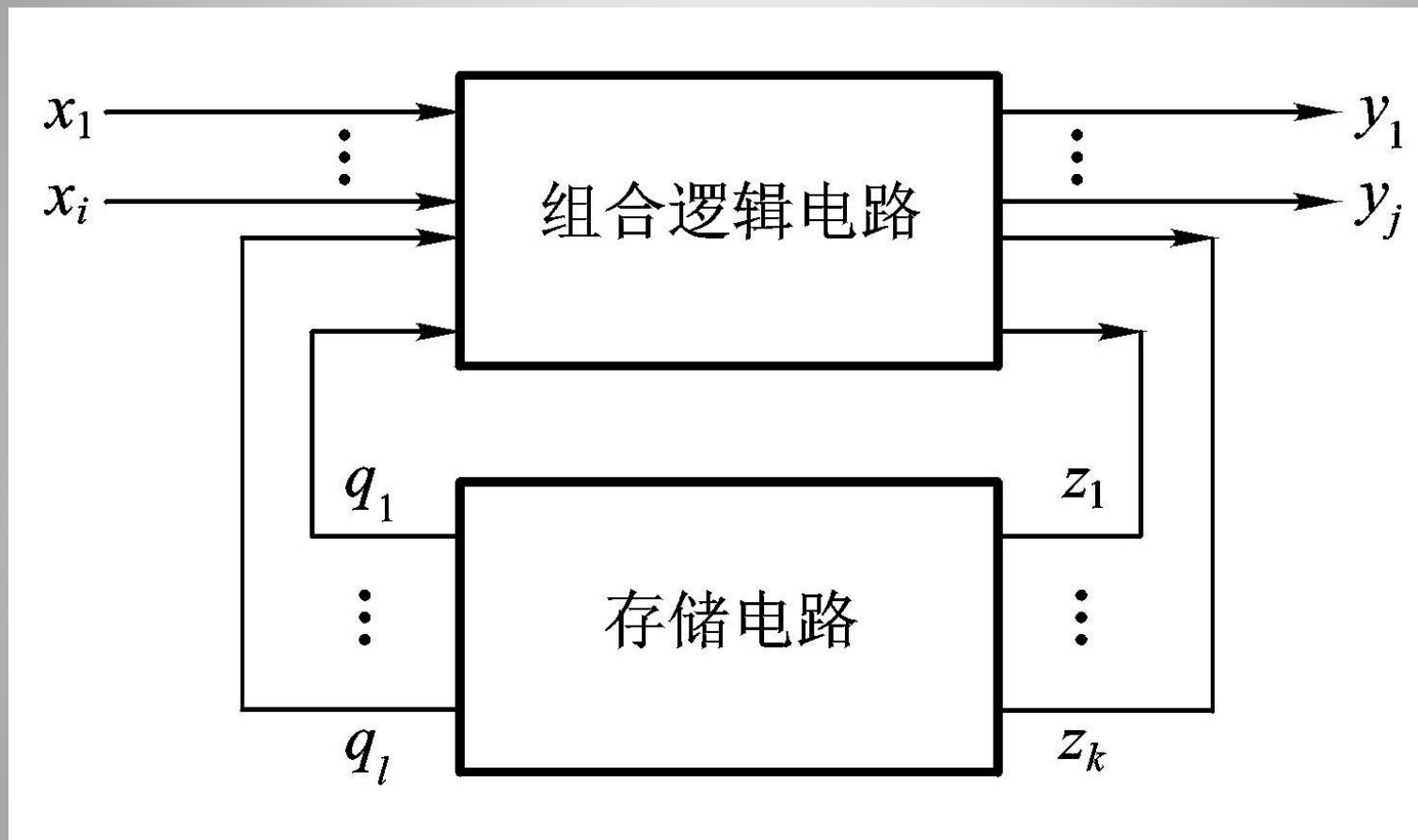
例：串行加法器，两个多位数从低位到高位逐位相加



2. 电路结构上

- ①包含存储电路和组合电路
- ②存储器状态和输入变量共同决定输出

二、时序电路的一般结构形式与功能描述方法



可以用三个方程组来描述:

$$\begin{cases} y_1 = f_1(x_1, x_2, \dots, x_i, q_1, q_2, \dots, q_l) \\ \vdots \\ y_j = f_j(x_1, x_2, \dots, x_i, q_1, q_2, \dots, q_l) \end{cases} \Rightarrow \text{输出方程 } Y = F(X, Q)$$

$$\begin{cases} z_1 = g_1(x_1, x_2, \dots, x_i, q_1, q_2, \dots, q_l) \\ \vdots \\ z_k = g_k(x_1, x_2, \dots, x_i, q_1, q_2, \dots, q_l) \end{cases} \Rightarrow \text{驱动方程 } Y = F(X, Q)$$

$$\begin{cases} q_1^* = h_1(z_1, z_2, \dots, z_i, q_1, q_2, \dots, q_l) \\ \vdots \\ q_l = h_l(z_1, z_2, \dots, z_i, q_1, q_2, \dots, q_l) \end{cases} \Rightarrow \text{状态方程 } Q^* = H(Z, Q)$$

三、时序电路的分类

1. 同步时序电路与异步时序电路

同步：存储电路中所有触发器的时钟使用统一的clk,状态变化发生在同一时刻

异步：没有统一的clk,触发器状态的变化有先有后

2. Mealy型和Moore型

Mealy型： $Y = F(X, Q)$

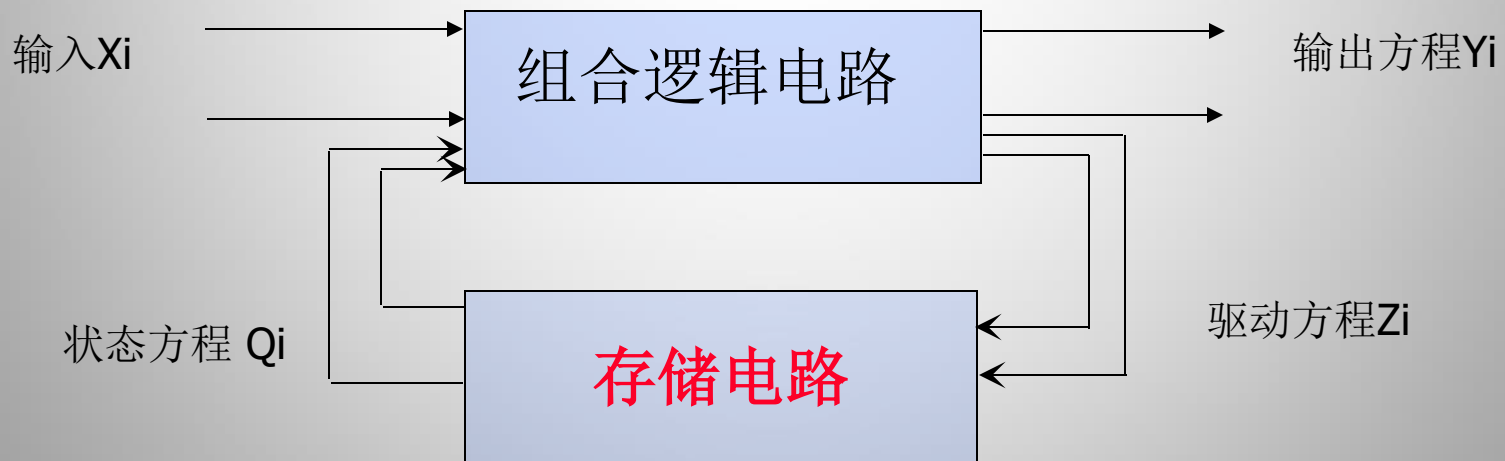
与 X 、 Q 有关

Moore型： $Y = F(Q)$

仅取决于电路状态

时序逻辑电路

- 时序电路通常包含组合电路和存储电路两部分.
- 存储电路的输出状态反馈到组合电路的输入端, 与输入信号一起, 共同决定组合逻辑电路的输出.
- 任一时刻的输出信号不仅取决于当时的输入信号, 还取决于电路原来的状态 (与以前的输入有关).



时序电路的结构框图

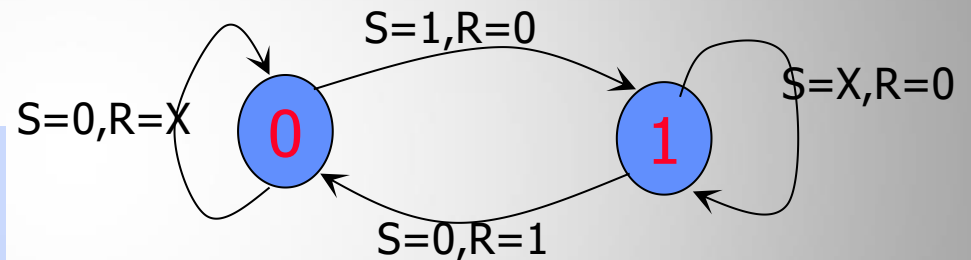
触发器Flip-Flop分类（复习）

- 逻辑功能分类
 - RS锁存器
 - JK触发器
 - T触发器
 - D触发器
- 逻辑功能指按触发器的次态和现态及输入信号之间的逻辑关系.
 - 特性表
 - 特性方程
 - 状态转换图

RS 锁存器（复习）

- 特性方程 $Q^{n+1} = S + R'Q^n$
- RS Latch 的状态转换图
- 特性表/真值表

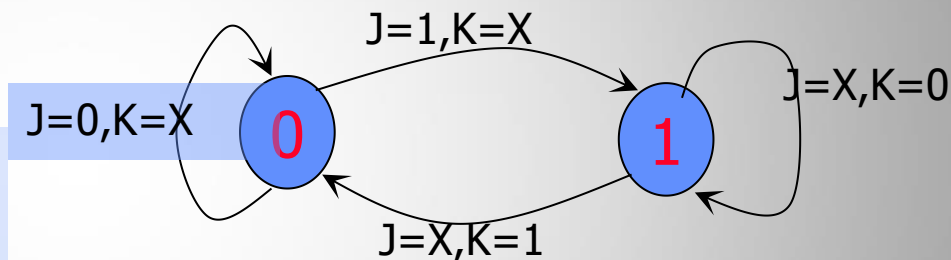
S	R	Q^n	Q^{n+1}	
0	0	0	0	} 保持
0	0	1	1	
0	1	0	0	} 复位
0	1	1	0	
1	0	0	1	} 置位
1	0	1	1	
1	1	0	0	} 不定
1	1	1	0	



JK 触发器

- 特性方程: $Q^{n+1} = JQ^n + K'Q^n$
- JK FF的状态转换图
- 特性表/真值表

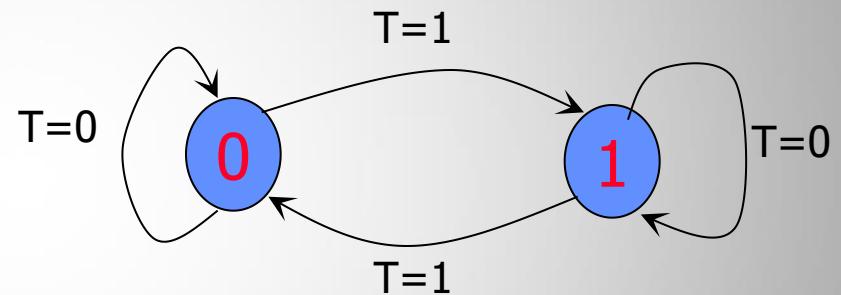
J	K	Q^n	Q^{n+1}	
0	0	0	0	} 保持
0	0	1	1	
0	1	0	0	} 复位
0	1	1	0	
1	0	0	1	} 置位
1	0	1	1	
1	1	0	1	} 翻转
1	1	1	0	



T 触发器

- 特性方程: $Q^{n+1} = TQ^n' + T'Q^n$
- T FF的状态转换图
- 特性表/真值表

T	Q^n	Q^{n+1}	
0	0	0	} 保持
0	1	1	
1	0	1	} 翻转
1	1	0	



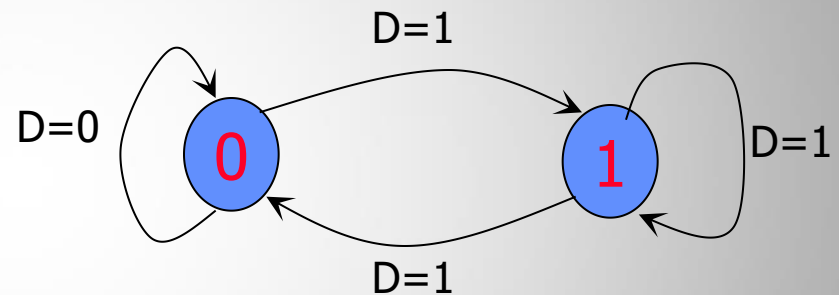
- T'触发器: $T=1, Q^{n+1} = Q^n'$

JK触发器的两个输入端连在一起作为T端, 可以构成T Flip-flop

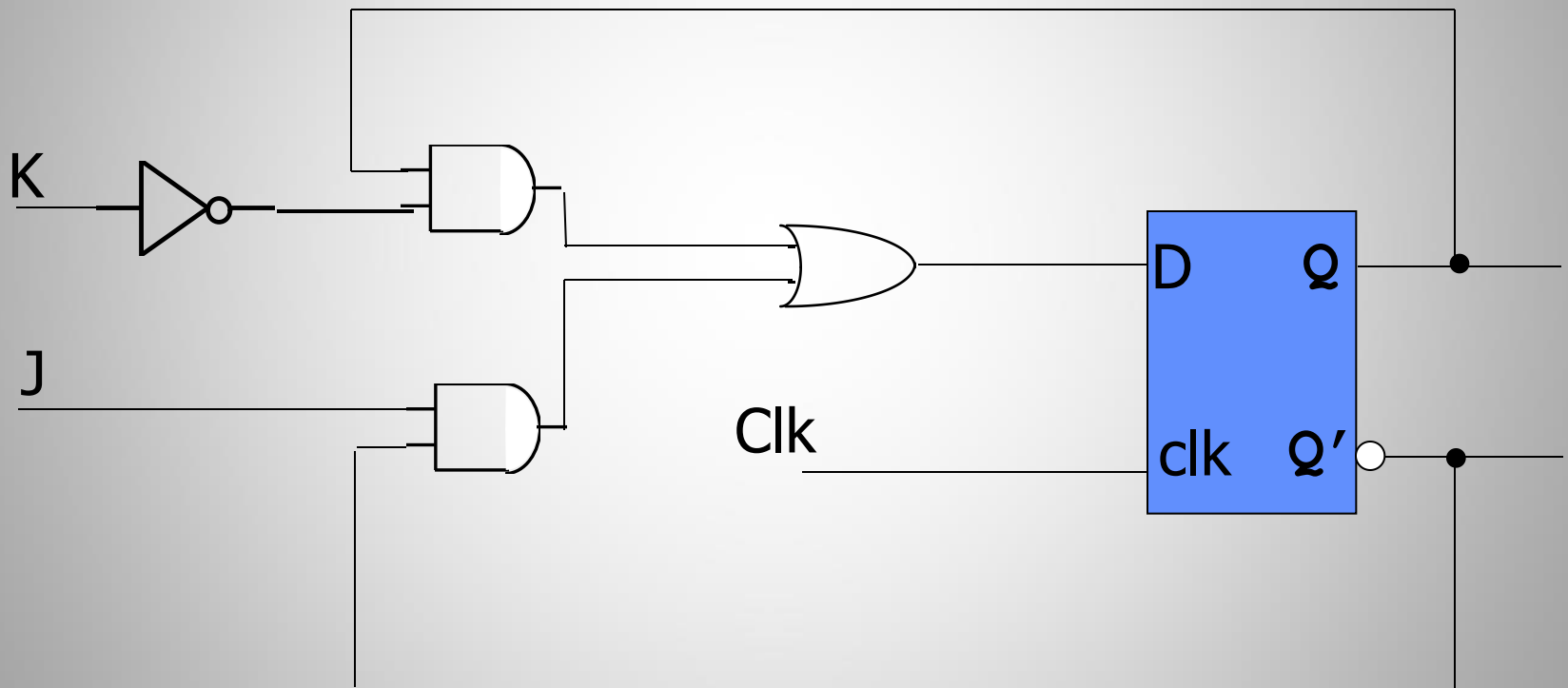
D 触发器（复习）

- 特性方程: $Q^{n+1} = D$
- D FF的状态转换图
- 特性表/真值表

D	Q^n	Q^{n+1}	
0	0	0	} reset
0	1	0	
1	0	1	} set
1	1	1	



采用D 触发器实现JK触发器（复习）



时序电路分类

- 同步时序电路

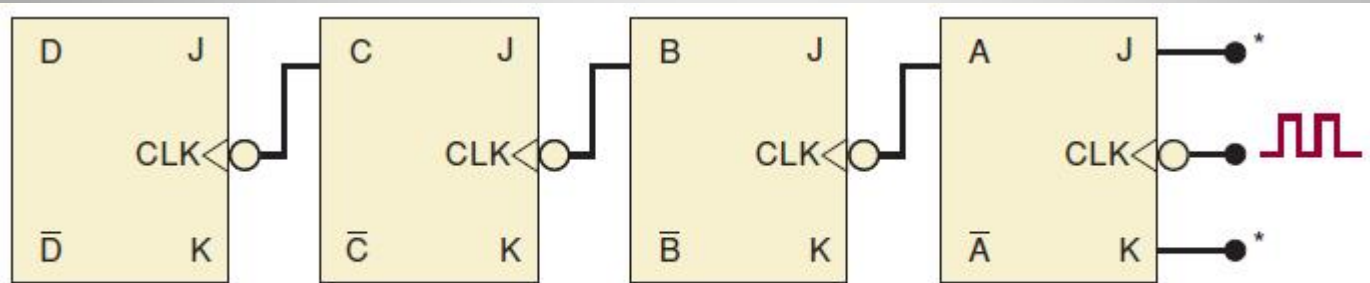
- 所有触发器状态的变化都是在同一个时钟信号下同时发生.

- 异步时序电路

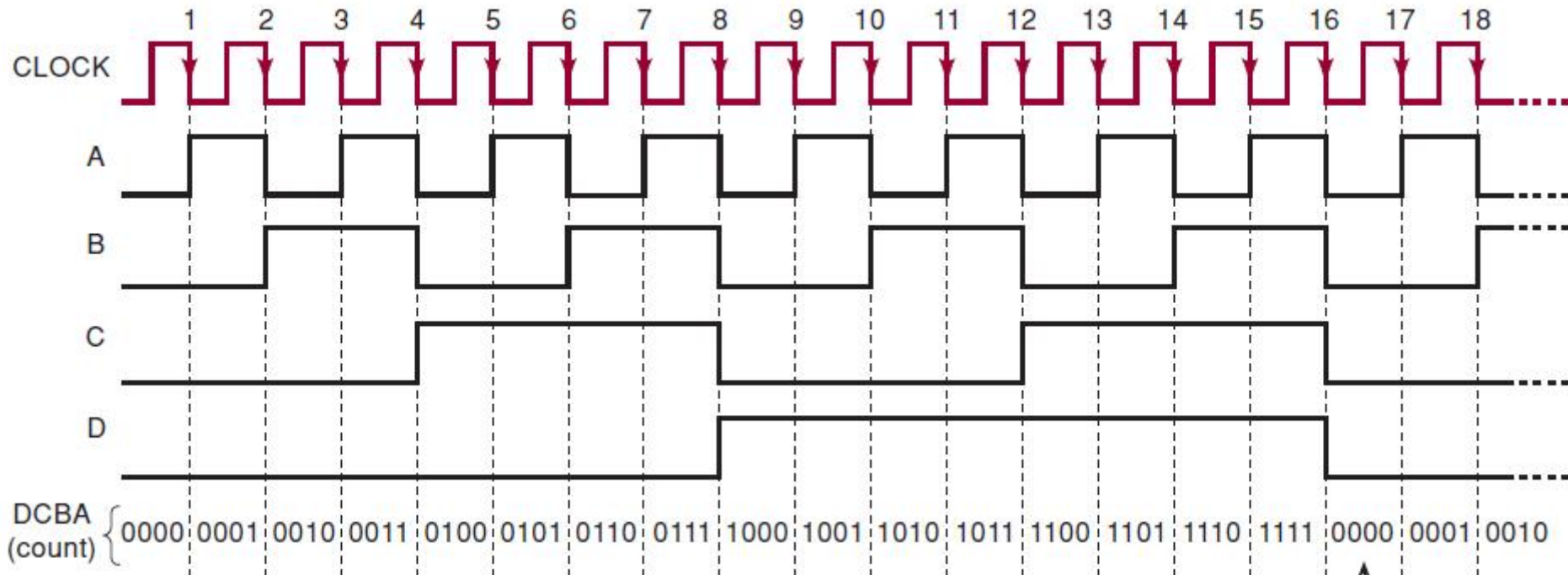
- 触发器状态的变化不是同时发生的.

异步时序电路

- ASYNCHRONOUS (RIPPLE) COUNTERS

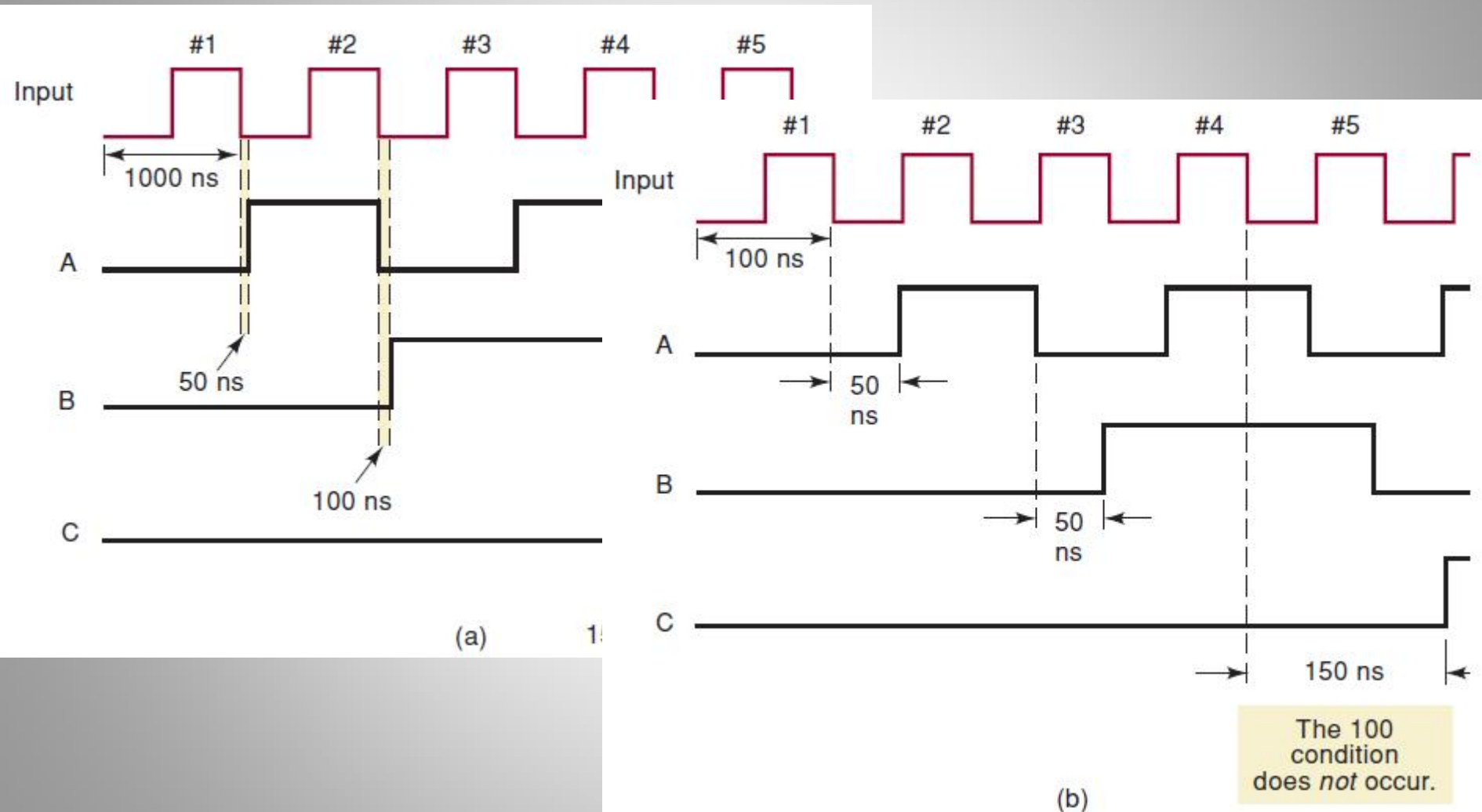


*All J and K inputs
assumed to be 1.



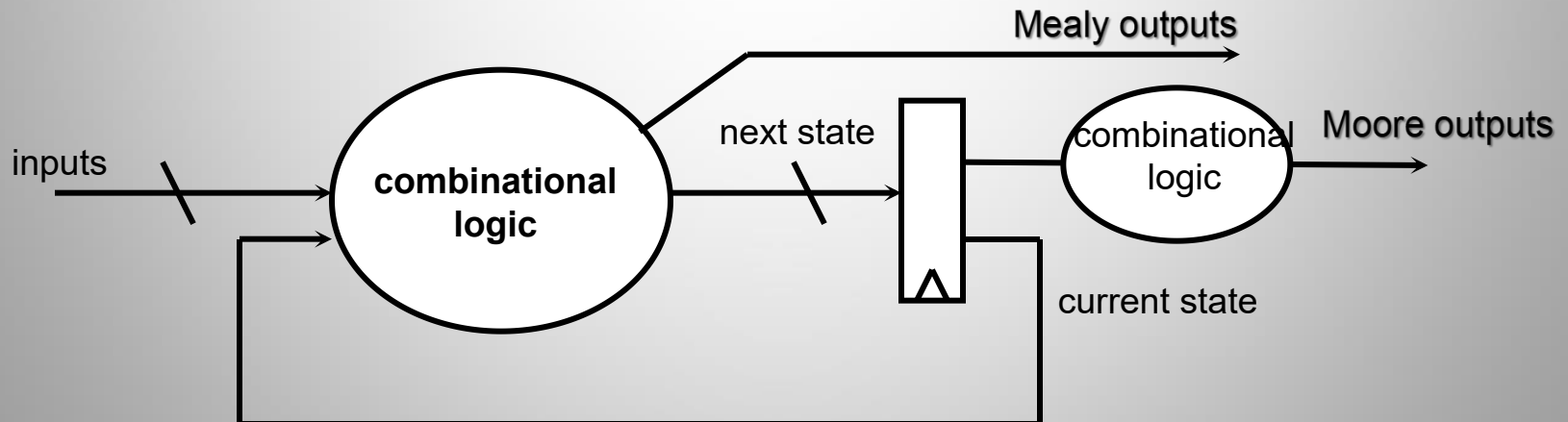
异步时序电路(2)

- PROPAGATION DELAY IN RIPPLE COUNTERS



FSM:有限状态机

- 采用输入信号和电路状态的逻辑函数去描述时序电路逻辑功能的方法
- **Mealy型**
 - 输出信号取决于存储电路状态和输入变量
- **Moore型**
 - 输出只是存储电路现态的函数
 - 输出与时钟同步



6.2 时序电路的分析方法

6.2.1 同步时序电路的分析方法

分析：找出给定时序电路的逻辑功能

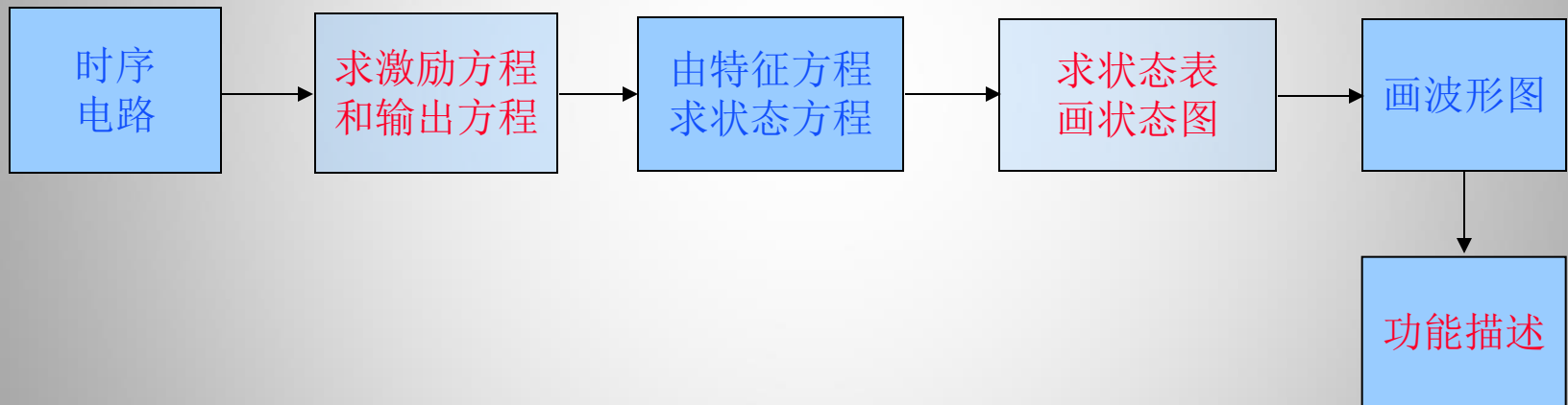
即找出在输入和CLK作用下，电路的次态和输出。

一般步骤：

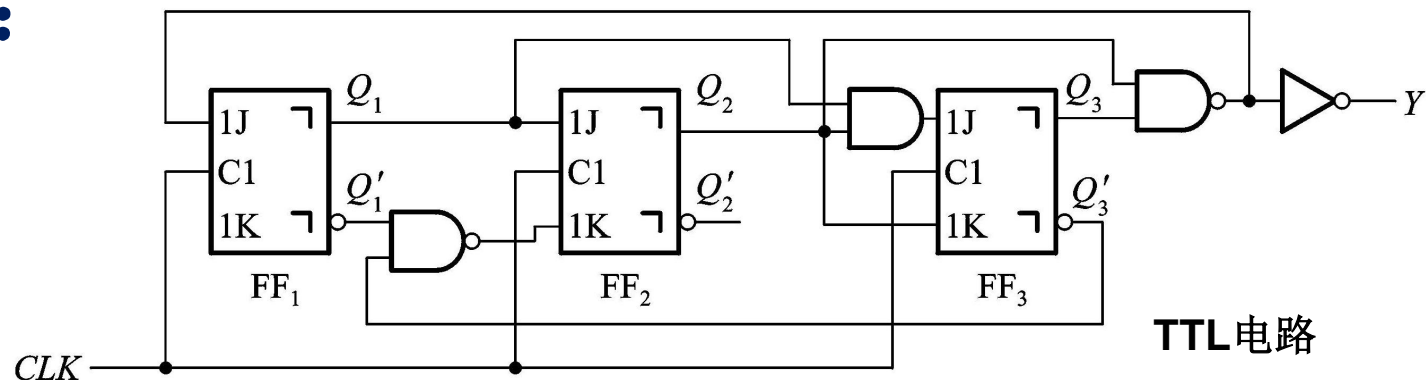
- ①从给定电路写出存储电路中每个触发器的驱动方程（输入的逻辑式），得到整个电路的驱动方程。
- ②将驱动方程代入触发器的特性方程，得到状态方程。
- ③从给定电路写出输出方程。

同步时序电路分析方法

- 目的是找出电路状态和输出信号的变换规律,
- 指出其逻辑功能



例1:



1.写驱动方程:

$$\begin{cases} J_1 = (Q_2 Q_3)', & K_1 = 1 \\ J_2 = Q_1, & K_2 = (Q_1' Q_3)' \\ J_3 = Q_1 Q_2, & K_3 = Q_2 \end{cases}$$

3.输出方程

$$Y = Q_2 Q_3$$

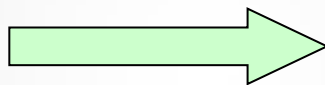
2.代入JK触发器的特性方程 ($Q^* = JQ' + K'Q$, 得状态方程:

$$\begin{cases} Q_1^* = (Q_2 Q_3)' \cdot Q_1' \\ Q_2^* = Q_1 Q_2' + Q_1' Q_3' Q_2 \\ Q_3^* = Q_1 Q_2 Q_3' + Q_2' Q_3 \end{cases}$$

6.2.2 时序电路的状态转换表、状态转换图、状态机流程图和时序图

一、状态转换表

Q_3	Q_2	Q_1	Q_3^*	Q_2^*	Q_1^*	Y
0	0	0	0	0	1	0
0	0	1	0	1	0	0
0	1	0	0	1	1	0
0	1	1	1	0	0	0
1	0	0	1	0	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	1	0	0	0	1

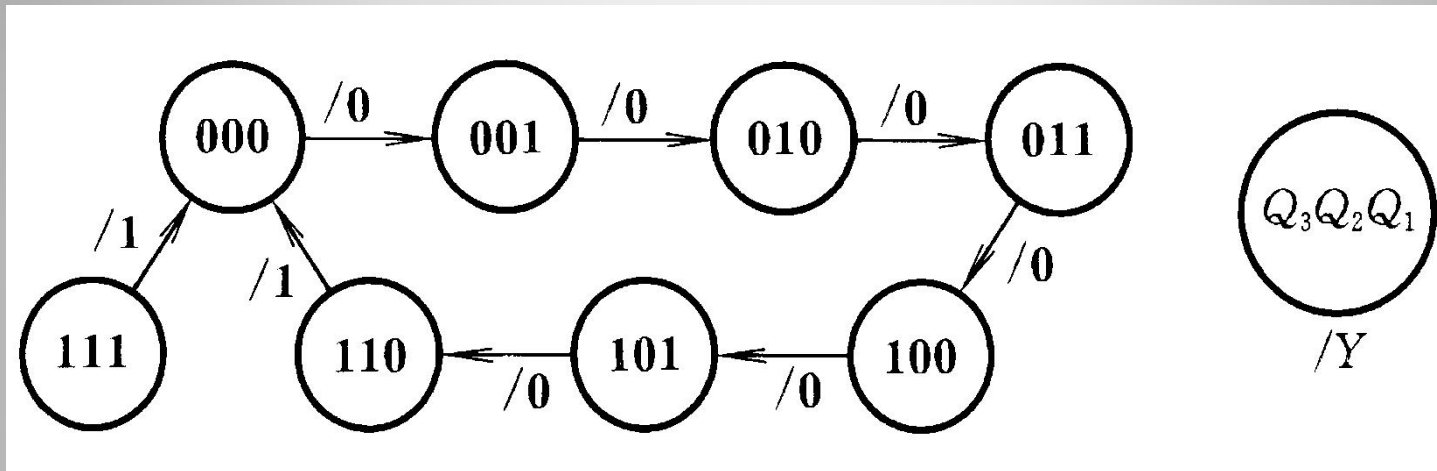


CLK	Q_3	Q_2	Q_1	Y
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	0
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
7	0	0	0	0
0	1	1	1	1
1	0	0	0	0

$$Y = Q_2 Q_3$$

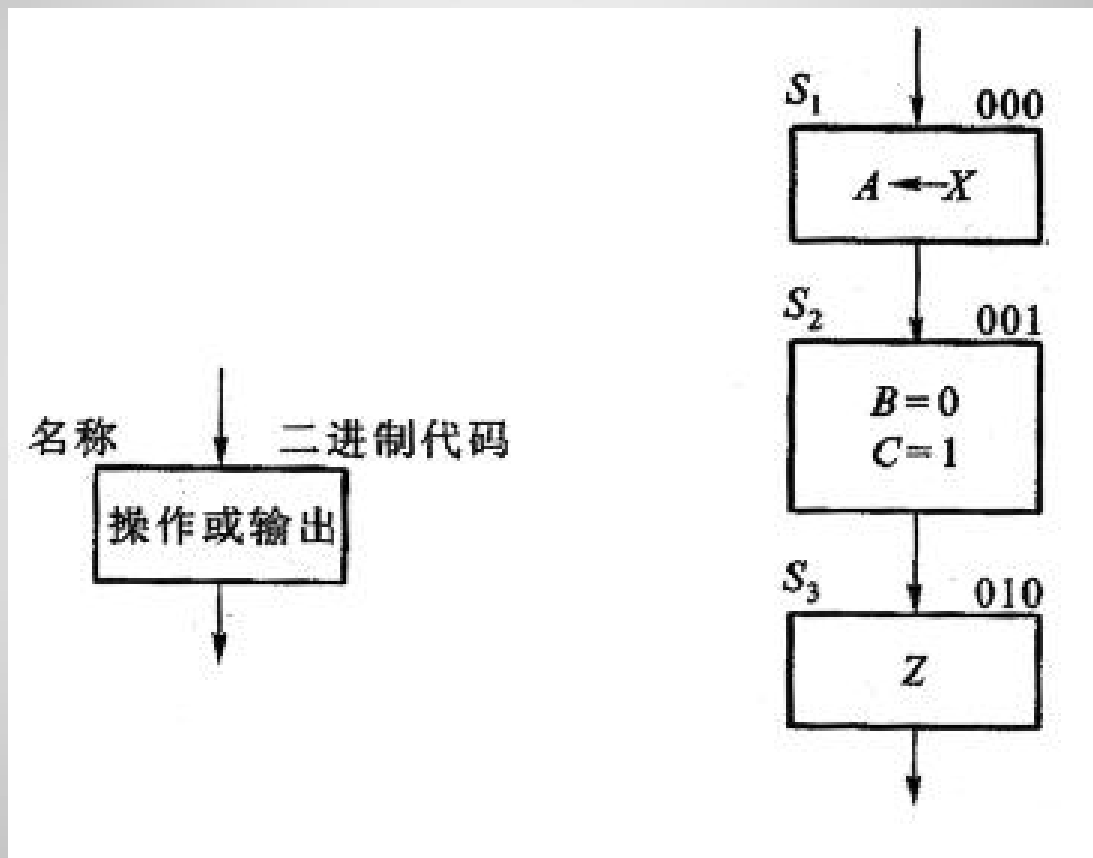
$$\begin{cases} Q_1^* = (Q_2 Q_3)' \cdot Q_1' \\ Q_2^* = Q_1 Q_2' + Q_1' Q_3' Q_2 \\ Q_3^* = Q_1 Q_2 Q_3' + Q_2' Q_3 \end{cases}$$

二、状态转换图



三、状态机流程图（State Machine Chart）

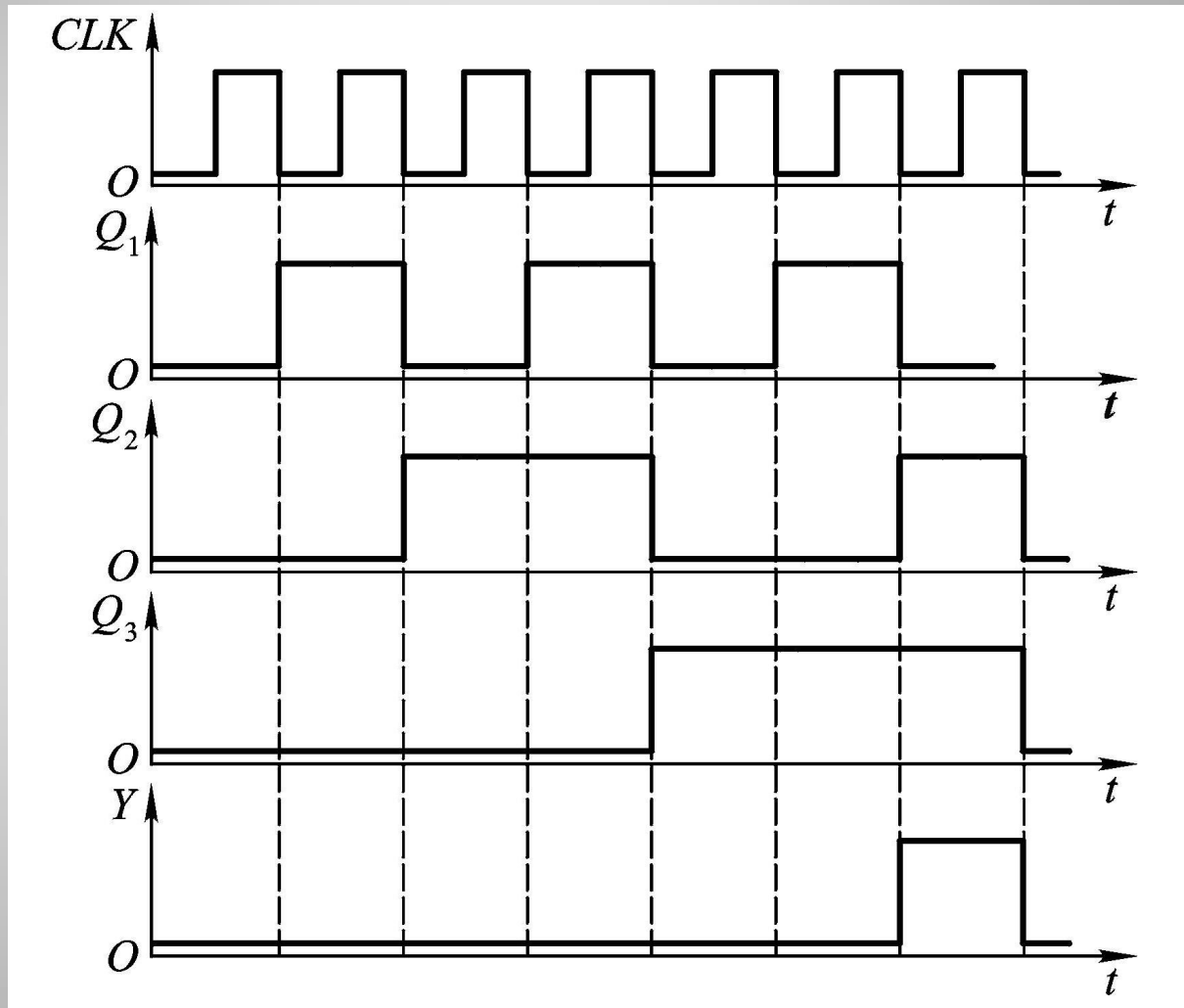
- ASM图符号/见书本267



(a)状态框符号

(b)状态框组成的ASM图实例
ASM图状态框

四、时序图



The diagram shows a 2-bit counter circuit. It consists of two D flip-flops, labeled FF_1 and FF_2 . The inputs are A and CLK . The output is Y . The circuit includes two 3-input AND gates, two 2-input OR gates, and two inverters. The inputs are connected to the D inputs of the flip-flops and the AND gates. The outputs of the flip-flops are connected to the OR gates and the AND gates. The output Y is the output of the second AND gate.

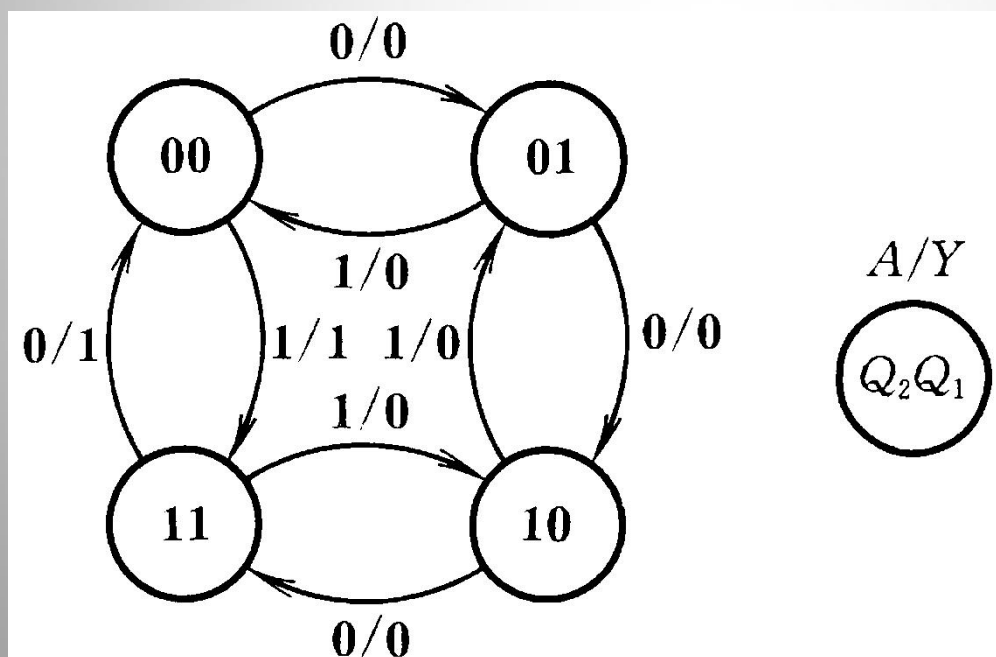
(2)状态方程 $\begin{cases} Q_1^*=D_1 \\ Q_2^*=A\oplus Q_1\oplus Q_2 \end{cases}$

$$Y = [(A'Q_1Q_2)'] \cdot (AQ_1'Q_2')' = A'Q_1Q_2 + AQ_1'Q_2'$$

(4) 列状态转换表:

$Q_2 * Q_1 \backslash A$	00	01	10	11
0	01/0	10/0	11/0	00/1
1	11/1	00/0	01/0	10/0

(5) 状态转换图



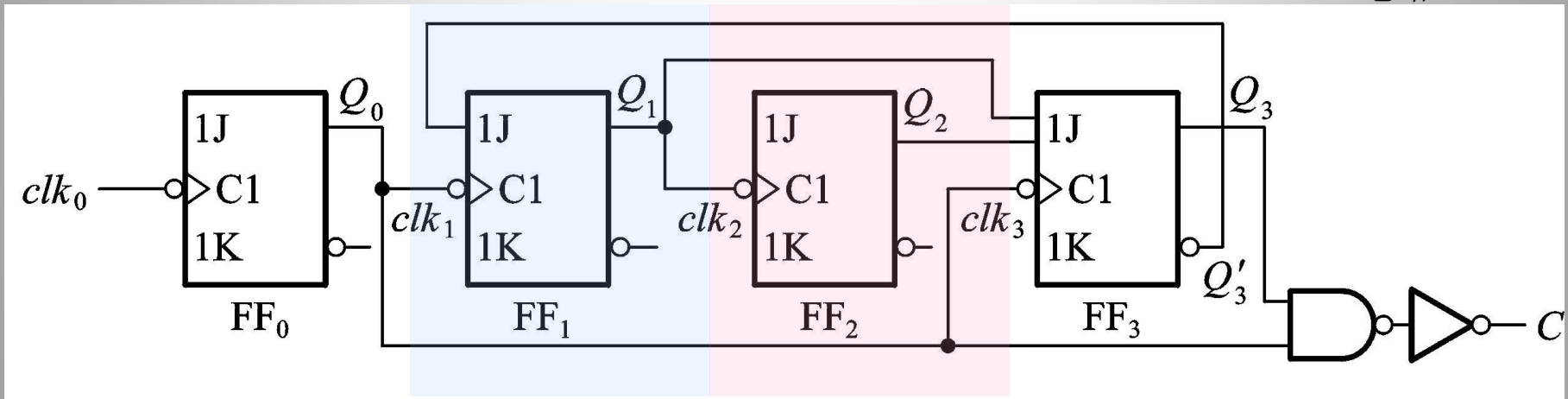
*6.2.3 异步时序逻辑电路的分析方法

各触发器的时钟不同时发生

例3:

$$Q_2^* = Q_2' \cdot clk_2$$

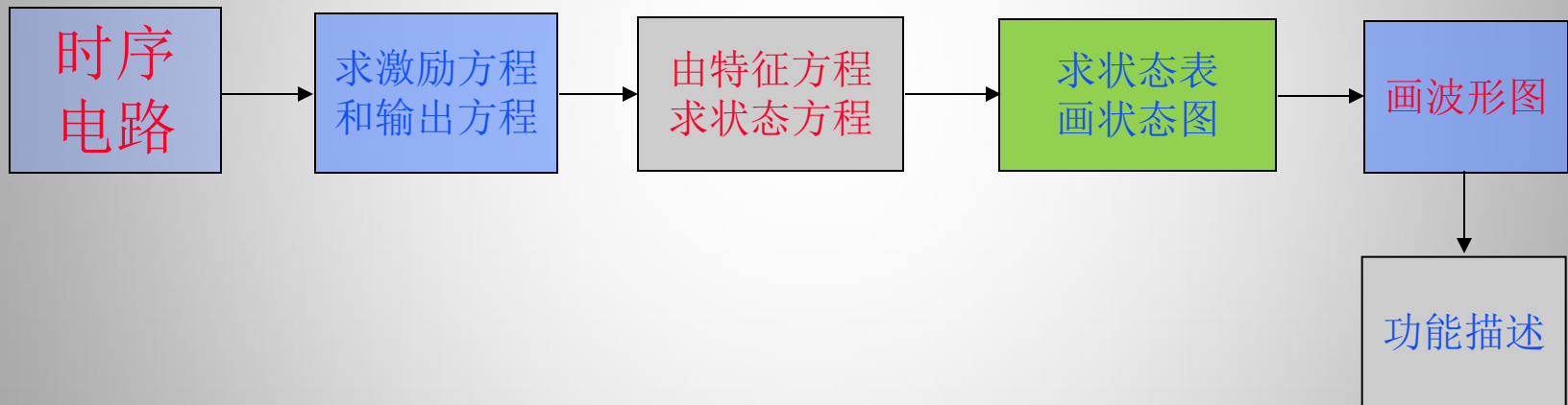
TTL 电路



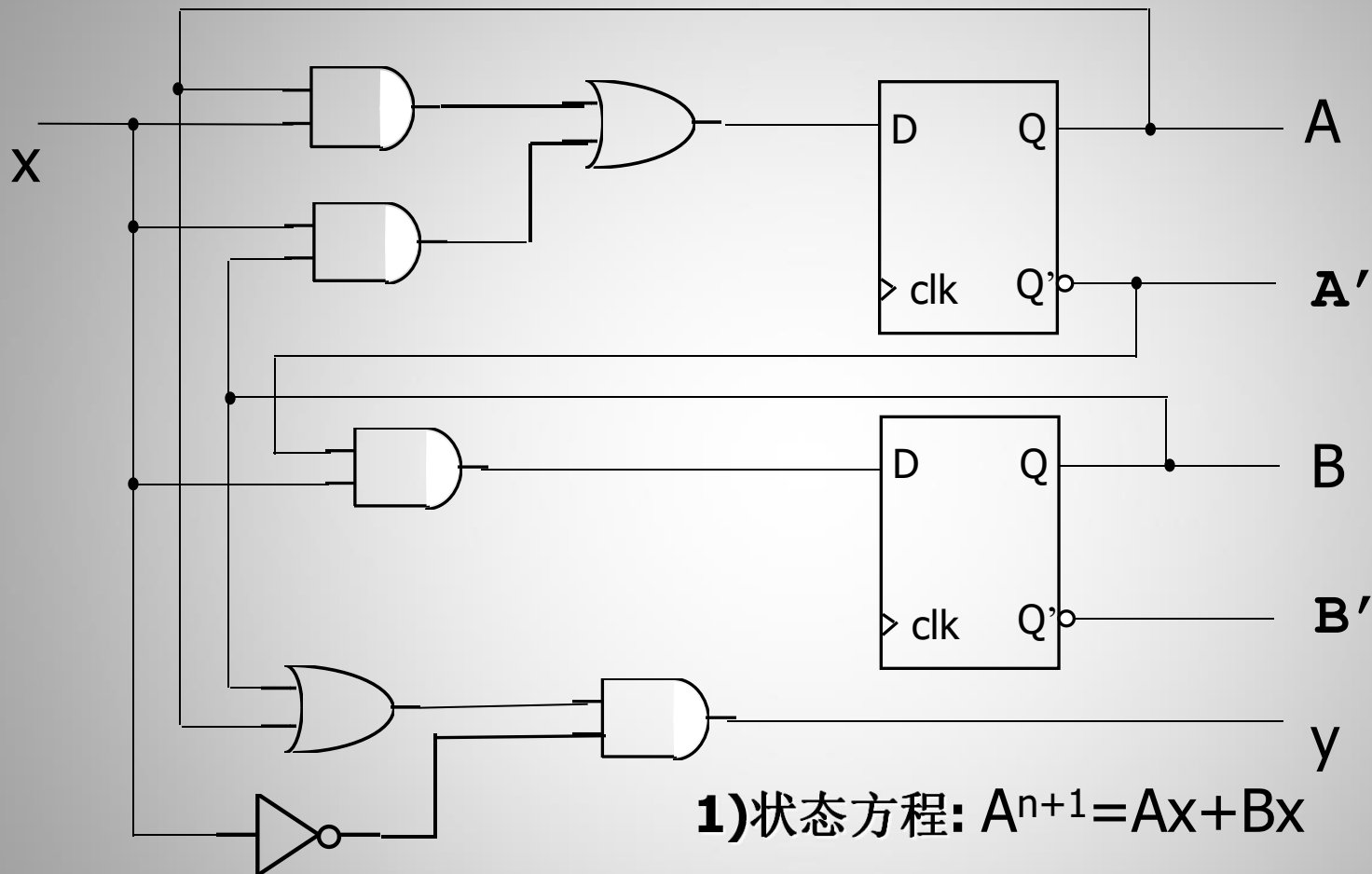
$$Q_1^* = Q_3' Q_1' \cdot clk_1$$

同步时序电路分析方法

- 目的是找出电路状态和输出信号的变换规律，指出其逻辑功能



同步时序电路分析 例-4

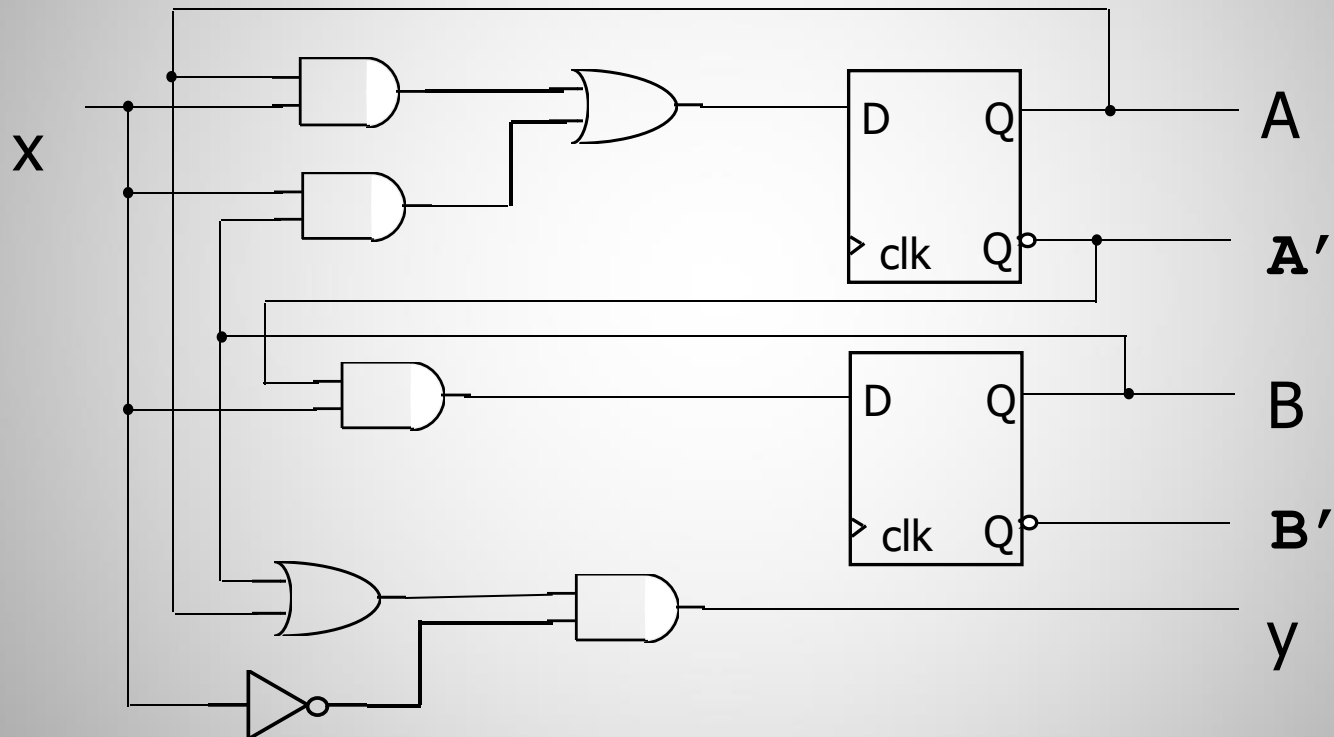


$$B^{n+1} = A'x$$

状态方程是确定触发器状态转移条件的表达式

分析例4

- 2) 输出方程 $y = (A+B) x'$

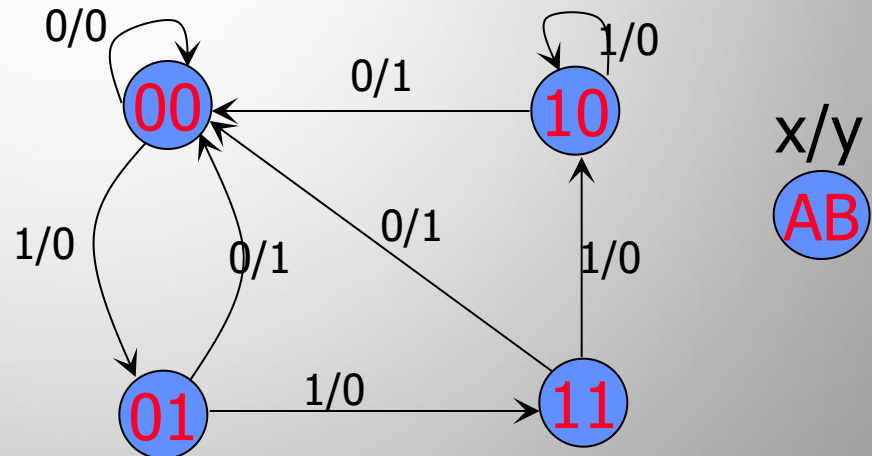


3)状态表描述/状态图

现态 输入			次态 输出		
A	B	x	A ⁿ⁺¹	B ⁿ⁺¹	Y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

现态	次态		输出	
	x=0	x=1	x=0	x=1
AB	AB	AB	Y	
00	00	01	0	0
01	00	11	1	0
10	00	10	1	0
11	00	10	1	0

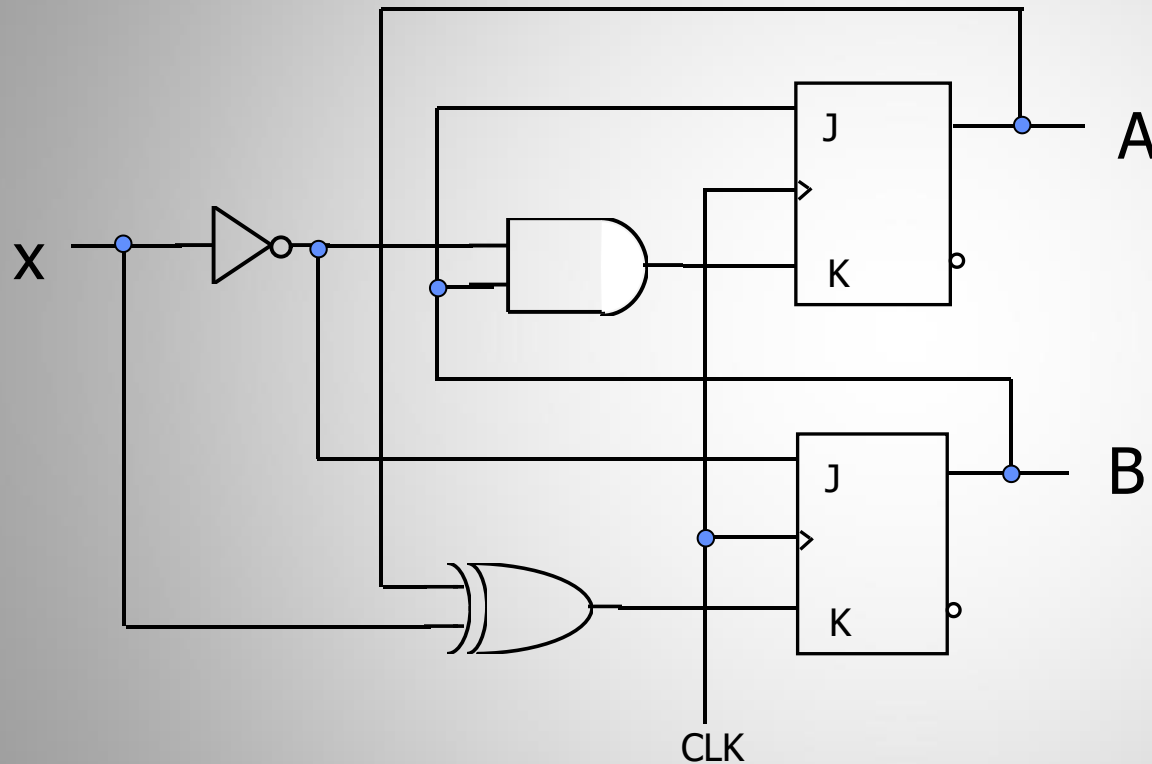
状态图



由JK触发器构成的时序电路分析

- 对D触发器,状态方程与输入方程一致.
- JK/T触发器,参考对应的特性表或特性方程来得到次态值.
 - 把触发器输入方程表示成现态和输入变量的函数.
 - 列出每个输入方程的二进制数值.
 - 利用对应触发器的特性表确定状态表中的次态值.

JK FF构成的时序电路分析-5



JK FF输入方程

$$J_A = B \quad K_A = Bx'$$

$$J_B = x' \quad K_B = A'x + Ax'$$

JK FF构成的时序电路分析(续)

- 把触发器的输入方程表示成现态和输入变量的函数.
- 将输入方程代入到触发器的特性方程中,得到状态方程.
- 使用对应的状态方程确定状态表中的次态.

JK FF特性方程:

$$Q^{n+1} = JQ^n + K'Q^n$$

JK FF输入方程:

$$J_A = B \quad K_A = Bx'$$

$$J_B = x' \quad K_B = A'x + Ax'$$

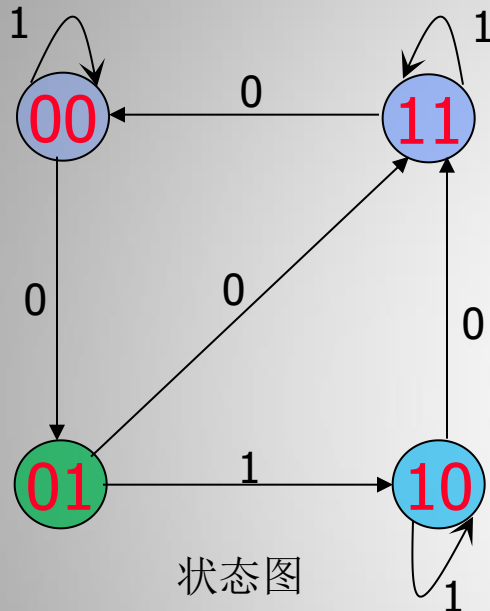
状态方程:

$$A^{n+1} = A'B + AB' + Ax$$

$$B^{n+1} = B'x' + ABx + A'Bx'$$

现态 输入			次态		触发器输入			
A	B	x	A	B	J _A	K _A	J _B	K _B
0	0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	0	1
0	1	0	1	1	1	1	1	0
0	1	1	1	0	1	0	0	1
1	0	0	1	1	0	0	1	1
1	0	1	1	0	0	0	0	0
1	1	0	0	0	1	1	1	1
1	1	1	1	1	0	0	0	0

JK FF构成的时序电路分析(续)



状态方程:

$$A^{n+1} = A' B + AB' + AX$$

$$B^{n+1} = B' x' + ABx + A' Bx'$$

现态 输入			次态		触发器输入			
A	B	x	A	B	J _A	K _A	J _B	K _B
0	0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	0	1
0	1	0	1	1	1	1	1	0
0	1	1	1	0	1	0	0	1
1	0	0	1	1	0	0	1	1
1	0	1	1	0	0	0	0	0
1	1	0	0	0	1	1	1	1
1	1	1	1	1	0	0	0	0

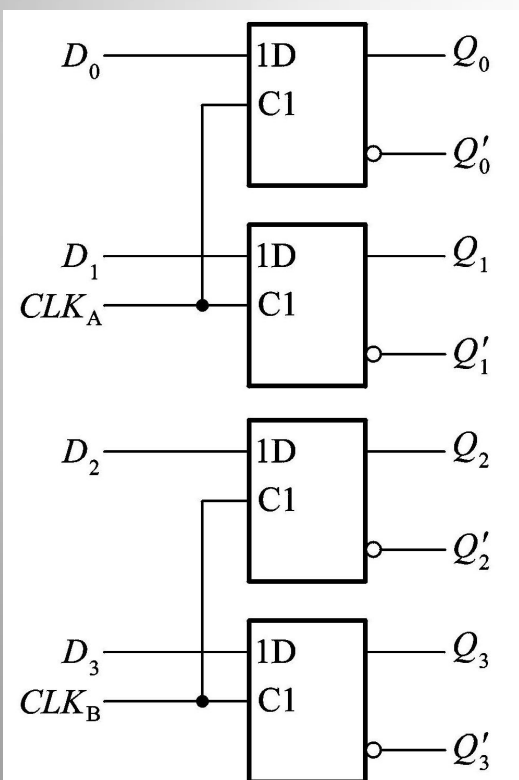
6.3 若干常用的时序逻辑电路

6.3.1 寄存器和移位寄存器

一、寄存器

- ①用于寄存一组二值代码，**N**位寄存器由**N**个触发器组成，可存放一组**N**位二值代码。
- ②只要求其中每个触发器可置1，置0。

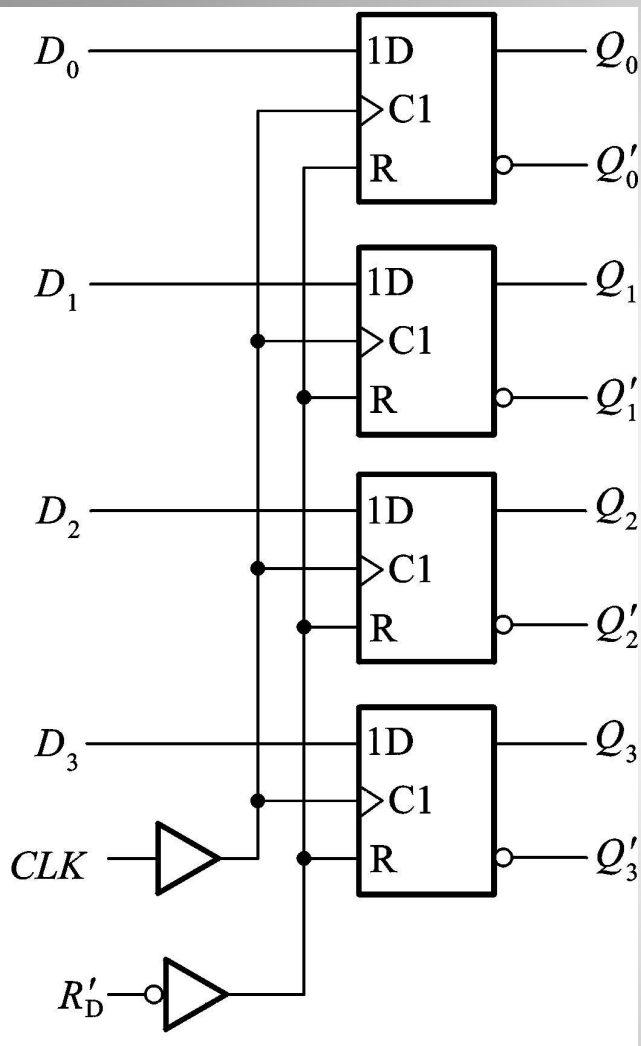
例1:



74 *LS* 75

clk 高电平期间 Q 随 D 改变

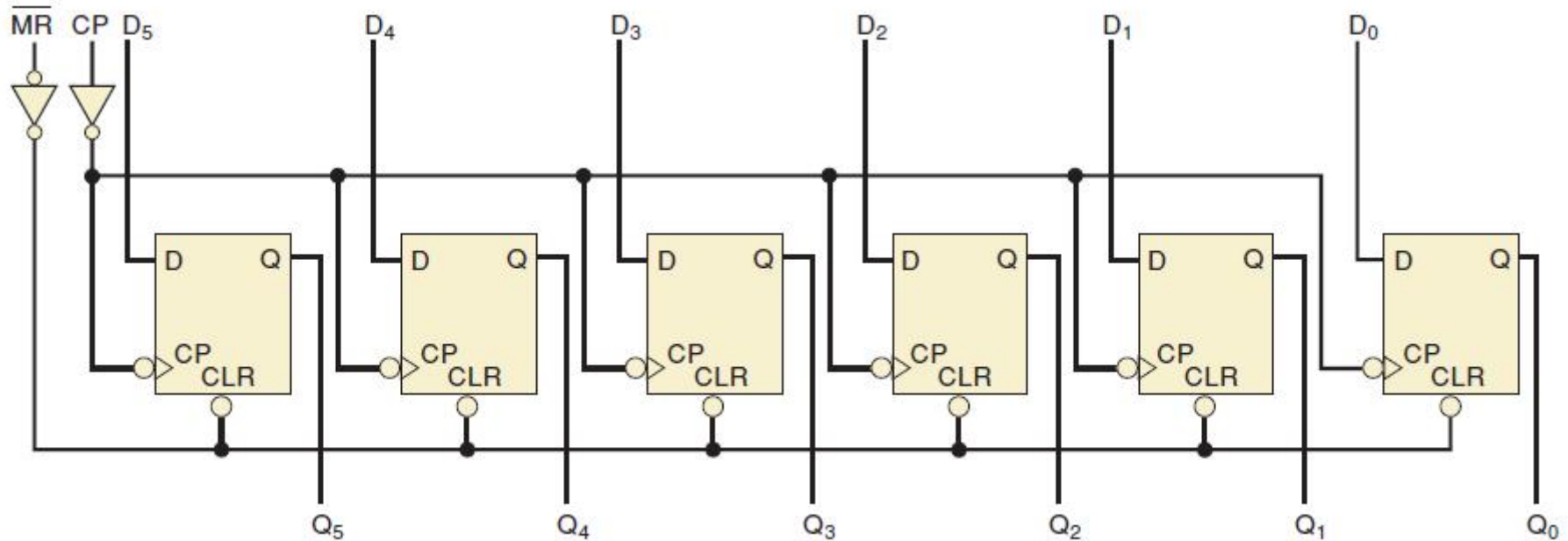
例2：用维-阻触发器结构的74HC175



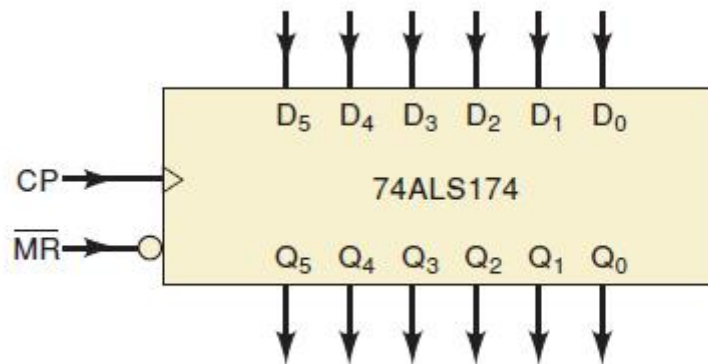
74HC175

$CLK \uparrow$ 时，将 $D_0 \sim D_3$ 存入，与此前后的 D 状态无关，有异步置0功能。

PARALLEL IN/PARALLEL OUT



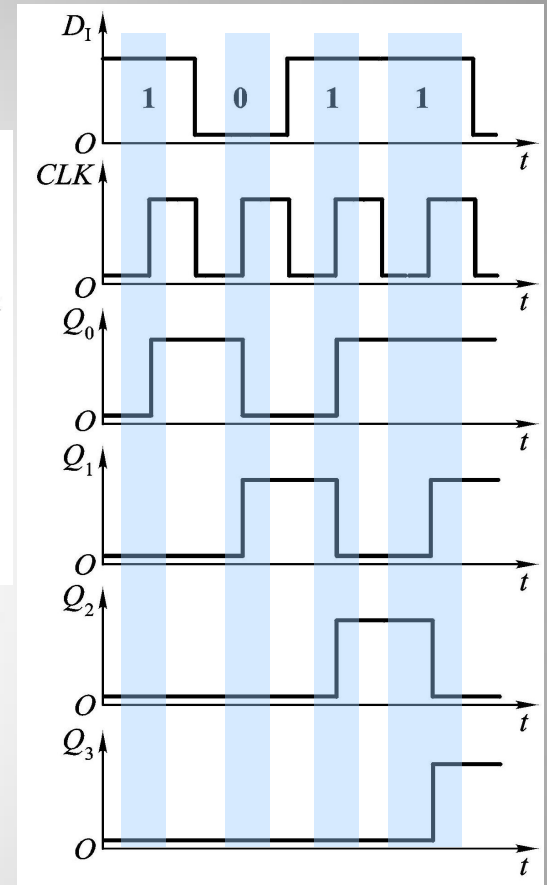
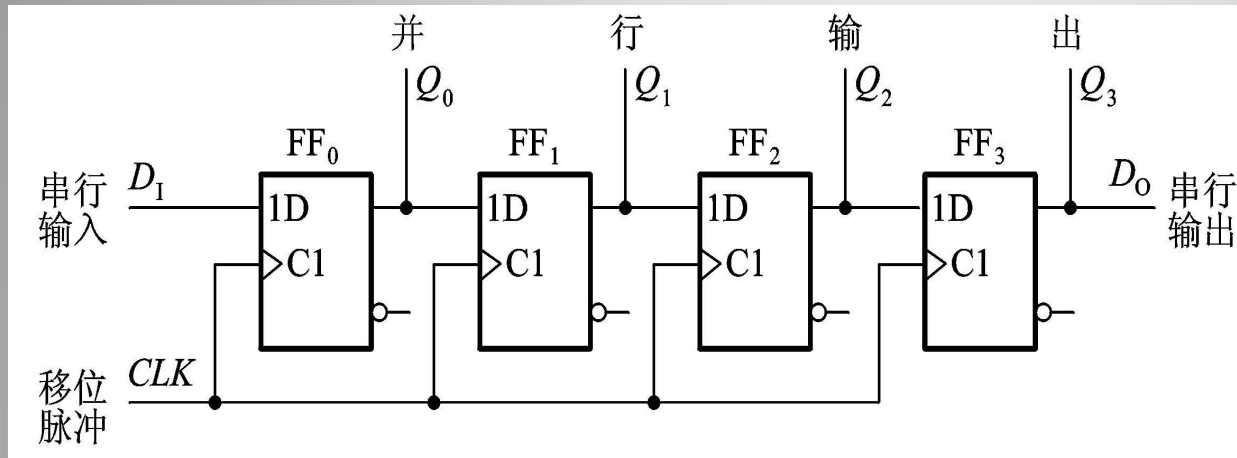
(a)



(b)

二、移位寄存器（代码在寄存器中左/右移动）

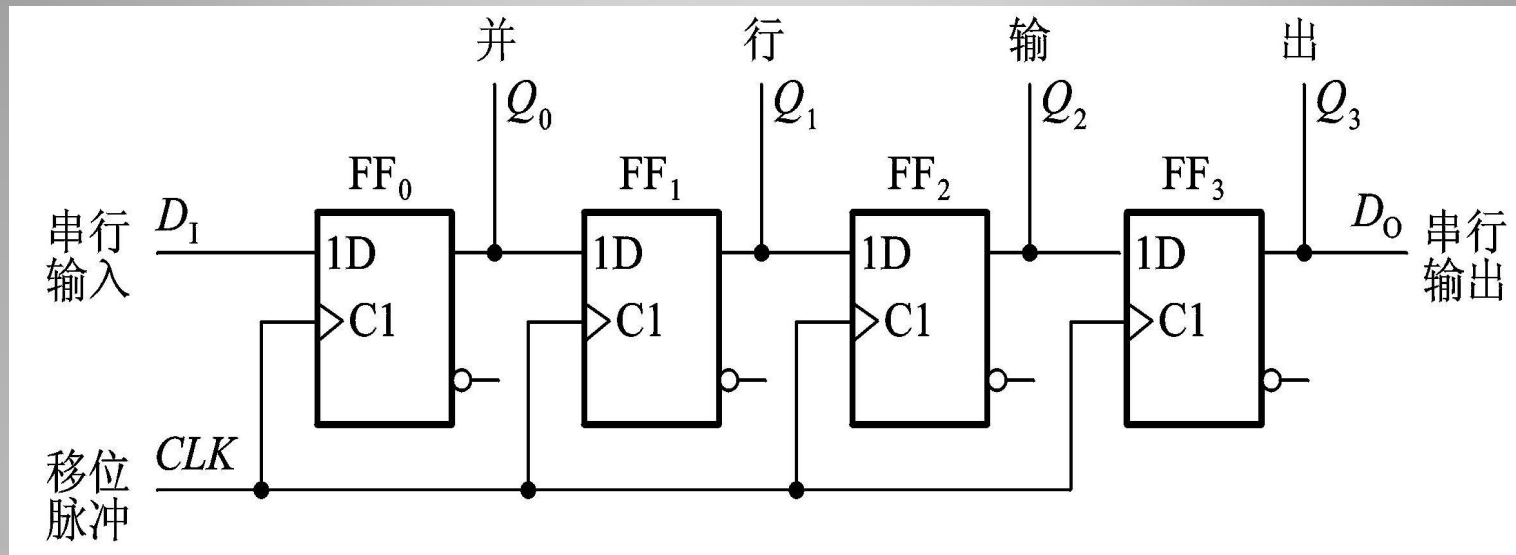
具有存储 + 移位功能



因为 触发器有延迟时间 t_{pd}

所以 $CLK \uparrow$ 到达时，各触发器按前一级触发器原来的状态 翻转

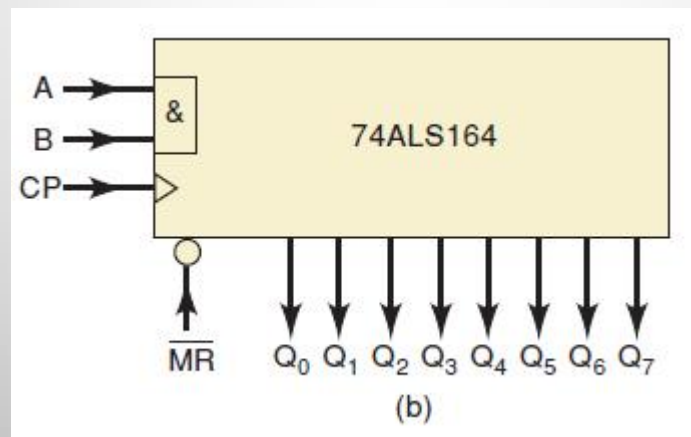
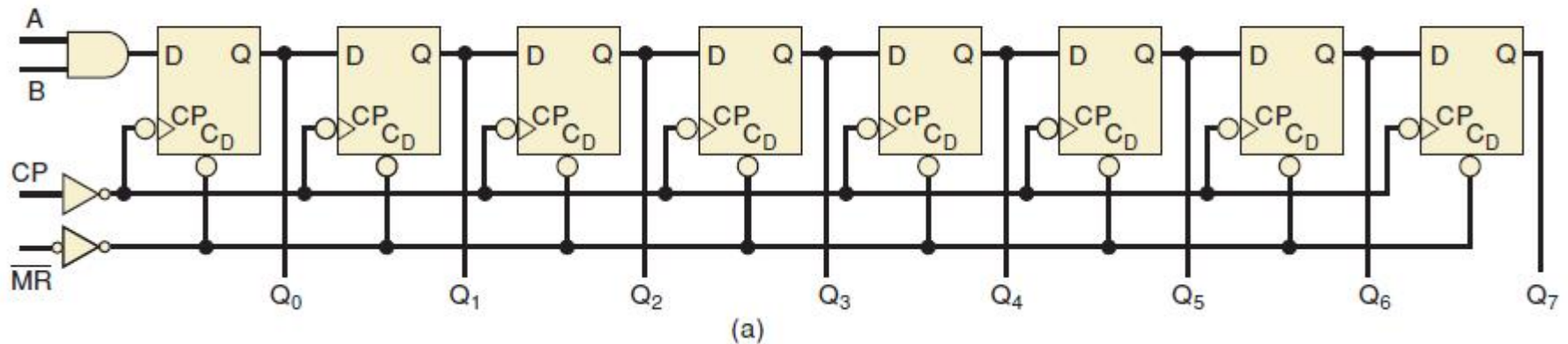
↓
数据依次右移1位



应用：
代码转换，串 $\Leftrightarrow$ 并
数据运算

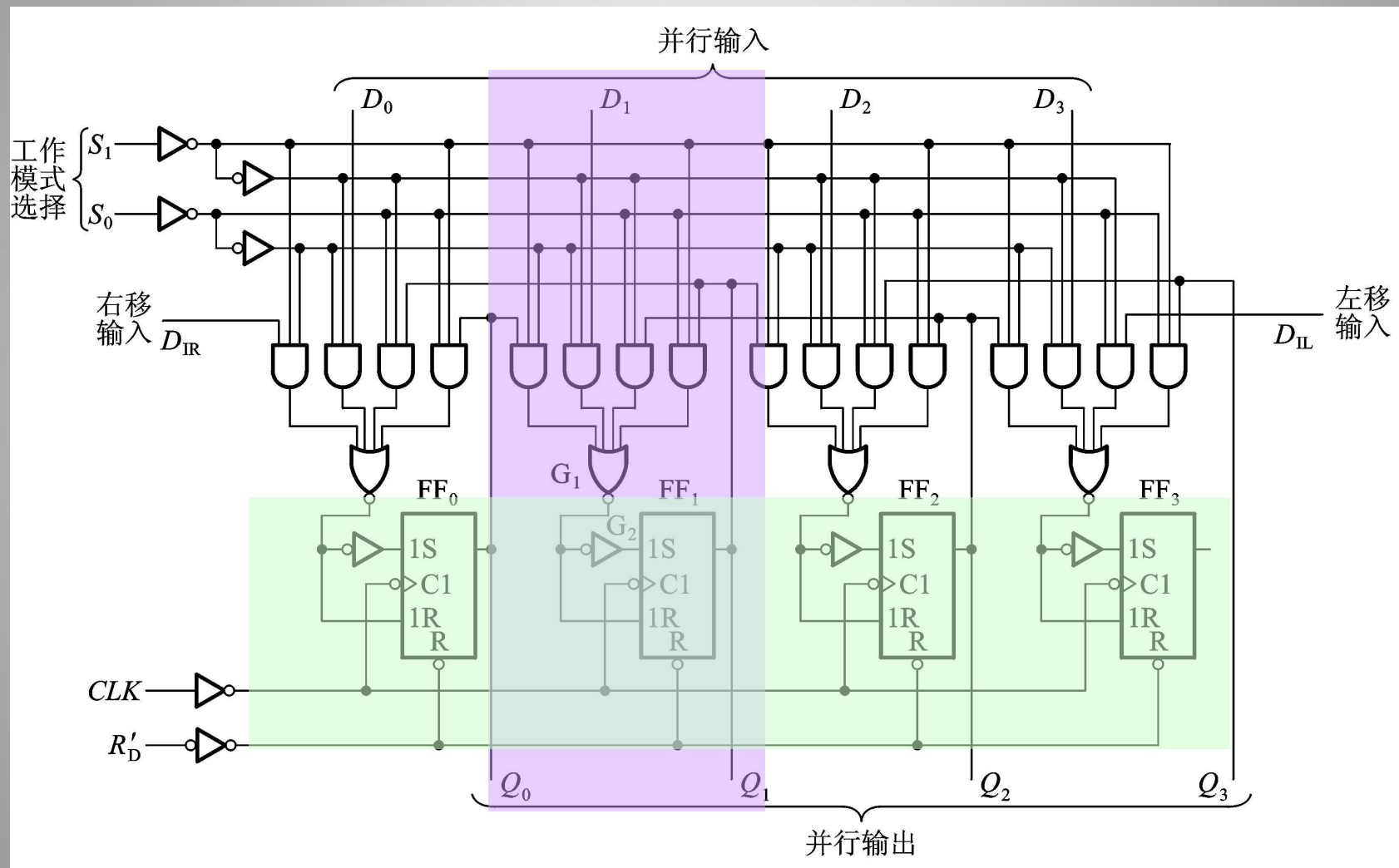
SERIAL IN/PARALLEL OUT

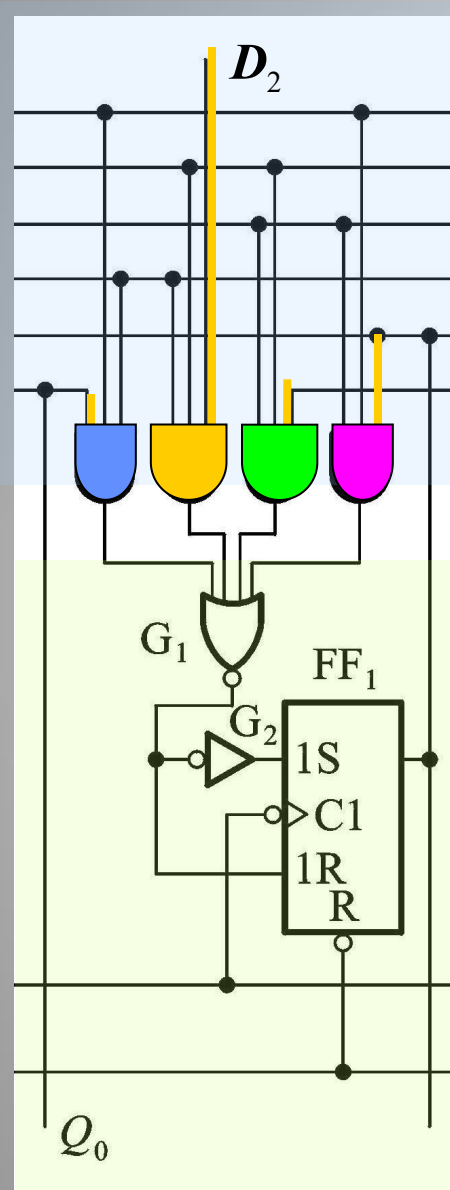
8-bit
shift register
74ALS164



器件实例：74HC194A

左/右移，并行输入，保持，异步置零等功能





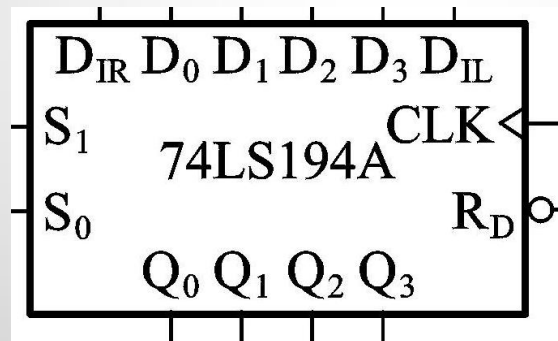
$$S_{Q1} = S'_1 S'_0 \cdot Q_1 + S'_1 S_0 \cdot Q_0 + S_1 S'_0 Q_2 + S_1 S_0 D_1$$

$$R_{Q1} = S'_{Q1}$$

$$Q_1^* = S_{Q1}$$

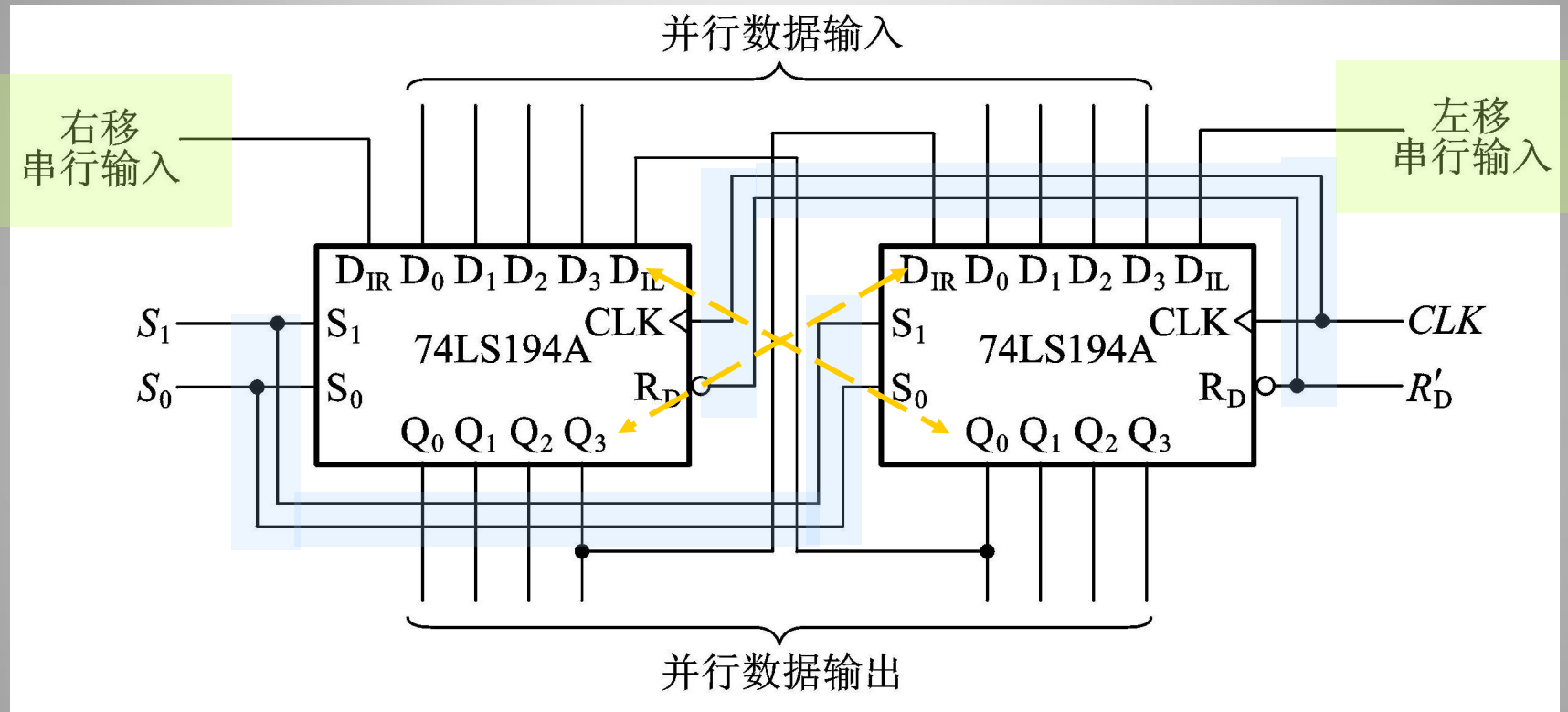


通过控制 $S_1 S_0$
就可以选择194的工作状态



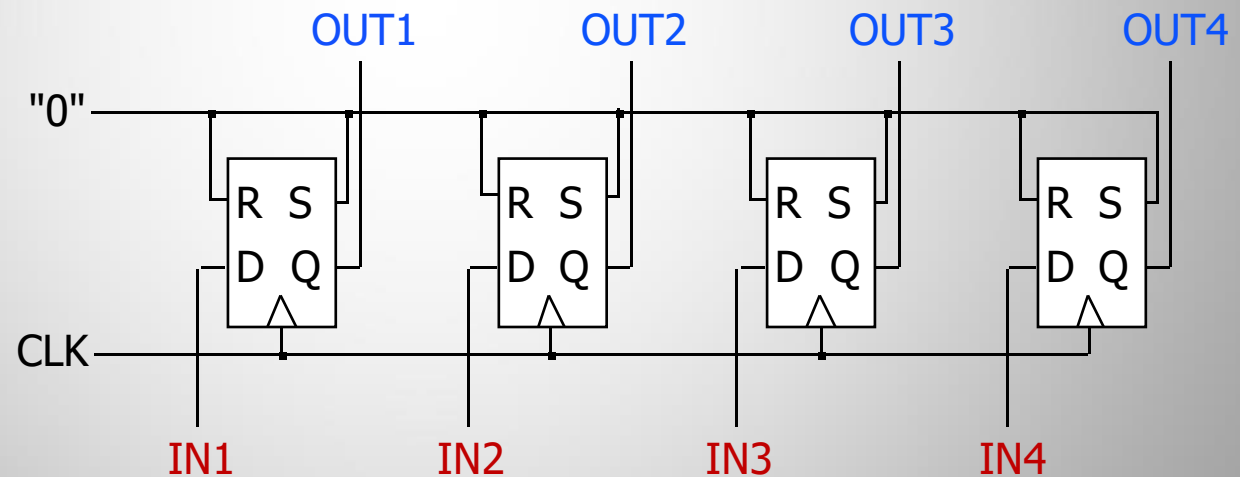
R'_D	S_1	S_0	工作状态
0	X	X	置零
1	0	0	保持
1	0	1	右移
1	1	0	左移
1	1	1	并行输入

扩展应用（4位 → 8位）

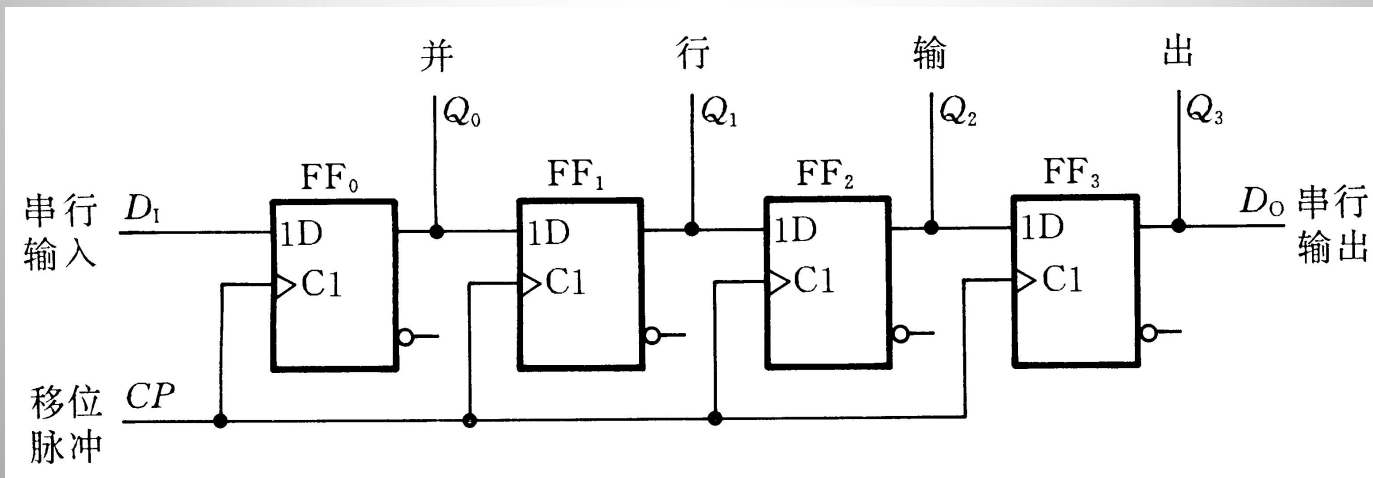
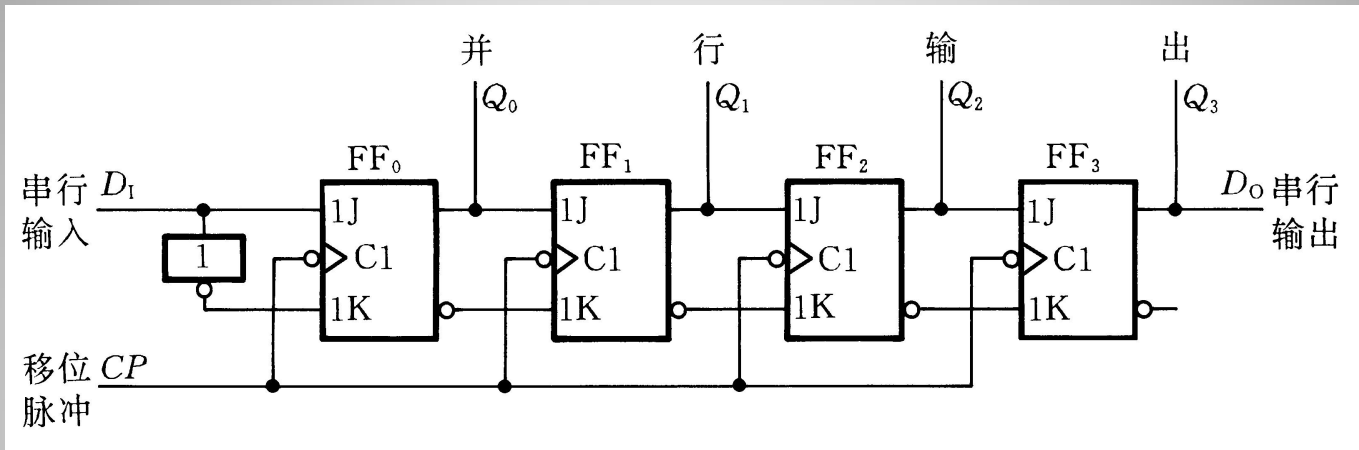


Registers

- **Collections of flip-flops with similar controls and logic**
 - Stored values somehow related (e.g., form binary value)
 - Share clock, reset, and set lines
 - Similar logic at each stage
- **Examples**
 - Shift registers
 - Counters

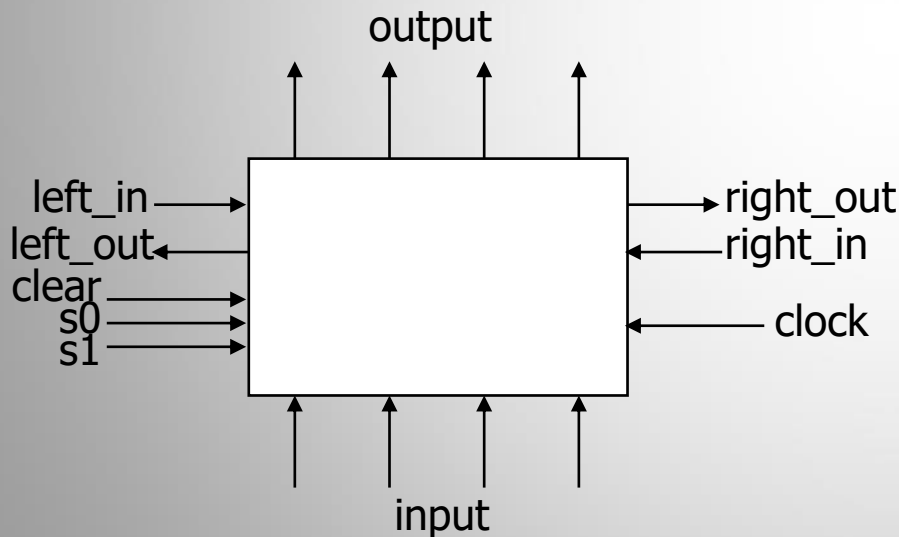


Shift Register: DFF and JK FF



Universal Shift Register

- **Holds 4 values**
 - Serial or parallel inputs
 - Serial or parallel outputs
 - Permits shift left or right
 - Shift in new values from left or right



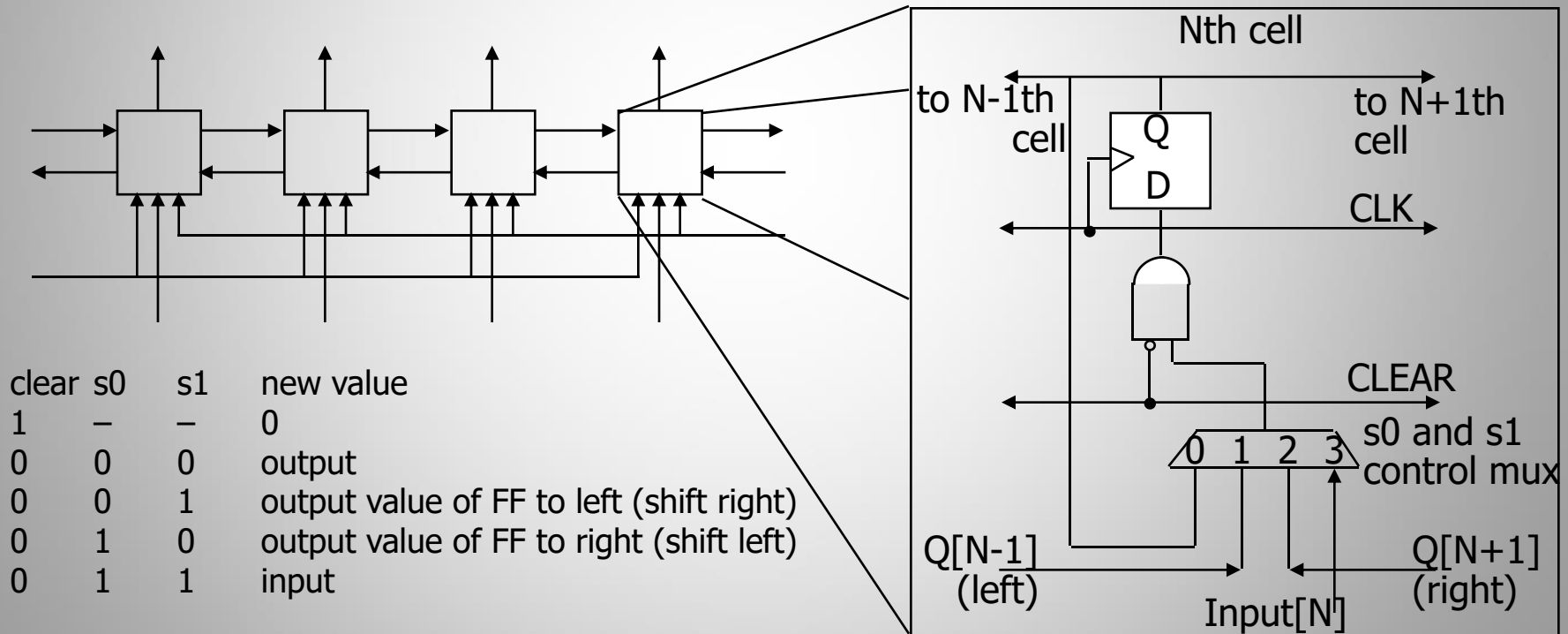
clear sets the register contents and output to 0

s1 and s0 determine the shift function

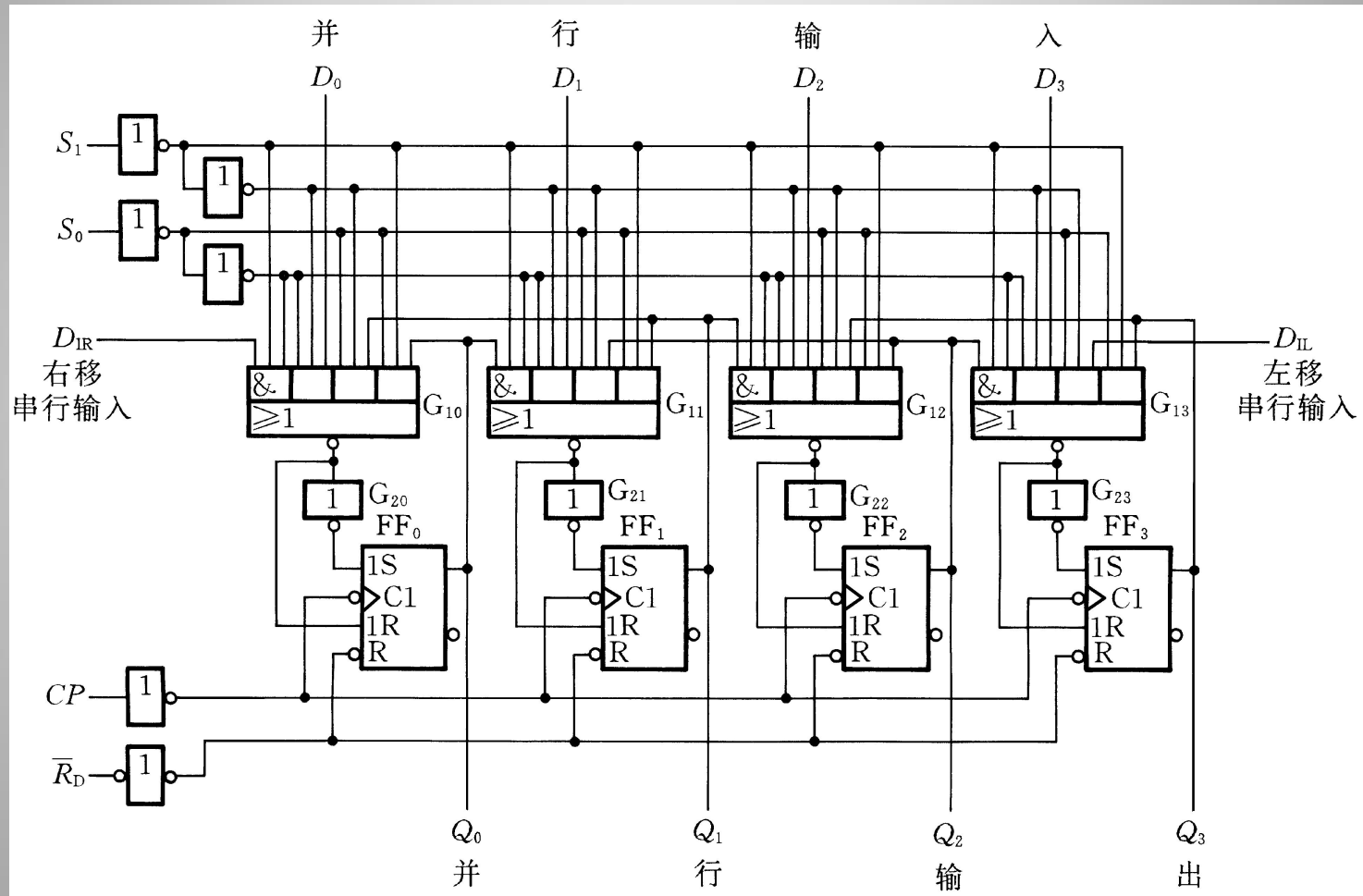
s0	s1	function
0	0	hold state
0	1	shift right
1	0	shift left
1	1	load new input

Design of Universal Shift Register

- Consider one of the four flip-flops
 - New value at next clock cycle:

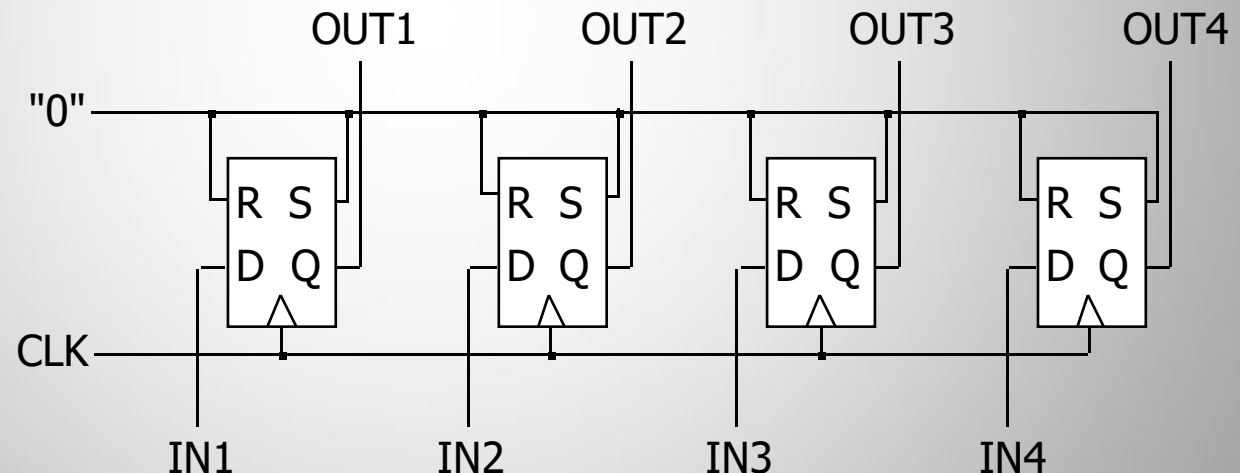


4位双向移位寄存器74HC194A的逻辑图



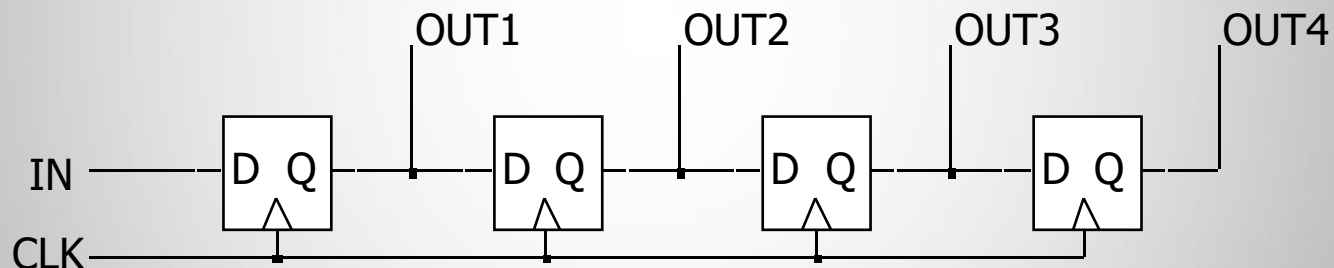
Registers

- **Collections of flip-flops with similar controls and logic**
 - Stored values somehow related (e.g., form binary value)
 - Share clock, reset, and set lines
 - Similar logic at each stage
- **Examples**
 - Shift registers
 - Counters



Shift Register

- **Holds samples of input**
 - Store last 4 input values in sequence
 - 4-bit shift register:



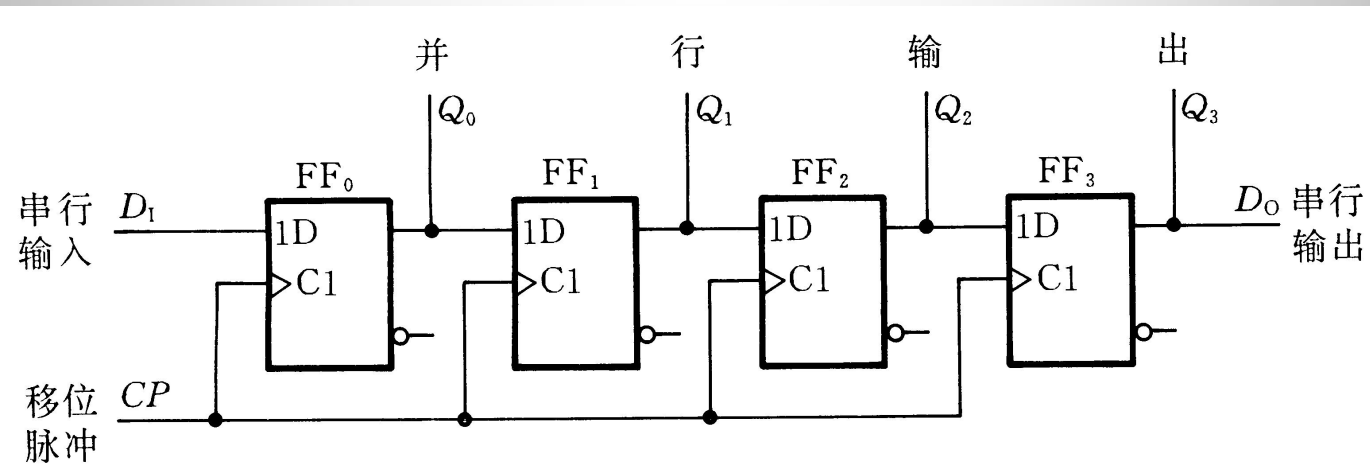
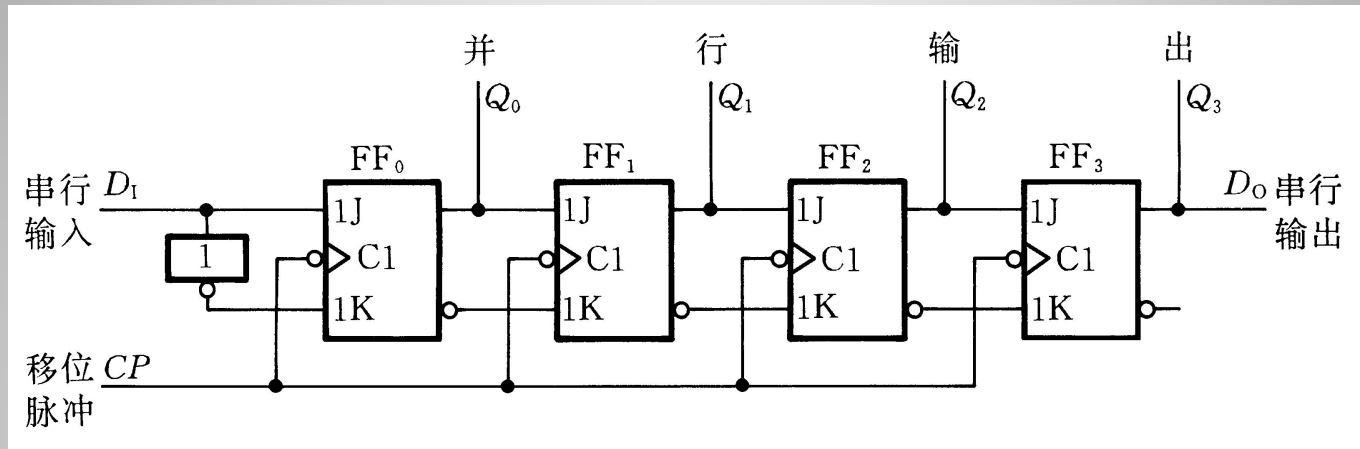
Shift Register Verilog

```
module shift_reg (out4, out3, out2, out1, in, clk);  
  output out4, out3, out2, out1;  
  input in, clk;  
  reg out4, out3, out2, out1;  
  
  always @(posedge clk)  
  begin  
    out4 <= out3;  
    out3 <= out2;  
    out2 <= out1;  
    out1 <= in;  
  end  
endmodule
```

Shift Register Verilog

```
module shift_reg (out, in, clk);  
    output [4:1] out;  
    input  in, clk;  
    reg   [4:1] out;  
  
    always @(posedge clk)  
    begin  
        out <= {out[3:1], in};  
    end  
endmodule
```

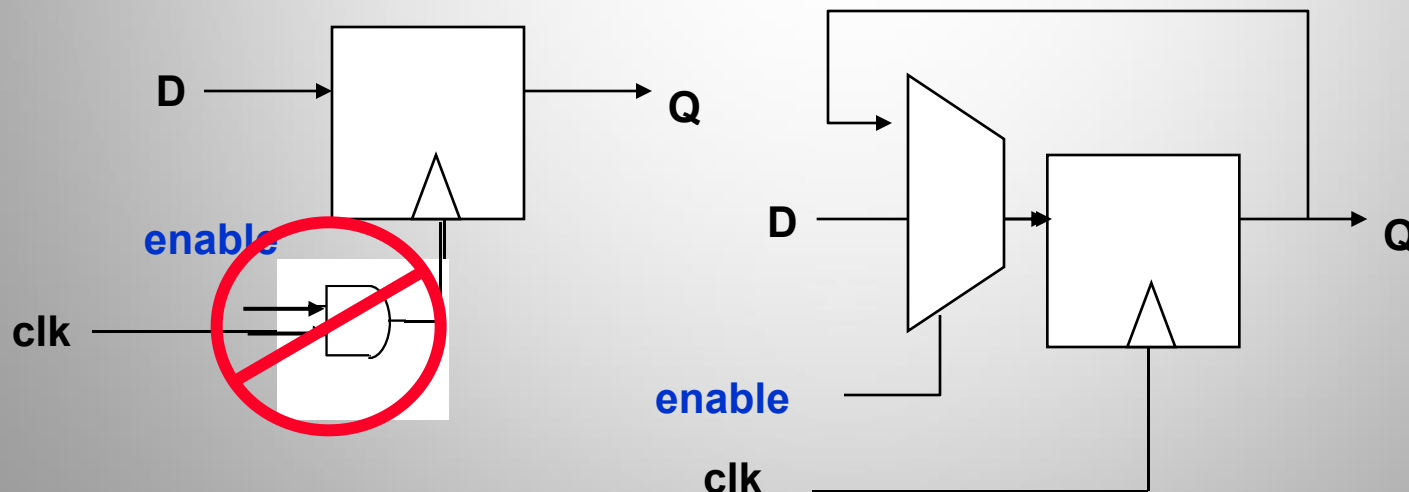
Shift Register: DFF and JK FF



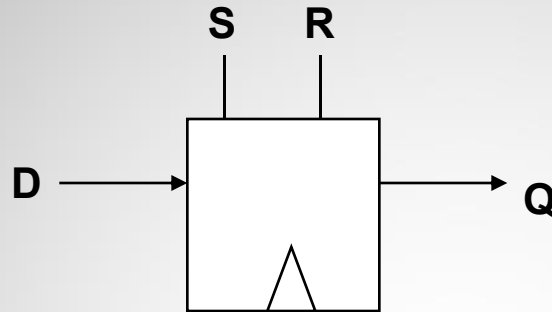
Register with selective load

- We often use registers to hold values for multiple clocks
 - Wait until needed
 - Used multiple times
- How do we modify our D flip-flop so that it holds the value till we are done with it?
- A very simple FSM

En	State	Next
0	Q	Q
1	Q	D



IQ: Design Register with Set/Reset

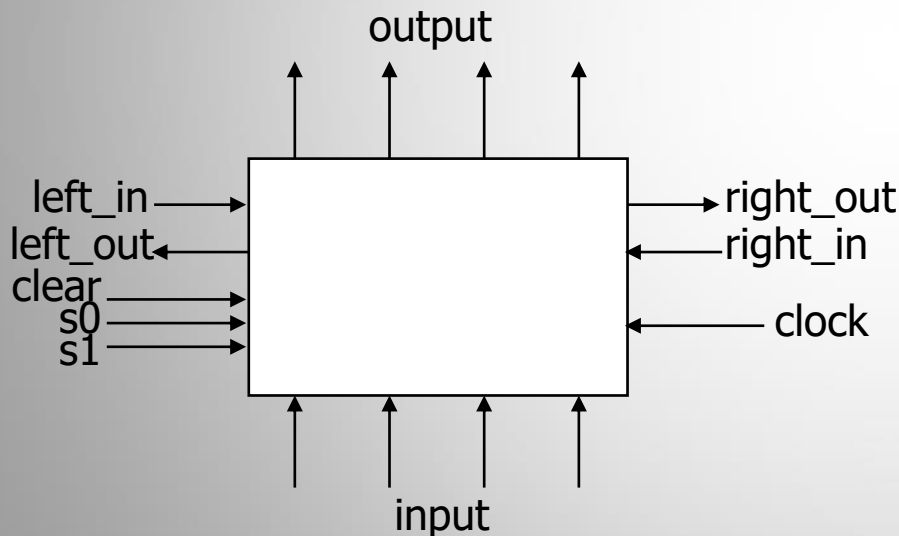


S	R	State	Next
0	0	Q	Q
0	1	Q	0
1	0	Q	1
1	1	Q	X

- Set forces state to 1
- Reset forces state to 0
- What might be a useful fourth option?

Universal Shift Register

- **Holds 4 values**
 - Serial or parallel inputs
 - Serial or parallel outputs
 - Permits shift left or right
 - Shift in new values from left or right



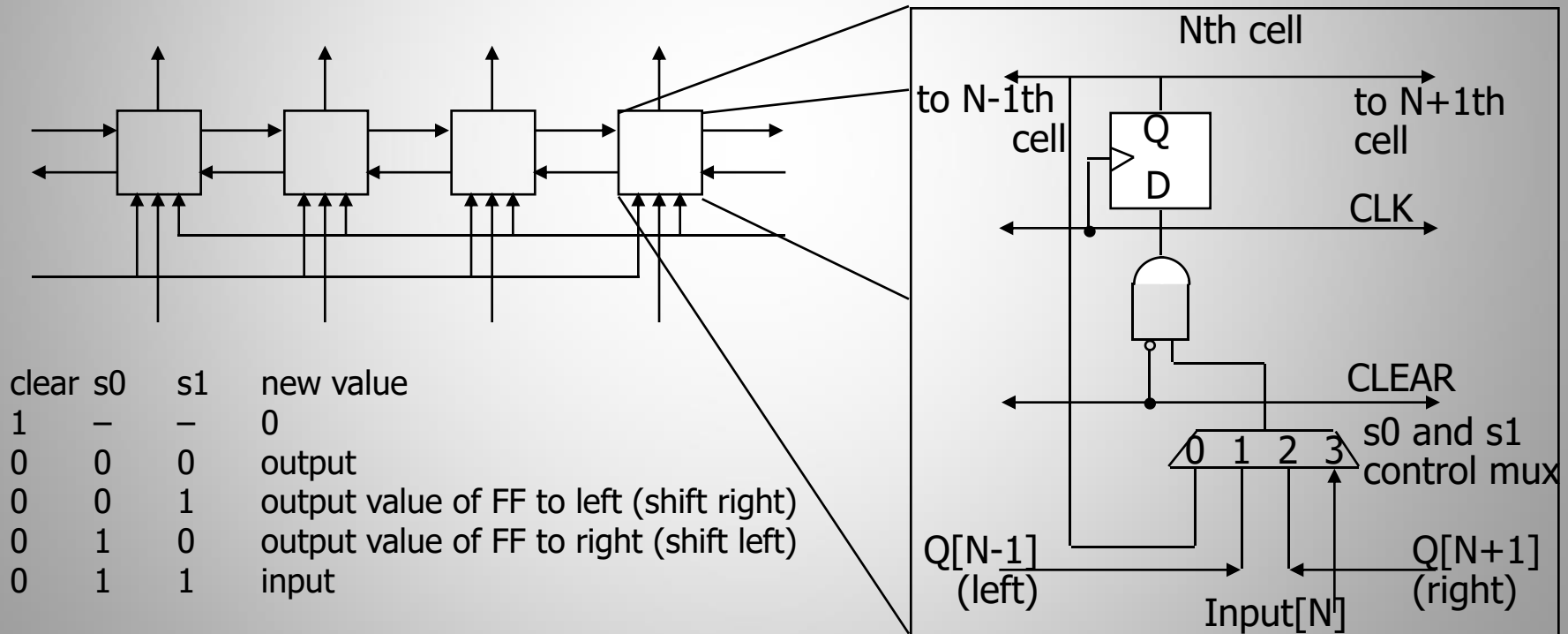
clear sets the register contents and output to 0

s1 and s0 determine the shift function

s0	s1	function
0	0	hold state
0	1	shift right
1	0	shift left
1	1	load new input

Design of Universal Shift Register

- Consider one of the four flip-flops
 - New value at next clock cycle:



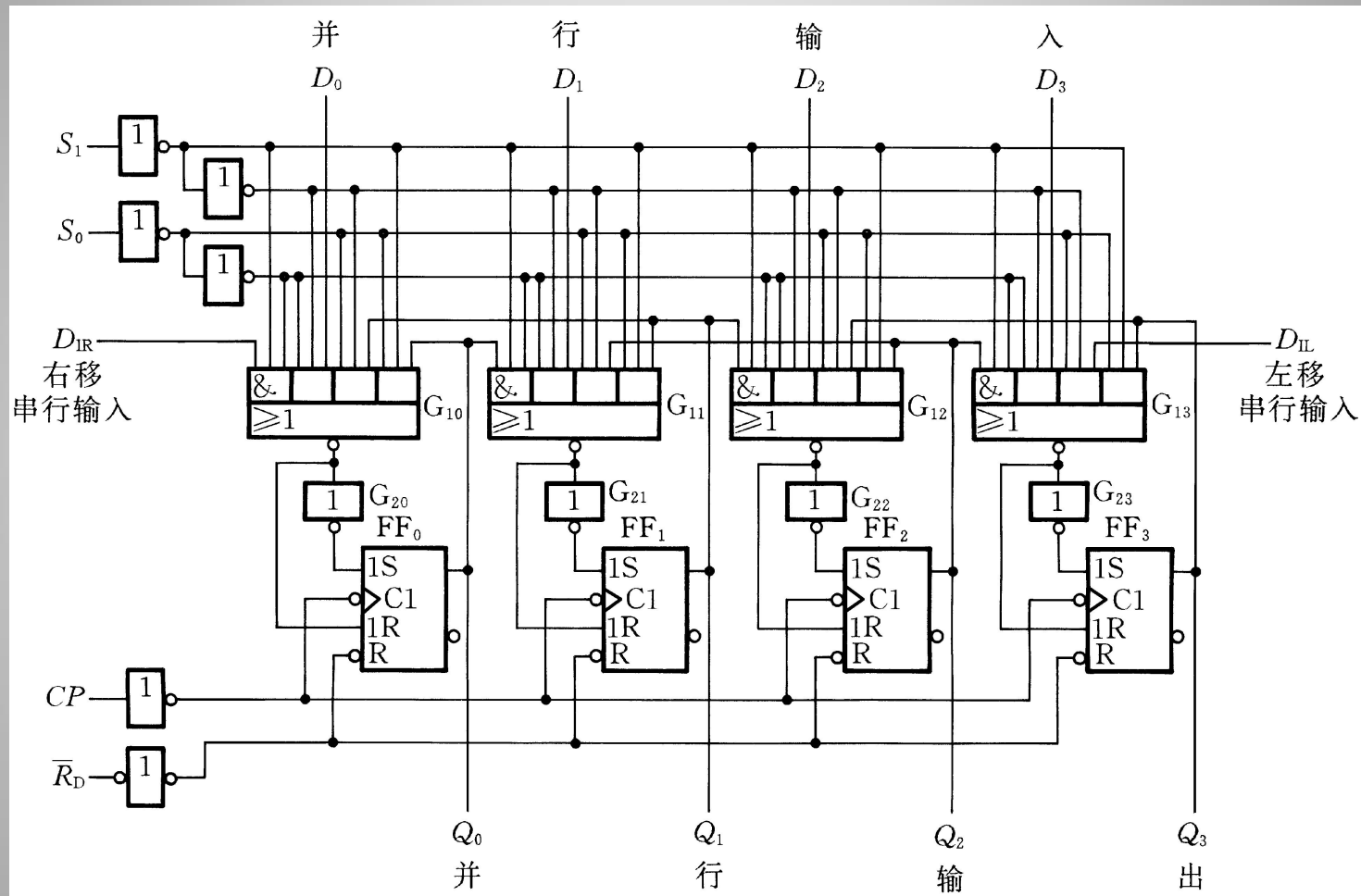
Universal Shift Register Verilog

```
module univ_shift (out, lo, ro, in, li, ri, s, clr, clk);
    output [3:0] out;
    output lo, ro;
    input [3:0] in;
    input [1:0] s;
    input li, ri, clr, clk;
    reg [3:0] out;

    assign lo = out[3];
    assign ro = out[0];

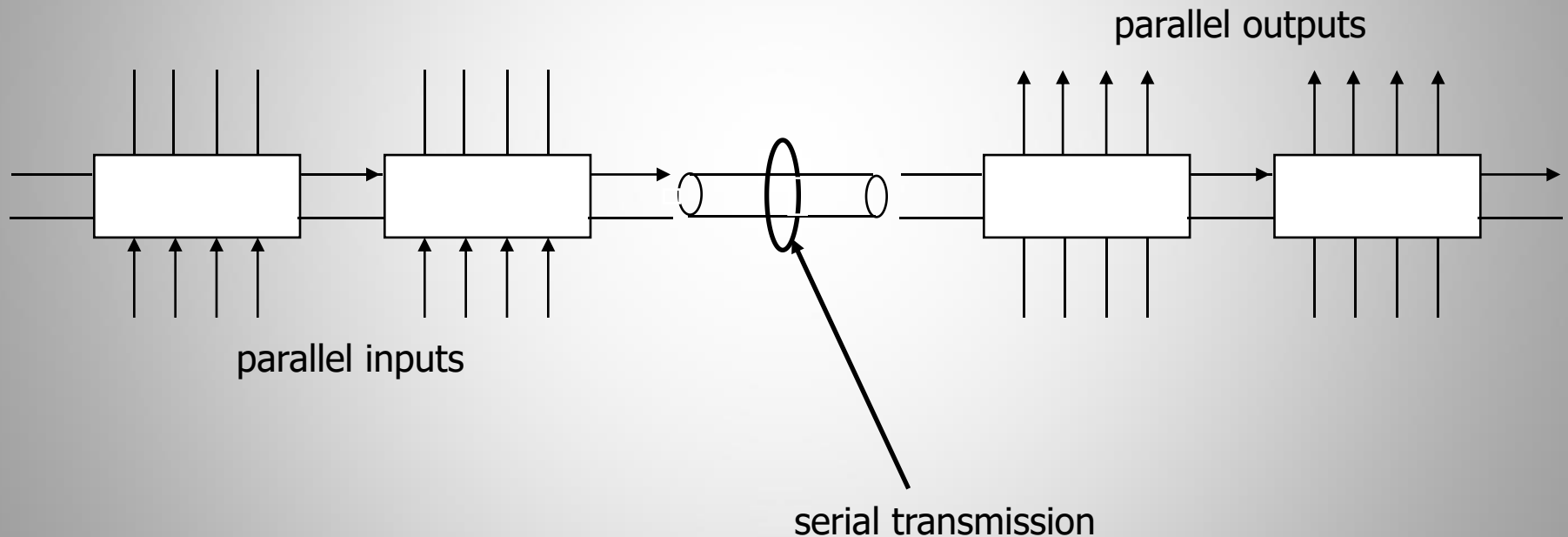
    always @(posedge clk or clr)
    begin
        if (clr) out <= 0;
        else
            case (s)
                3: out <= in;
                2: out <= {out[2:0], ri};
                1: out <= {li, out[3:1]};
                0: out <= out;
            endcase
        end
    end
endmodule
```

4位双向移位寄存器74HC194A的逻辑图



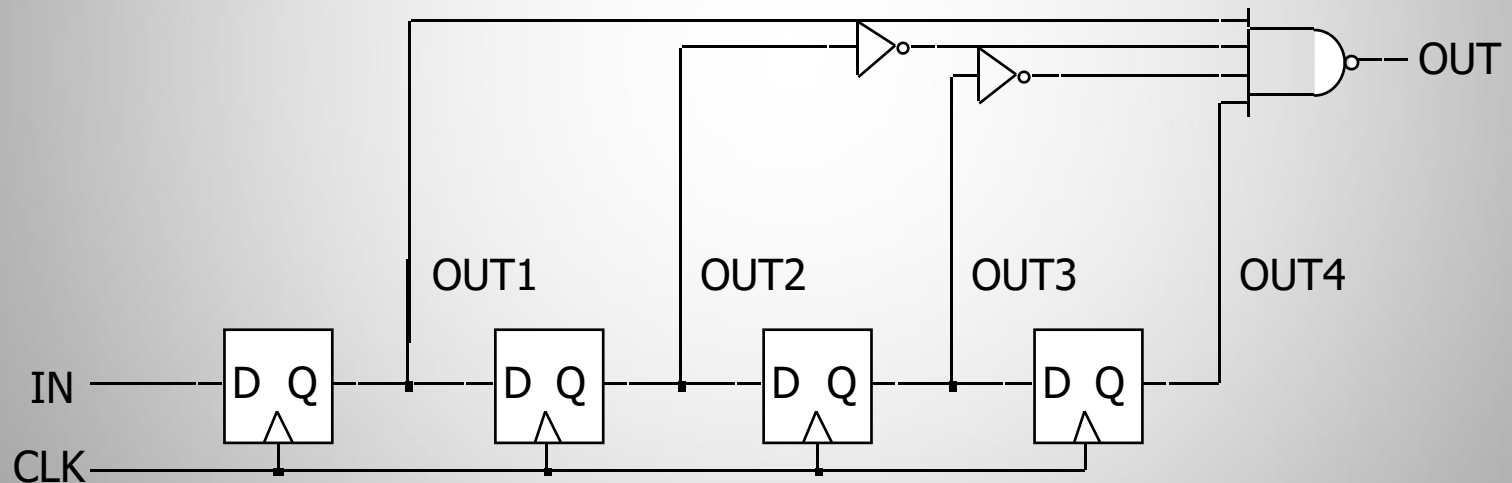
Shift Register Application

- **Parallel-to-serial conversion for serial transmission**



Pattern Recognizer

- **Combinational function of input samples**
 - In this case, recognizing the pattern 1001 on the single input signal



Another example

- **Door combination lock:**
 - punch in 3 values in sequence and the door opens; if there is an error the lock must be reset; once the door opens the lock must be reset
 - inputs: sequence of input values, reset
 - outputs: door open/close
 - memory: must remember combination
or always have it available as an input

Implementation in software

```
integer combination_lock ( ) {  
    integer v1, v2, v3;  
    integer error = 0;  
    static integer c[3] = 3, 4, 2;  
  
    while (!new_value( ));  
    v1 = read_value( );  
    if (v1 != c[1]) then error = 1;  
  
    while (!new_value( ));  
    v2 = read_value( );  
    if (v2 != c[2]) then error = 1;  
  
    while (!new_value( ));  
    v3 = read_value( );  
    if (v3 != c[3]) then error = 1;  
  
    if (error == 1) then return(0); else return (1);  
}
```

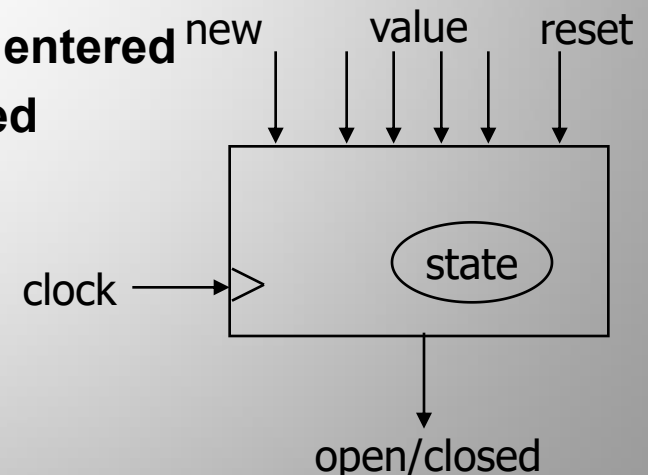
Implementation as a sequential digital system

- **Encoding:**

- how many bits per input value?
- how many values in sequence?
- how do we know a new input value is entered?
- how do we represent the states of the system?

- **Behavior:**

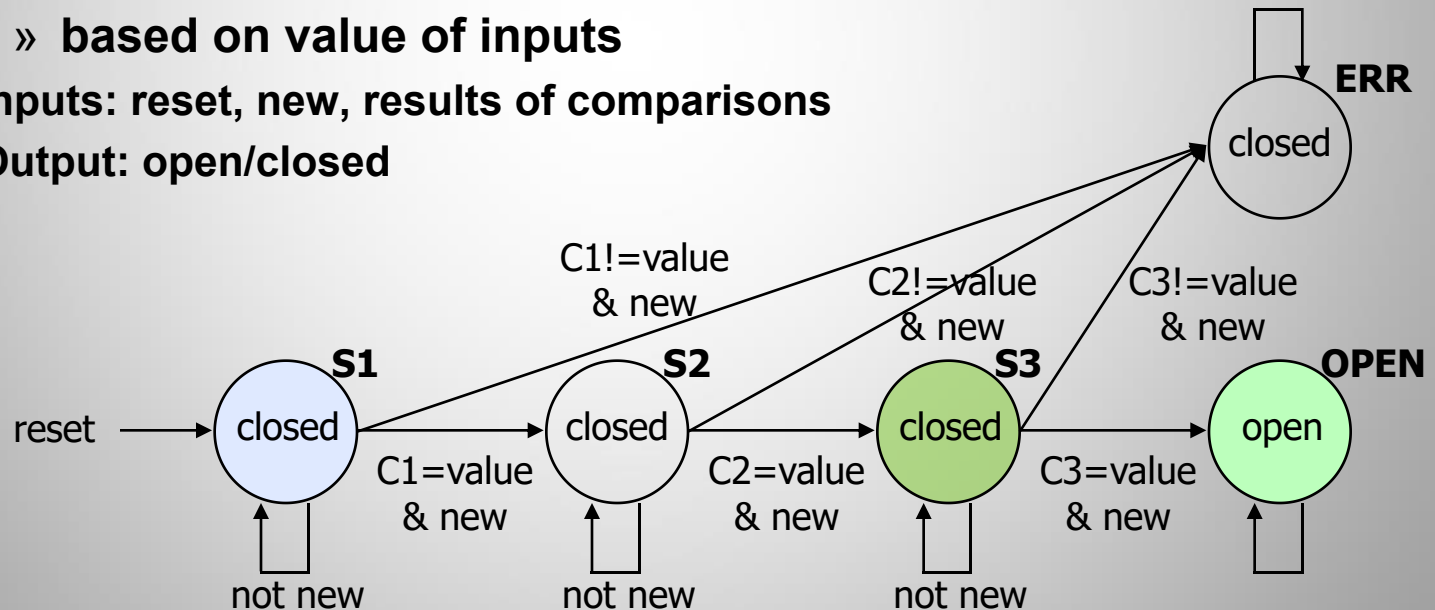
- clock wire tells us when it's ok to look at inputs (i.e., they have settled after change)
- sequential: sequence of values must be entered
- sequential: remember if an error occurred
- finite-state specification



Sequential example: abstract control

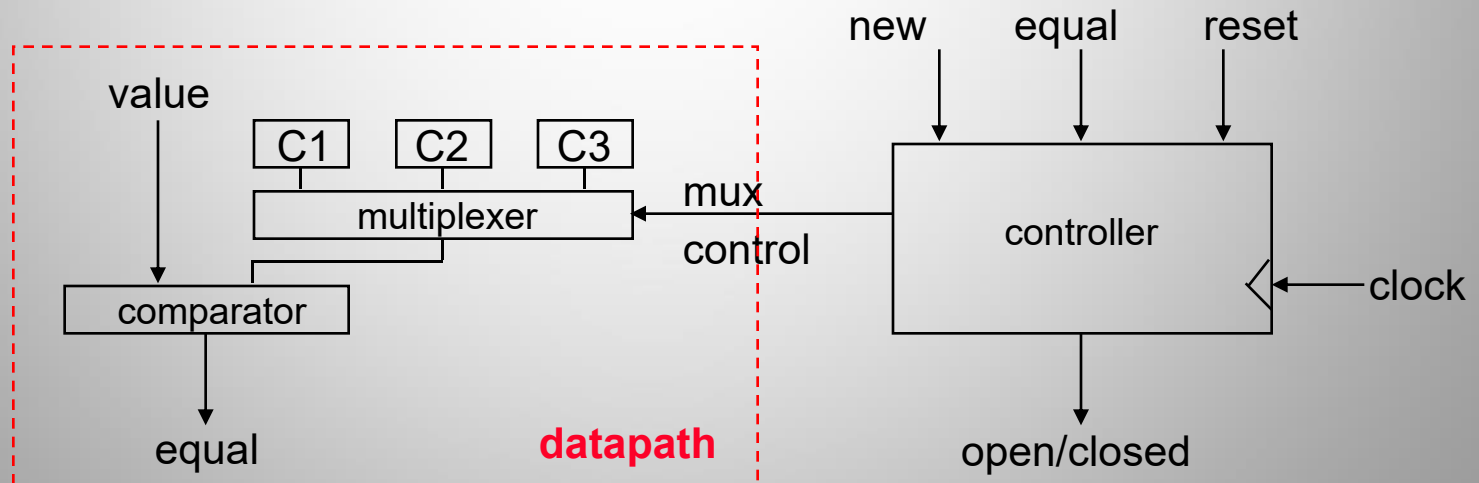
- **Finite-state diagram**

- **States: 5 states**
 - » represent point in execution of machine
 - » each state has outputs
- **Transitions: 6 from state to state, 5 self transitions, 1 global**
 - » changes of state occur when clock says it's ok
 - » based on value of inputs
- **Inputs: reset, new, results of comparisons**
- **Output: open/closed**



data-path vs. control

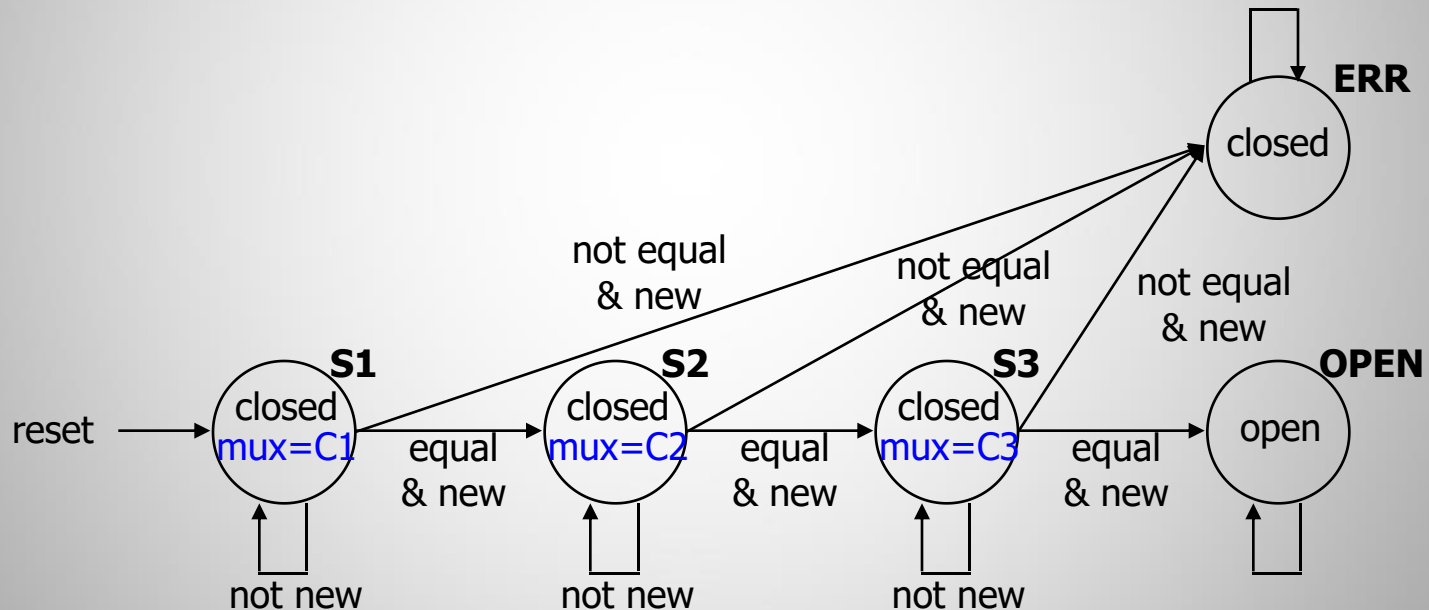
- **Internal structure**
 - data-path
 - » storage for combination
 - » comparators
 - control
 - » finite-state machine controller
 - » control for data-path
 - » state changes controlled by clock



Sequential example :finite-state machine

- **Finite-state machine**

- refine state diagram to include internal structure

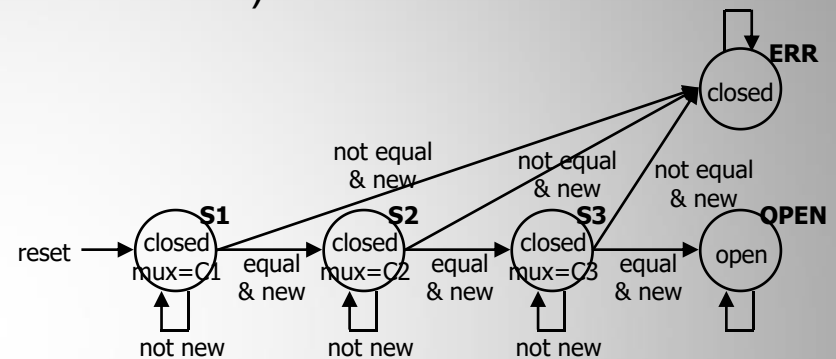


Sequential example: finite-state machine

- Finite-state machine

- generate state table (much like a truth-table)

Symbolic states



reset	new	equal	state	next state	mux	open/closed
1	-	-	-	S1	C1	closed
0	0	-	S1	S1	C1	closed
0	1	0	S1	ERR	-	closed
0	1	1	S1	S2	C2	closed
0	0	-	S2	S2	C2	closed
0	1	0	S2	ERR	-	closed
0	1	1	S2	S3	C3	closed
0	0	-	S3	S3	C3	closed
0	1	0	S3	ERR	-	closed
0	1	1	S3	OPEN	-	open
0	-	-	OPEN	OPEN	-	open
0	-	-	ERR	ERR	-	closed

Encoding?

Sequential example: encoding

- **Encode state table**

- state can be: S1, S2, S3, OPEN, or ERR

- » needs at least 3 bits to encode: 000, 001, 010, 011, 100

- » and as many as 5: 00001, 00010, 00100, 01000, 10000

- » choose 4 bits: 0001, 0010, 0100, 1000, 0000

binary

One-hot

hybrid

- **Encode outputs**

- output mux can be: C1, C2, or C3

- » needs 2 to 3 bits to encode

- » choose 3 bits: 001, 010, 100

- output open/closed can be: open or closed

- » needs 1 or 2 bits to encode

- » choose 1 bits: 1, 0

Sequential example :encoding

- **Encode state table**
 - state can be: S1, S2, S3, OPEN, or ERR
 - » choose 4 bits: 0001, 0010, 0100, 1000, 0000
 - output mux can be: C1, C2, or C3
 - » choose 3 bits: 001, 010, 100
 - output open/closed can be: open or closed
 - » choose 1 bits: 1, 0

reset	new	equal	state	next state	mux	open/closed
1	-	-	-	0001	001	0
0	0	-	0001	0001	001	0
0	1	0	0001	0000	-	0
0	1	1	0001	0010	010	0
0	0	-	0010	0010	010	0
0	1	0	0010	0000	-	0
0	1	1	0010	0100	100	0
0	0	-	0100	0100	100	0
0	1	0	0100	0000	-	0
0	1	1	0100	1000	-	1
0	-	-	1000	1000	-	1
0	-	-	0000	0000	-	0

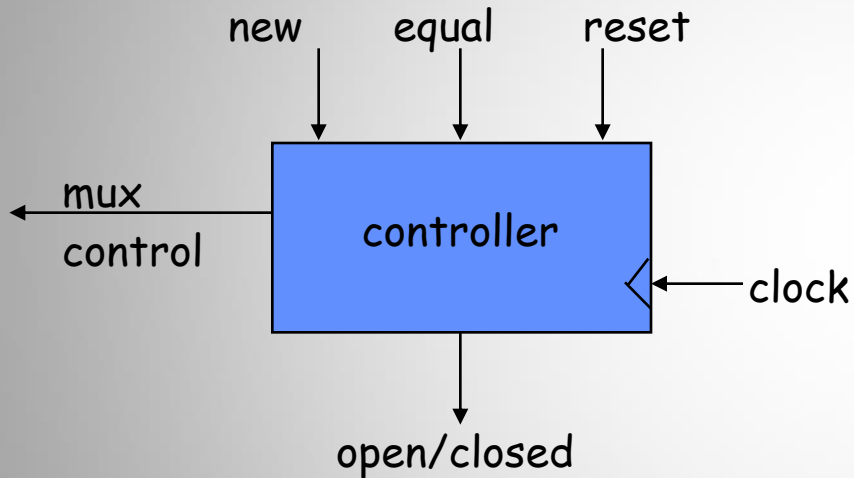
good choice of encoding!

mux is identical to
last 3 bits of next state

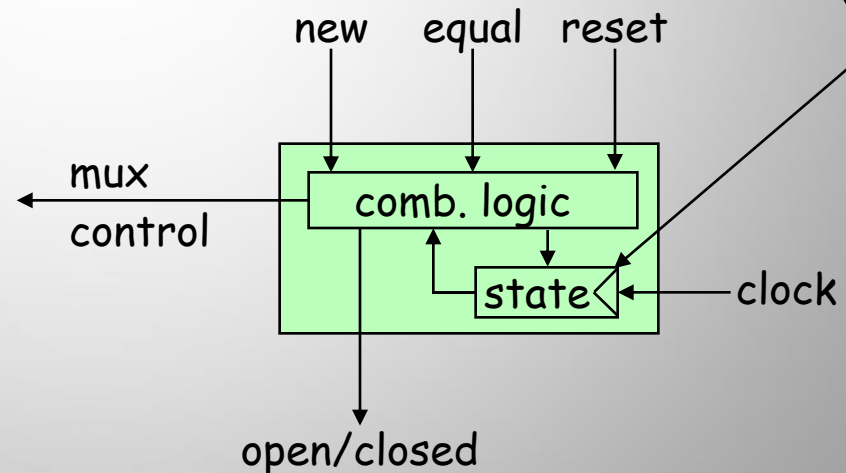
open/closed is
identical to first bit
of state

Sequential example :controller implementation

- Implementation of the controller

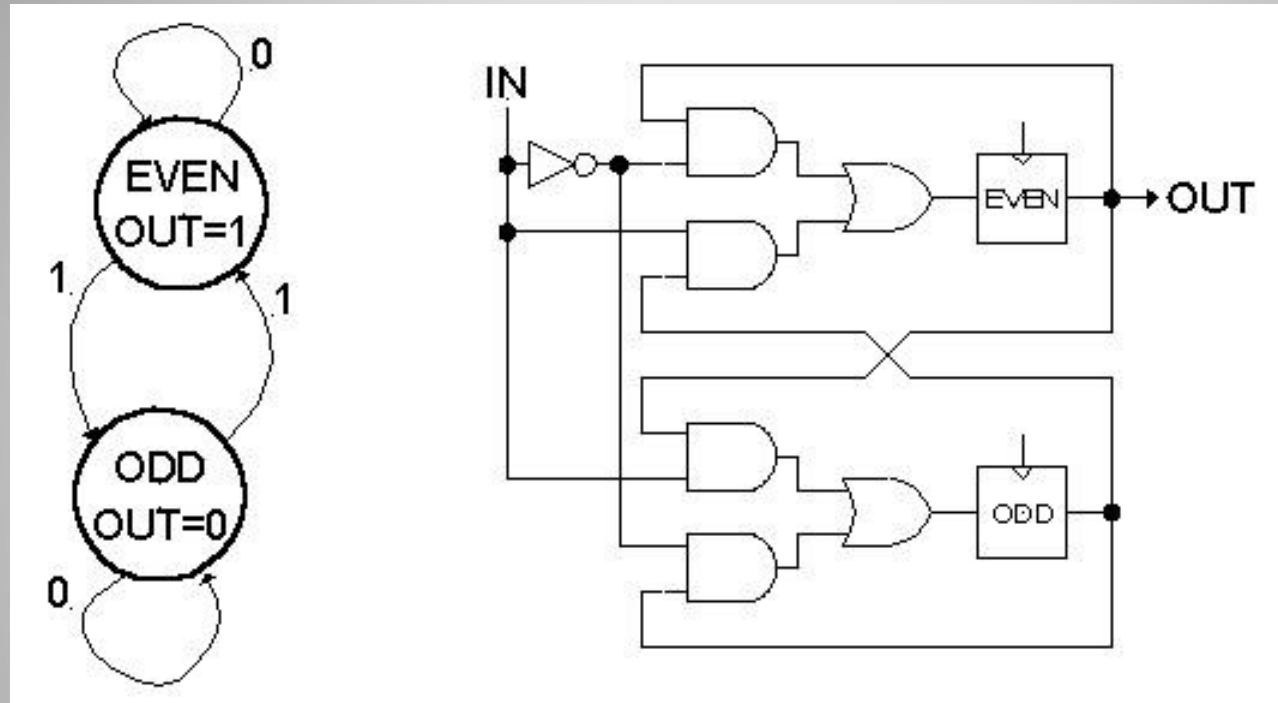


special circuit element,
called a register, for
remembering inputs
when told to by clock

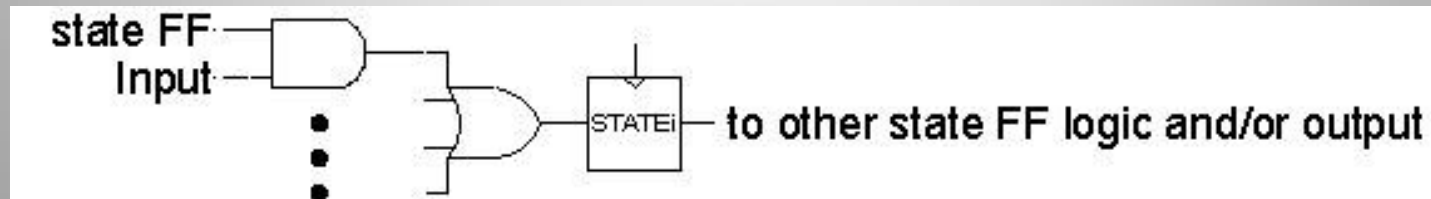


One-hot encoded FSM

- Even Parity Checker

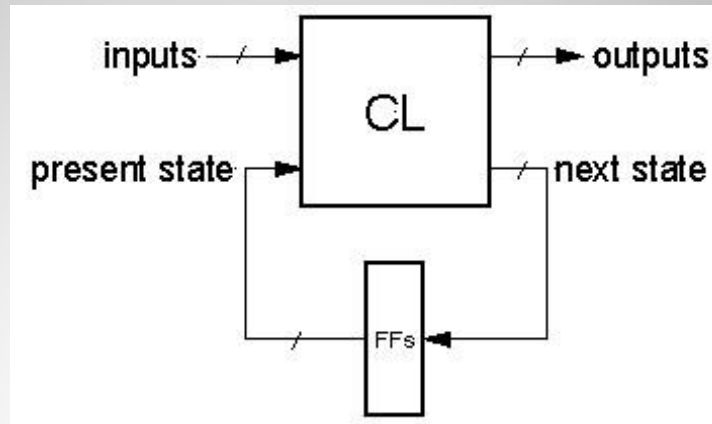


- FFs must be initialized for correct operation (only one 1)
- In General:



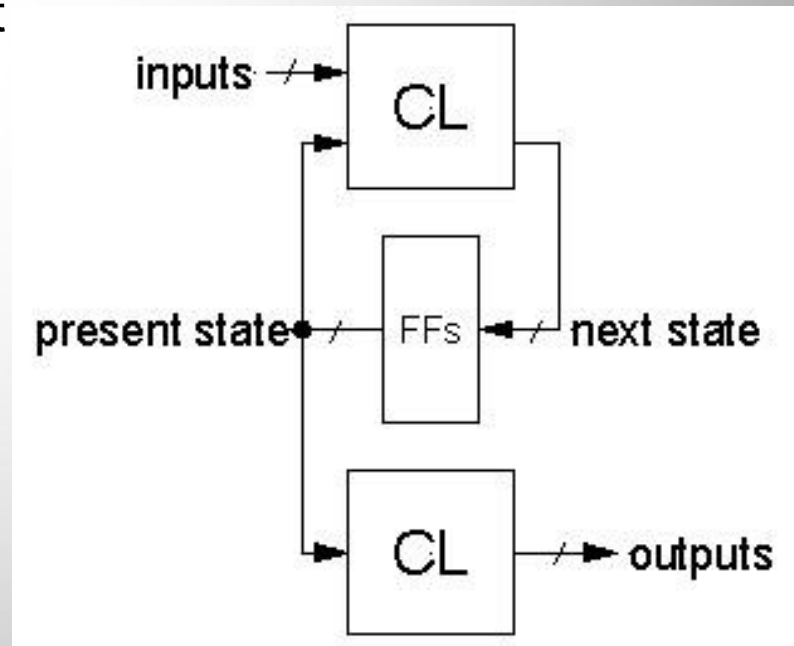
FSM Implementation Notes

- General FSM form:

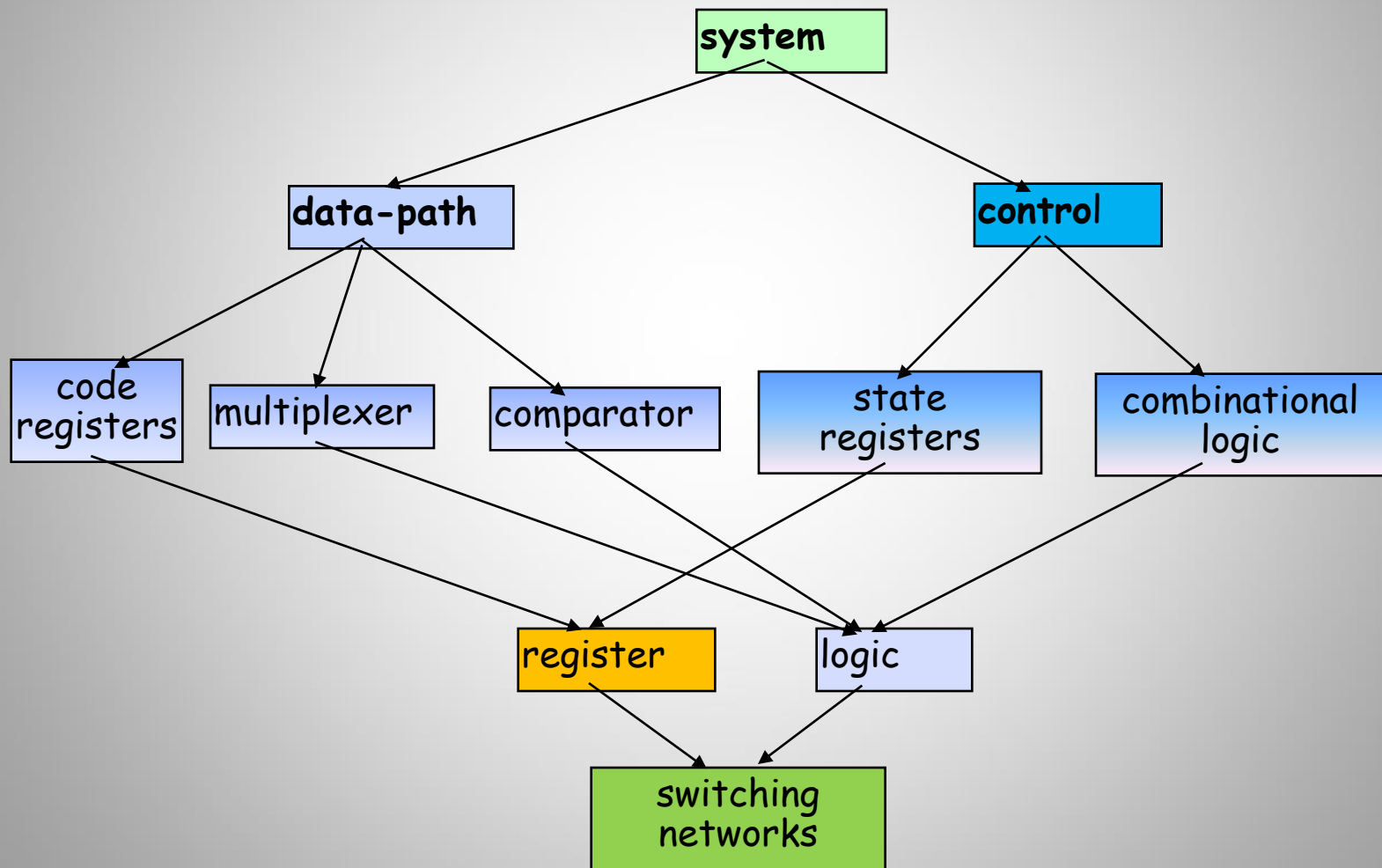


- All examples so far generate output based only on the present state:

- Commonly called **Moore Machine**
(If output functions include both present state and input then called a *Mealy Machine*)



System Design Hierarchy



6.3.2 计数器

- 用于计数、分频、定时、产生节拍脉冲等
- 分类：按时钟分，同步、异步
 - 按计数过程中数字增减分，加、减和可逆
 - 按计数器中的数字编码分，二进制、二-十进制和循环码...
 - 按计数容量分，十进制，六十进制...

一、同步计数器

1. 同步二进制计数器

①同步二进制加法计数器

原理：根据二进制加法运算规则可知：在多位二进制数末位加1，若第*i*位以下皆为1时，则第*i*位应翻转。

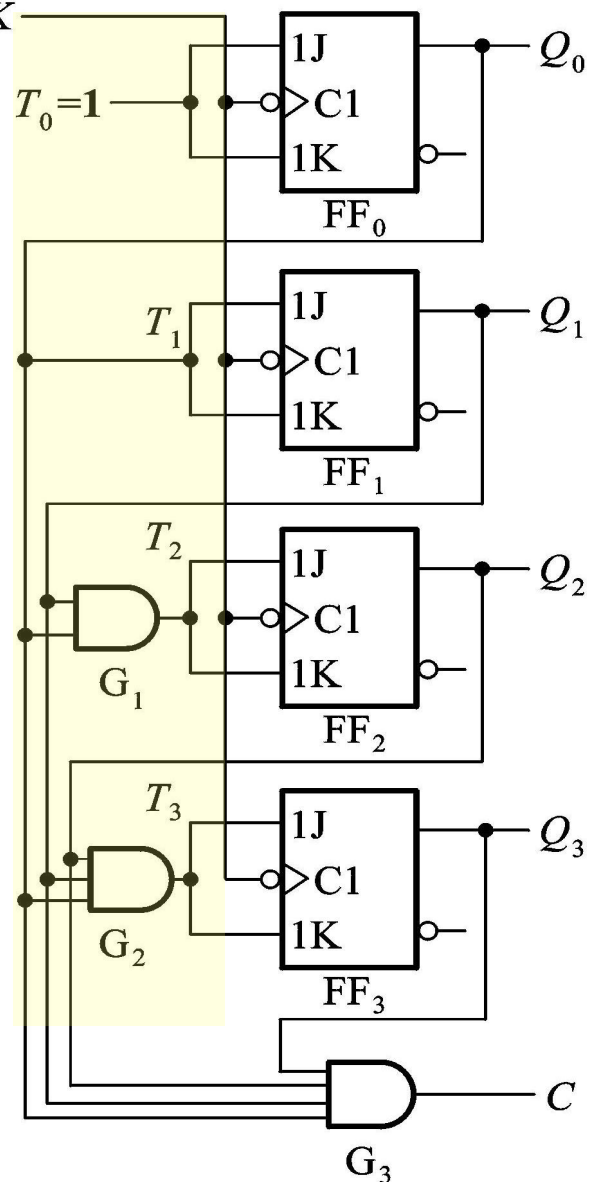
由此得出规律，若用T触发器构成计数器，则第*i*位触发器输入端*T_i*的逻辑式应为：

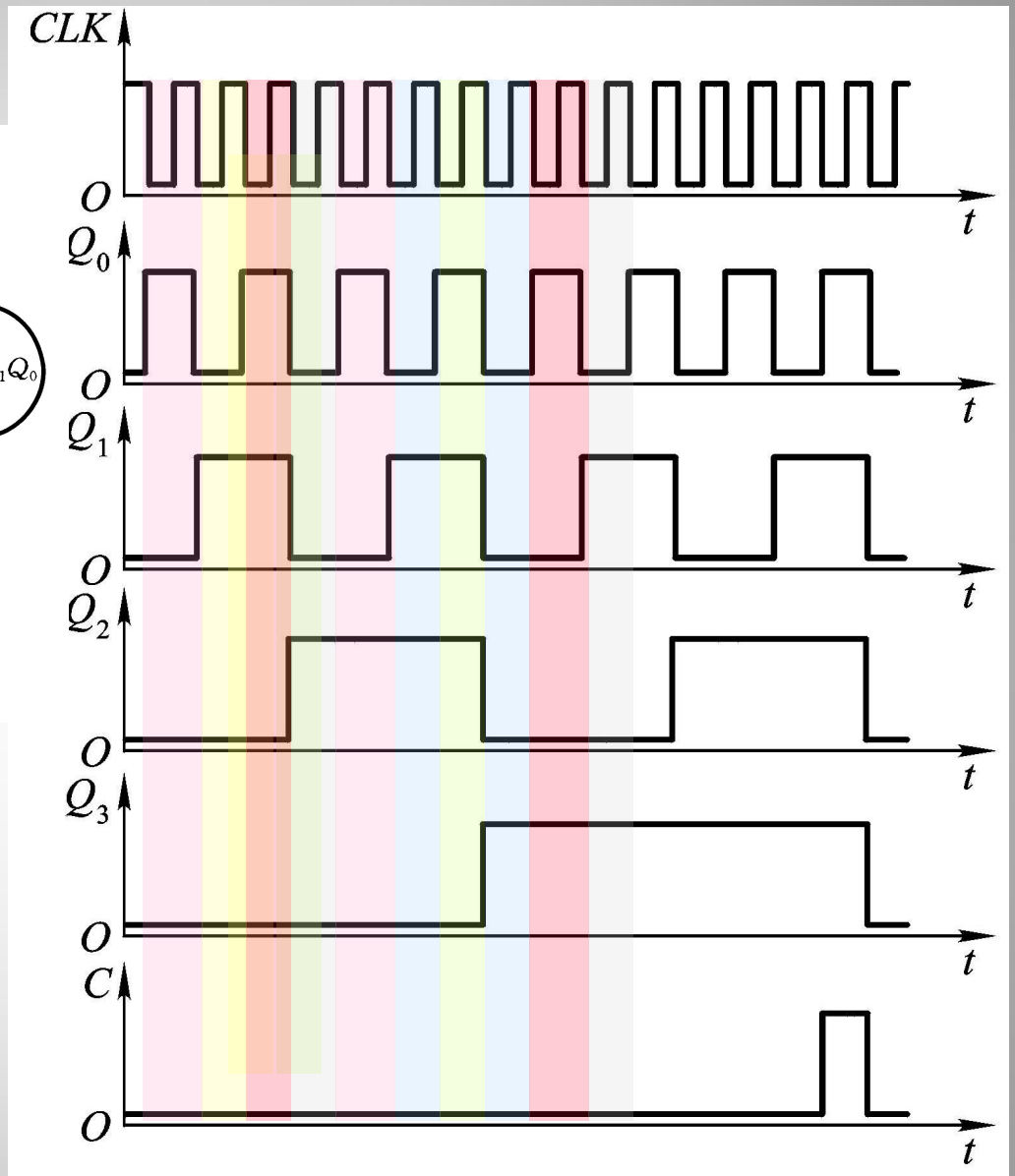
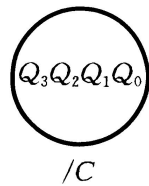
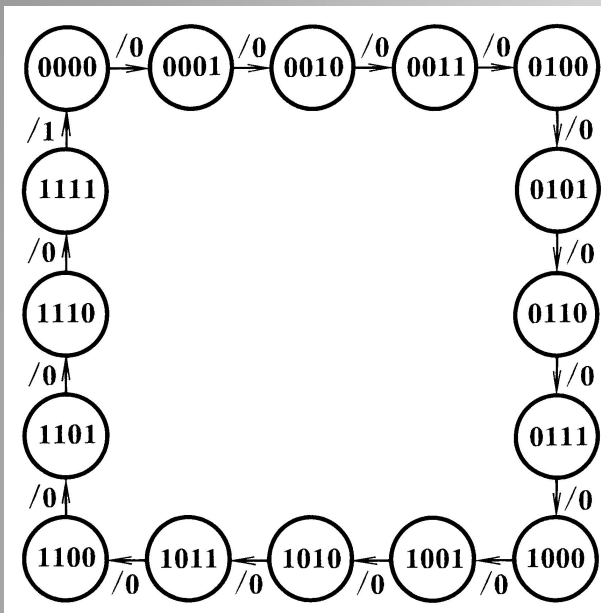
$$T_i = Q_{i-1}Q_{i-2}\dots Q_0$$

$$T_0 \equiv 1$$

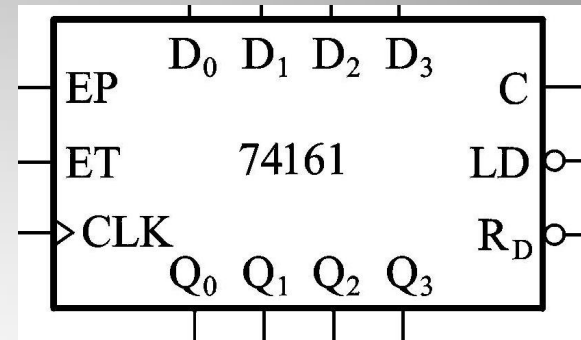
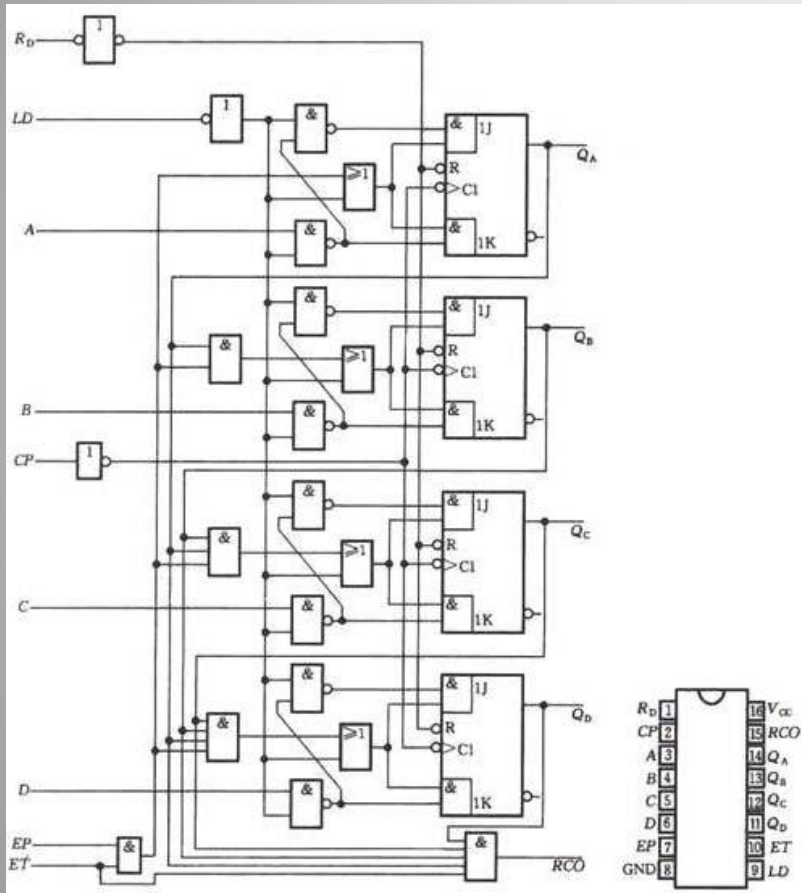
计数脉冲

CLK



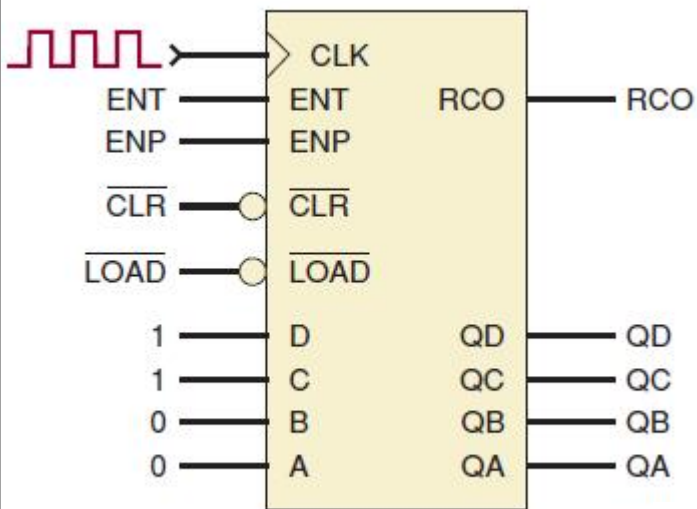


器件实例：74161

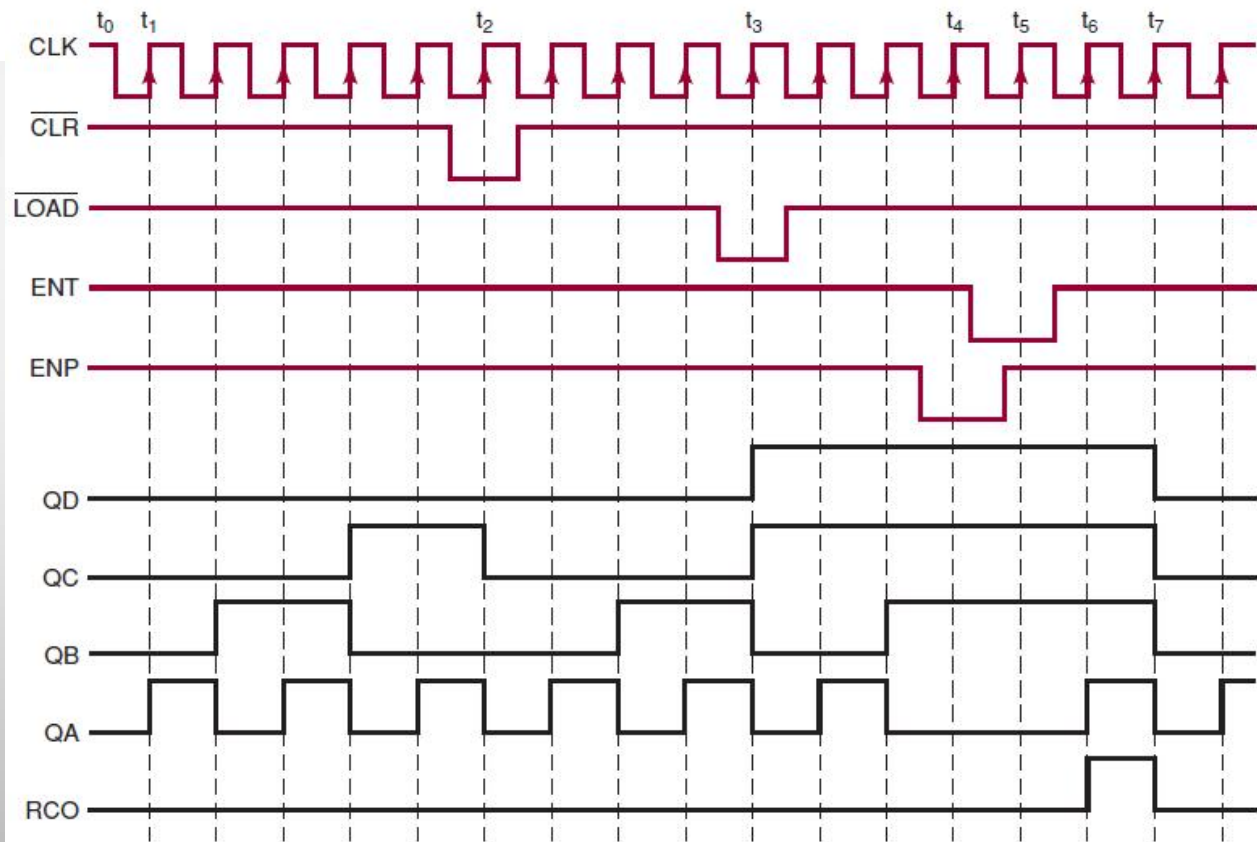


CLK	R'_D	LD'	EP	ET	工作状态
X	0	X	X	X	置 0 (异步)
	1	0	X	X	预置数 (同步)
X	1	1	0	1	保持 (包括C)
X	1	1	X	0	保持 (C=0)
	1	1	1	1	计数

74HC163



(a)



(b)

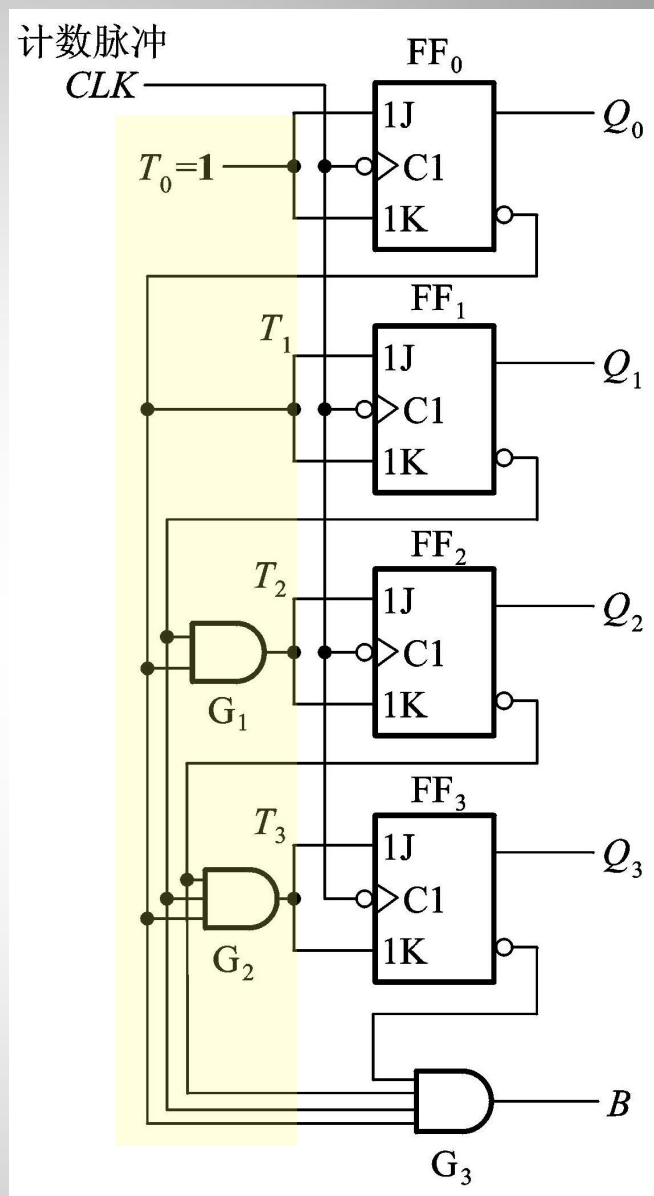
②同步二进制减法计数器

原理：根据二进制减法运算规则可知：在多位二进制数末位减1，若第*i*位以下皆为0时，则第*i*位应翻转。

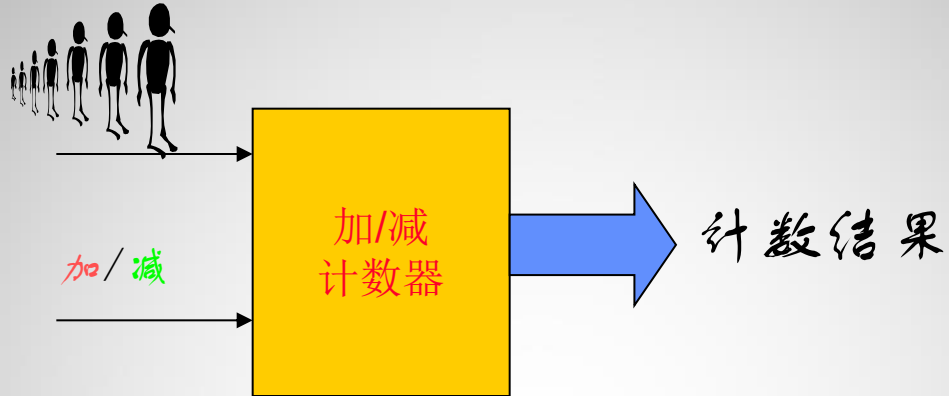
由此得出规律，若用T触发器构成计数器，则第*i*位触发器输入端*T_i*的逻辑式应为：

$$T_i = Q'_{i-1} Q'_{i-2} \cdots Q'_0$$

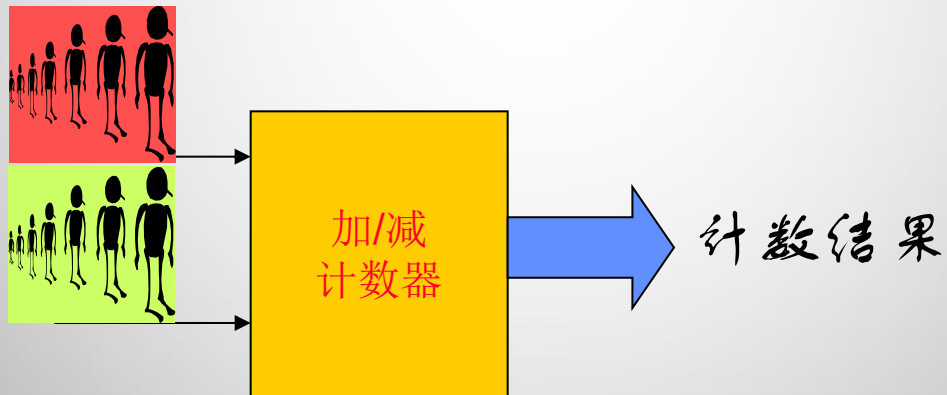
$$T_0 \equiv 1$$



③同步加减计数器



 两种解决方案



a. 单时钟方式

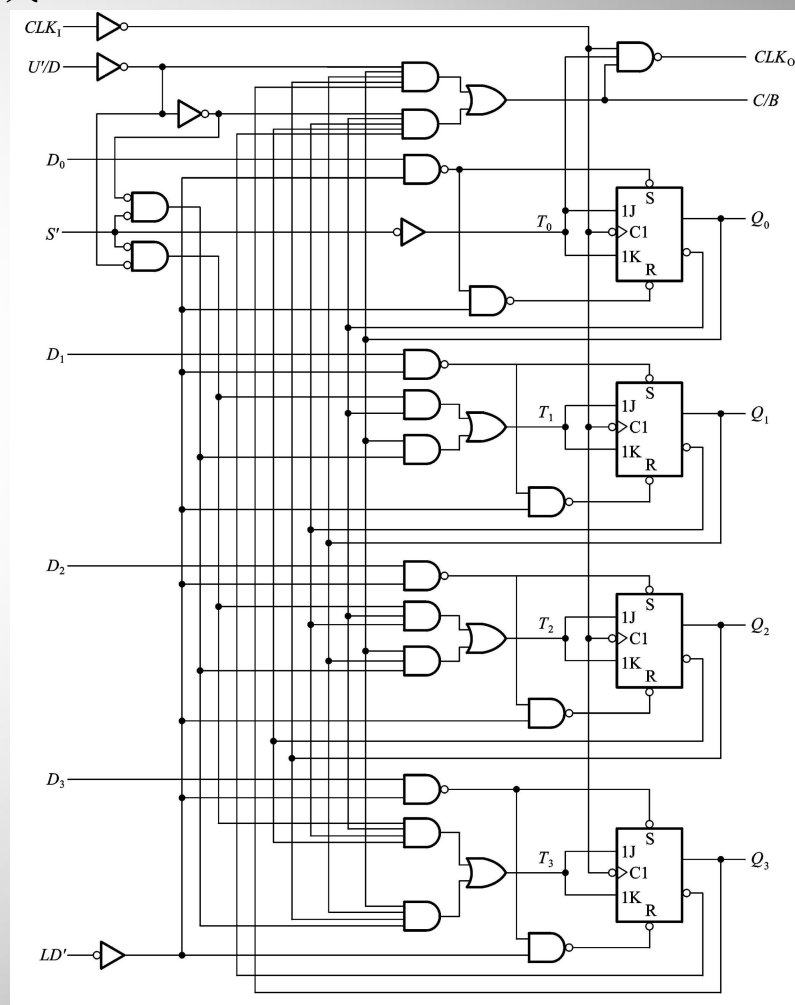
加/减脉冲用同一输入端，

由加/减控制线的高低电平决定加/减

器件实例：74LS191（用T触发器）

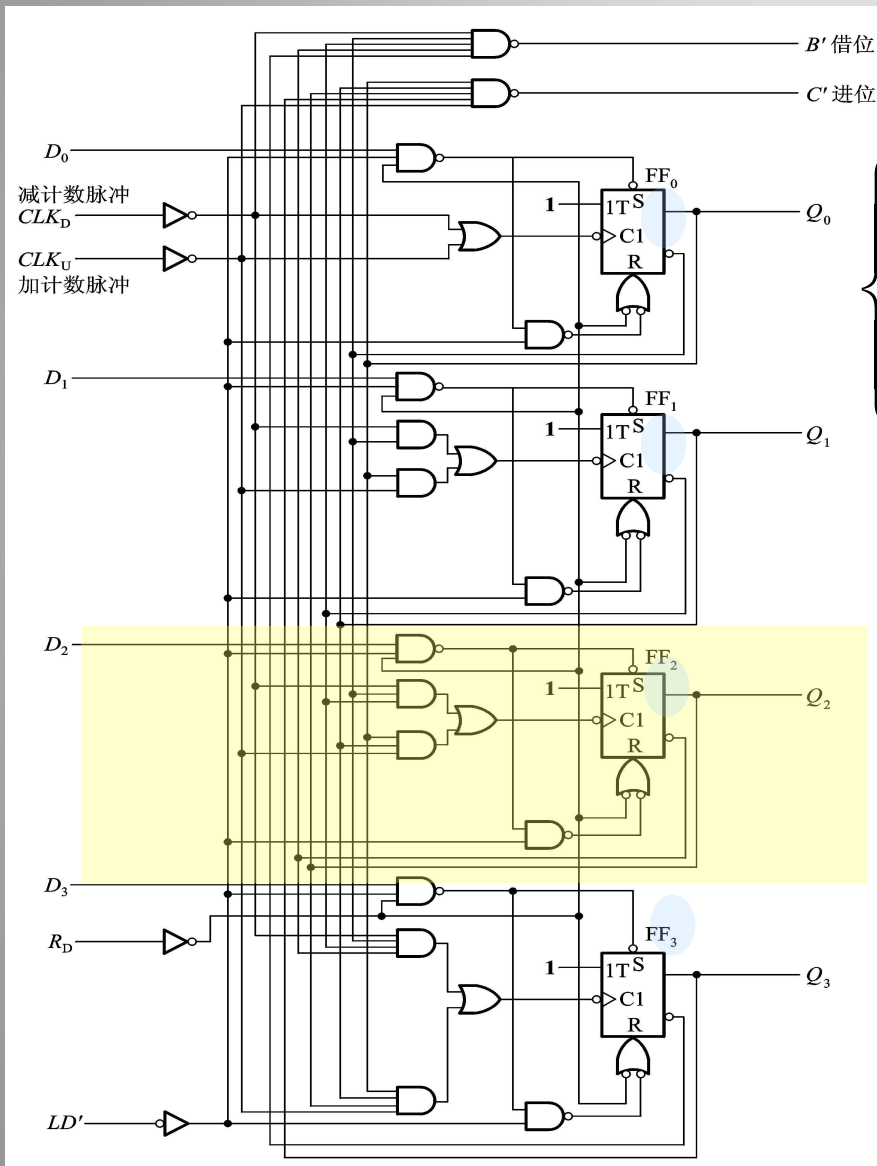
$$\begin{cases} T_i = (U'/D)' \prod_{j=0}^{i-1} Q_j + (U'/D) \prod_{j=0}^{i-1} Q'_j \\ T_0 = 1 \end{cases}$$

CLK_I	S'	LD'	U'/D	工作状态
X	1	1	X	保持
X	X	0	X	预置数(异步)
	0	1	0	加计数
	0	1	1	减计数



b. 双时钟方式

器件实例：74LS193（采用T'触发器，即T=1）



$$\left\{ \begin{aligned} CLK_i &= CLK_U \prod_{j=0}^{i-1} Q_j + CLK_D \prod_{j=0}^{i-1} Q'_j \\ CLK_0 &= CLK_U + CLK_D \end{aligned} \right.$$

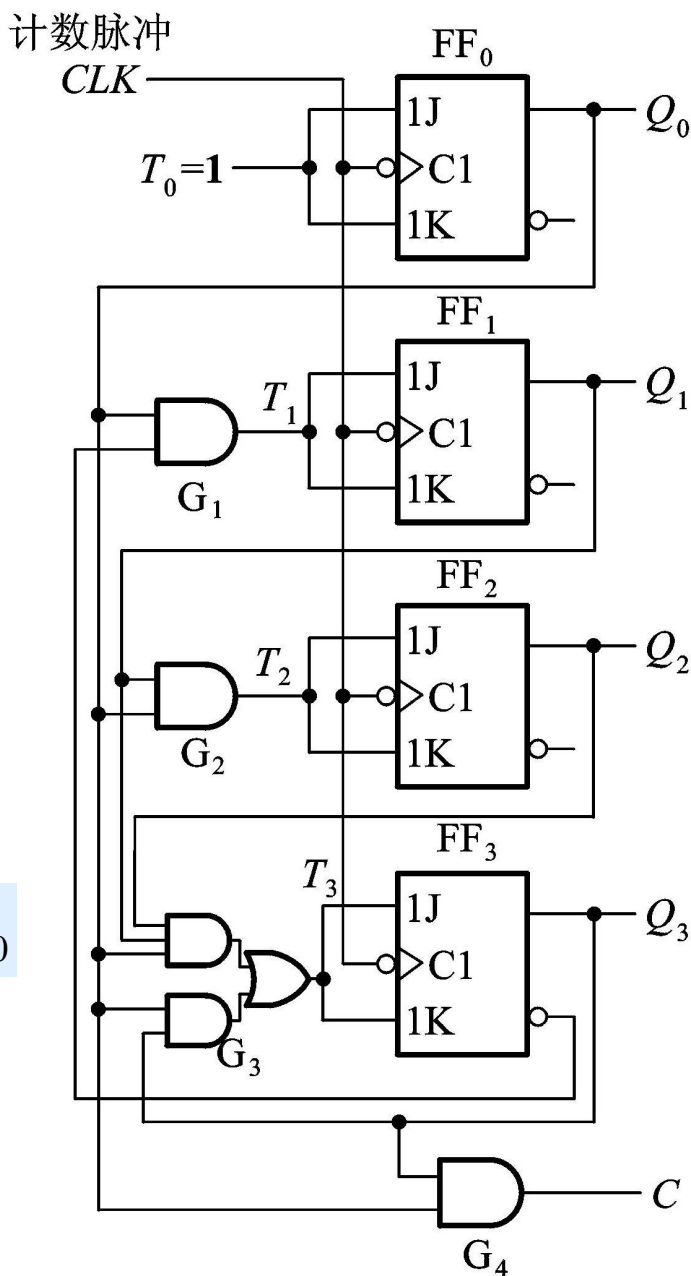
$$CLK_2 = CLK_U Q_1 Q_0 + CLK_D Q'_1 Q'_0$$

2. 同步十进制计数器

①加法计数器

基本原理：在四位二进制计数器基础上修改，当计到1001时，则下一个CLK电路状态回到0000。

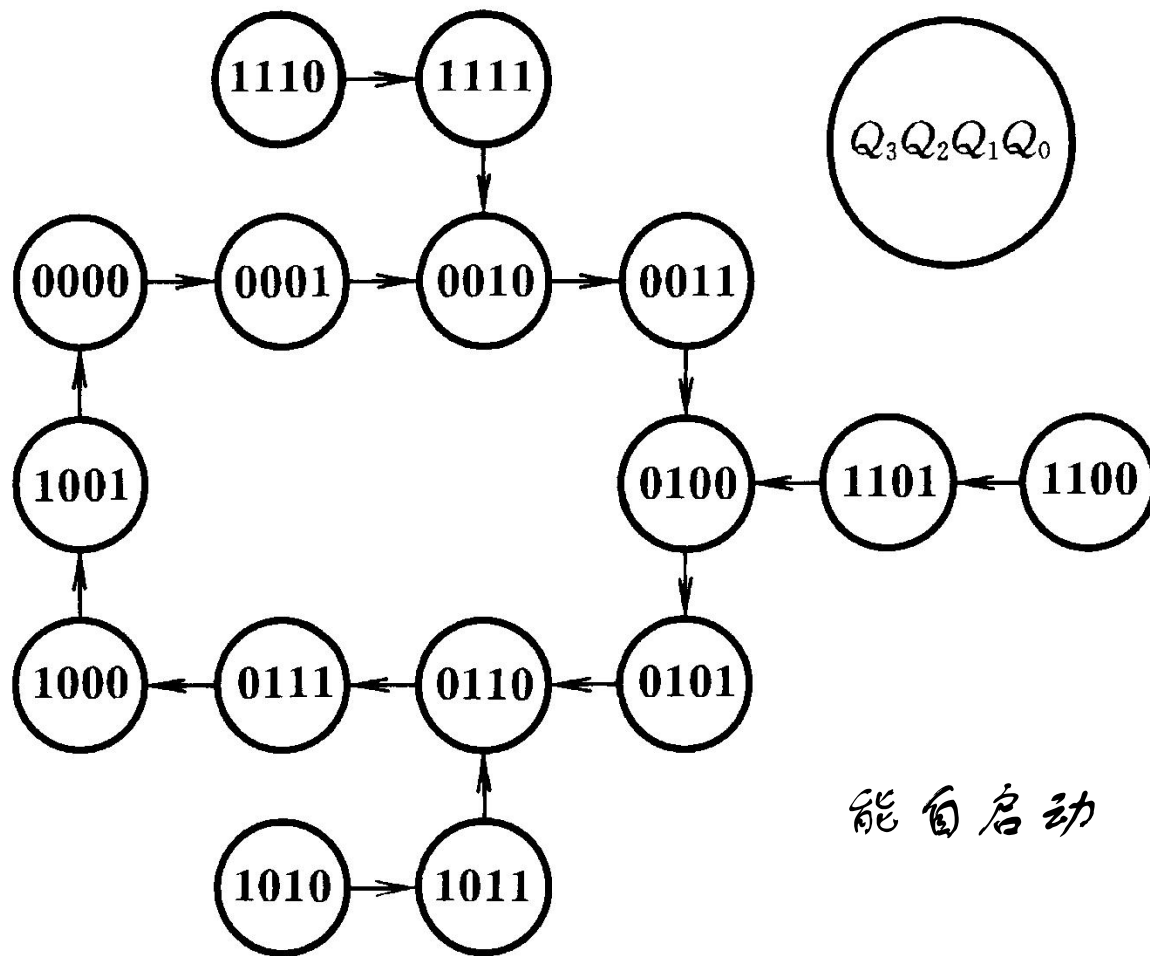
$$T_3 = Q_2 Q_1 Q_0 \Rightarrow Q_2 Q_1 Q_0 + Q_3 Q_0$$



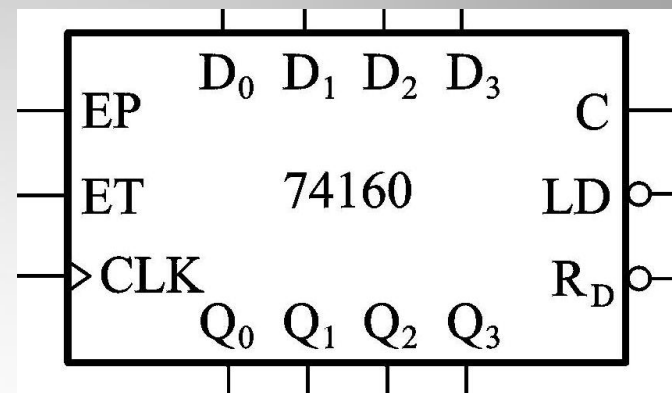
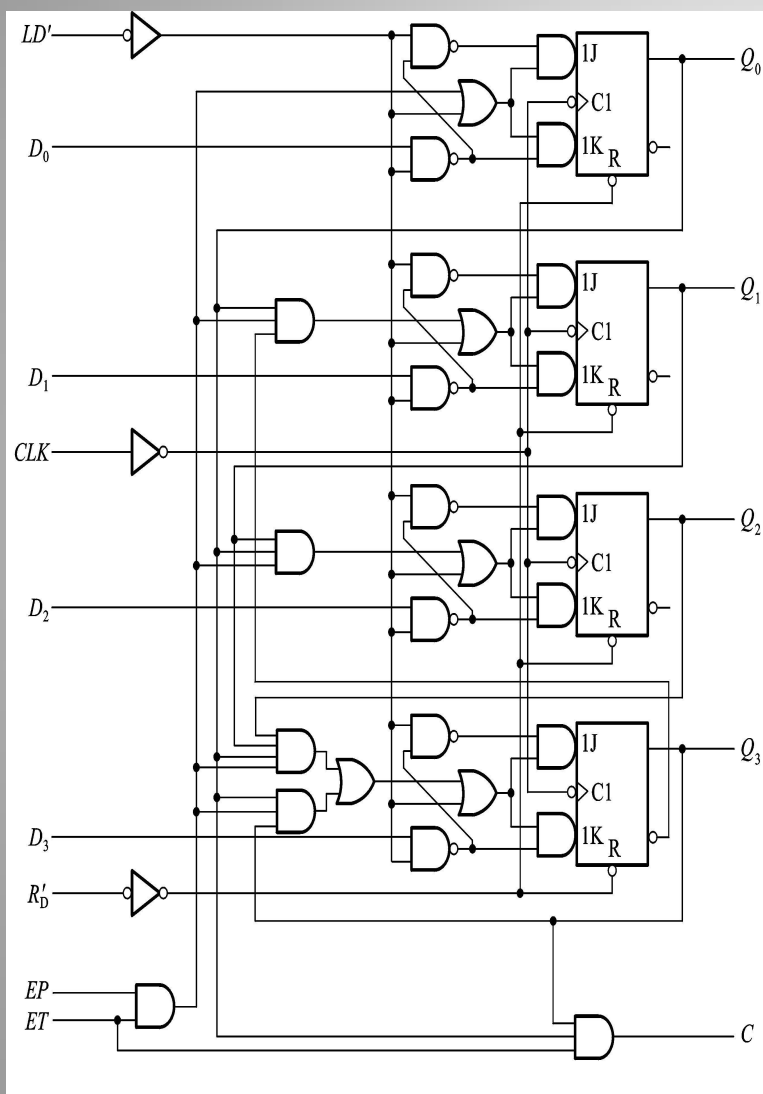
$$T_0 = 1$$

$$T_1 = Q_0 \Rightarrow Q_0 Q'_3$$

$$T_2 = Q_1 Q_2$$

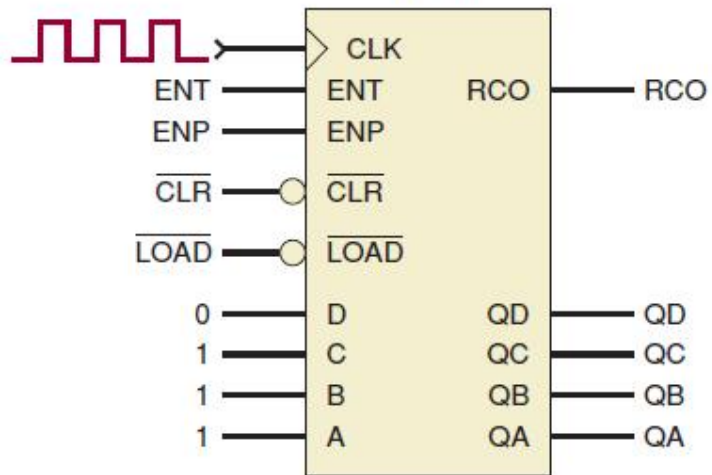


器件实例：74 160

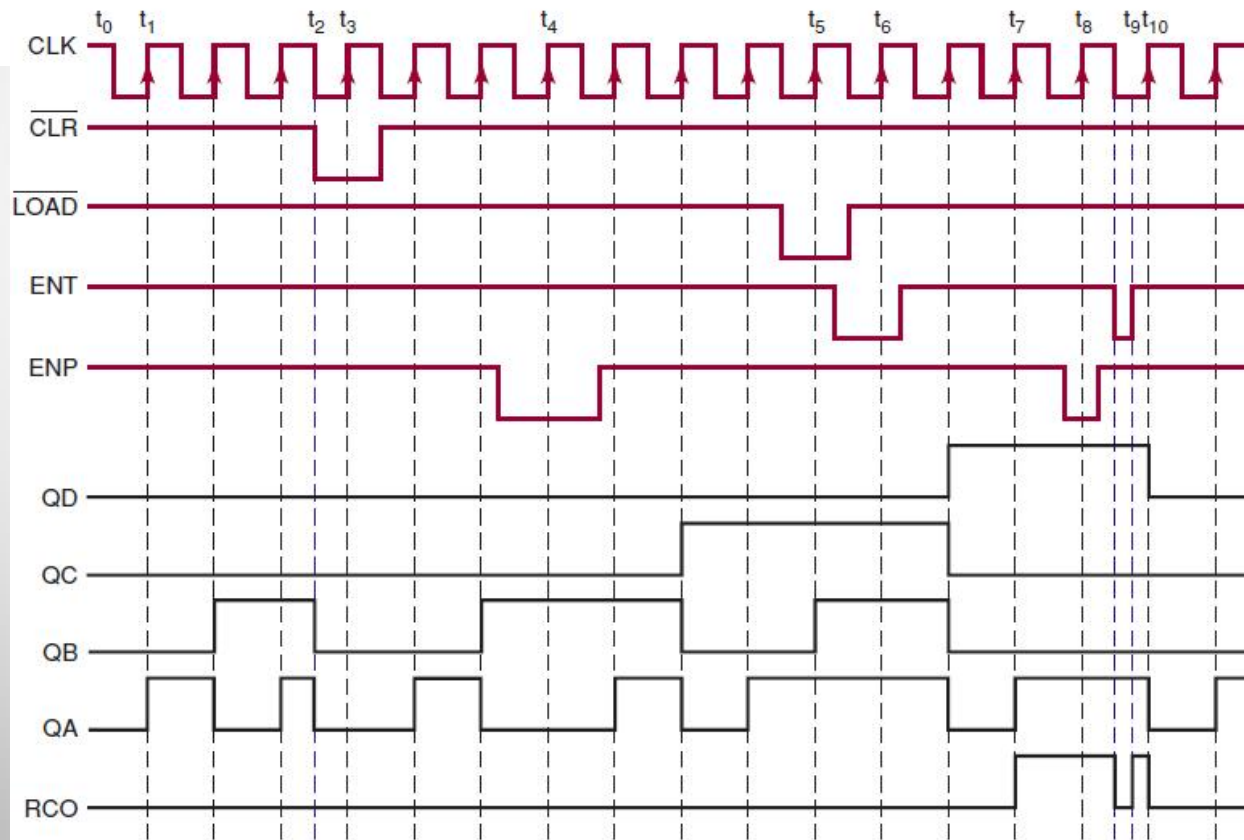


CLK	R'_D	LD'	EP	ET	工作状态
X	0	X	X	X	置 0 (异步)
	1	0	X	X	预置数 (同步)
X	1	1	0	1	保持 (包括C)
X	1	1	X	0	保持 (C=0)
	1	1	1	1	计数

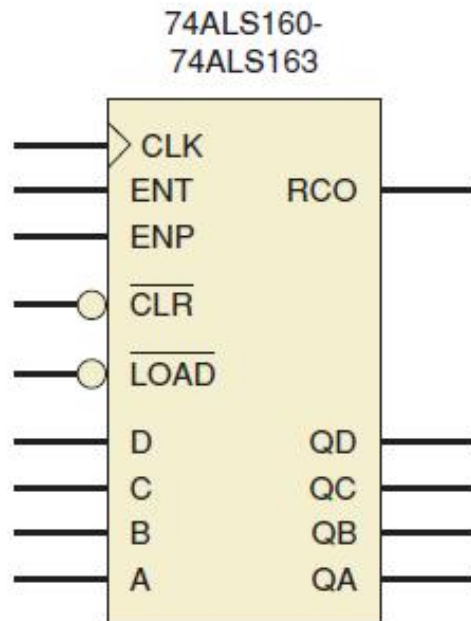
74HC160



(a)



(b)



(a)

Part Number	Modulus
74ALS160	10
74ALS161	16
74ALS162	10
74ALS163	16

(b)

74ALS160-74ALS163 Function Table

$\overline{\text{CLR}}$	$\overline{\text{LOAD}}$	ENP	ENT	CLK	Function	Part Numbers
L	X	X	X	X	Asynch. Clear	74ALS160 & 74ALS161
L	X	X	X	$\uparrow$	Synchr. Clear	74ALS162 & 74ALS163
H	L	X	X	$\uparrow$	Synchr. Load	All
H	H	H	H	$\uparrow$	Count up	All
H	H	L	X	X	No change	All
H	H	X	L	X	No change	All

(c)

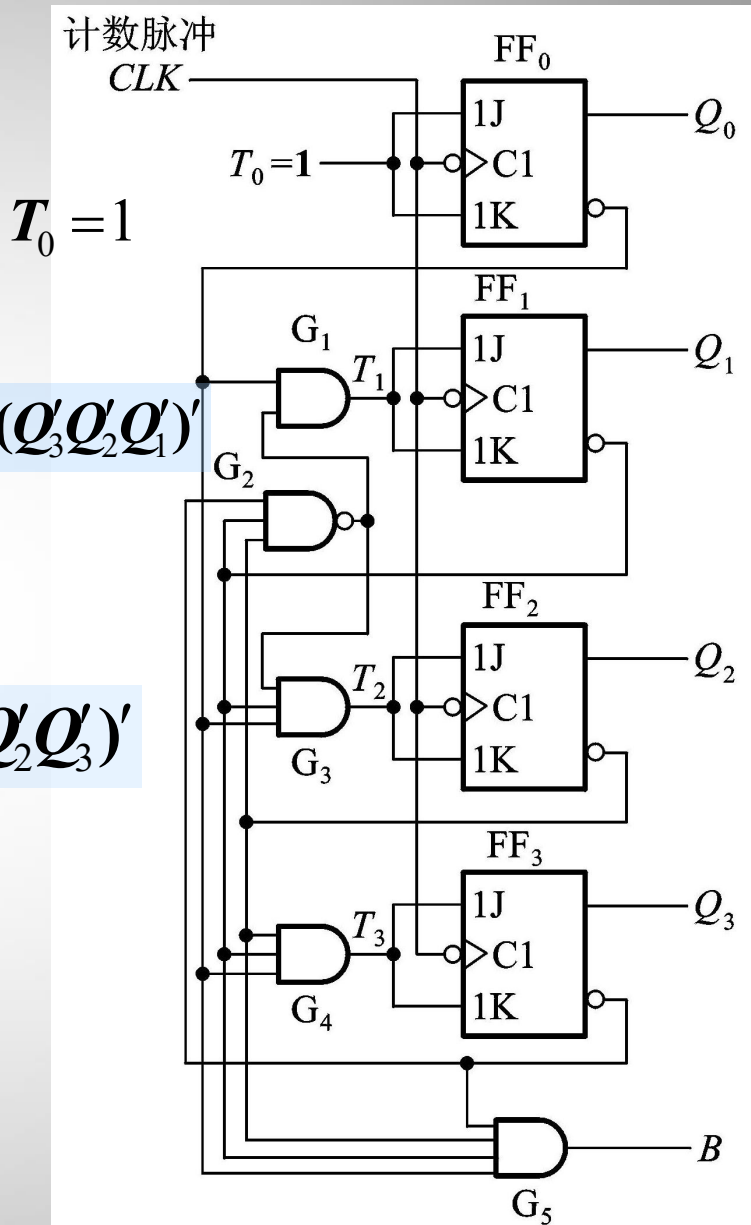
②减法计数器

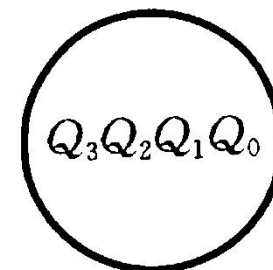
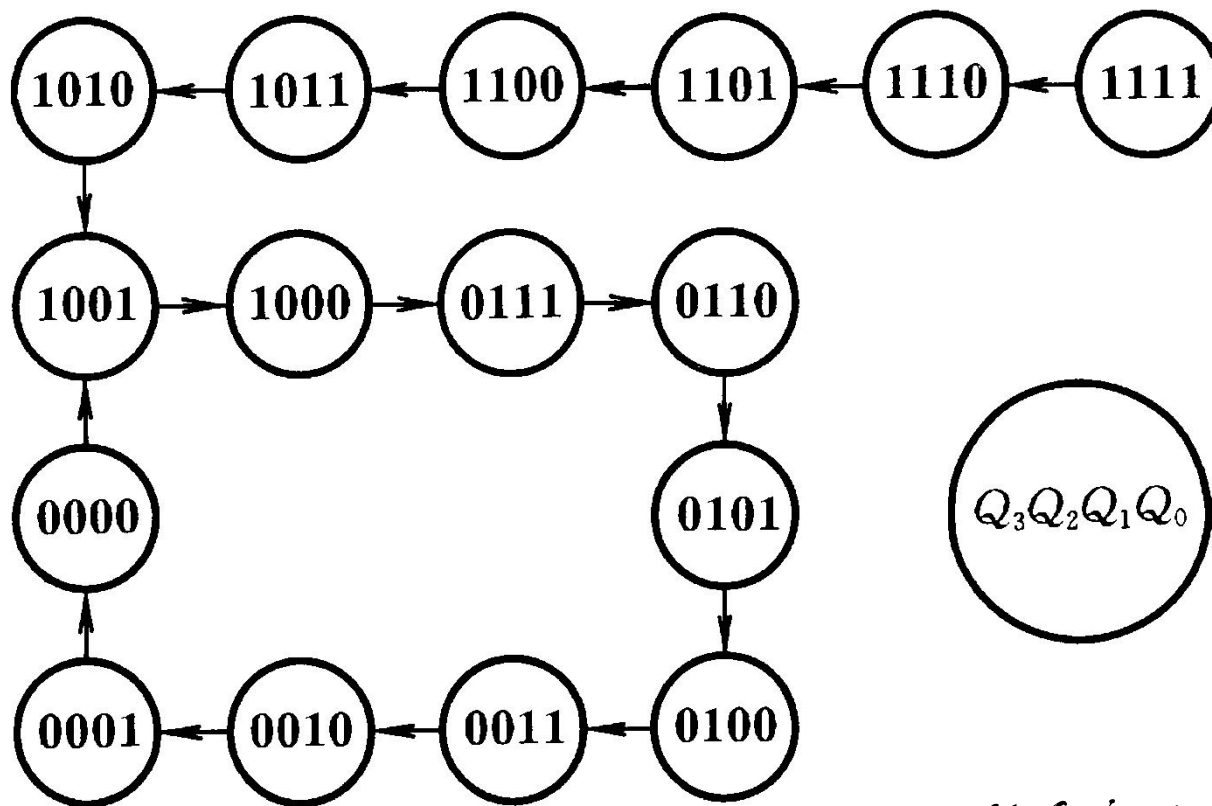
基本原理：对二进制减法计数器进行修改，在0000时减“1”后跳变为1001，然后按二进制减法计数就行了。

$$T_1 = Q'_0 \Rightarrow Q'_0(Q'_3Q'_2Q'_1)'$$

$$T_2 = \underline{Q}'_1 \underline{Q}'_0 \Rightarrow \underline{Q}'_1 \underline{Q}'_0 \cdot (\underline{Q}'_1 \underline{Q}'_2 \underline{Q}'_3)'$$

$$T_3 = Q_2 Q_1 Q_0$$





能自启动

③十进制可逆计数器

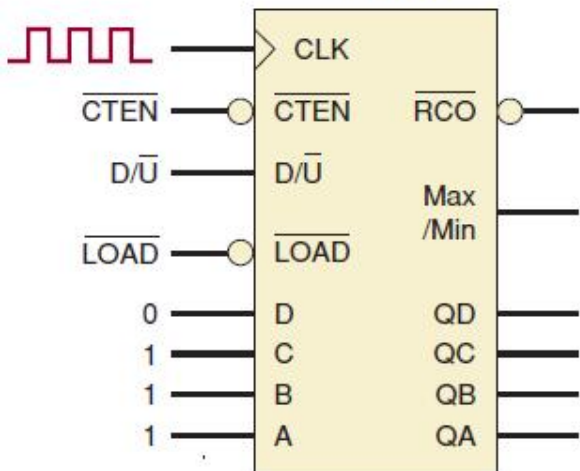
基本原理一致，电路只用到0000~1001的十个状态

实例器件

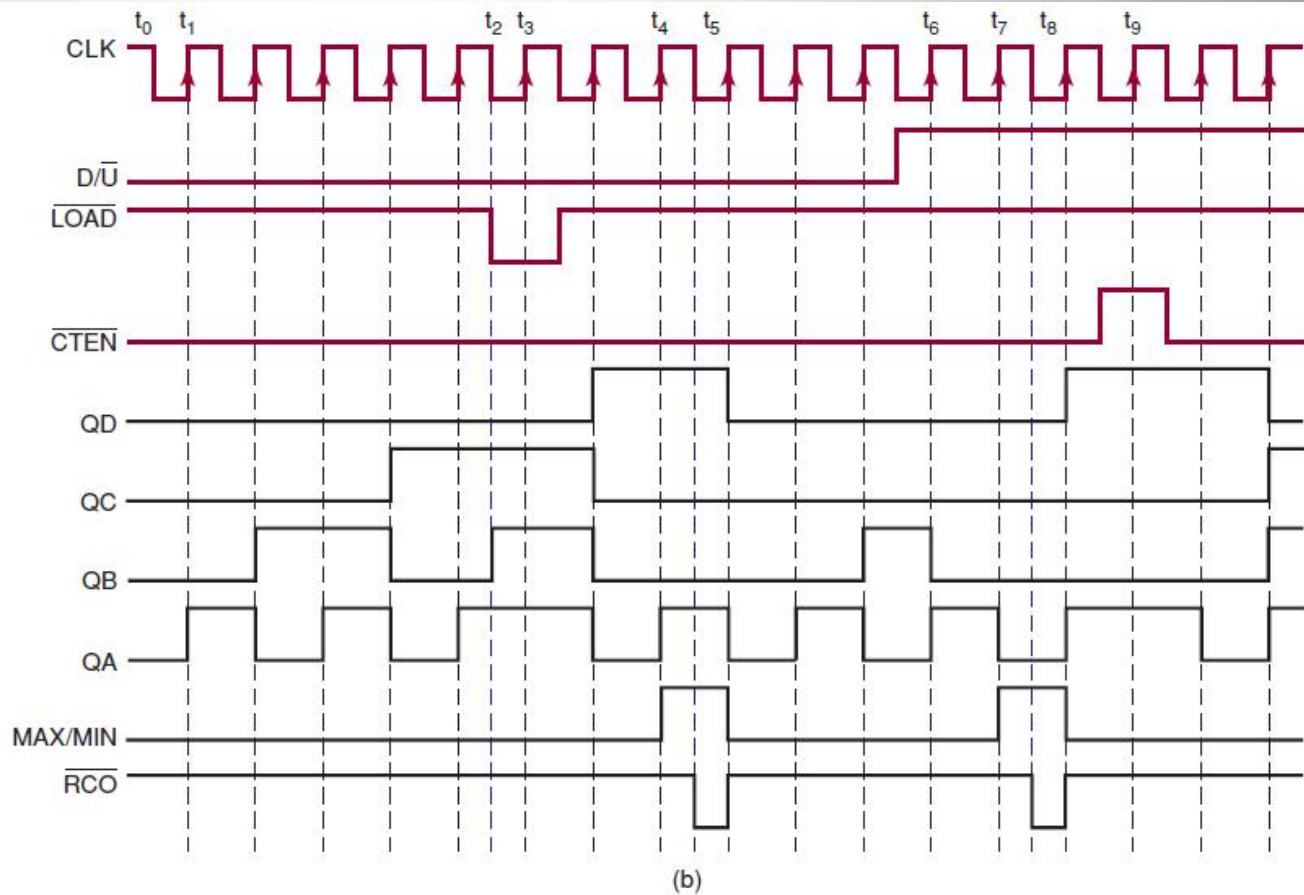
单时钟：74190,168

双时钟：74192

74HC190



(a)



(b)

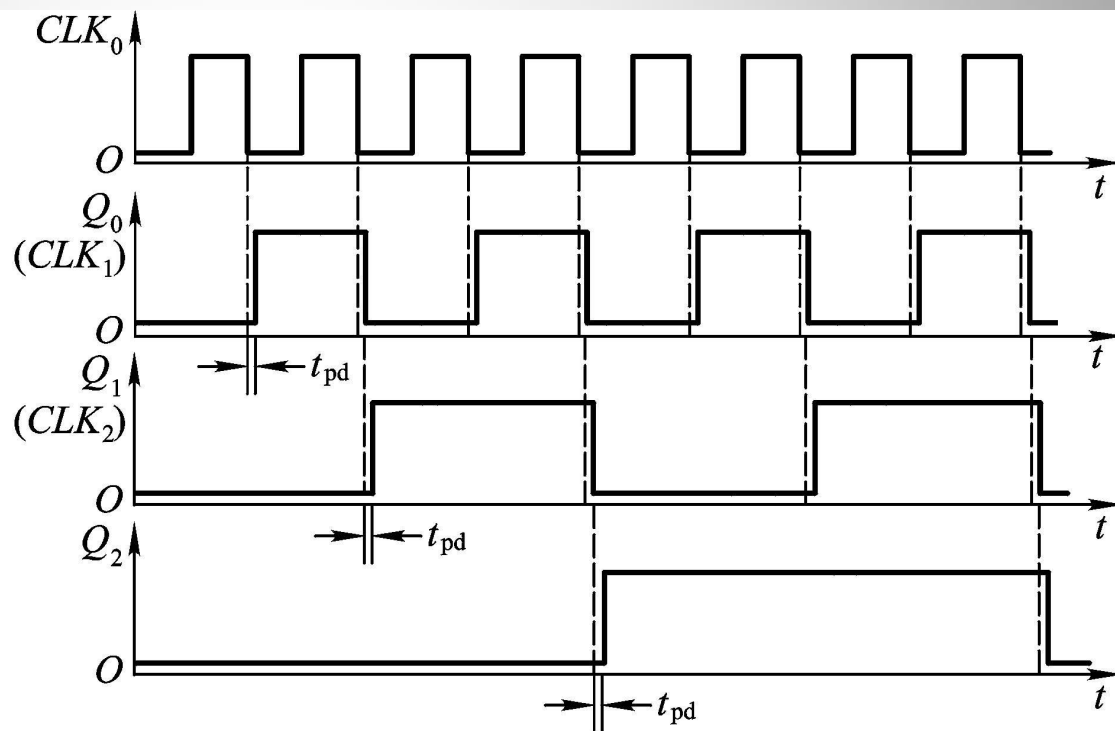
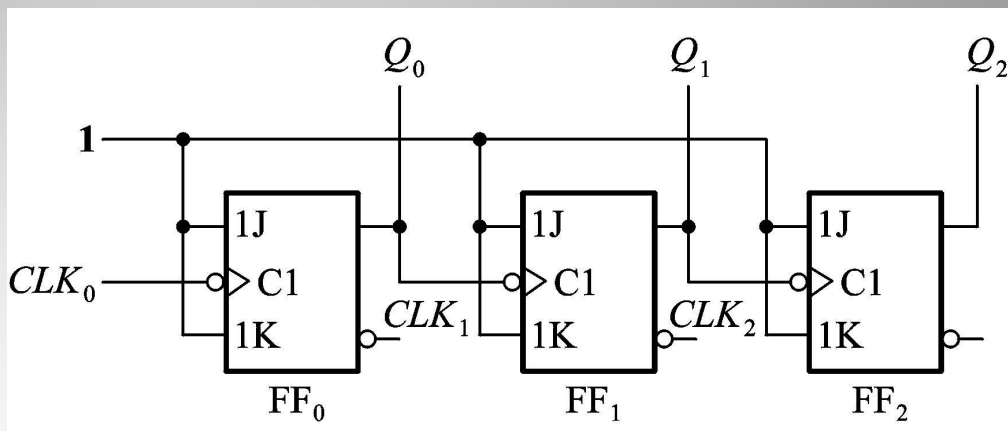
二. 异步计数器

1. 二进制计数器

① 异步二进制加法计数器

在末位+1时，从低位到高位逐位进位方式工作。

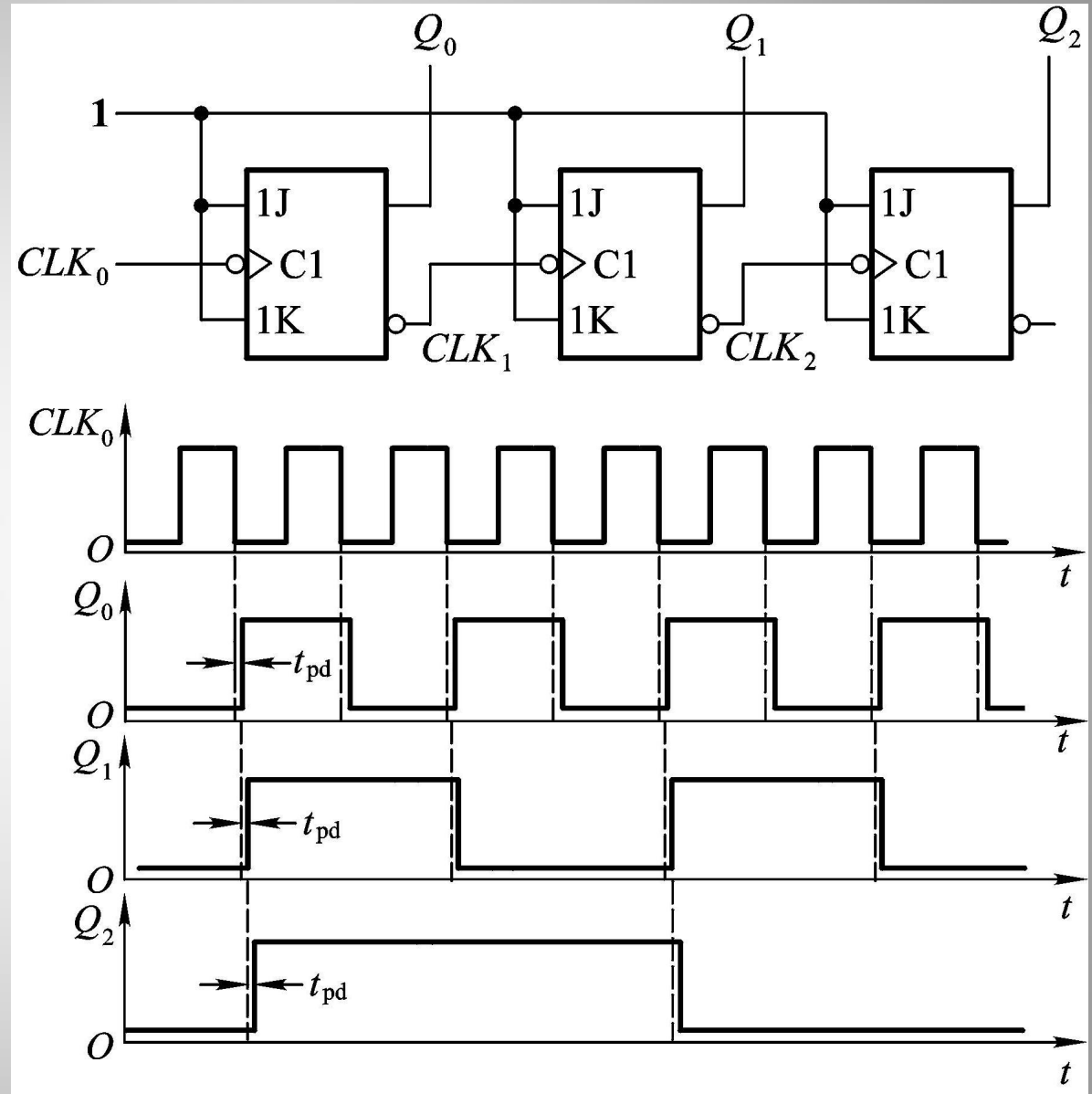
原则：每1位从“1”变“0”时，向高位发出进位，使高位翻转



②异步二进制减法计数器

在末位-1时，从低位到高位逐位借位方式工作。

原则：每1位从“0”变“1”时，向高位发出进位，使高位翻转



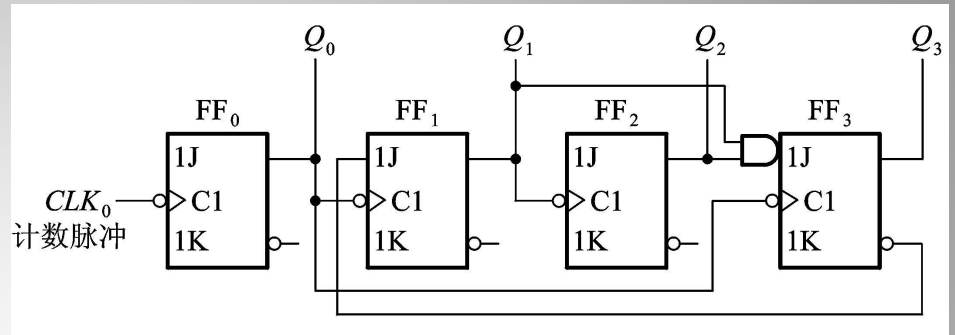
2、异步十进制加法计数器

原理：

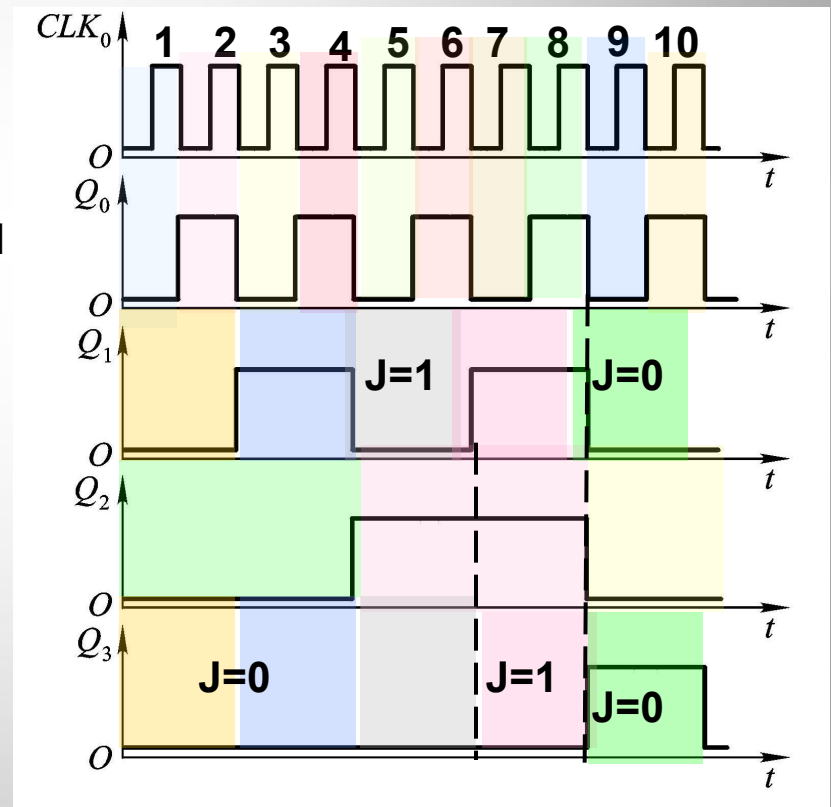
在4位二进制异步加法计数器上修改而成，

要跳过

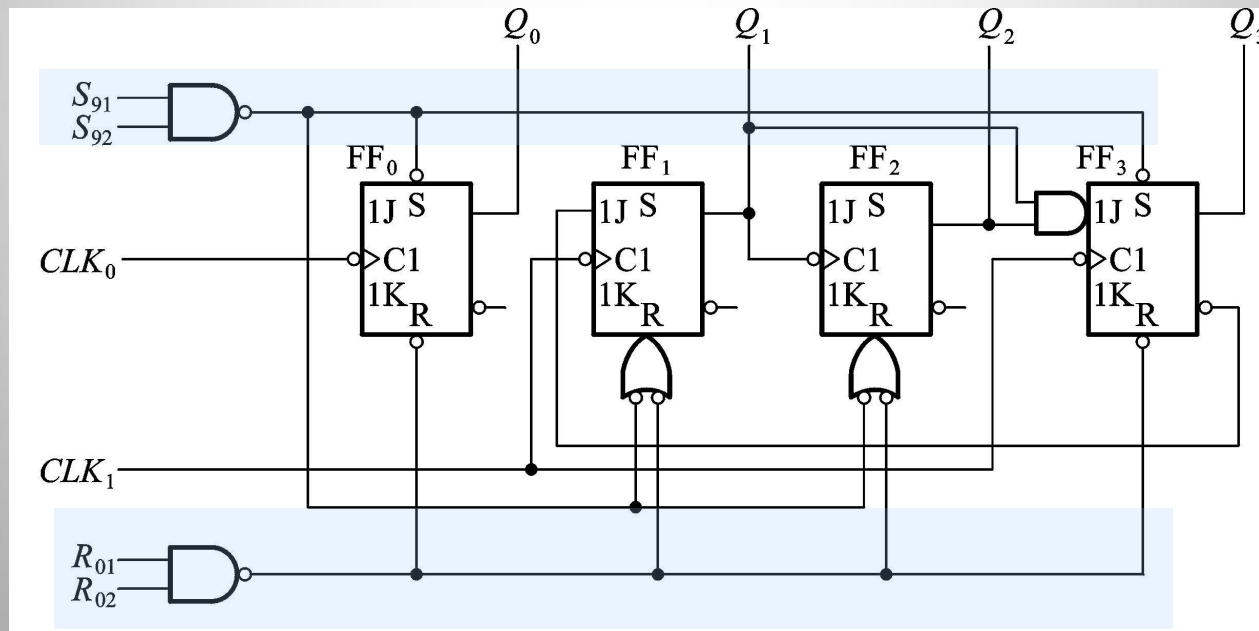
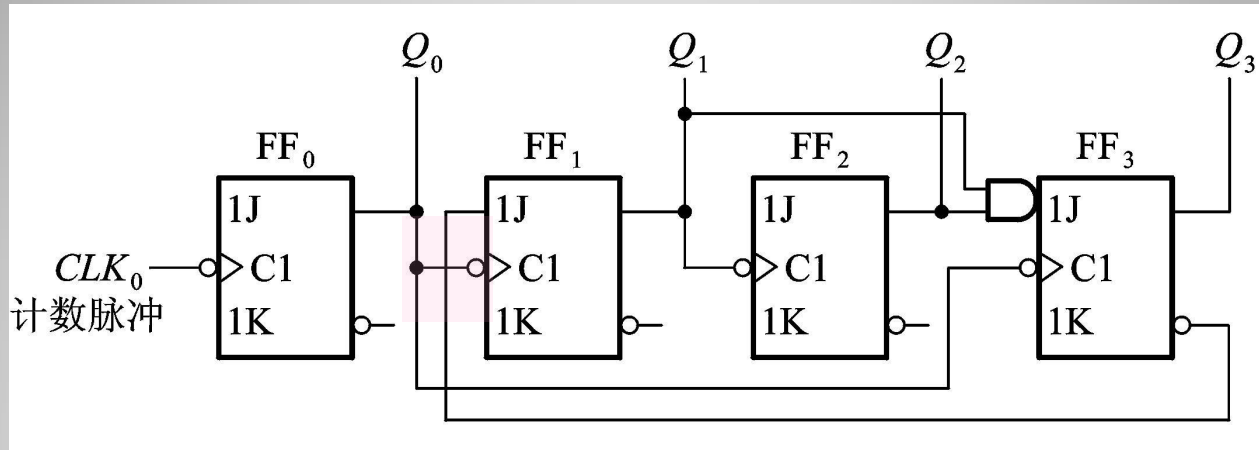
1010 ~ 1111这六个状态



$J=K=1$

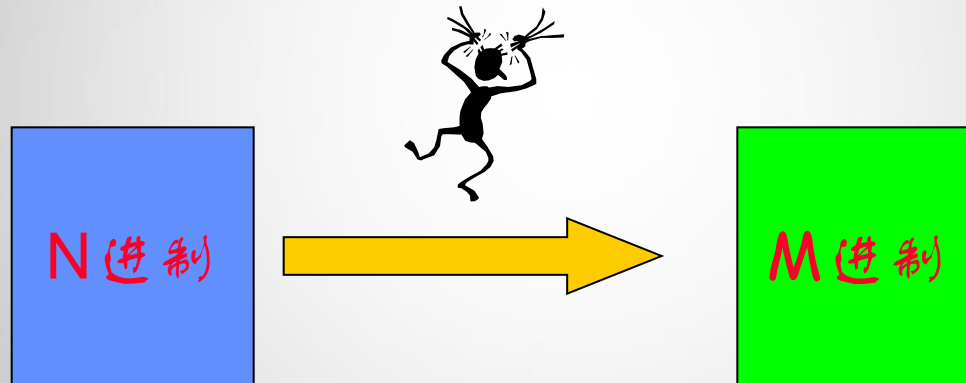


器件实例：二一五一十进制异步计数器74LS290



三、任意进制计数器的构成方法

用已有的**N**进制芯片，组成**M**进制计数器，是常用的方法。



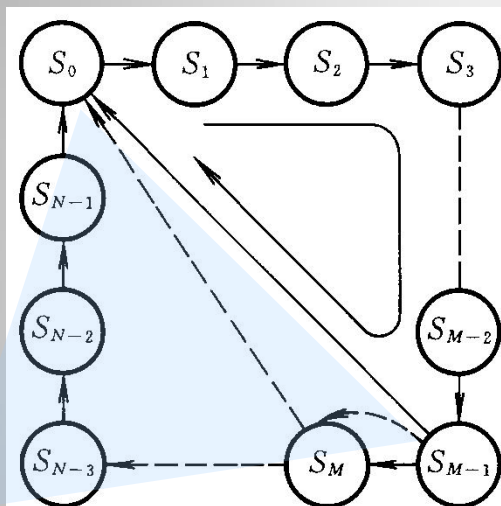
$$\begin{cases} N > M \\ N < M \end{cases}$$

1. $N > M$

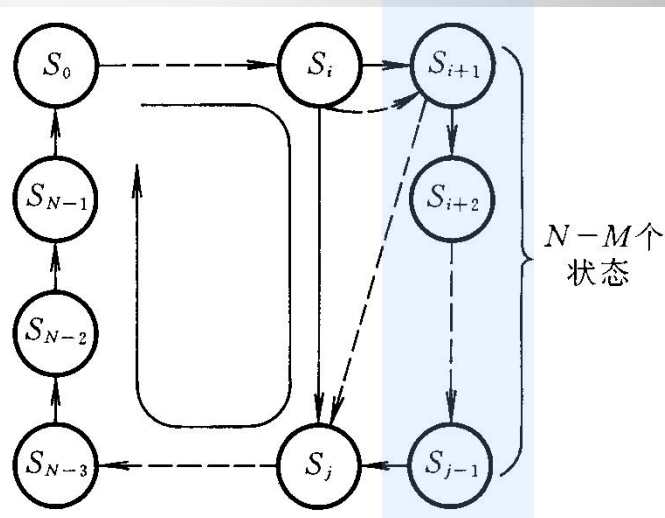
原理：计数循环过程中设法跳过 $N-M$ 个状态。

具体方法：置零法

置数法



(a)



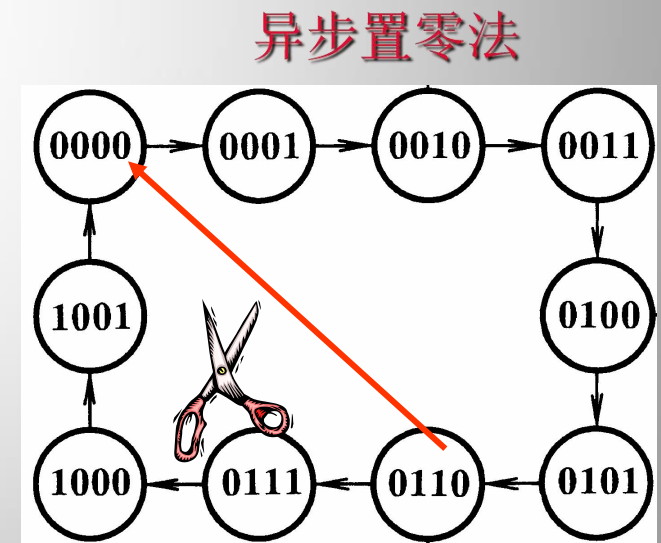
(b)

{ 异步置零法
{ 同步置零法

{ 异步预置数法
{ 同步预置数法

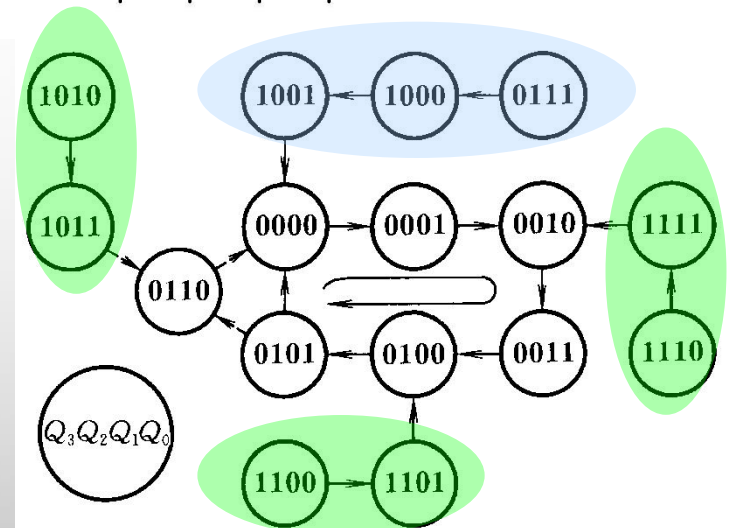
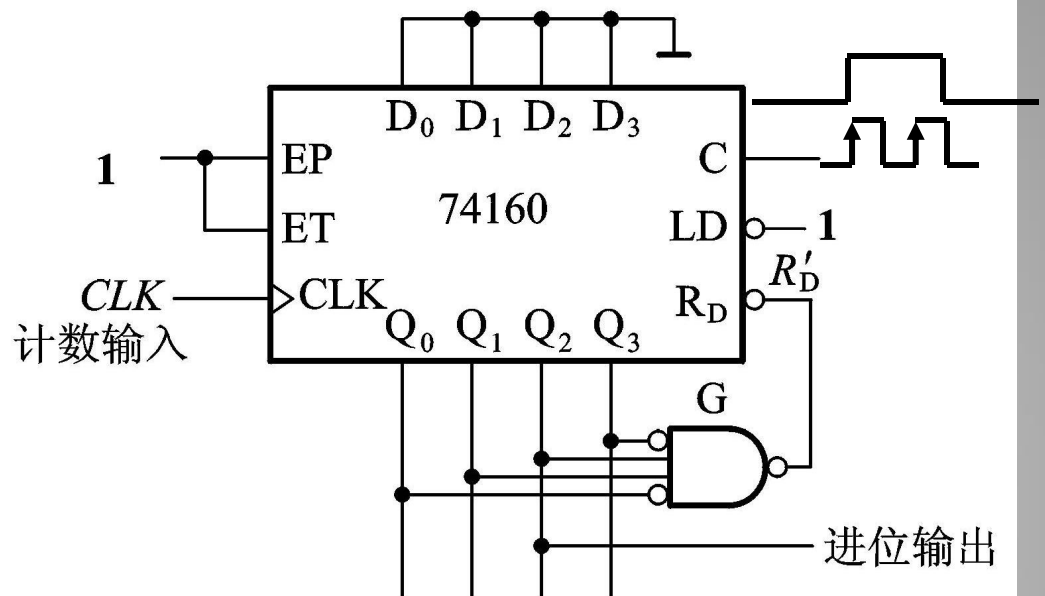
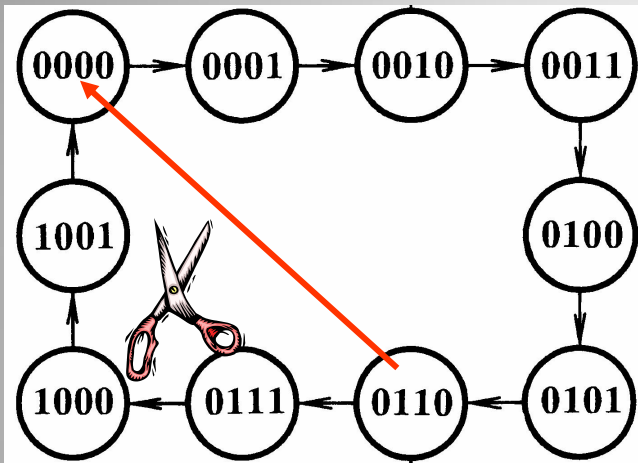
例：将十进制的74160接成六进制计数器

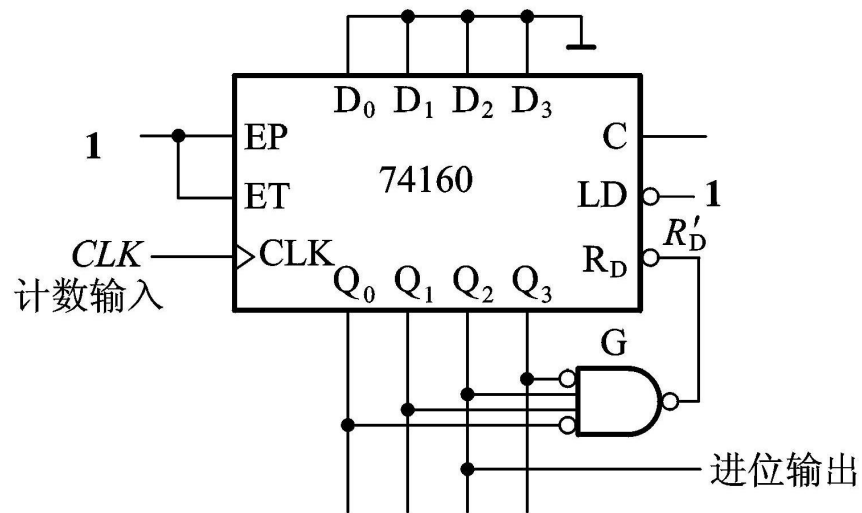
CLK	R'_D	LD'	EP	ET	工作状态
X	0	X	X	X	置 0（异步）
	1	0	X	X	预置数（同步）
X	1	1	0	1	保持（包括C）
X	1	1	X	0	保持（C=0）
	1	1	1	1	计数



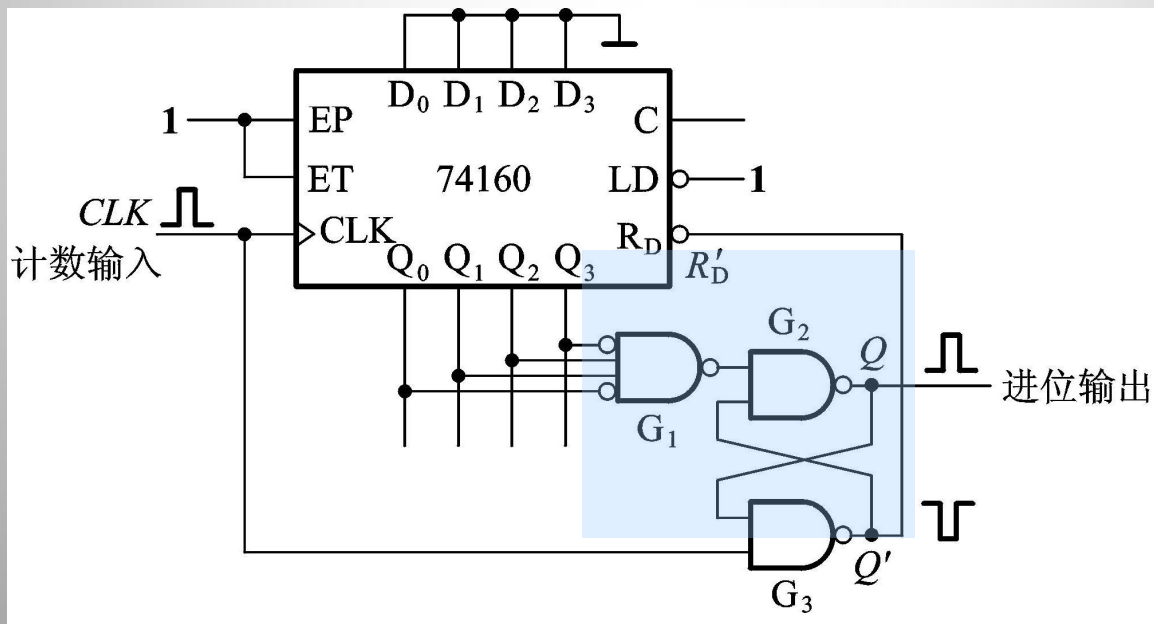
例：将十进制的74160接成六进制计数器

异步置零法





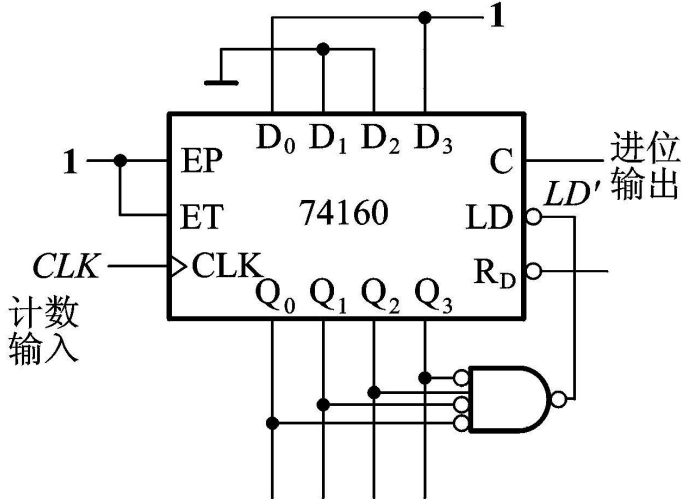
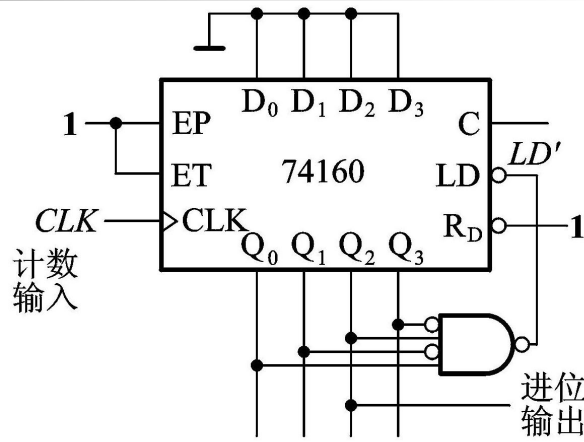
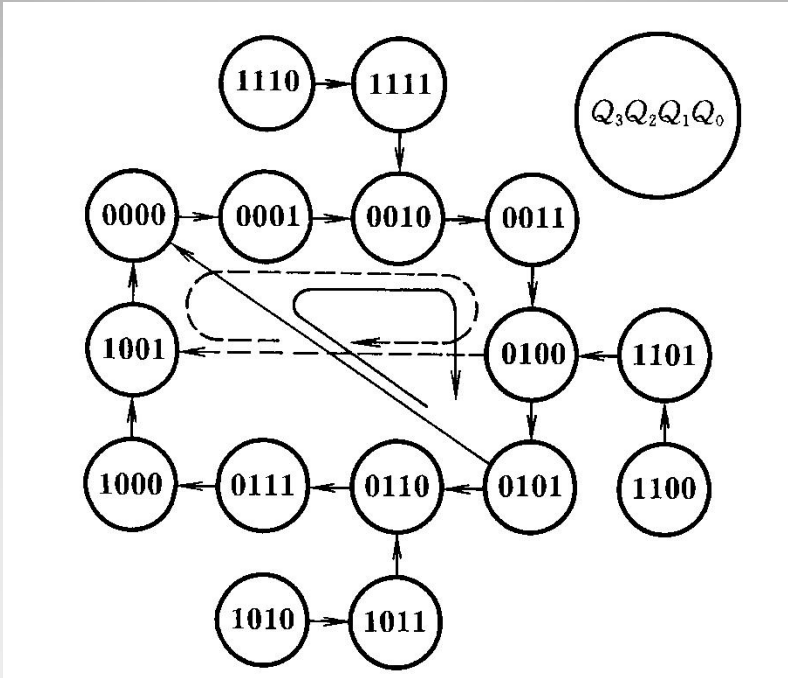
缺点：置 0 信号作用时间短



置数法

(a)置入0000

(b)置入1001



2. $N < M$

① $M = N1 \times N2$

先用前面的方法分别接成N1和N2两个计数器。

N1和N2间的连接有两种方式：

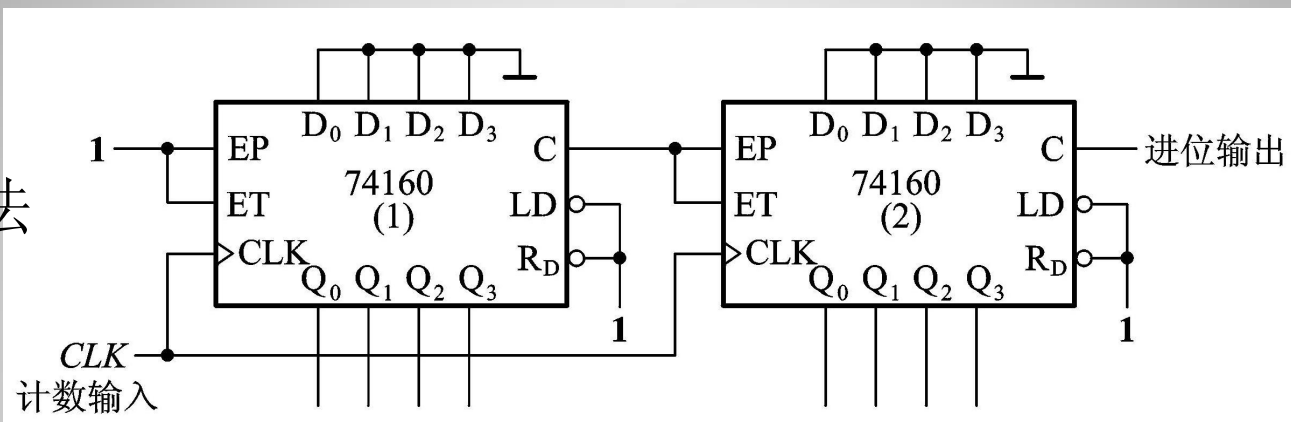
- a. **并行进位方式**：用同一个CLK，低位片的进位输出作为高位片的计数控制信号（如74160的EP和ET）
- b. **串行进位方式**：低位片的进位输出作为高位片的CLK，两片始终同时处于计数状态

例：用74160接成一百进制

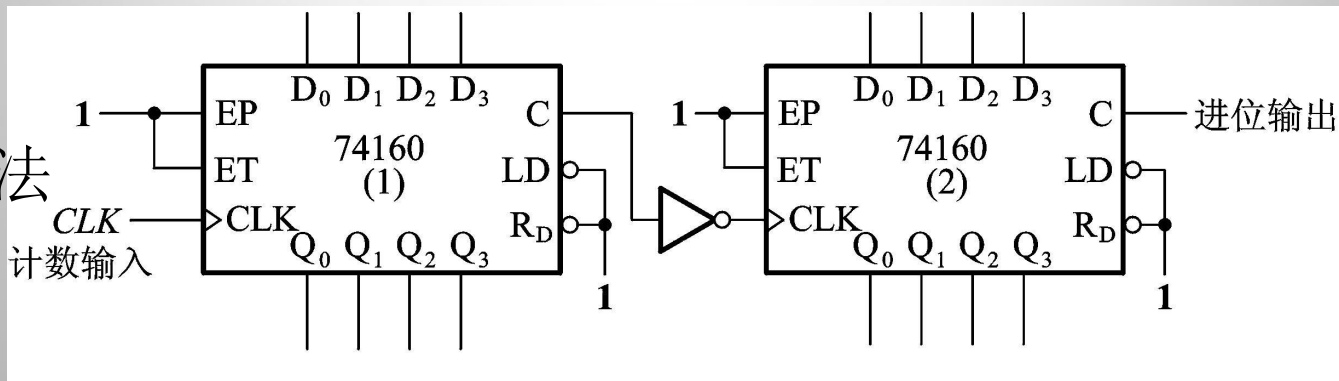
CLK	R'_D	LD'	EP	ET	工作状态
X	0	X	X	X	置 0（异步）
	1	0	X	X	预置数（同步）
X	1	1	0	1	保持（包括C）
X	1	1	X	0	保持（C=0）
	1	1	1	1	计数

例：用两片74160接成一百进制计数器

并行进位法



串行进位法



②M不可分解

采用整体置零和整体置数法：

先用两片接成 $M' > M$ 的计数器

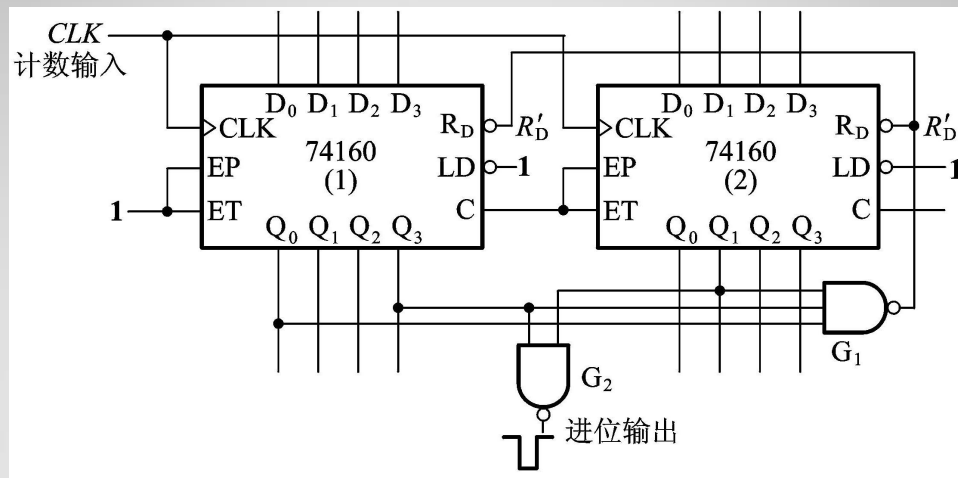
然后再采用置零或置数的方法

例：用74160接成二十九进制

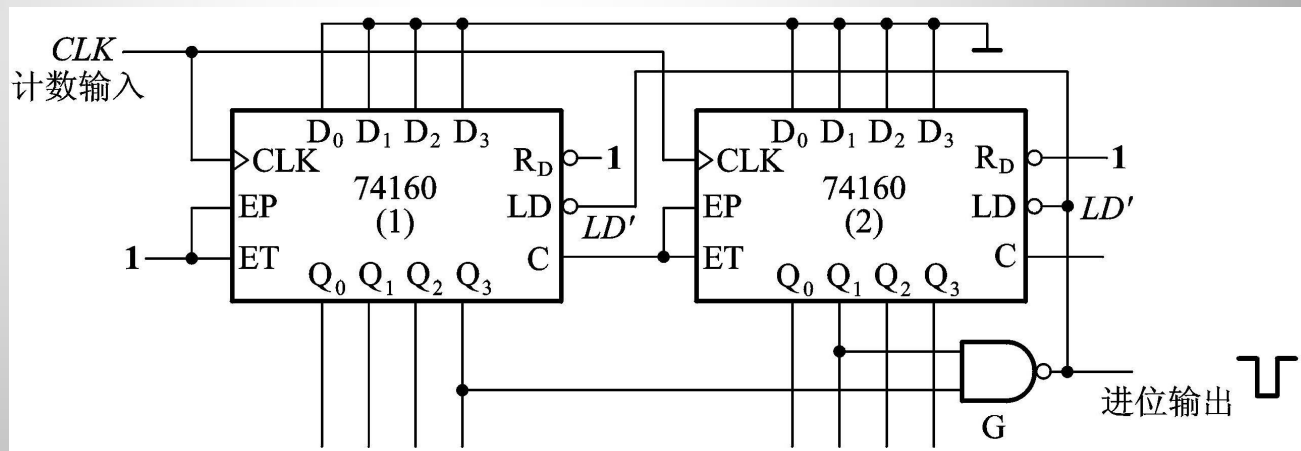
CLK	R'_D	LD'	EP	ET	工作状态
X	0	X	X	X	置 0（异步）
	1	0	X	X	预置数（同步）
X	1	1	0	1	保持（包括C）
X	1	1	X	0	保持（C=0）
	1	1	1	1	计数

例：用74160接成二十九进制

整体置零
(异步)

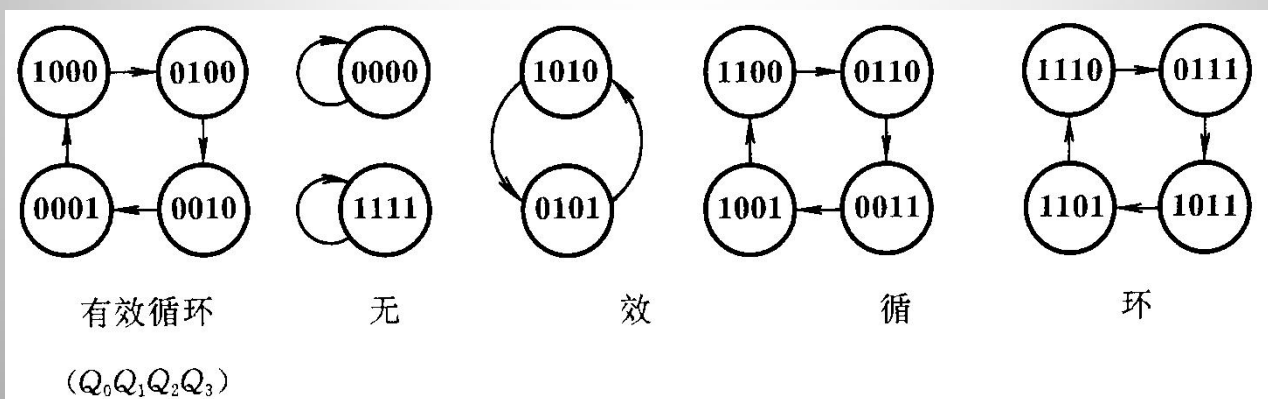
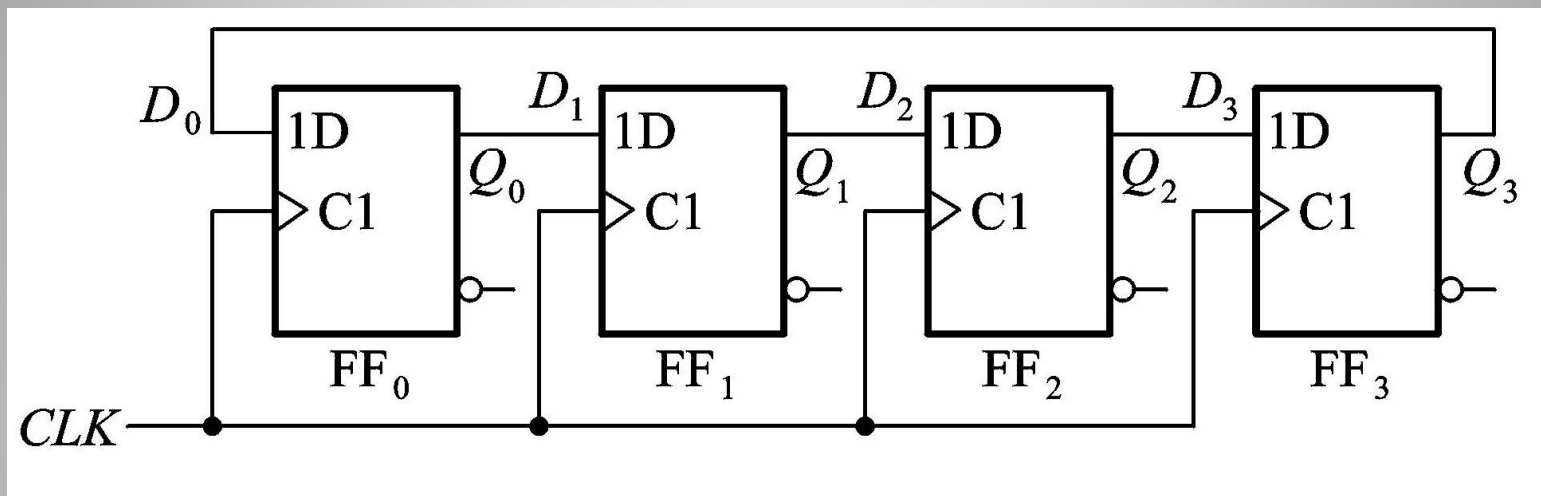


整体置数
(同步)

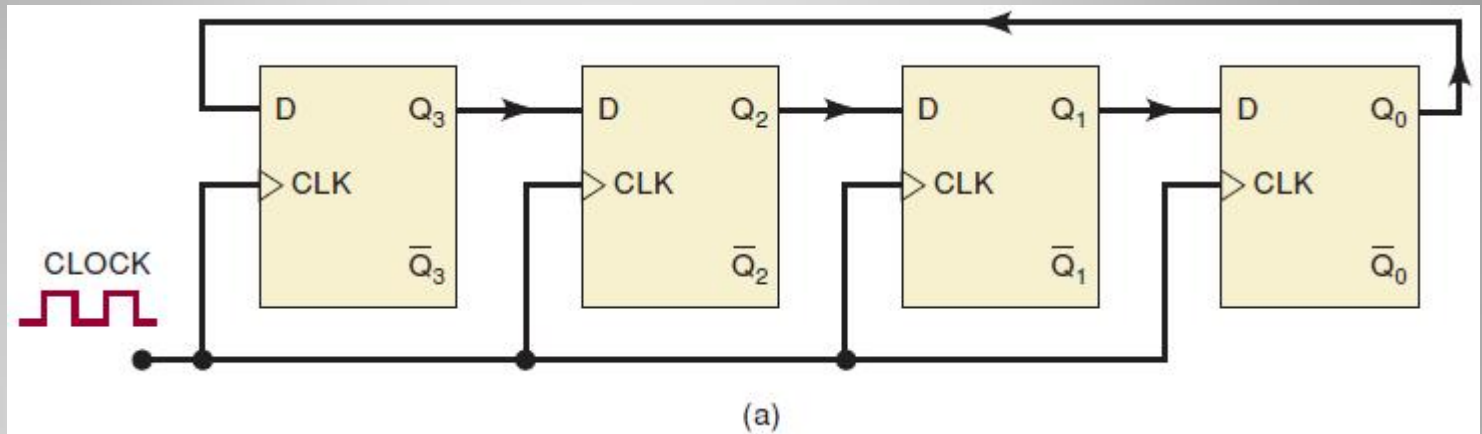


四、移位寄存器型计数器

1. 环形计数器

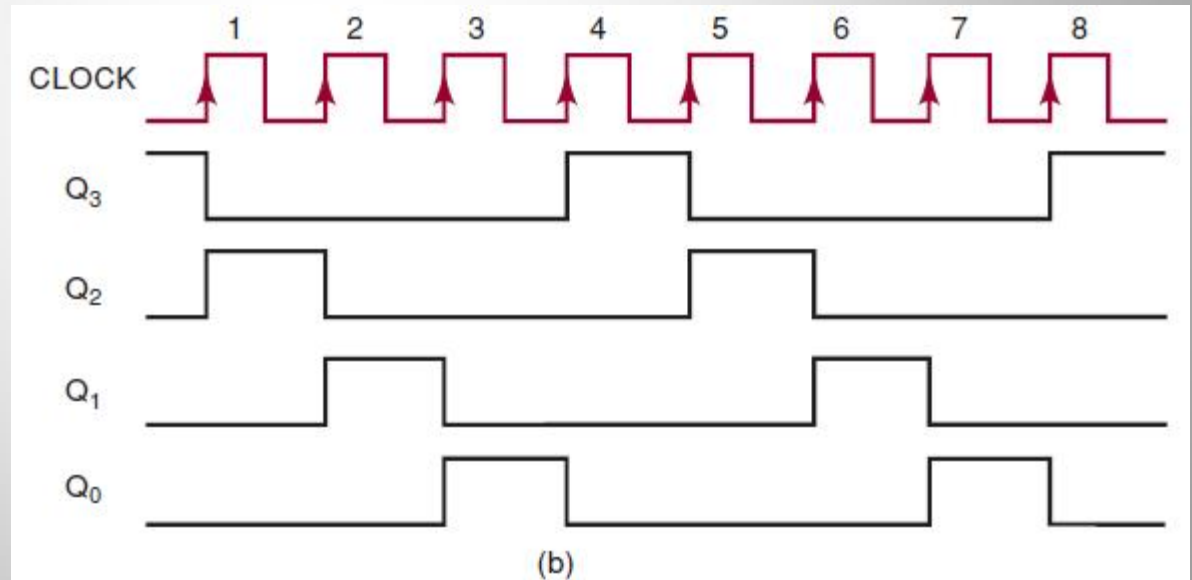


Ring Counter

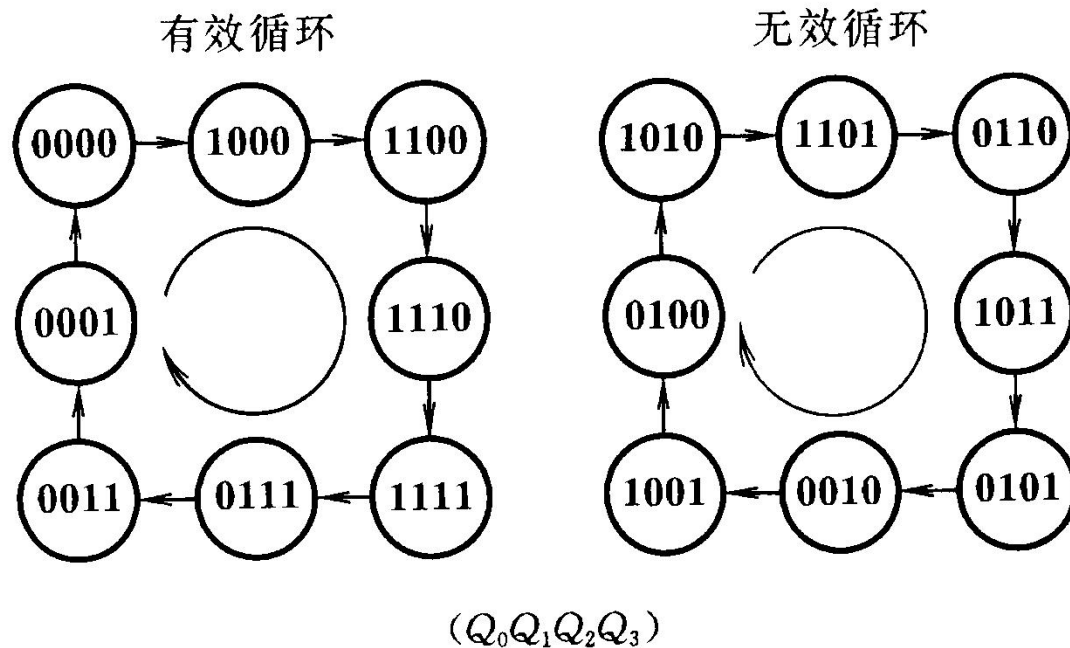
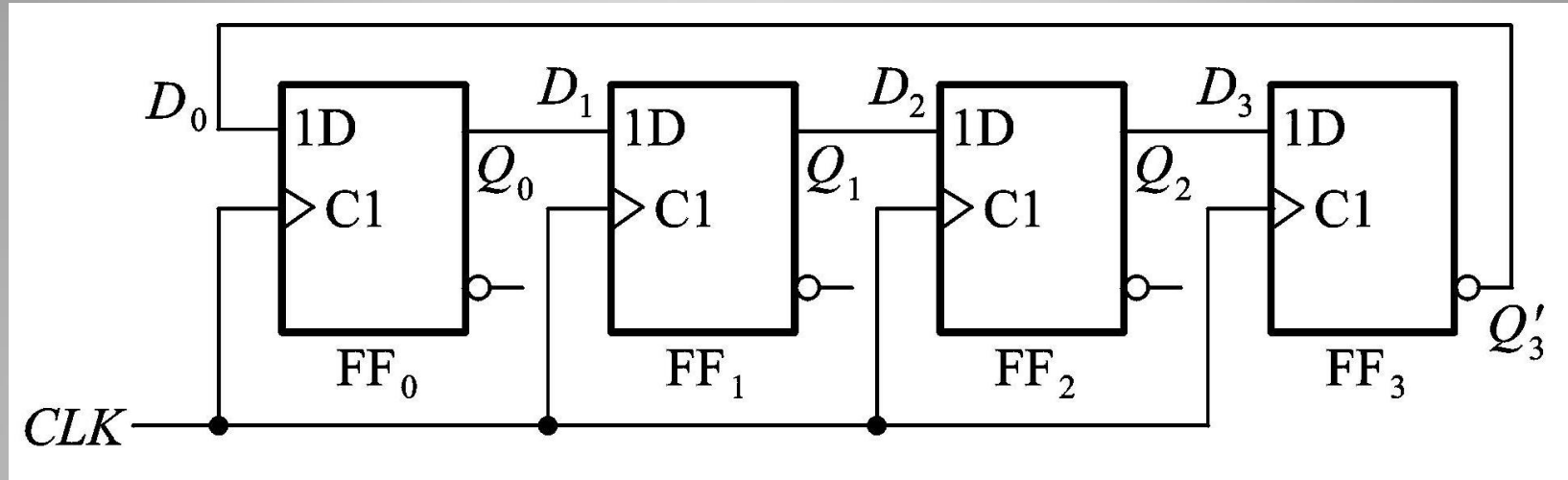


Q_3	Q_2	Q_1	Q_0	CLOCK pulse
1	0	0	0	0
0	1	0	0	1
0	0	1	0	2
0	0	0	1	3
1	0	0	0	4
0	1	0	0	5
0	0	1	0	6
0	0	0	1	7
.	.	.	.	.
.	.	.	.	.

(c)

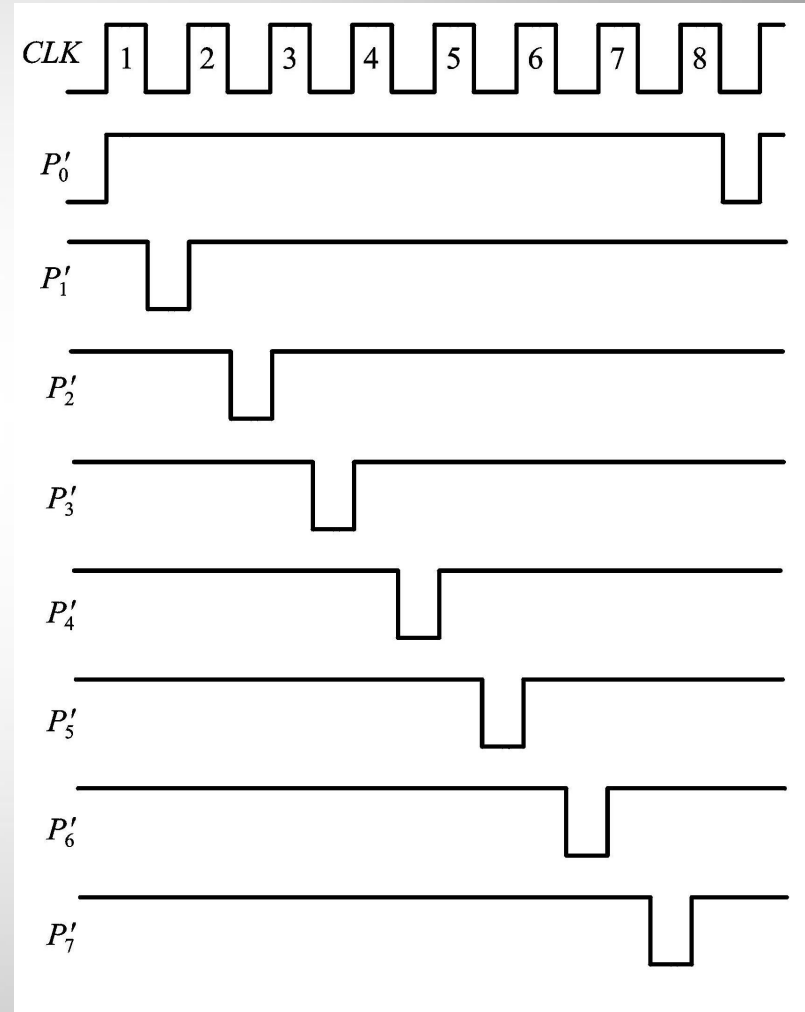
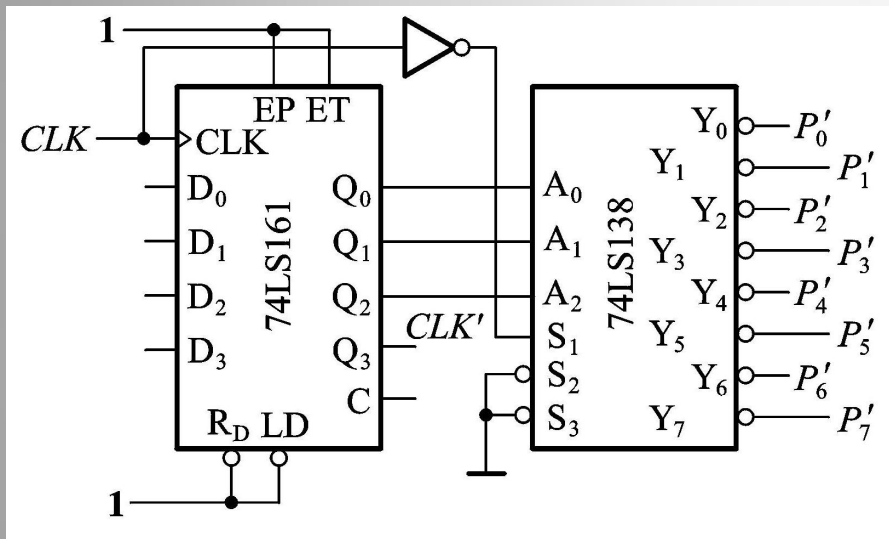


2. 扭环形计数器 Johnson Counter

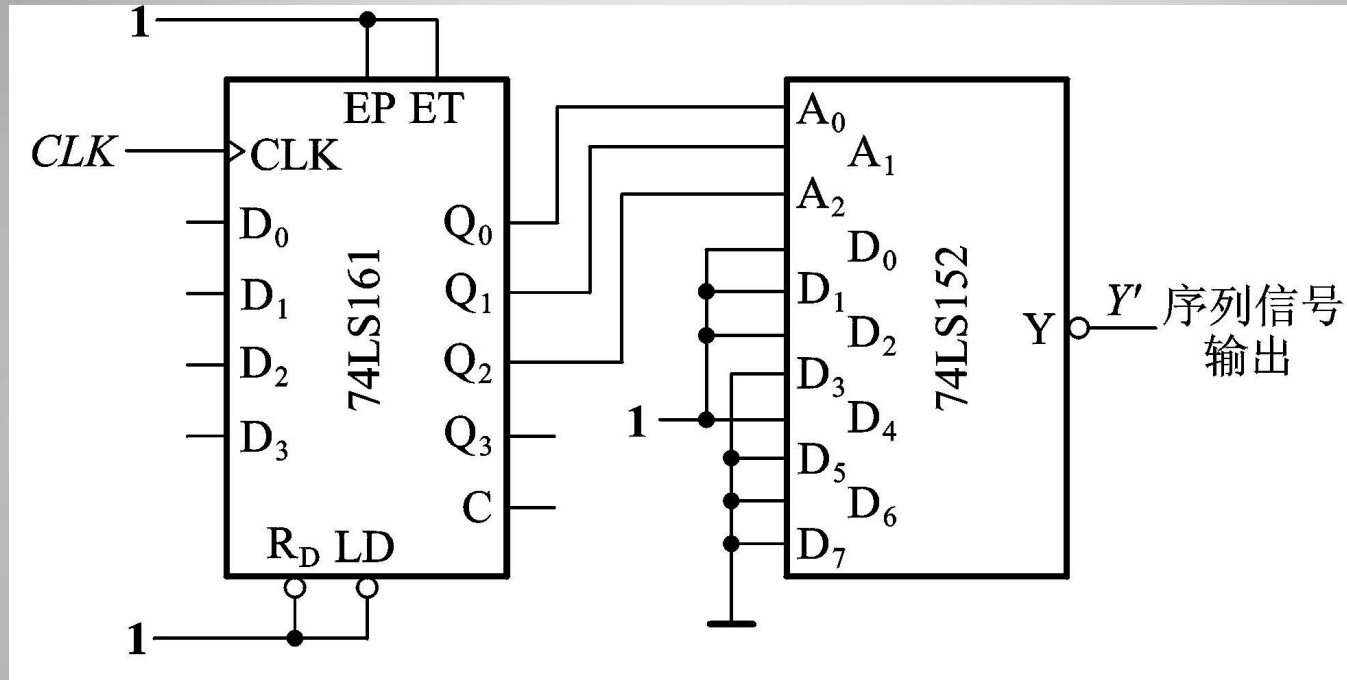


五、计数器应用实例

例1，计数器+译码器→顺序节拍脉冲发生器

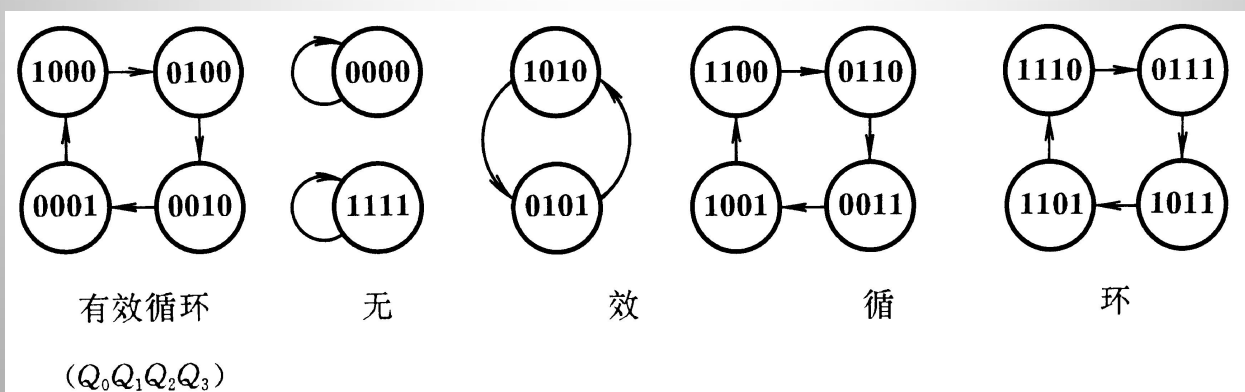
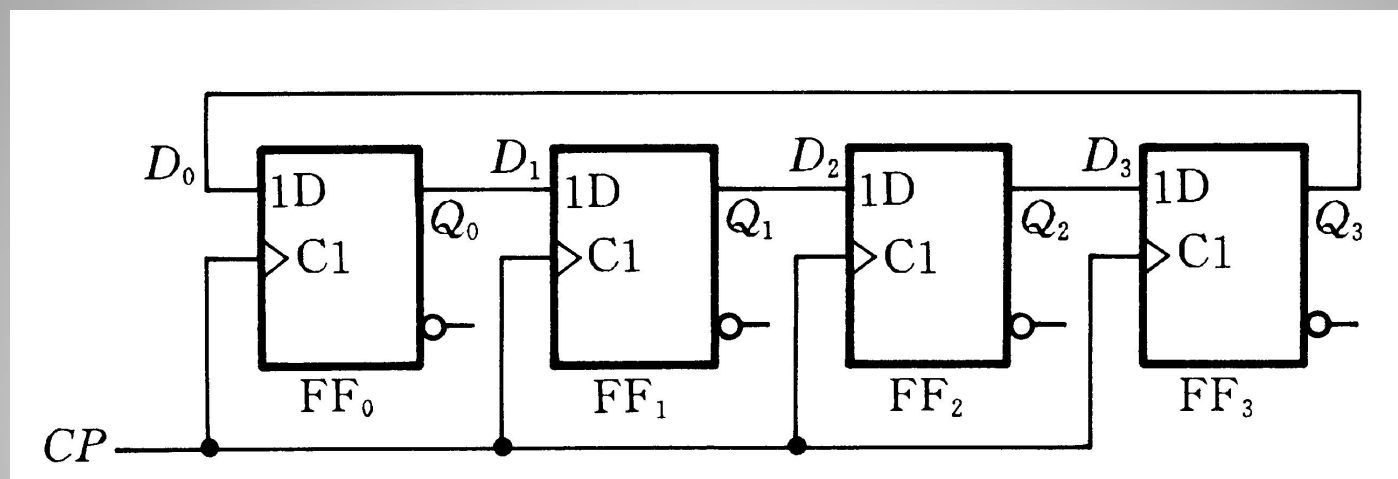


例2，计数器+数据选择器→序列脉冲发生器

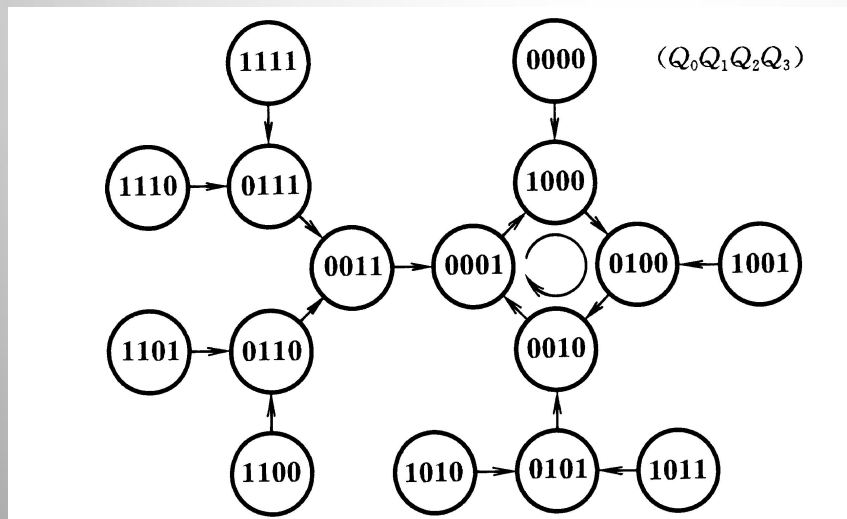
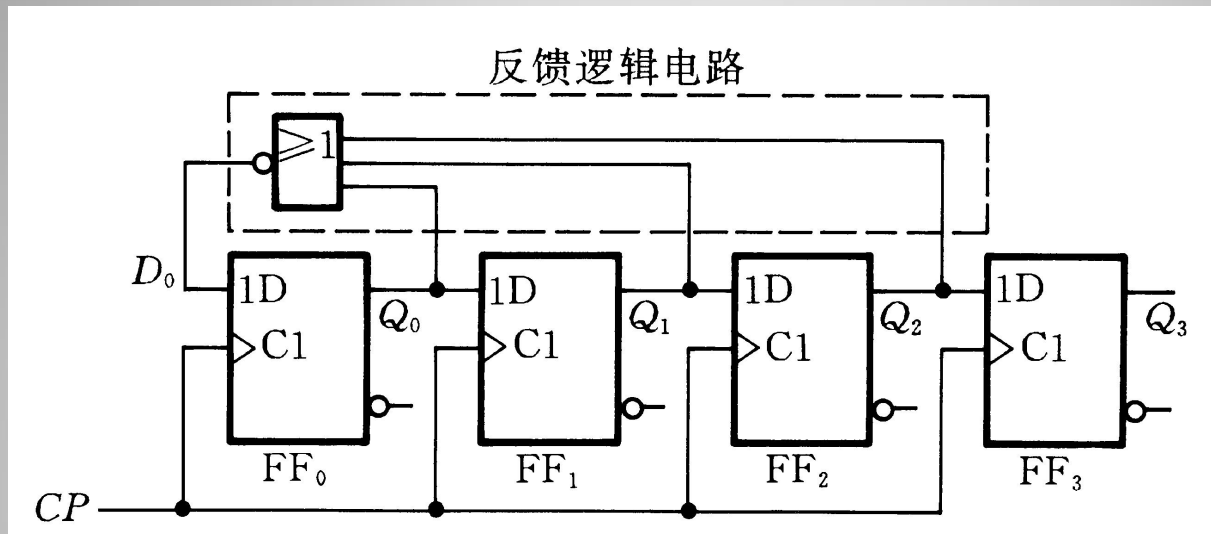


发生的序列：00010111? 11101000 ...

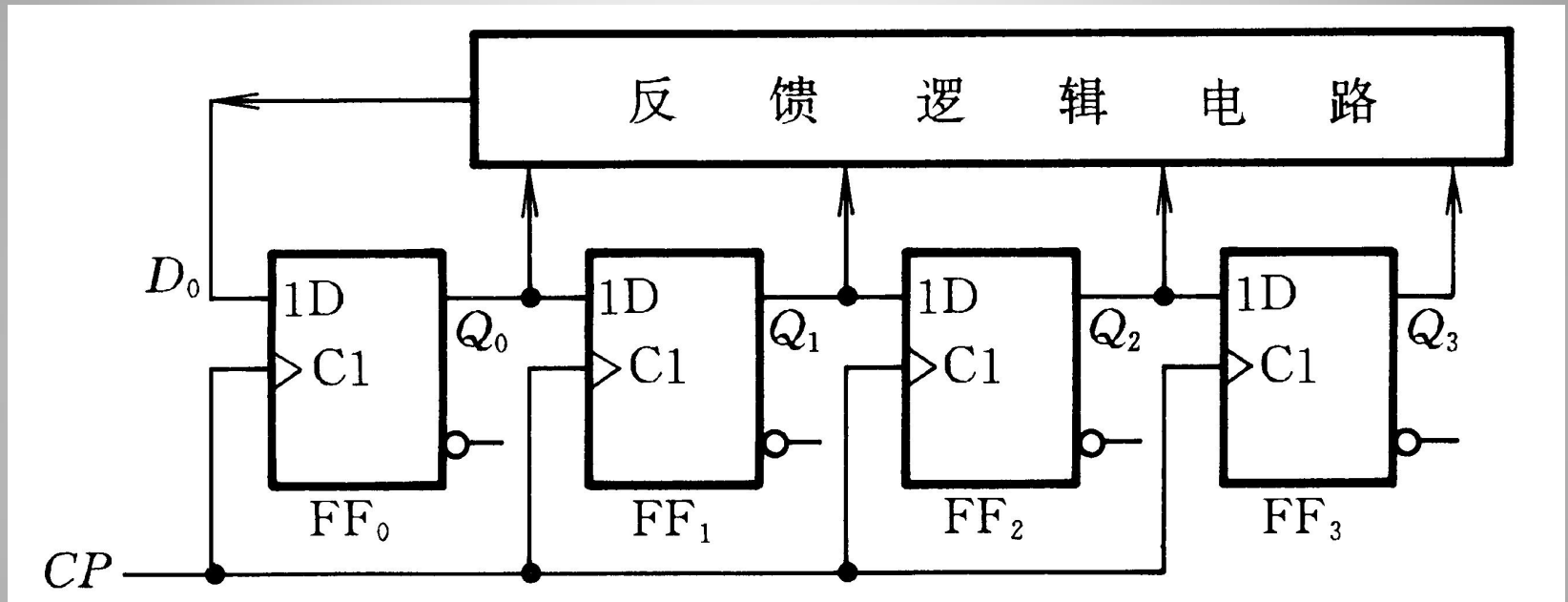
环形计数器电路



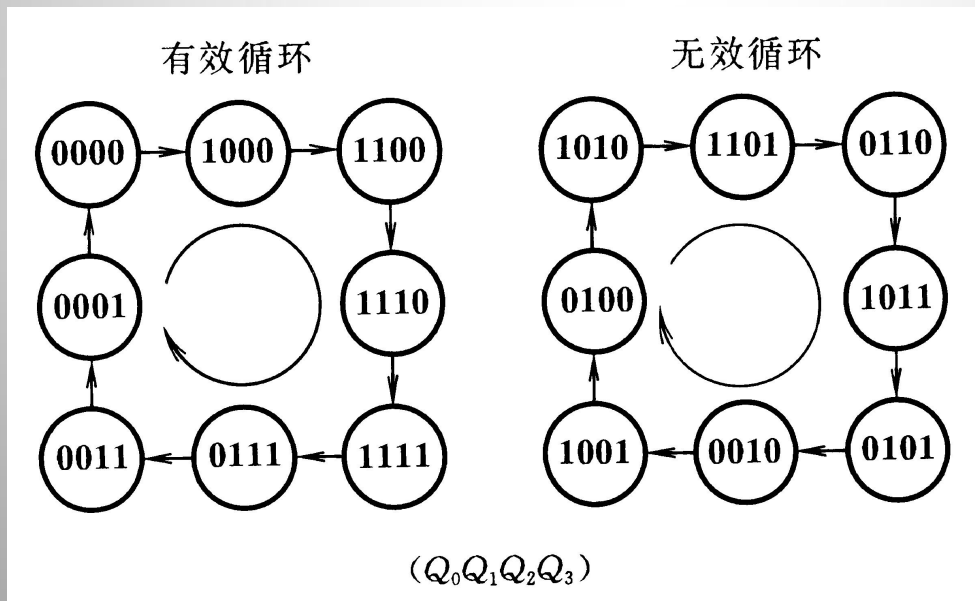
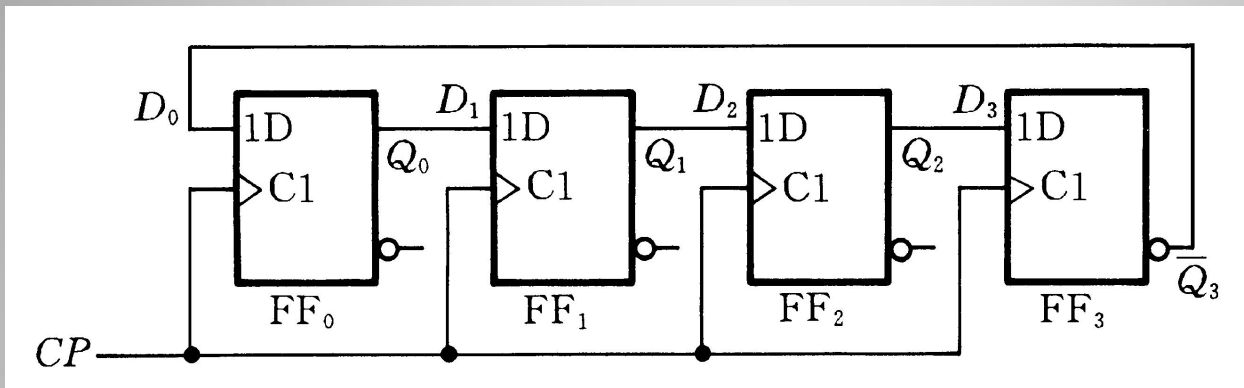
能自启动的环形计数器电路



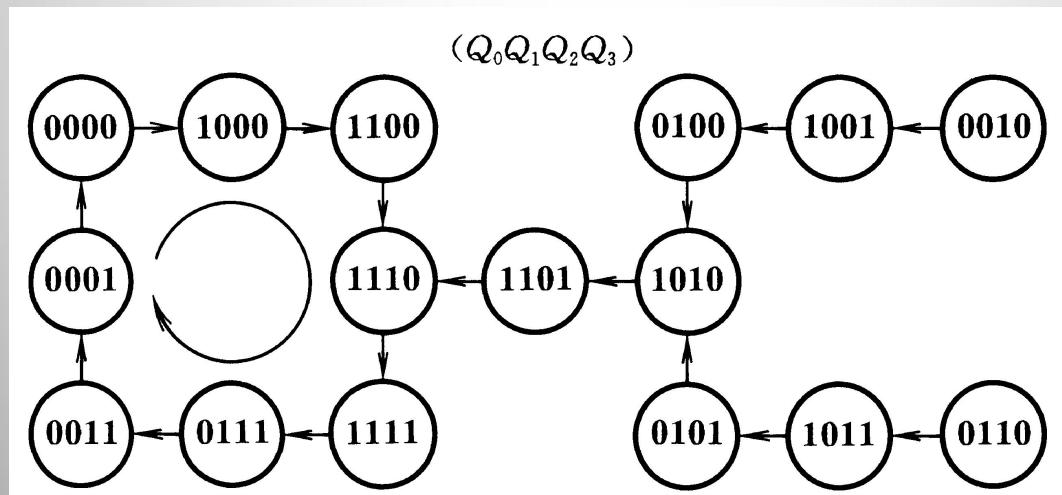
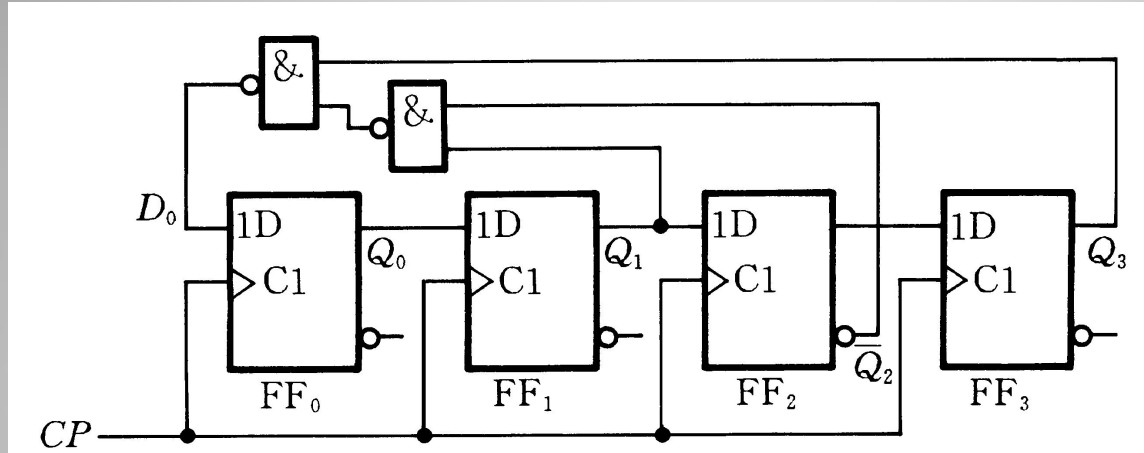
移位寄存器型计数器的一般结构形式



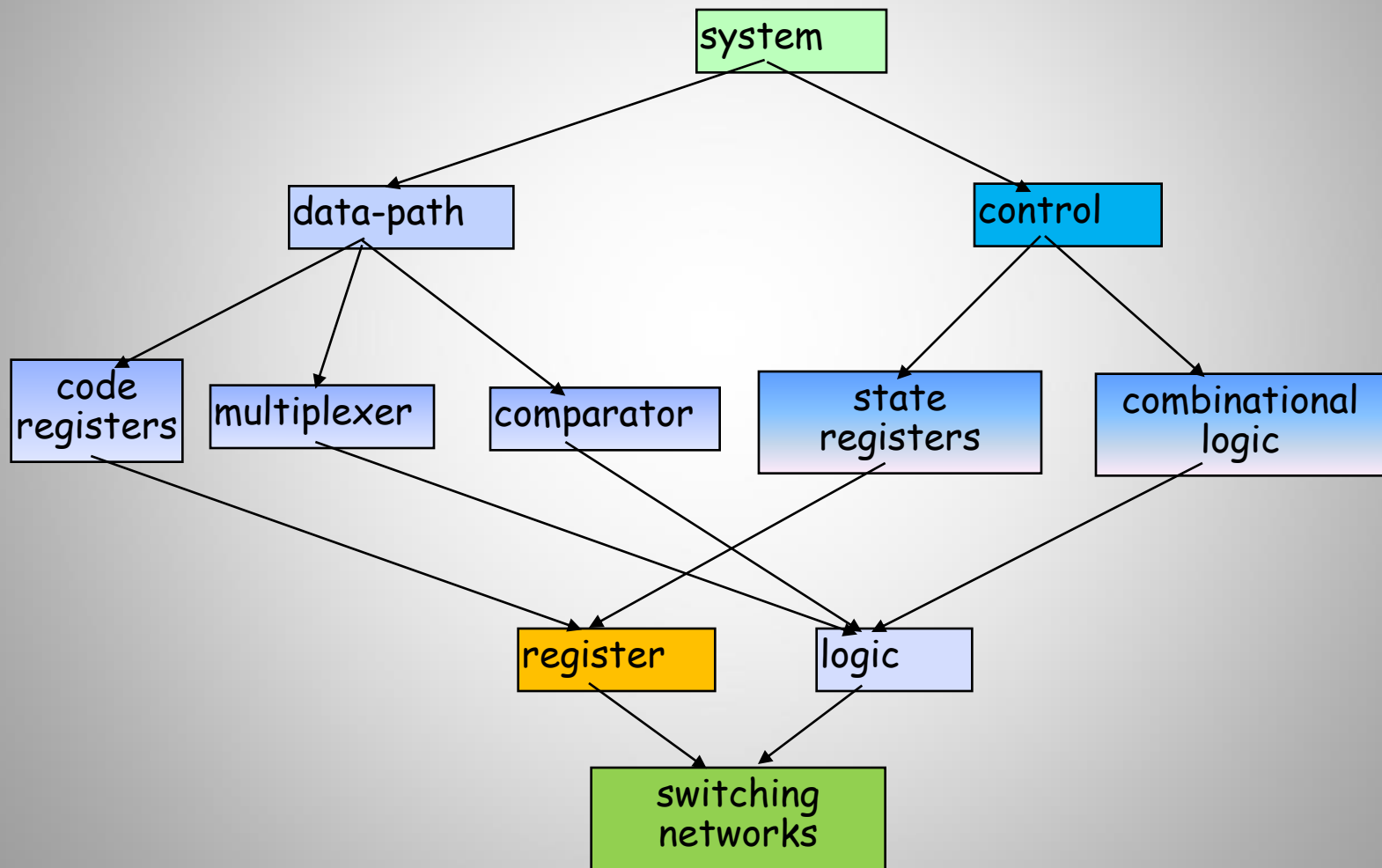
扭环型计数器电路



能自启动的扭环形计数器



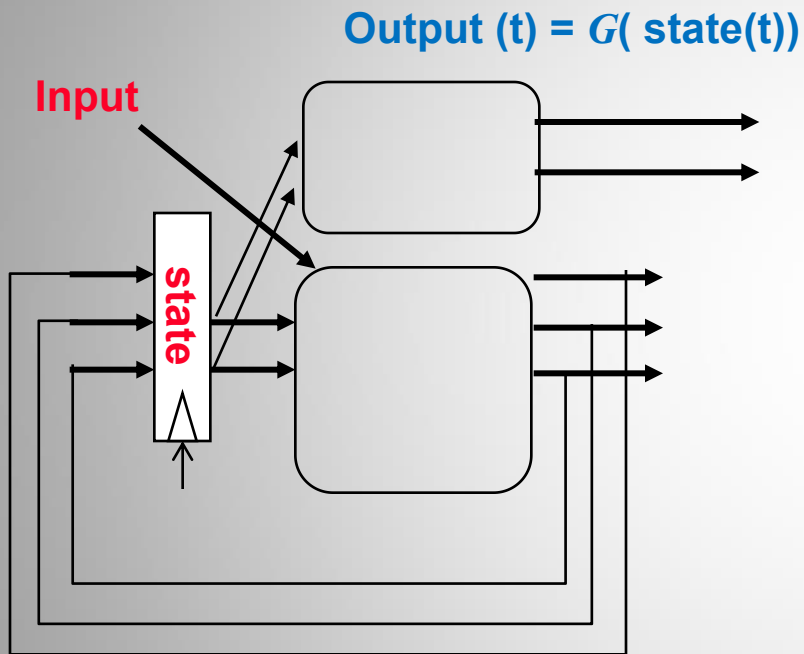
System Design Hierarchy



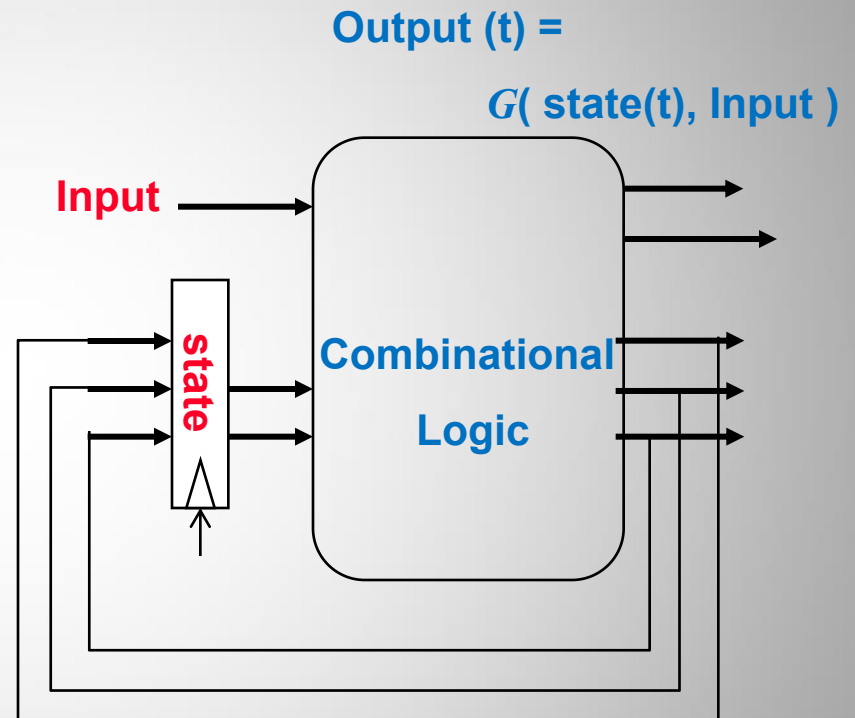
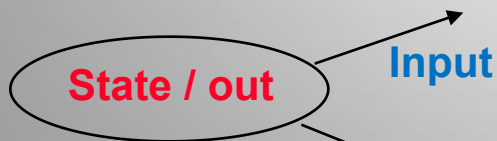
Two Kinds of FSMs

• Moore Machine vs

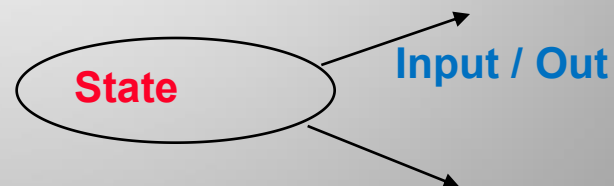
Mealy Machine



$$\text{state}(t+1) = F(\text{state}(t), \text{input}(t))$$



$$\text{state}(t+1) = F(\text{state}(t), \text{input})$$



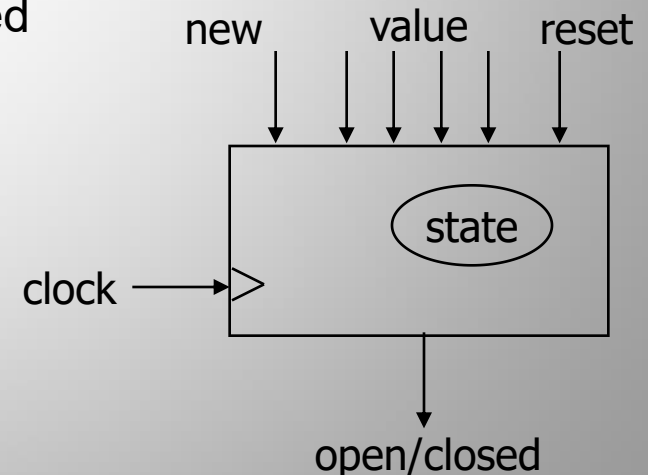
Implementation as a sequential digital system

- **Encoding:**

- how many bits per input value?
- how many values in sequence?
- how do we know a new input value is entered?
- how do we represent the states of the system?

- **Behavior:**

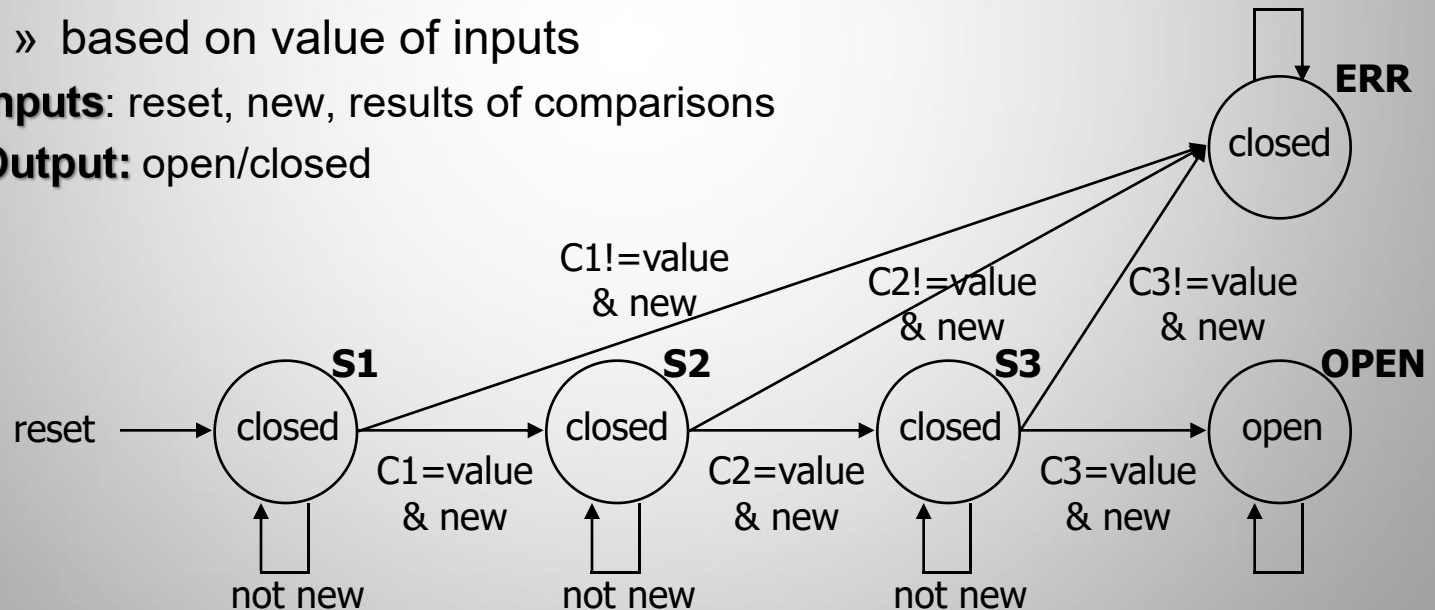
- clock wire tells us when it's ok to look at inputs (i.e., they have settled after change)
- sequential: sequence of values must be entered
- sequential: remember if an error occurred
- finite-state specification



Sequential example: abstract control

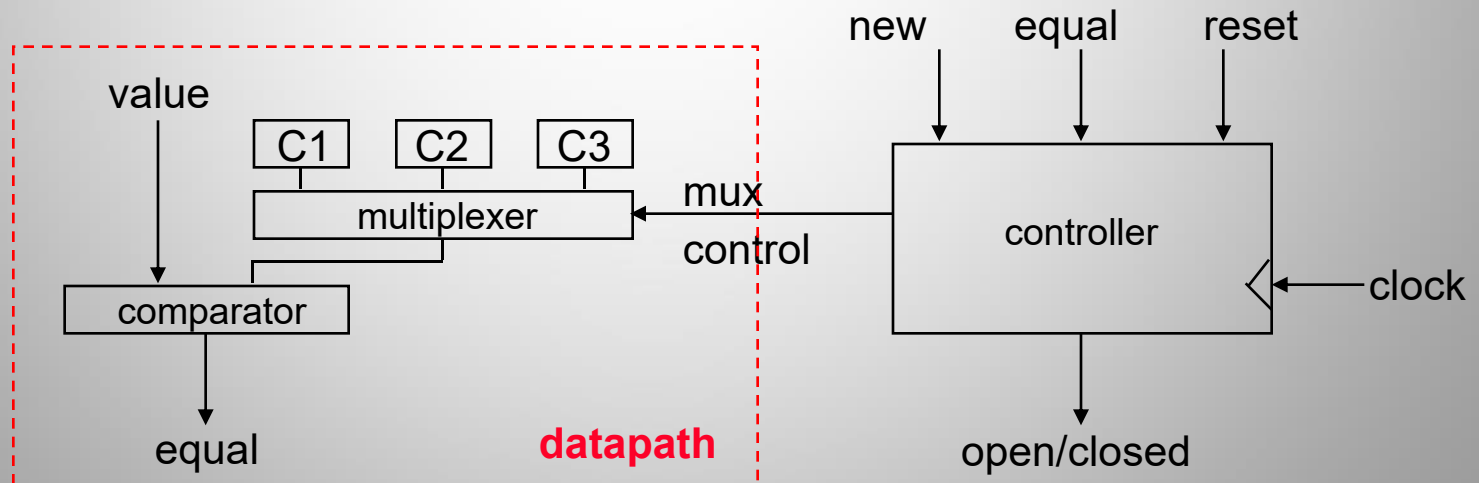
- **Finite-state diagram**

- **States:** 5 states
 - » represent point in execution of machine
 - » each state has outputs
- **Transitions:** 6 from state to state, 5 self transitions, 1 global
 - » changes of state occur when clock says it's ok
 - » based on value of inputs
- **Inputs:** reset, new, results of comparisons
- **Output:** open/closed



data-path vs. control

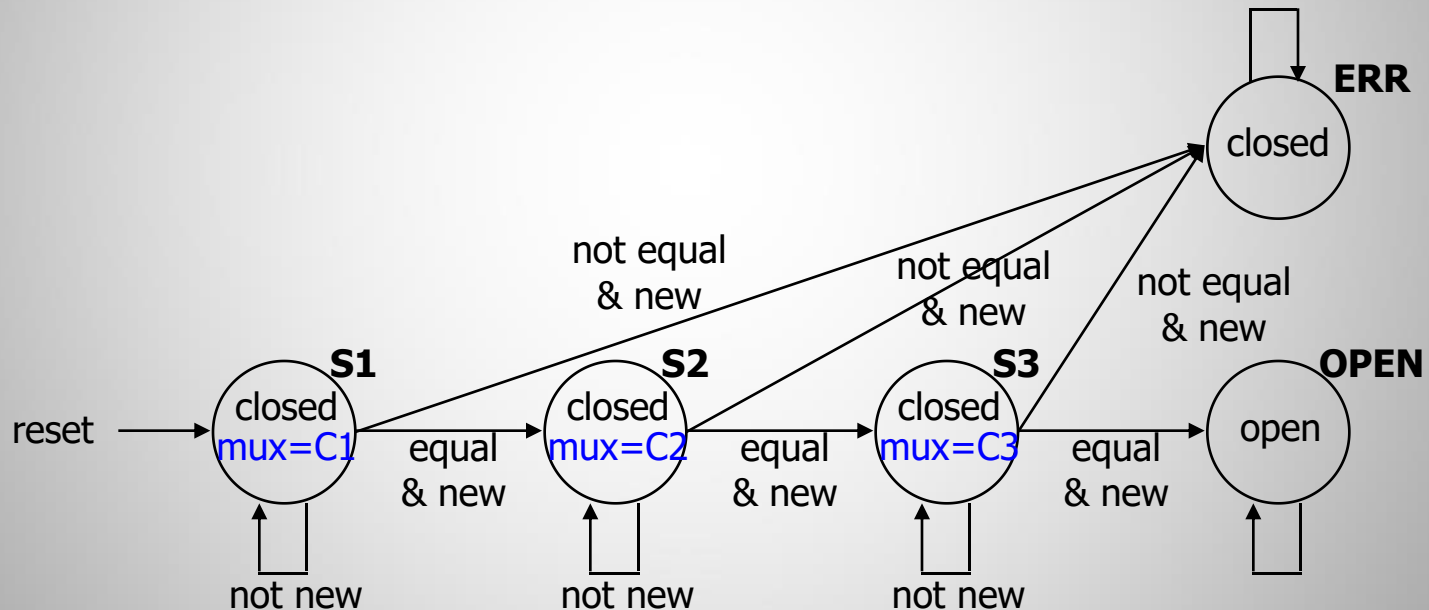
- **Internal structure**
 - data-path
 - » storage for combination
 - » comparators
 - control
 - » finite-state machine controller
 - » control for data-path
 - » state changes controlled by clock



Sequential example :finite-state machine

- **Finite-state machine**

- refine state diagram to include internal structure



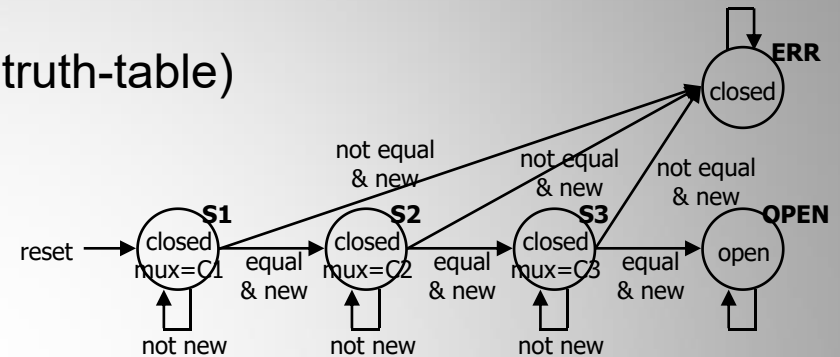
Sequential example: finite-state machine

- Finite-state machine

- generate state table (much like a truth-table)

Symbolic states

reset	new	equal	state	next state	mux	open/closed
1	-	-	-	S1	C1	closed
0	0	-	S1	S1	C1	closed
0	1	0	S1	ERR	-	closed
0	1	1	S1	S2	C2	closed
0	0	-	S2	S2	C2	closed
0	1	0	S2	ERR	-	closed
0	1	1	S2	S3	C3	closed
0	0	-	S3	S3	C3	closed
0	1	0	S3	ERR	-	closed
0	1	1	S3	OPEN	-	open
0	-	-	OPEN	OPEN	-	open
0	-	-	ERR	ERR	-	closed



Encoding?

Sequential example: encoding

- **Encode state table**

- state can be: S1, S2, S3, OPEN, or ERR
 - » needs at least 3 bits to encode: 000, 001, 010, 011, 100
 - » and as many as 5: 00001, 00010, 00100, 01000, 10000
 - » choose 4 bits: 0001, 0010, 0100, 1000, 0000

binary

One-hot

hybrid

- **Encode outputs**

- output mux can be: C1, C2, or C3
 - » needs 2 to 3 bits to encode
 - » choose 3 bits: 001, 010, 100
- output open/closed can be: open or closed
 - » needs 1 or 2 bits to encode
 - » choose 1 bits: 1, 0

Sequential example :encoding

- Encode state table
 - state can be: S1, S2, S3, OPEN, or ERR
 - » choose 4 bits: 0001, 0010, 0100, 1000, 0000
 - output mux can be: C1, C2, or C3
 - » choose 3 bits: 001, 010, 100
 - output open/closed can be: open or closed
 - » choose 1 bits: 1, 0

reset	new	equal	state	next state	mux	open/closed
1	-	-	-	0001	001	0
0	0	-	0001	0001	001	0
0	1	0	0001	0000	-	0
0	1	1	0001	0010	010	0
0	0	-	0010	0010	010	0
0	1	0	0010	0000	-	0
0	1	1	0010	0100	100	0
0	0	-	0100	0100	100	0
0	1	0	0100	0000	-	0
0	1	1	0100	1000	-	1
0	-	-	1000	1000	-	1
0	-	-	0000	0000	-	0

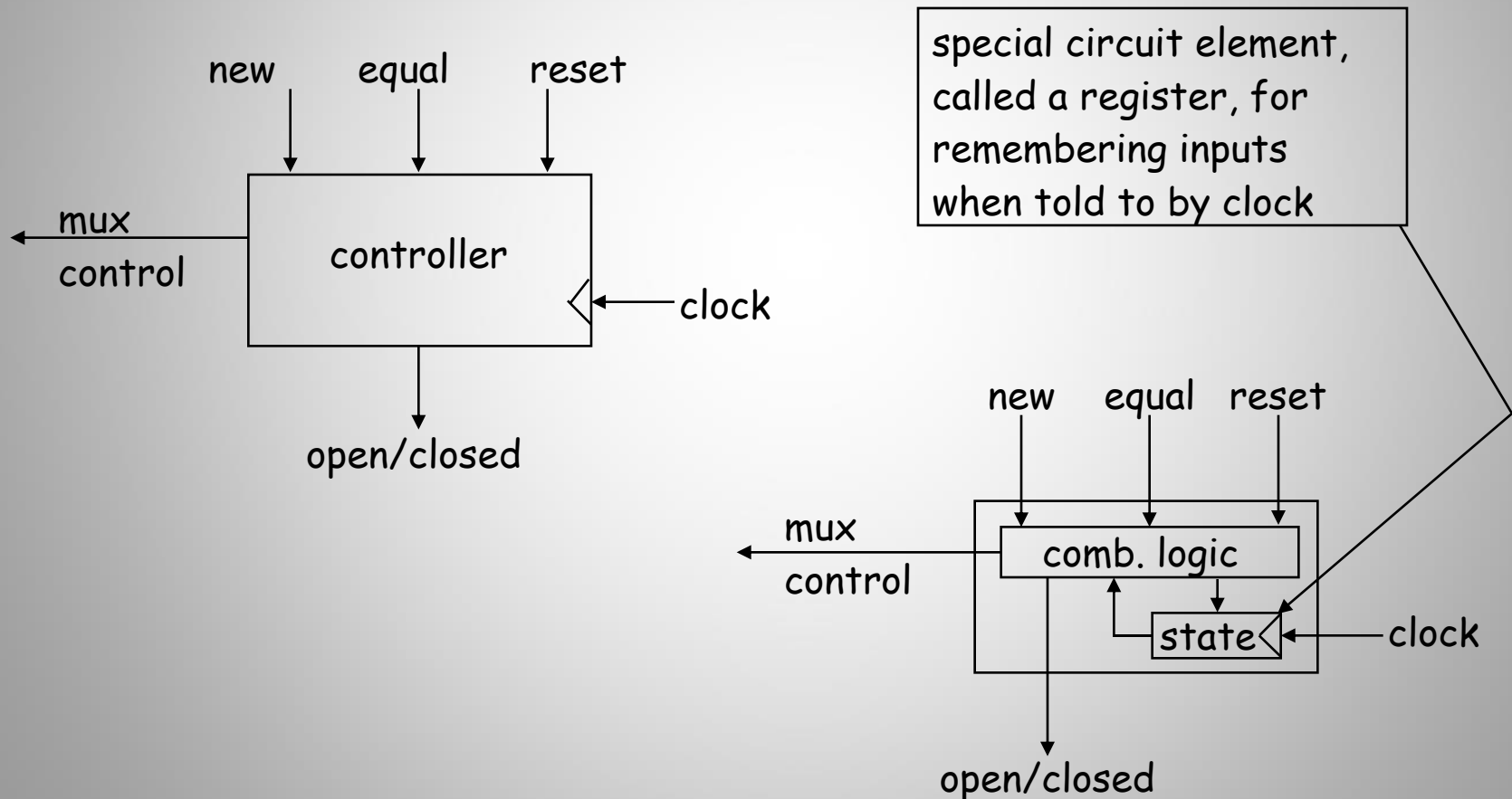
good choice of encoding!

mux is identical to last 3 bits of next state

open/closed is identical to first bit of state

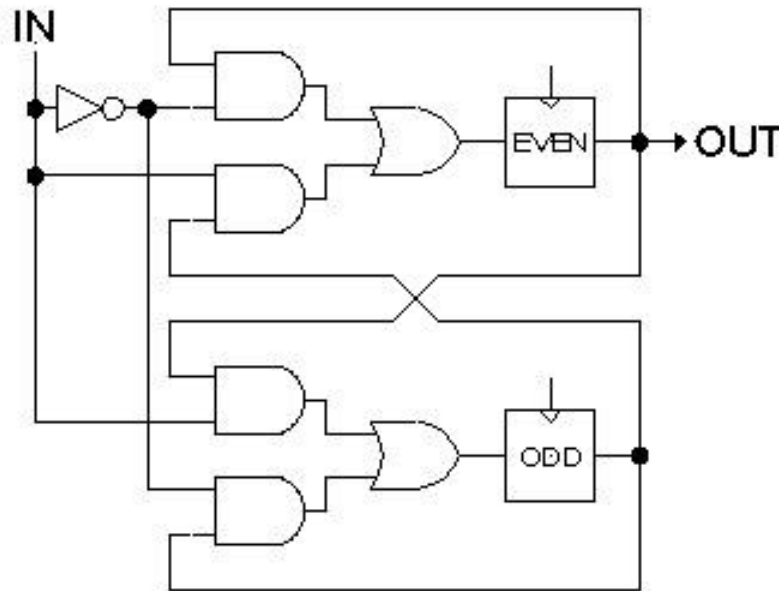
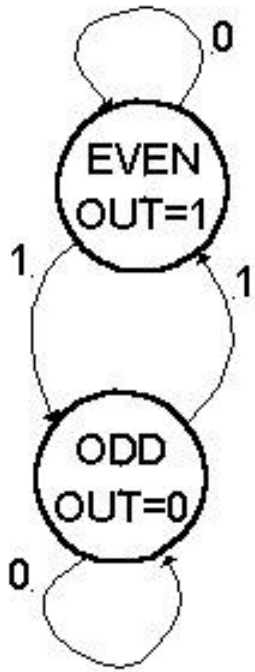
Sequential example:controller implementation

- Implementation of the controller

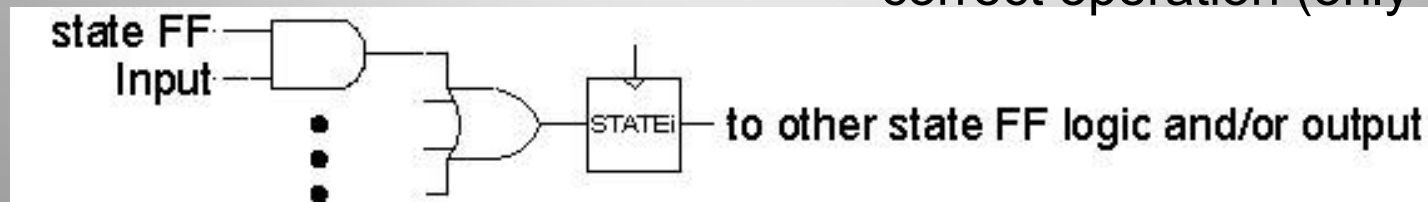


One-hot encoded FSM

- **Even Parity Checker Circuit:**

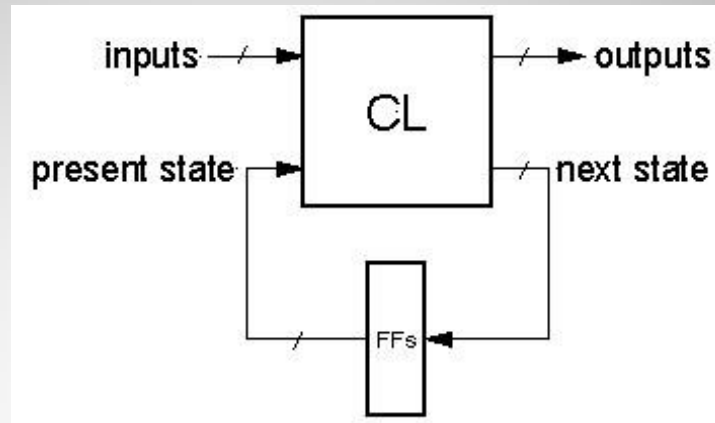


- FFs must be initialized for correct operation (only one 1)

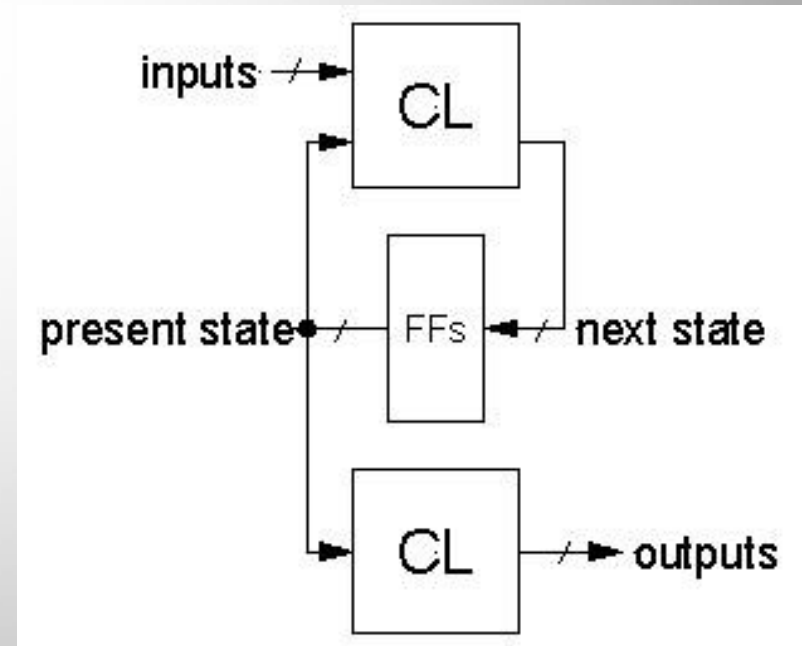


FSM Implementation Notes

- **General FSM form:**

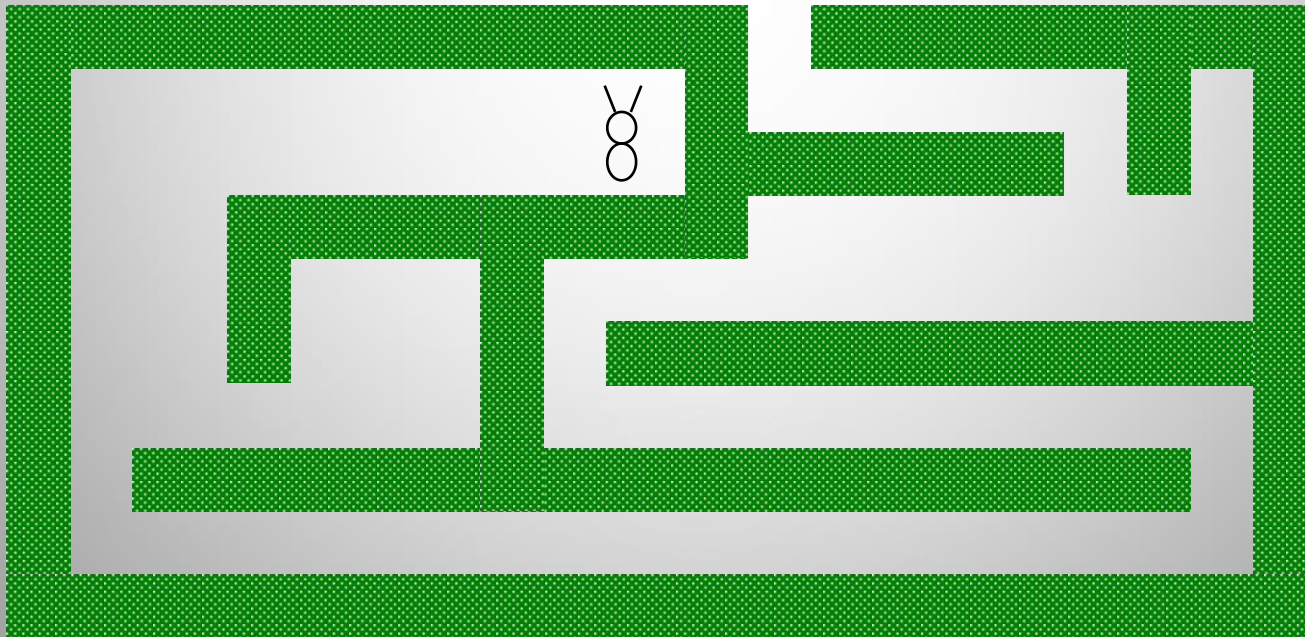


- All examples so far generate output based only on the present state:
- Commonly called **Moore Machine**
(If output functions include both present state and input then called a *Mealy Machine*)

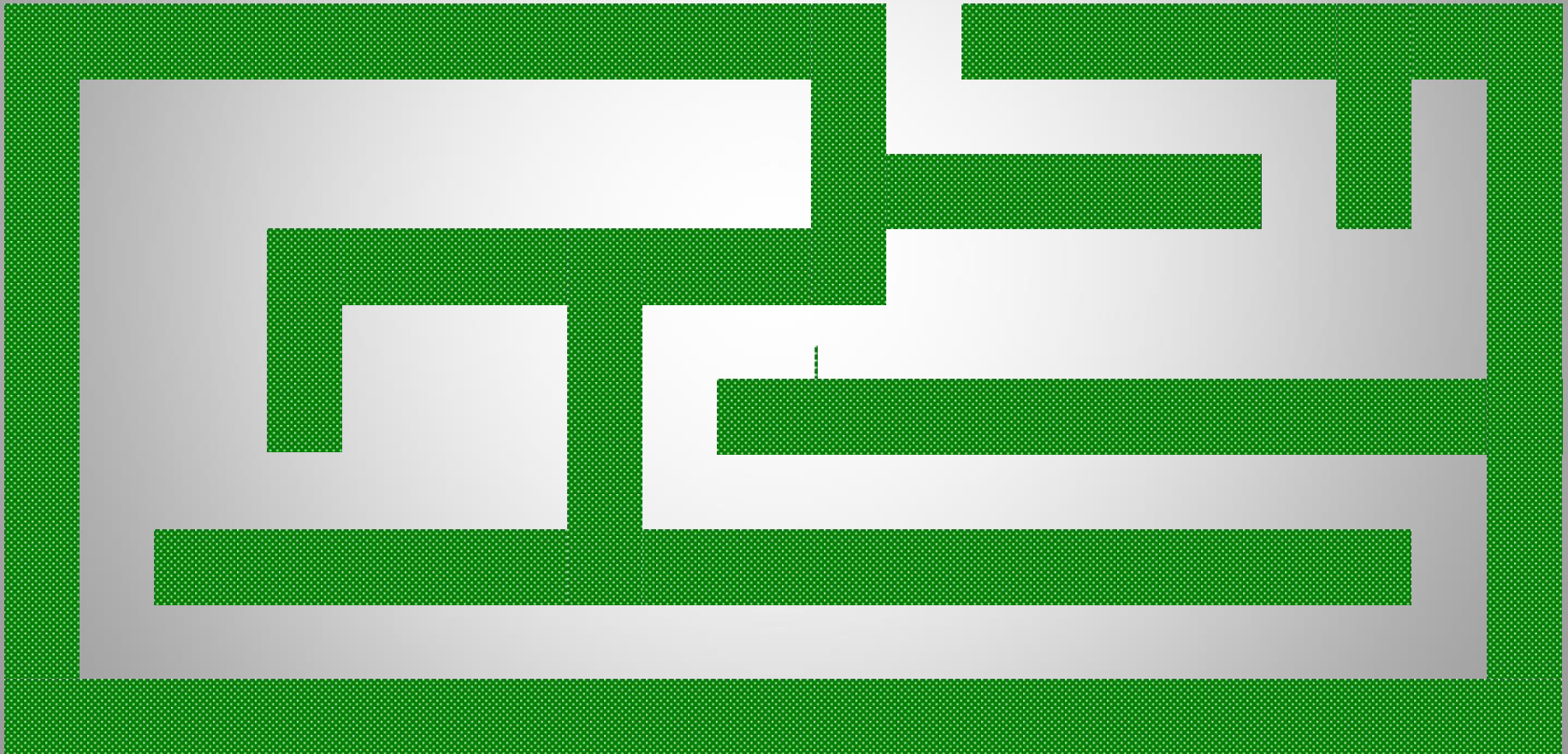


Example: Ant Brain (自学)

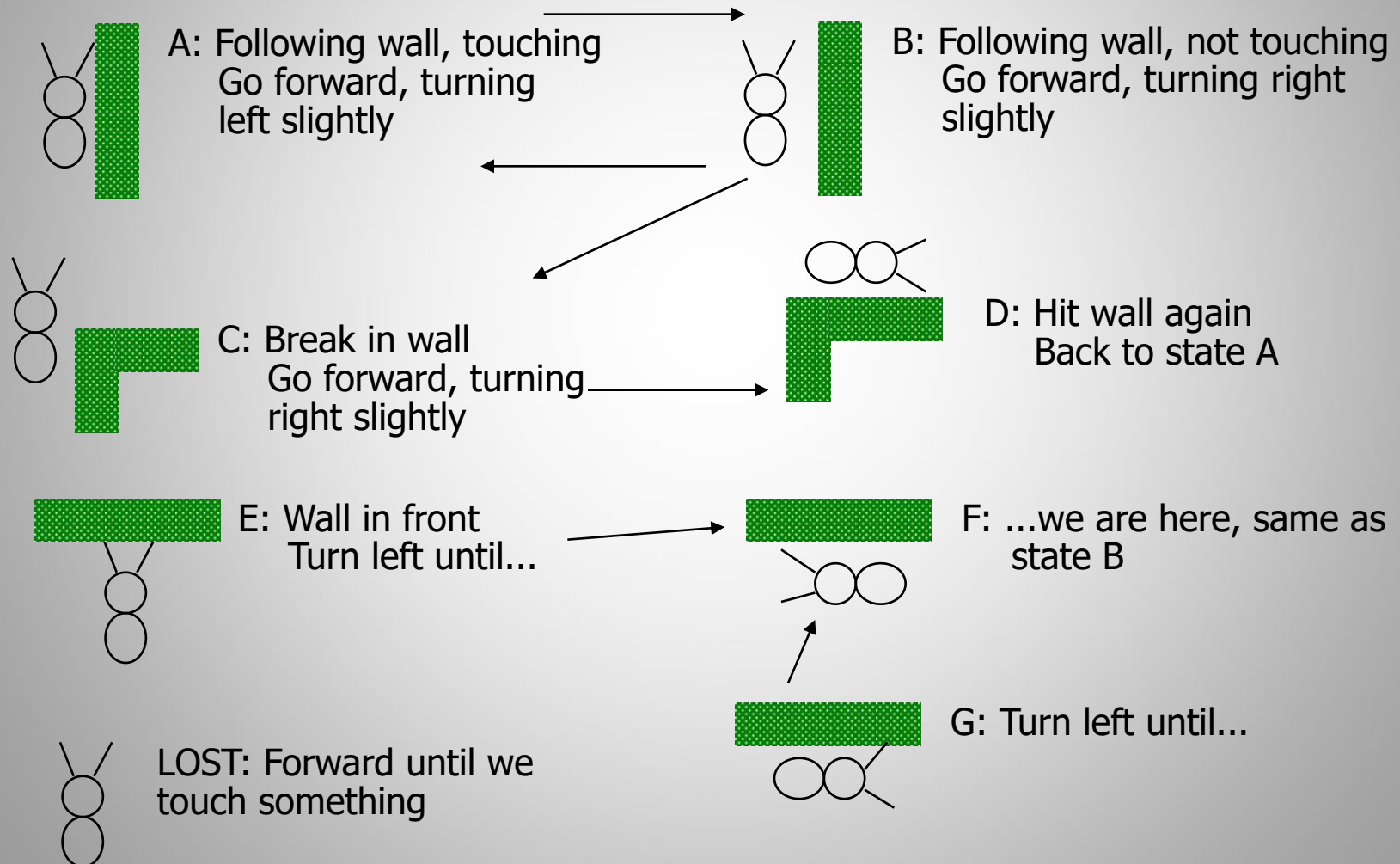
- Sensors: L and R antennae, 1 if in touching wall
- Actuators: F - forward step, TL/TR - turn left/right slightly
- Goal: find way out of maze
- Strategy: keep the wall on the right



Ant Brain

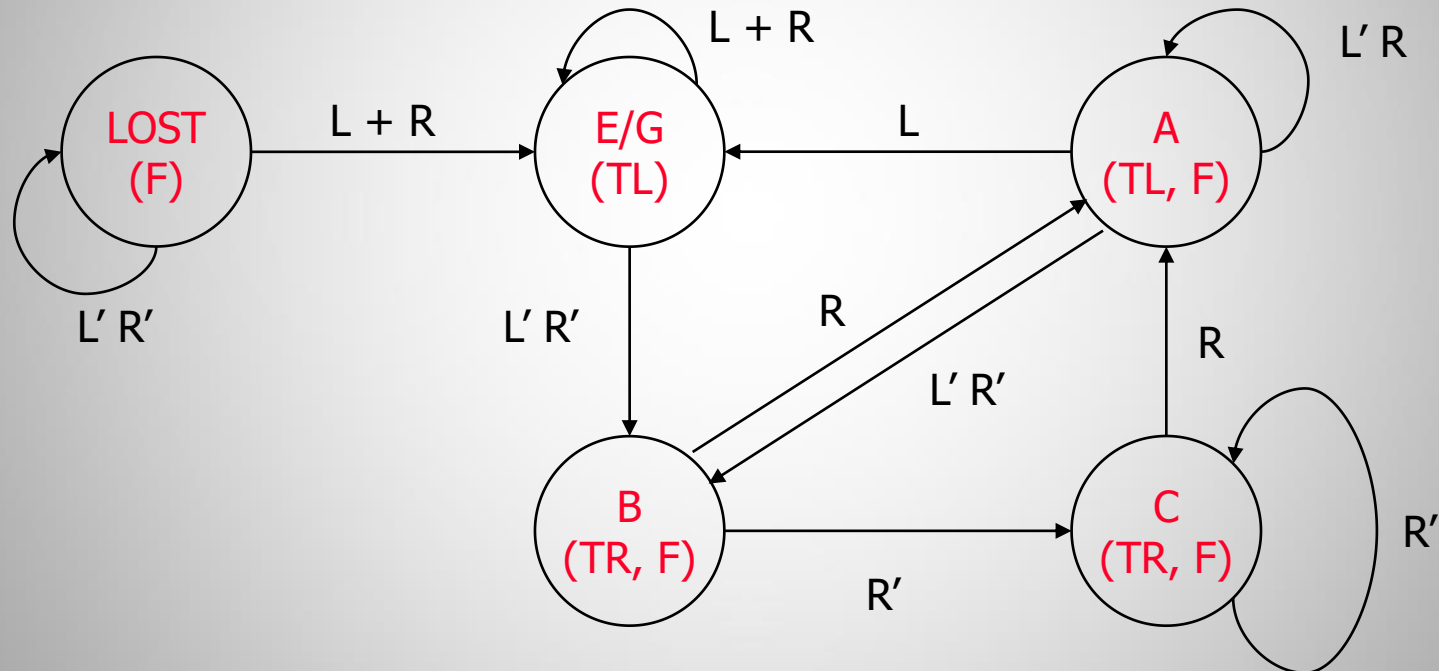


Ant Behavior



Designing an Ant Brain

- State Diagram

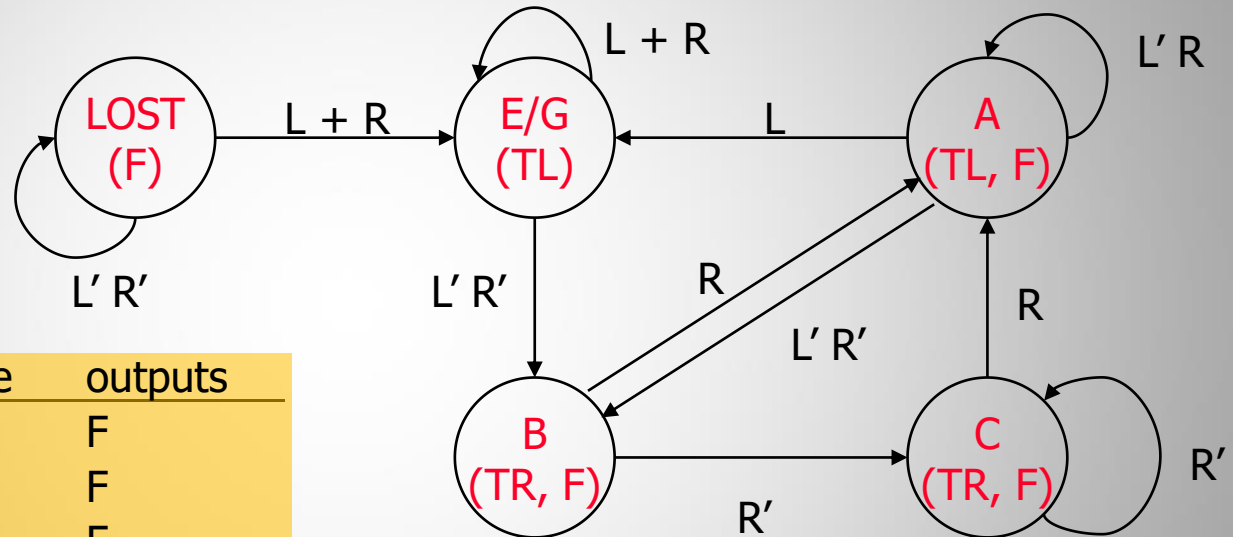


Synthesizing the Ant Brain Circuit

- **Encode States Using a Set of State Variables**
 - Arbitrary choice - may affect cost, speed
- **Use Transition Truth Table**
 - Define next state function for each state variable
 - Define output function for each output
- **Implement next state and output functions using combinational logic**
 - 2-level logic (ROM/PLA/PAL)
 - Multi-level logic
 - Next state and output functions can be optimized together

Transition Truth Table

- Using symbolic states and outputs



state	L	R	next state	outputs
LOST	0	0	LOST	F
LOST	–	1	E/G	F
LOST	1	–	E/G	F
A	0	0	B	TL, F
A	0	1	A	TL, F
A	1	–	E/G	TL, F
B	–	0	C	TR, F
B	–	1	A	TR, F
...	...	...	...	...

Synthesis

- 5 states : at least 3 state variables required (X, Y, Z)
 - State assignment (in this case, arbitrarily chosen)

LOST - 000
 E/G - 001
 A - 010
 B - 011
 C - 100

state X,Y,Z	L	R	next state X', Y', Z'	outputs F TR TL		
0 0 0	0	0	0 0 0	1	0	0
0 0 0	0	1	0 0 1	1	0	0
...	...	...	...	...		
0 1 0	0	0	0 1 1	1	0	1
0 1 0	0	1	0 1 0	1	0	1
0 1 0	1	0	0 0 1	1	0	1
0 1 0	1	1	0 0 1	1	0	1
0 1 1	0	0	1 0 0	1	1	0
0 1 1	0	1	0 1 0	1	1	0
...	...	...	...	...		

it now remains
 to synthesize
 these 6 functions

Synthesis of Next State and Output Functions

state X,Y,Z	inputs L R	next state X ⁺ ,Y ⁺ ,Z ⁺	outputs F TR TL
0 0 0	0 0	0 0 0	1 0 0
0 0 0	- 1	0 0 1	1 0 0
0 0 0	1 -	0 0 1	1 0 0
0 0 1	0 0	0 1 1	0 0 1
0 0 1	- 1	0 1 0	0 0 1
0 0 1	1 -	0 1 0	0 0 1
0 1 0	0 0	0 1 1	1 0 1
0 1 0	0 1	0 1 0	1 0 1
0 1 0	1 -	0 0 1	1 0 1
0 1 1	- 0	1 0 0	1 1 0
0 1 1	- 1	0 1 0	1 1 0
1 0 0	- 0	1 0 0	1 1 0
1 0 0	- 1	0 1 0	1 1 0

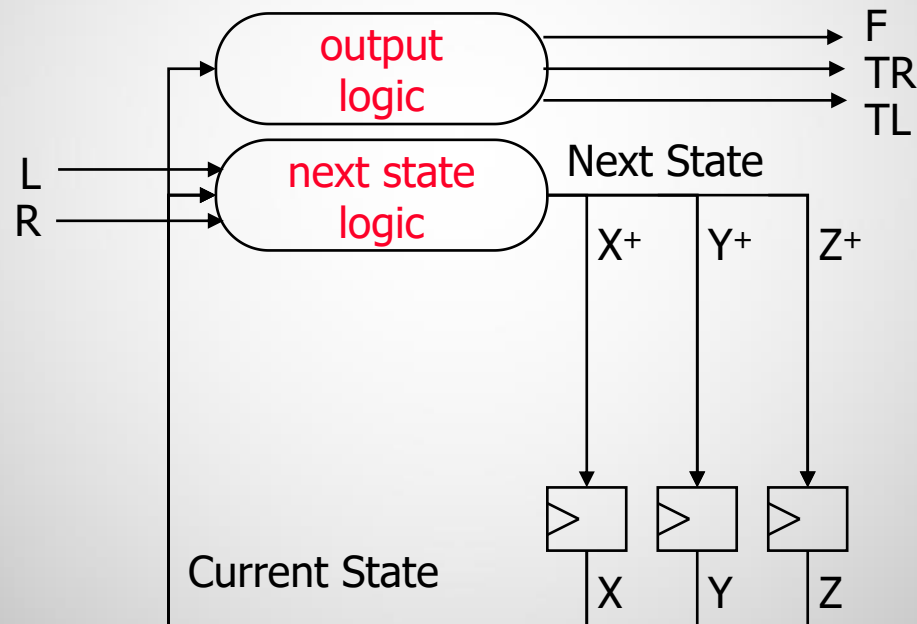
e.g.

$$TR = X + Y Z$$

$$X^+ = X R' + Y Z R' = R' TR$$

Circuit Implementation

- **Outputs are a function of the current state only - Moore machine**



Verilog Sketch

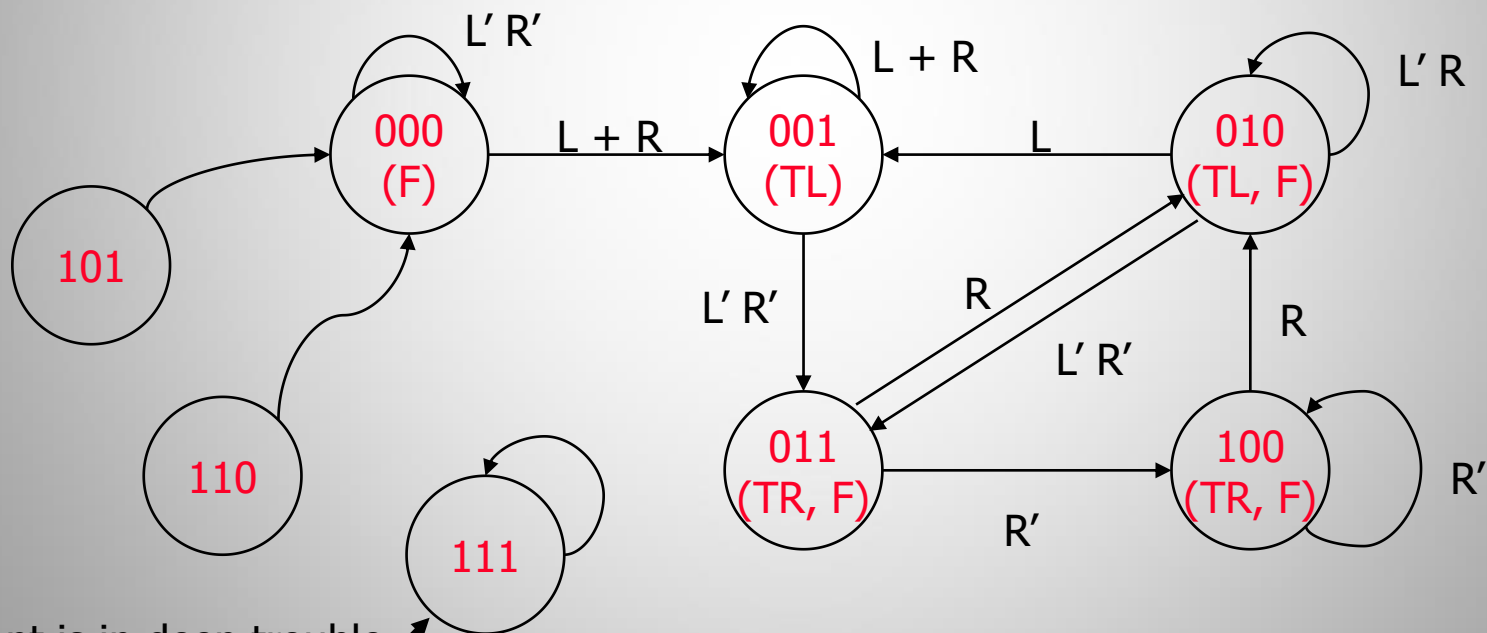
```
module ant_brain (F, TR, TL, L, R)
  inputs      L, R;
  outputs     F, TR, TL;
  reg        X, Y, Z;

  assign F = function(X, Y, Z, L, R);
  assign TR = function(X, Y, Z, L, R);
  assign TL = function(X, Y, Z, L, R);

  always @(posedge clk)
    begin
      X <= function (X, Y, Z, L, R);
      Y <= function (X, Y, Z, L, R);
      Z <= function (X, Y, Z, L, R);
    end
endmodule
```

Don't Cares in FSM Synthesis

- What happens to the "unused" states (101, 110, 111)?
- Exploited as don't cares to minimize the logic
 - If states can't happen, then don't care what the functions do
 - if states do happen, we may be in trouble



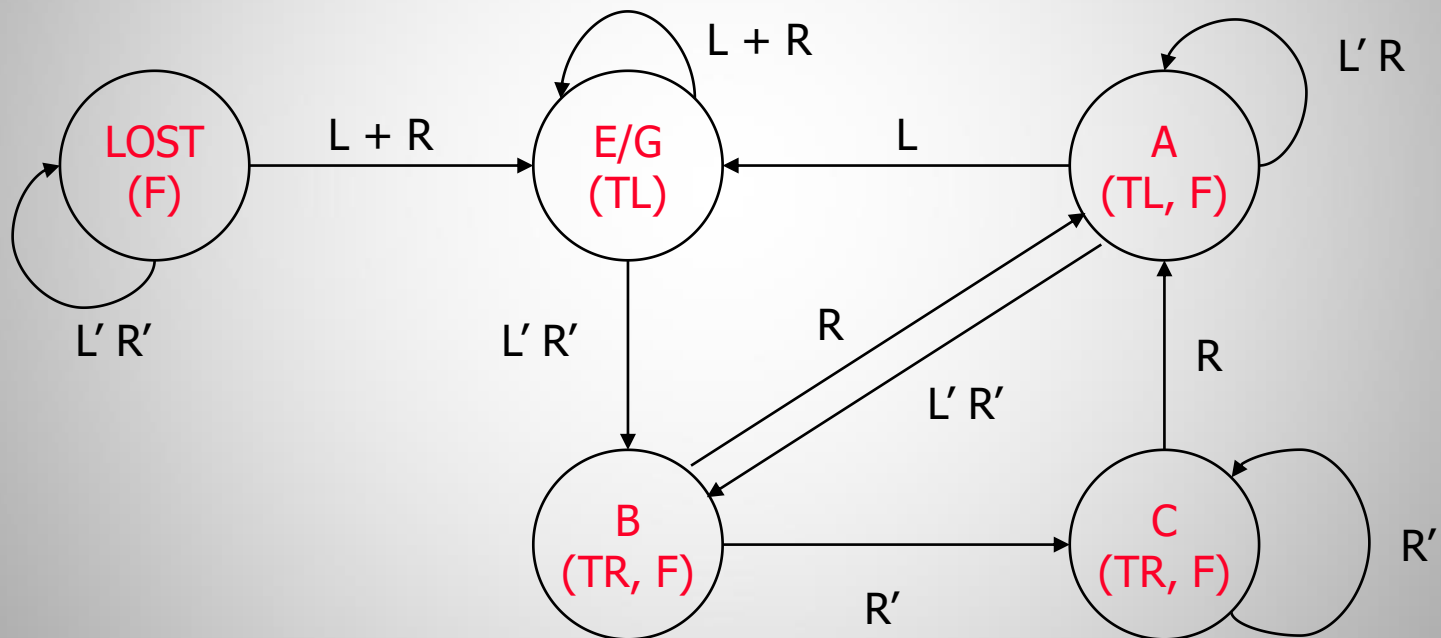
Ant is in deep trouble
if it gets in this state

State Minimization

- **Fewer states may mean fewer state variables**
- High-level synthesis may generate many redundant states
- Two states are equivalent if they are impossible to distinguish from the outputs of the FSM, i. e., for any input sequence the outputs are the same
- **Two conditions for two states to be equivalent:**
 - 1) Output must be the same in both states
 - 2) Must transition to equivalent states for all input combinations

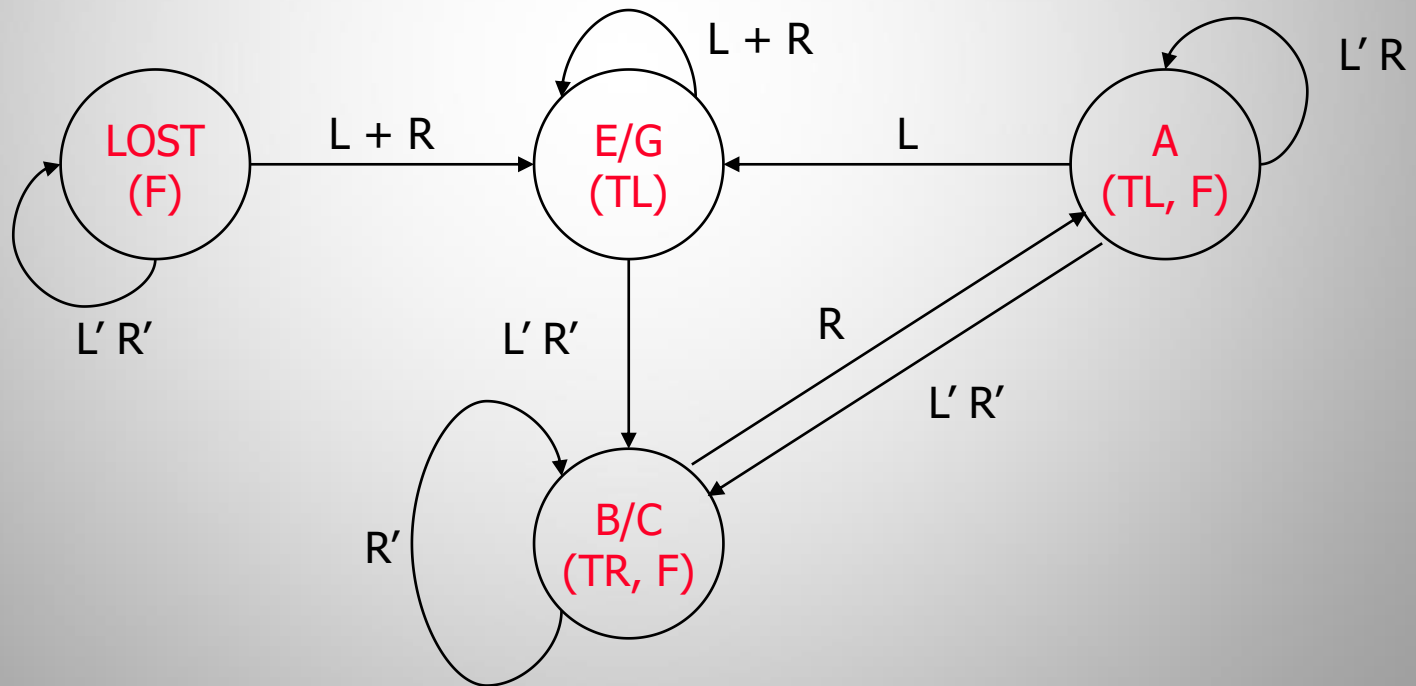
Ant Brain Revisited

- Any equivalent states?



New Improved Brain

- Merge equivalent B and C states
- Behavior is exactly the same as the 5-state brain
- We now need only 2 state variables rather than 3



New Brain Implementation

state X,Y	inputs L R	next state X',Y'	outputs F TR TL
00	0 0	00	1 0 0
00	- 1	01	1 0 0
00	1 -	01	1 0 0
01	0 0	11	0 0 1
01	- 1	01	0 0 1
01	1 -	01	0 0 1
10	0 0	11	1 0 1
10	0 1	10	1 0 1
10	1 -	01	1 0 1
11	- 0	11	1 1 0
11	- 1	10	1 1 0

X+

X			
0	1	1	1
0	0	1	1
0	0	1	0
0	0	1	0

L | R

Y

Y+

X			
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	1

L | R

Y

F

X			
1	0	1	1
1	0	1	1
1	0	1	1
1	0	1	1

L | R

Y

TR

X			
0	0	1	0
0	0	1	0
0	0	1	0
0	0	1	0

L | R

Y

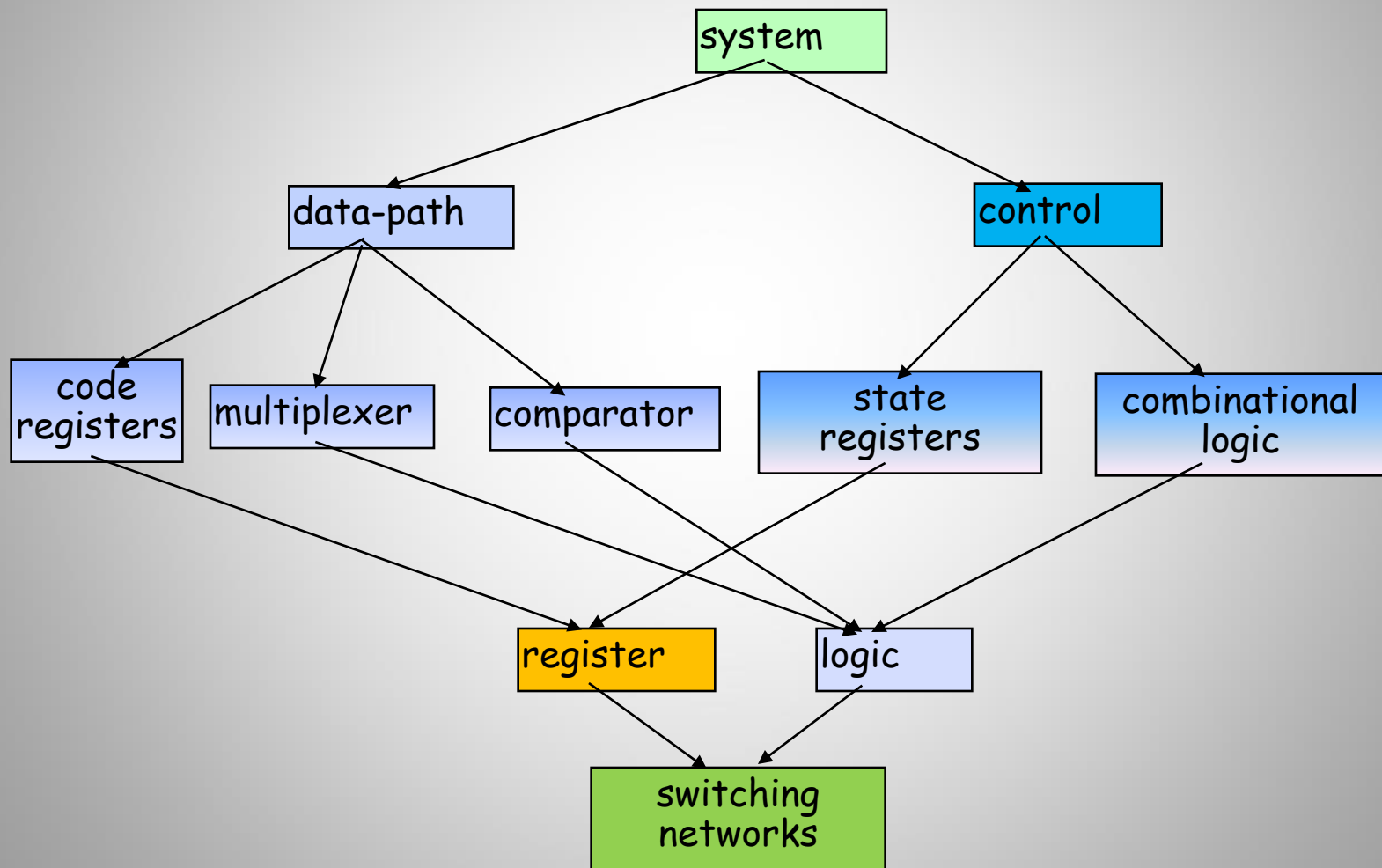
TL

X			
0	1	0	1
0	1	0	1
0	1	0	1
0	1	0	1

L | R

Y

System Design Hierarchy



Sequential Logic Implementation Summary

- **Models for representing sequential circuits**
 - Abstraction of sequential elements
 - Finite state machines and their state diagrams
 - Inputs/outputs
 - Mealy, Moore, and synchronous Mealy machines
- **Finite state machine design procedure**
 - Deriving state diagram
 - Deriving state transition table
 - Determining next state and output functions
 - Implementing combinational logic

6.4 时序逻辑电路的设计方法

6.4.1 同步时序逻辑电路的设计方法

设计的一般步骤

一、逻辑抽象，求出状态转换图或状态转换表

1. 确定输入/输出变量、电路状态数。
2. 定义输入/输出逻辑状态以及每个电路状态的含意，并对电路状态进行编号。
3. 按设计要求列出状态转换表，或画出状态转换图。

二、状态化简

若两个状态在相同的输入下有相同的输出，并转换到同一个次态，则称为等价状态；等价状态可以合并。

三、状态分配（编码）

1. 确定触发器数目。
2. 给每个状态规定一个代码。

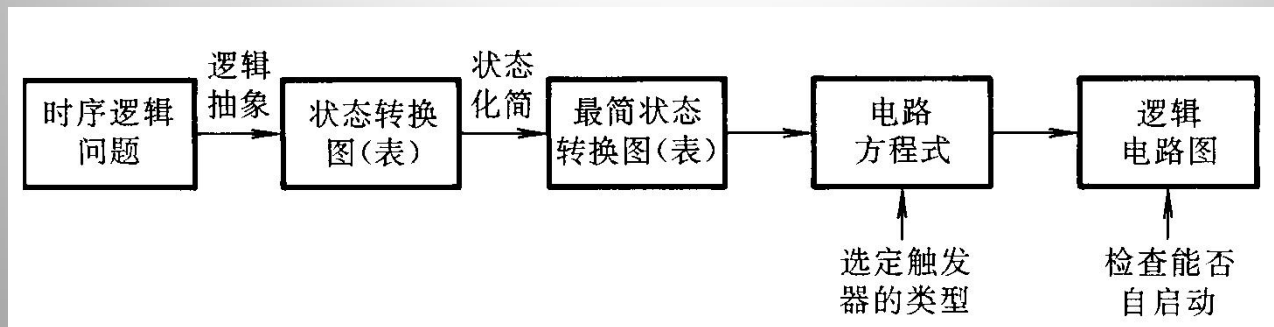
（通常编码的取法、排列顺序都依照一定的规律）

四、选定触发器类型

求出状态方程，驱动方程，输出方程。

五、画出逻辑图

六、检查自启动



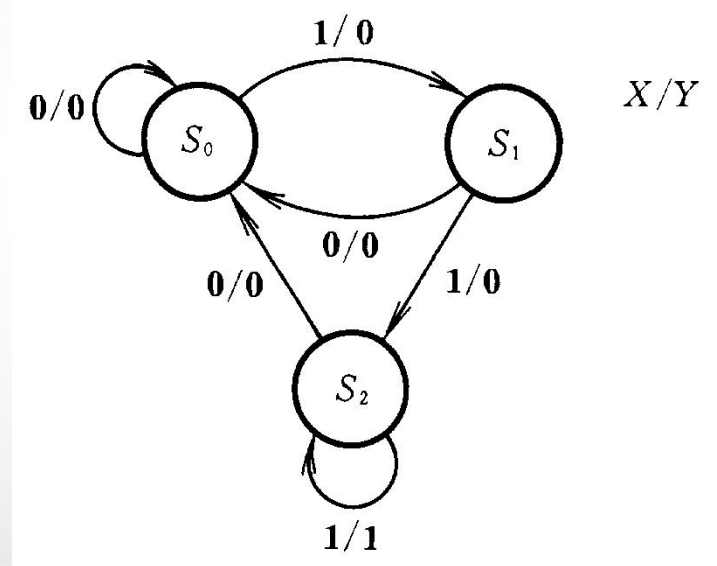
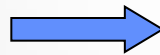
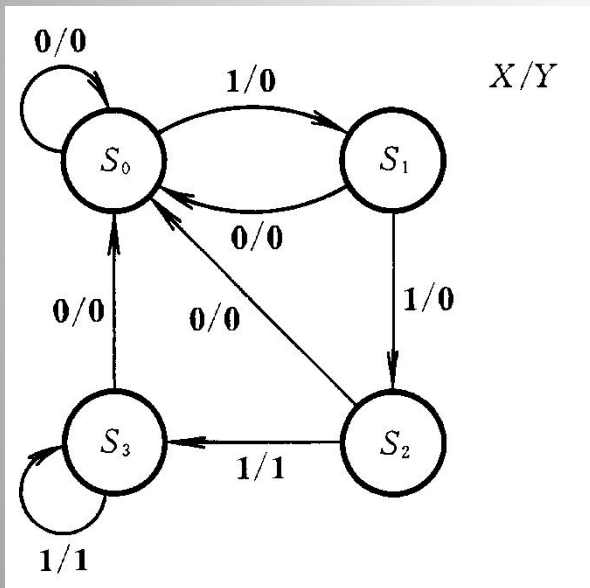
例1： 设计一个串行数据检测器，要求在连续输入三个或三个以上“1”时输出为1，其余情况下输出为0。

一、抽象、画出状态转换图

二、状态化简

用X（1位）表示输入数据

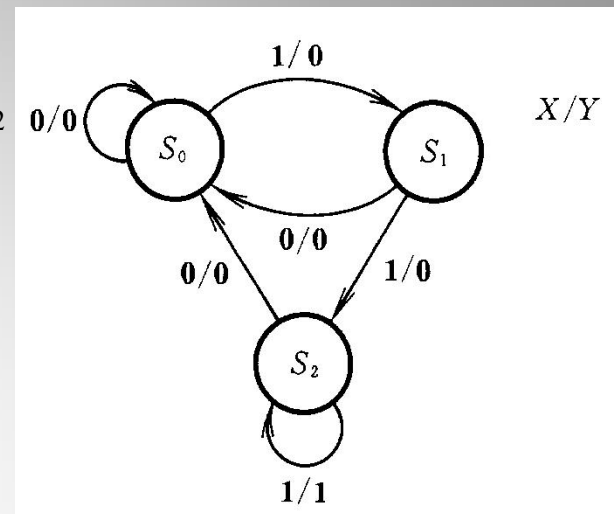
用Y（1位）表示输出（检测结果）



三、状态分配

取 $n=2$ ，令 q_1q_0 的00、01、10为 s_0 、 s_1 、 s_2 则，

q_1q_0		X			
		00	01	11	10
X	0	00/0	00/0	××/×	00/0
	1	01/0	10/0	××/×	10/1



q_1q_0		X			
		00	01	11	10
X	0	0	0	×	0
	1	0	1	×	1

(a) Q_1^*

q_1q_0		X			
		00	01	11	10
X	0	0	0	×	0
	1	1	0	×	0

(b) Q_0^*

q_1q_0		X			
		00	01	11	10
X	0	0	0	×	0
	1	0	0	×	1

(c) Y

$$Q_1^* = XQ_1 + XQ_0 \quad Q_0^* = XQ_1'Q_0'$$

$$Y = XQ_1$$

四、选用JK触发器，求方程组

$$Q_1^* = XQ_1 + XQ_0$$

$$\mathbf{Q}_0^* = \mathbf{X}\mathbf{Q}_1'\mathbf{Q}_0'$$

$$Y = XQ_1$$

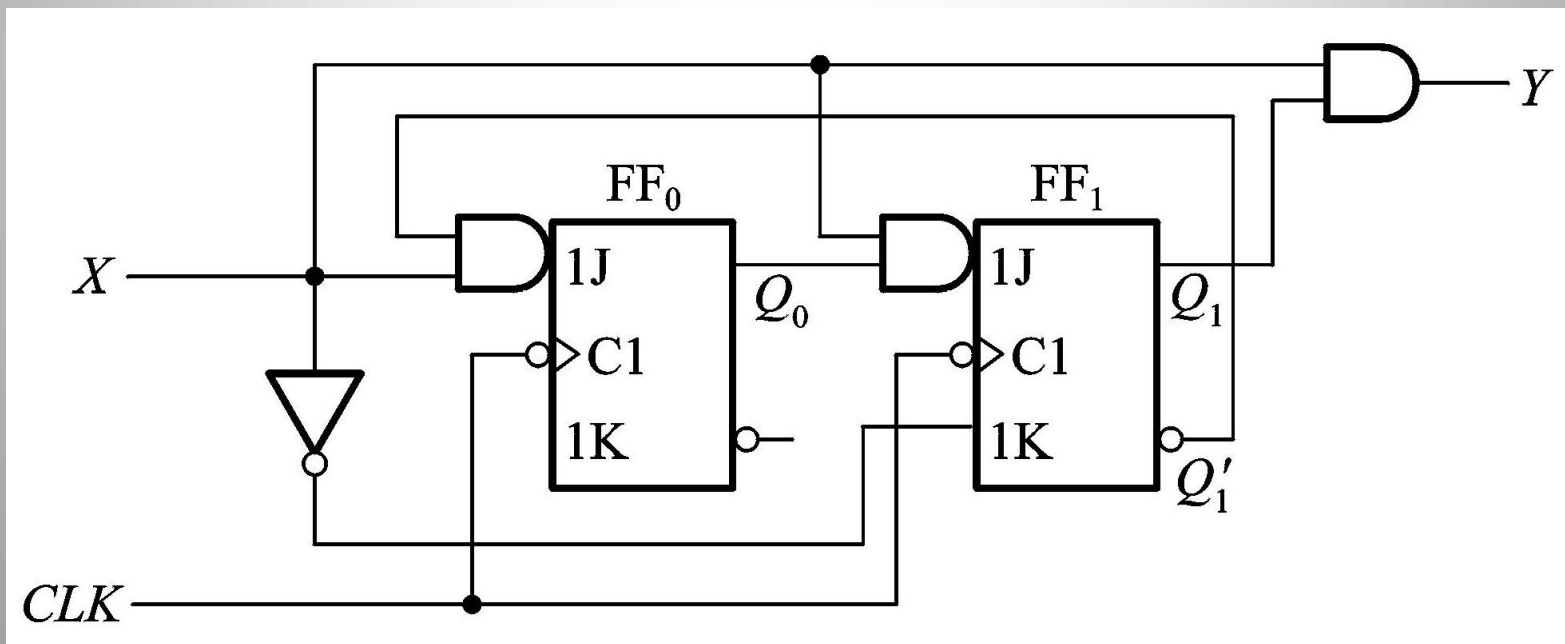
$$\mathbf{Q}_1^* = X\mathbf{Q}_1 + X\mathbf{Q}_0(\mathbf{Q}_1 + \mathbf{Q}_1')$$

$$\mathcal{Q}_0^* = X\mathcal{Q}'_1\mathcal{Q}'_0$$

$$= (XQ_0)Q_1' + (X')'Q_1$$

$$= (XQ'_1)Q'_0 + 1'Q_0$$

五、画逻辑图



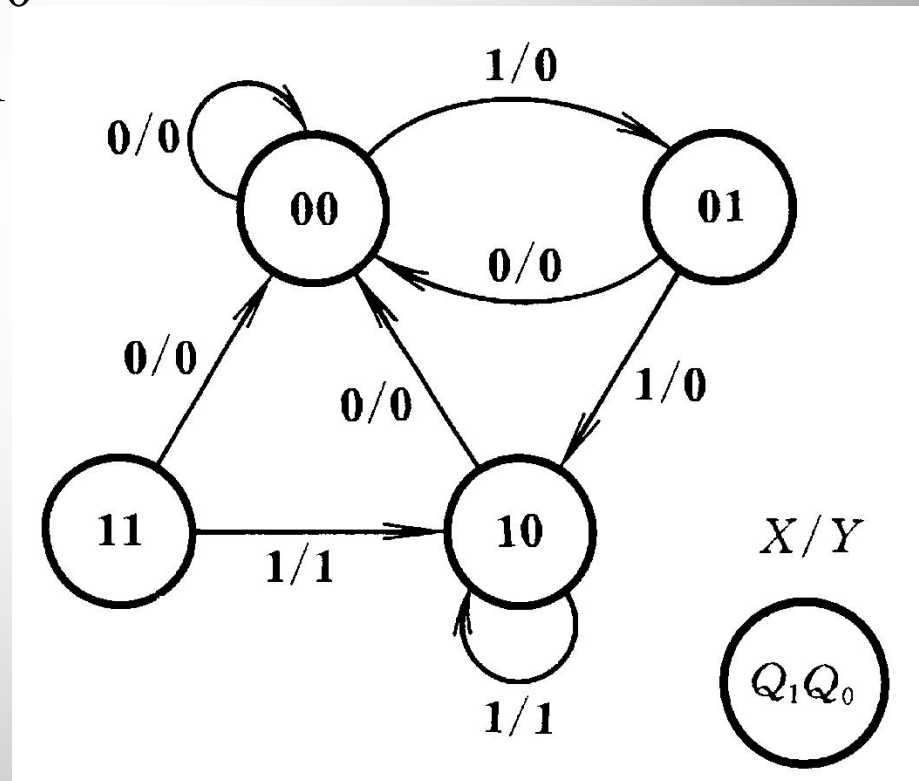
• 六、检查电路能否自启动

- 将状态“11”代入状态方程和输出方程，分别求 $X=0/1$ 下的次态和现态下的输出，得到：

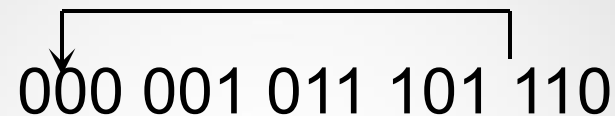
$X = 0$ 时， $Q_1^* Q_0^* = 00$, $Y = 0$

$X = 1$ 时， $Q_1^* Q_0^* = 10$, $Y = 1$

能自启动



【例2】 用JK触发器设计一个五进制同步计数器，要求状态转换关系为：



解： 本例属于给定状态时序电路设计问题。

① 列状态表。

根据题意，该时序电路有三个状态变量，设状态变量为 Q_2 、 Q_1 、 Q_0 ，可作出二进制状态表如表所示。

状态表一

Q_2	Q_1	Q_0	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	0	0	0	1
0	0	1	0	1	1
0	1	0	×	×	×
0	1	1	1	0	1
1	0	0	×	×	×
1	0	1	1	1	0
1	1	0	0	0	0
1	1	1	×	×	×

② 确定激励函数和输出函数。

Q_2Q_1		00	01	11	10
Q_0	0	0	×	0	×
	1	0	1	×	1

(a)

Q_2Q_1		00	01	11	10
Q_0	0	0	×	0	×
	1	1	0	×	1

(b)

Q_2Q_1		00	01	11	10
Q_0	0	1	×	0	×
	1	1	1	×	0

(c)

次态卡诺图

(a) Q_2^{n+1} ;

(b) Q_1^{n+1} ;

(c) Q_0^{n+1}

由次态卡诺图求出其状态方程和激励函数如下：

$$Q_2^{n+1} = Q_1 \bar{Q}_2 + \bar{Q}_1 Q_2, \quad J_2 = Q_1, \quad K_2 = Q_1$$

$$Q_1^{n+1} = Q_0 \bar{Q}_1, \quad J_1 = Q_0, \quad K_1 = 1$$

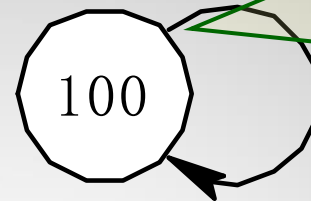
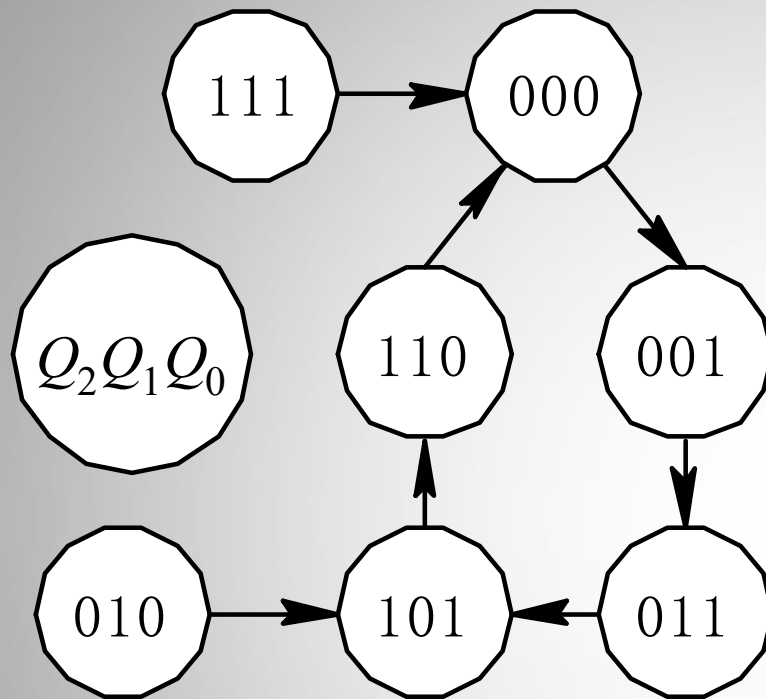
$$Q_0^{n+1} = \bar{Q}_2 \bar{Q}_0 + \bar{Q}_2 Q_0, \quad J_0 = \bar{Q}_2, \quad K_0 = Q_2$$

③ 自启动检查。

根据以上状态方程，检查多余状态的转移情况如表所示，其完整的状态图如下图所示。

Q_2	Q_1	Q_0	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	1	0	1	0	1
1	0	0	1	0	0
1	1	1	0	0	0

多余状态转移表



可以看出，该电路一旦进入状态100，就不能进入计数主循环，因而该电路不能实现自启动，需要修改设计。

在非完全描述时序电路中，由于存在无效状态，使得在激励函数的获取过程中出现了任意项。在求取激励函数时，如果某任意项被圈入，则该任意项被确认为1，否则被确认为0。

由于圈法的随意性，故无效状态的转移可能出现死循环而使电路不能自启动。当电路不能自启动时，解决的方法有多种。

第一种方法，将原来的非完全描述时序电路中没有描述的状态的转移情况加以定义，使其成为完全描述时序电路。

如将表状态表中的无效状态的转移方向均定义为000，则可得到一个完全描述时序电路的状态表如下表所示。

显然，按照下表设计的时序电路，不存在死循环问题，因为它是完全描述的。**这种方法由于失去了任意项，会增加电路的复杂程度。**

状态表二

Q_2	Q_1	Q_0	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	0	0	0	1
0	0	1	0	1	1
0	1	0	0	0	0
0	1	1	1	0	1
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	0	0	0
1	1	1	0	0	0

第二种方法，改变原来的圈法。

如果盲目地改变所有激励函数的圈法，则其工作量大，效果差。

若在分析观察的基础上改变某激励函数的圈法，则能获得较满意的效果。

观察原来的次态卡诺图，如果希望能尽量使用任意项，只能对(a)和(c)的圈法作修改。

现对(c)的圈法作修改，它仅改变 Q_0 的转移，新的圈法见下页。

Q_2Q_1		00	01	11	10
Q_0	0	0	×	0	×
	1	0	1	×	1

(a)

Q_2Q_1		00	01	11	10
Q_0	0	0	×	0	×
	1	1	0	×	1

(b)

Q_2Q_1		00	01	11	10
Q_0	0	1	×	0	×
	1	1	1	×	0

(c)

由新圈法得

$$Q_0^{n+1} = \overline{Q_1}\overline{Q_0} + \overline{Q_2}Q_0$$

$$J_0 = \overline{Q_1}, K_0 = Q_2$$

Q_2Q_1		00	01	11	10
Q_0	0	1	×	0	×
	1	1	1	×	0

(c) Q_0^{n+1}

分析新圈法可知：状态010将转移到100（原转移到101；

现在最后一位 Q_0 转为0），状态100将转移到101（原转移到100，现最后一位 Q_0 转为1）。

由分析可以看出，新圈法将克服死循环，也不增加激励函数的复杂程度。

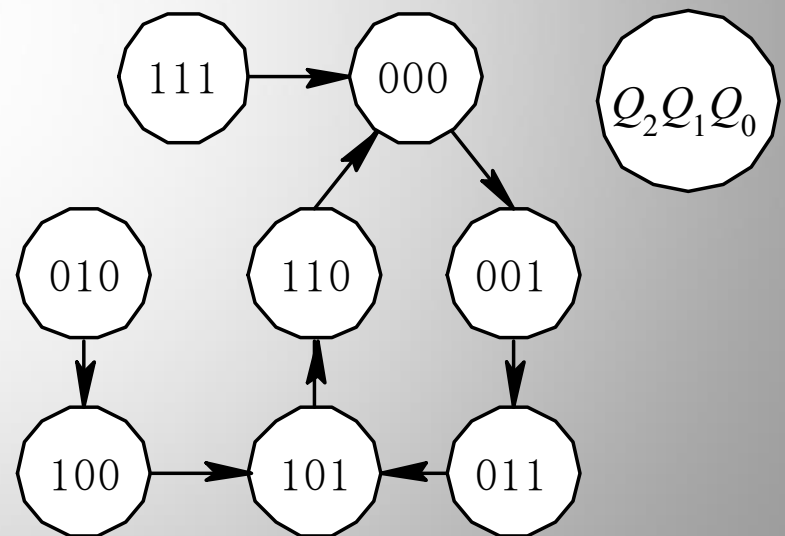
重新检查多余状态的转移情况如下表所示，其状态图如下图所示所示，可以看到该电路具有自启动能力。

如果修改次态卡诺图中(a)的圈法，也可以得到同样的效果。

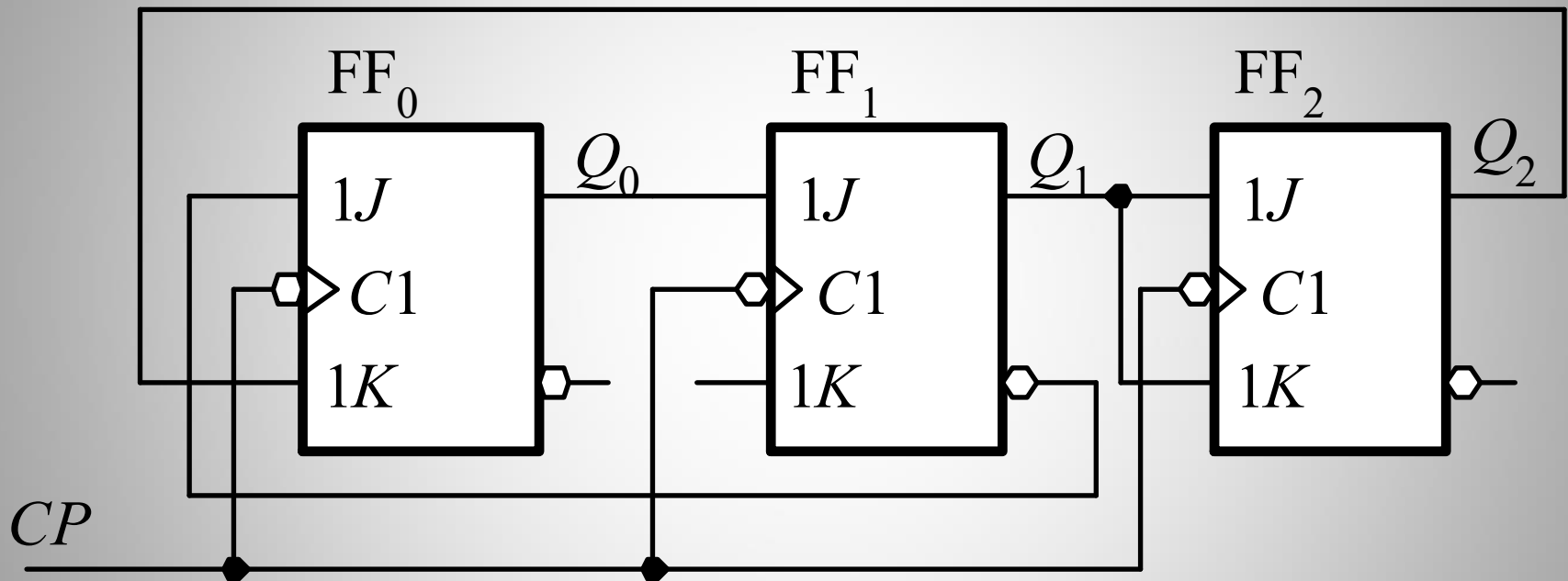
多余状态转移表

Q_2	Q_1	Q_0	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	1	0	1	0	0
1	0	0	1	0	1
1	1	1	0	0	0

状态图



④ 画逻辑图。



Sequential Logic Implementation

- **Models for representing sequential circuits**
 - Abstraction of sequential elements
 - Finite state machines and their state diagrams
 - Inputs/outputs
 - Mealy, Moore, and synchronous Mealy machines
- **Finite state machine design procedure**
 - Verilog specification
 - Deriving state diagram
 - Deriving state transition table
 - Determining next state and output functions
 - Implementing combinational logic

设计举例3

例3 设计一个彩色球控制器。要求彩色球能发出3种颜色的光，红灯亮5S，绿灯亮9S，黄灯亮16S，不断循环。

解：(1)确定系统方案，建立ASM图。

三色球的工作的3个阶段用控制器的3个状态T0、T1、T2表示：

红灯(R)亮5S:—— T0, 编码为00, 时间 $T_S=5S$

绿灯(R)亮9S:—— T1, 编码为01, 时间 $T_M=9S$

黄灯(Y)亮16S:——T2, 编码为10, 时间 $T_L=16S$

回零信号: —— W。

例3

设计一个彩色球控制器。

解：(1)确定系统方案，建立ASM图。

红灯 (R) 亮5S:— T_0 , 编码为00, 时间 $T_s=5S$

绿灯 (R) 亮9S:— T_1 , 编码为01, 时间 $T_M=9S$

黄灯 (Y) 亮16S:— T_2 , 编码为10, 时间 $T_L=16S$

回零信号: —— W 。

控制器的ASM图如图所示:

(2)控制器设计

由ASM图列出状态转换表

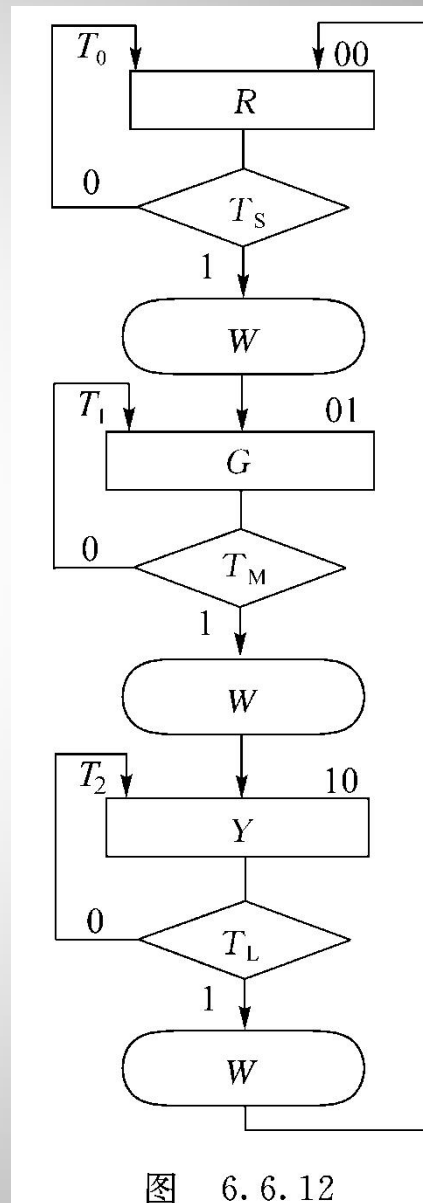


图 6.6.12

例3 设计一个彩色球控制器。

解： (2) 控制器设计

输入					输出		
现态		状态转换条件			次态		转换信号
Q_1^n	Q_0^n	$T_S=5s$	$T_M=5s$	$T_L=5s$	Q_1^{n+1}	Q_0^{n+1}	W
0	0	0	×	×	0	0	0
0	0	1	×	×	0	1	1
0	1	×	0	×	0	1	0
0	1	×	1	×	1	0	1
1	0	×	×	0	1	0	0
1	0	×	×	1	0	0	1

由状态表得状态方程：

$$Q_1^{n+1} = \overline{Q_1}Q_0T_M + Q_1\overline{Q_0}\overline{T_L}$$

$$Q_0^{n+1} = \overline{Q_1}\overline{Q_0}T_S + \overline{Q_1}Q_0\overline{T_M}$$

$$W = \overline{Q_1}\overline{Q_0}T_S + \overline{Q_1}Q_0T_M + Q_1\overline{Q_0}T_L$$

例3 设计一个彩色球控制器。

解： (2) 控制器设计

状态方程：

$$Q_1^{n+1} = \overline{Q}_1 Q_0 T_M + Q_1 \overline{Q}_0 \overline{T}_L$$

$$Q_0^{n+1} = \overline{Q}_1 \overline{Q}_0 T_S + \overline{Q}_1 Q_0 \overline{T}_M$$

$$W = \overline{Q}_1 \overline{Q}_0 T_S + \overline{Q}_1 Q_0 T_M + Q_1 \overline{Q}_0 T_L$$

由状态方程可以设计出控制器电路

该电路由4选1数字选择器和D触发器组成。

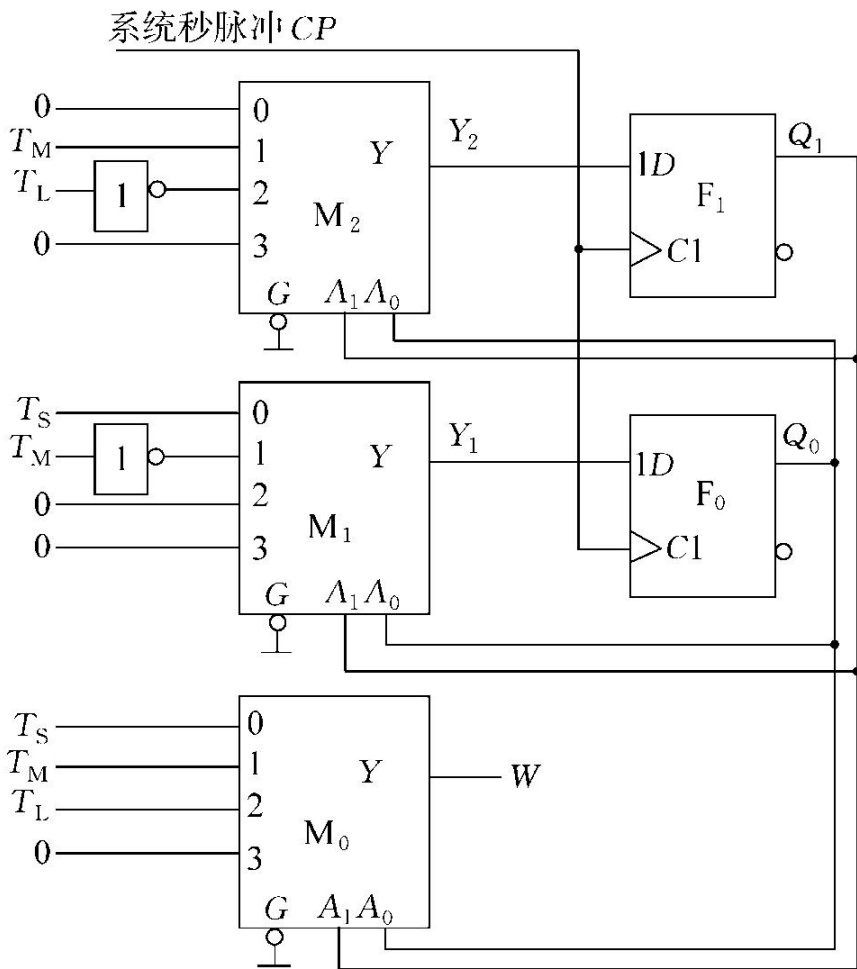
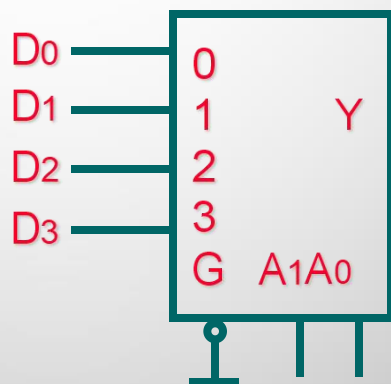


图 6.6.13



$A_1A_0 = 00$, $Y = D_0$

$A_1A_0 = 01$, $Y = D_1$

$A_1A_0 = 10$, $Y = D_2$

$A_1A_0 = 11$, $Y = D_3$

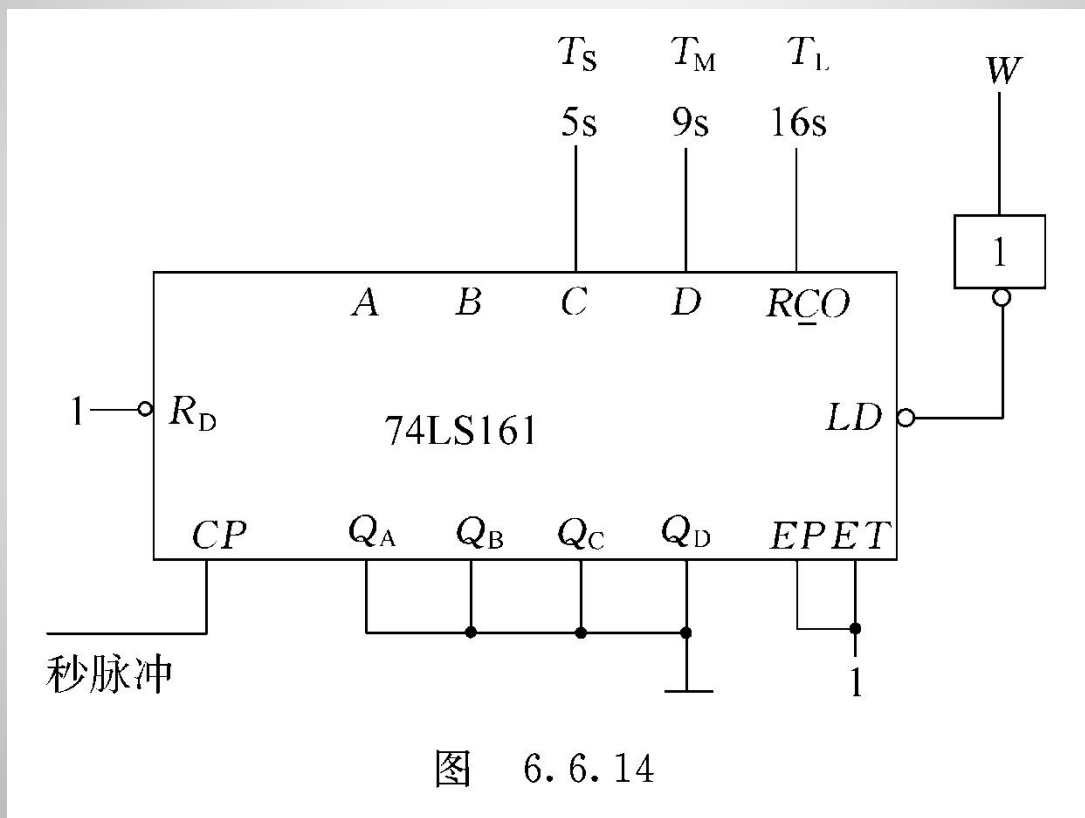
于此 $A_1A_0 = Q_1Q_0$,
对照图可自行分析。

例3 设计一个彩色球控制器。

解： (3)定时器设计

选用集成计数器74LS161，构成能提供5s、9s、16s定时信号的定时器。

电路如图：



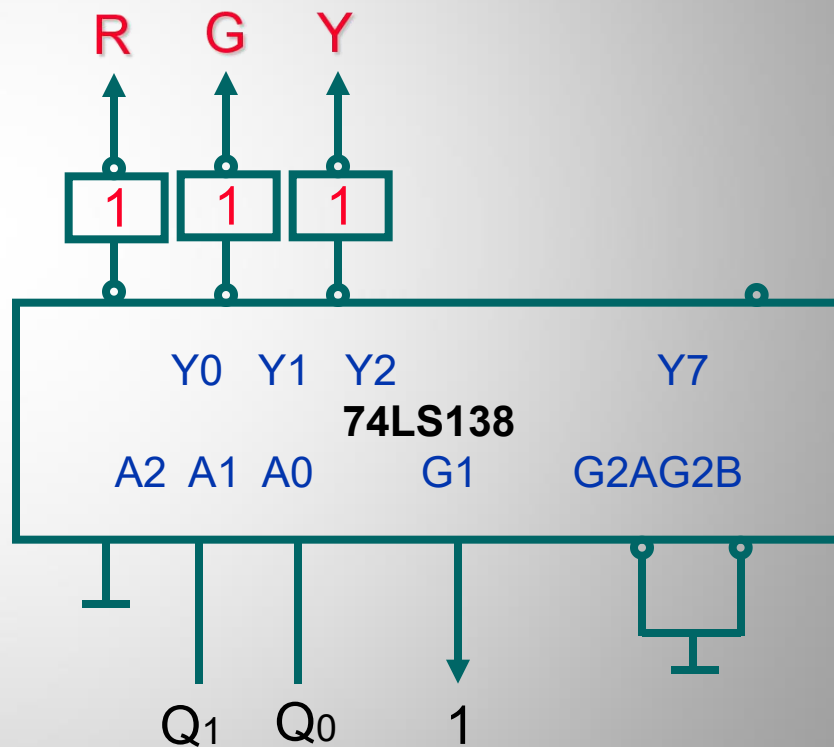
例3 设计一个彩色球控制器。

解： (4) 译码器设计

控制器状态与信号灯的关系如表所示：

状态 Q_1Q_0	R	G	Y
0 0	1	0	0
0 1	0	1	0
1 0	0	0	1

选用74LS138译码器构成译码电路：



Sequential Logic Implementation Summary

- **Design Principle 1:** Simplicity favors regularity.
- **Design Principle 2:** Make the common case fast.
- **Design Principle 3:** Smaller is faster.
- **Design Principle 4:** Good design demands good compromises.

自学内容（重点）

- **6.4.3 异步时序逻辑电路的设计方法（P324）**

- 时钟选择
- 重点掌握---课本内容
- 见本ppt--26, 27, 38

- **6.4.4 复杂时序逻辑电路的设计（P328）**

- 模块分解
- 综合练习---历年考研题目

【例1】应用系统设计--复杂电路设计

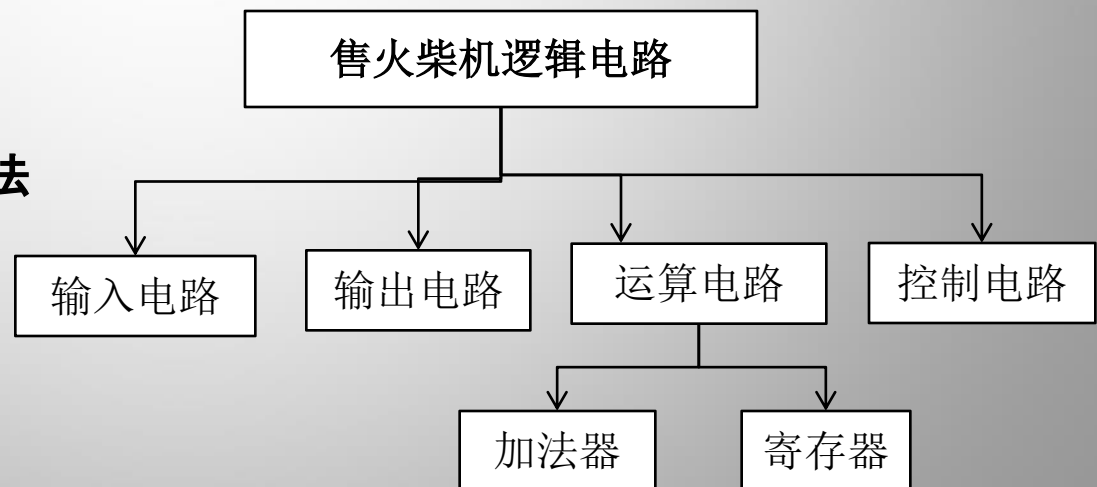
- 设计一个自动售火柴机的逻辑电路。每次可投入一枚1角、2角、5角的硬币，累计超过8角以后，输出一盒火柴，同时找回多于8角的钱。

- 【解】

- 1) 逻辑抽象为逻辑函数问题：
 - 输入信号：投币信号为变量 I_1 (1角)、 I_2 (2角)、 I_5 (5角)；
 - 输出信号：控制火柴盒弹出的信号 Y ；
 - 输出信号：找零 Z_1 (1角)、 Z_2 (2角)、 Z_4 (4角=2X2角)；

- 2) 层次化结构设计

- 模块电路划分
- 自顶向下Top-down的方法



【例1】

- 设计一个自动售火柴机的逻辑电路。每次可投入一枚1角、2角、5角的硬币，累计超过8角以后，输出一盒火柴，同时找回多于8角的钱。

- 【解】

- 3) 运算电路功能分析：

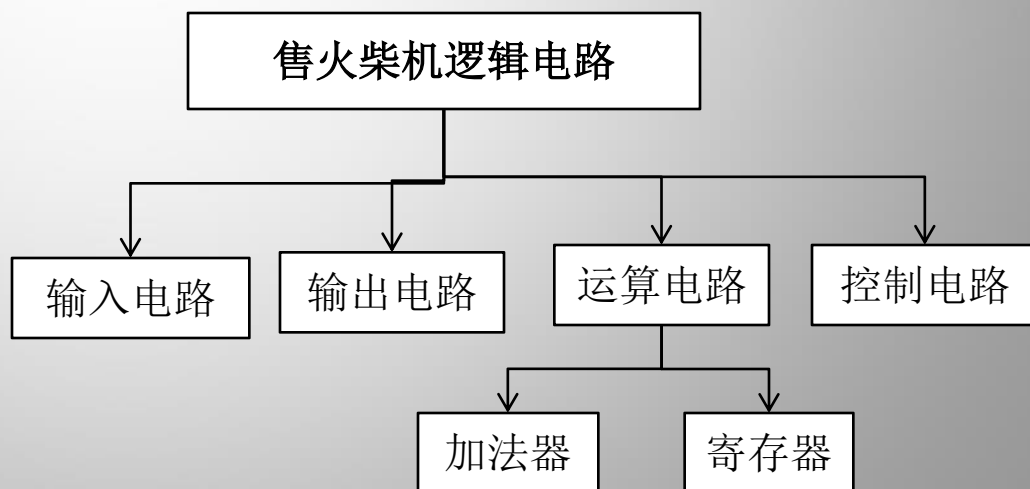
- 每次输入都要做加法-----累加器；
- 加法结果-----金额要暂存起来-----寄存器；
- 当寄存器的值大于等于8时-----输出信号Y、Z；

- 4) 输入电路分析

- 投币信号-----脉冲整形；
- 编码转为二进制数；

- 5) 控制电路分析

- 加法器是组合电路；
- 每次输出需要清零信号；
- 触发脉冲CLK由投币产生；



【例1】

- 设计一个自动售火柴机的逻辑电路。每次可投入一枚1角、2角、5角的硬币，累计超过8角以后，输出一盒火柴，同时找回多于8角的钱。

【解】

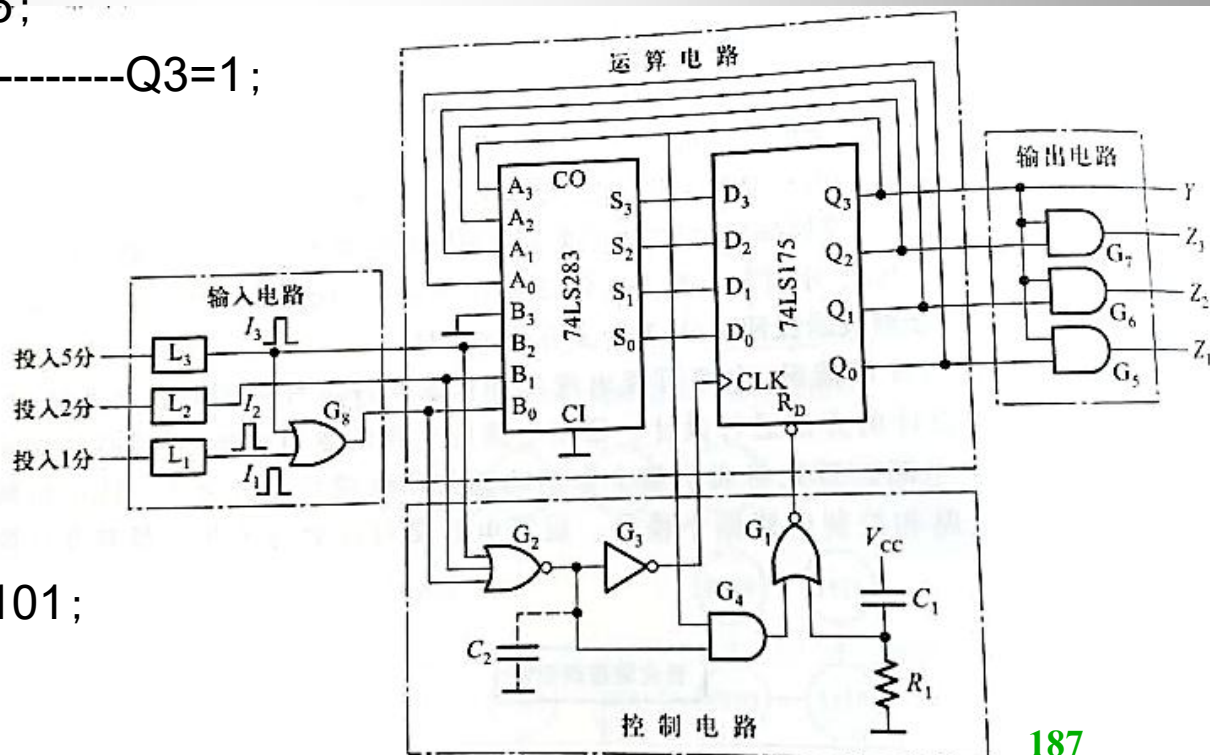
6) 元件选择:

- 4位加法器-----74LS283;
- 4位寄存器-----74LS175;
- 寄存器的值大于等于8时----- $Q_3=1$;
- 异步清零;

7) 电路图

8) 电路分析

- 输入整形电路
- $L_1 L_2 L_3$ ----74LS14;
- $R_1 C_1$ ----上电复位;
- 1角=001, 2角=010, 5角=101;
- C_2 防止竞争冒险;
- G_2 脉冲-- $\rightarrow$ CLK;



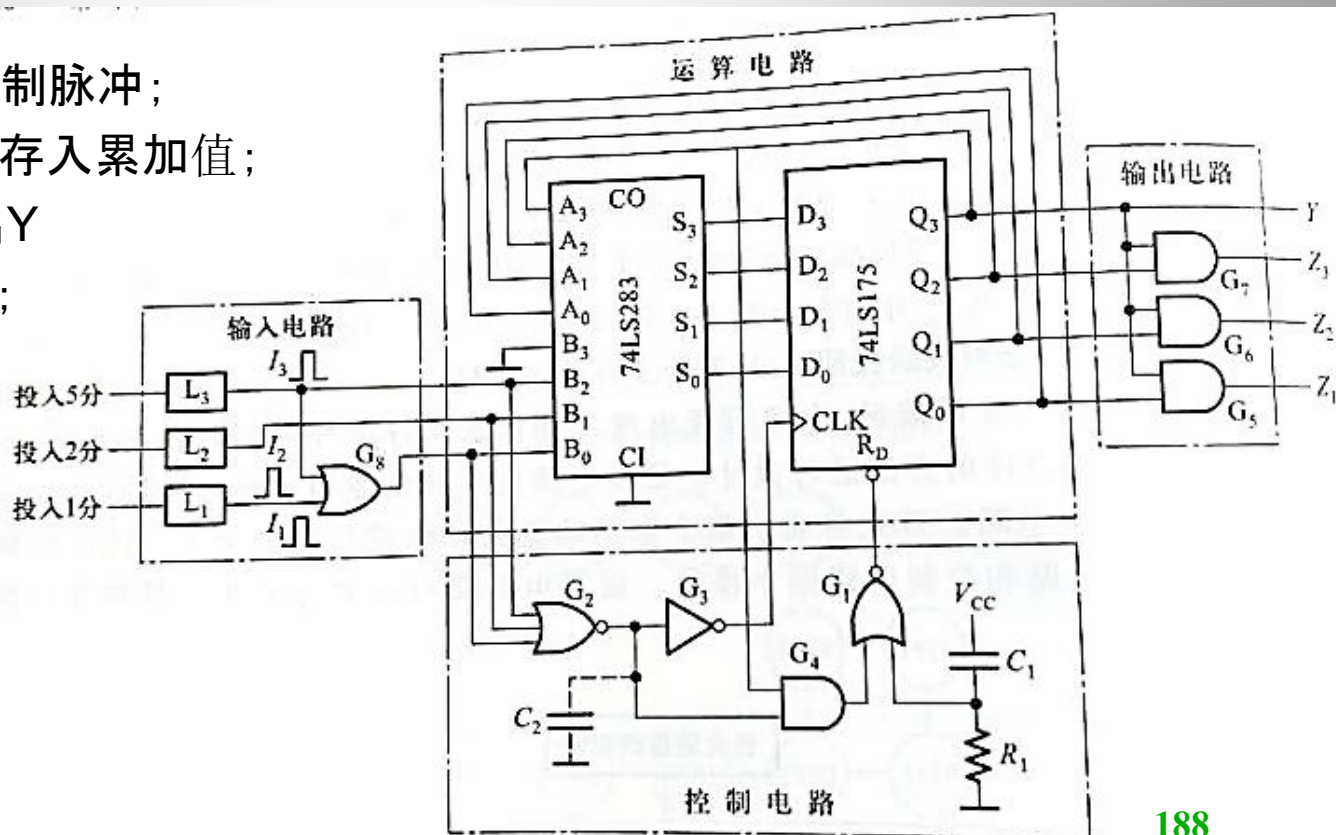
【例3】

- 设计一个自动售火柴机的逻辑电路。每次可投入一枚1角、2角、5角的硬币，累计超过8角以后，输出一盒火柴，同时找回多于8角的钱。

【解】

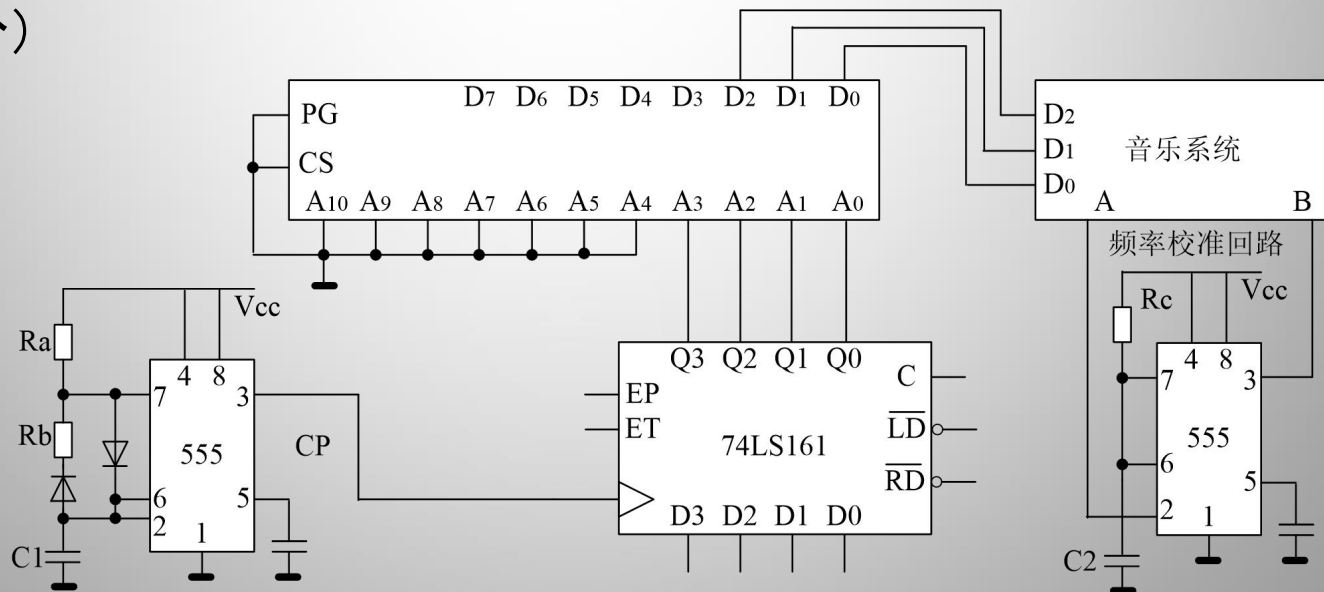
9) 电路功能描述。

- 上电清0寄存器；
- 每次投币产生二进制脉冲；
- 每次投币产生CLK存入累加值；
- 一旦 $Q_3=1$ ，则输出Y
- 及找零(8的余数)；
- 同时清0寄存器；



【例2】应用系统分析与设计---综合电路

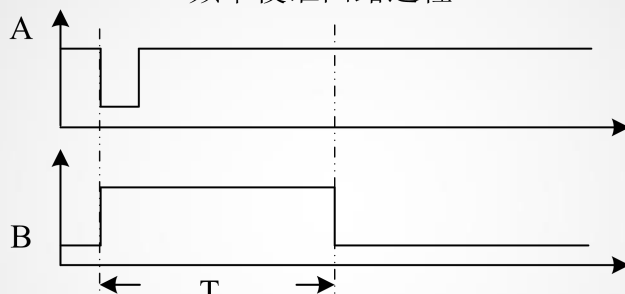
- 有一个音乐处理系统，能够将并行输入的三位二进制码转化成音乐播放输出。璐璐打算采用一片EPROM存储器2716、555定时器产生时钟、74LS161以及必要的门电路完成以下功能：循环贝多芬名曲的《欢乐颂》中的一段3-3-4-5-5-4-3-2-1-1-2-3-2-1-1（注序列的循环长度是15）电路如图所示：
- (1) 74LS161的时钟由左边的555定时器产生。若要求时钟周期=0.07ms，并且要求时钟占空比为50%。已知 $R_b = 50\text{K}\Omega$ ，请求出 R_a 电阻、 C_1 电容的参数。（取 $\ln 2 = 0.7$ ）（8分）
- (2) 音乐系统为了能够校正内部的频率，设有一个外部频率校正系统。从A端输出一个，较细的下降脉冲，截止当B接收到一个下降沿期间，时间为 T 。请写出电路图右边555定时器实现上述功能的详细过程，并且用 R_c 以及 C_2 表示时间 T 。随着 T 的宽度增加，请回答外部频率校准回路的精度和稳定度是上升还是下降？（8分）



【例2】

- (2) 音乐系统为了能够校正内部的频率，设有一个外部频率校正系统。从A端输出一个，较细的下降脉冲，截止当B接收到一个下降沿期间，时间为T。请写出电路图右边555定时器实现上述功能的详细过程，并且用Rc以及C2表示时间T。随着T的宽度增加，请回答外部频率校准回路的精度和稳定度是上升还是下降？(8分)

频率校准回路过程



- (3) 璐璐在烧写EPROM的时候发现，其地址为1000的位置，数据不能写进去。她只好将数据顺延往下填入到EPROM中。现在要利用74LS161计数器完成EPROM中音乐数据的读出。请写画出系统的状态转换图。(提示：序列长度是15，因此状态个数也为15)。画出74LS161设计图。(补全图示74LS161的管脚即可，允许使用必要的门电路)。(8分)

地址				数据		
A3	A2	A	A0	D2	D1	D0
0	0	0	0	0	1	1
0	0	0	1	0	1	1
0	0	1	0	1	0	0
0	0	1	1	1	0	1
0	1	0	0	1	0	1
0	1	0	1	1	0	0
0	1	1	0	0	1	1
0	1	1	1	0	1	0
1	0	0	0	X	X	X
1	0	0	1	0	0	1
1	0	1	0	0	0	1
1	0	1	1	0	1	0
1	1	0	0	0	1	1
1	1	0	1	0	1	0
1	1	1	0	0	0	1
1	1	1	1	0	0	1

小 结

- 1.时序逻辑电路的特点：**任一时刻输出状态不仅取决于当时的输入信号，还与电路的原状态有关。因此时序电路中必须含有存储器件。
- 2.描述时序逻辑电路逻辑功能的方法：**有逻辑方程式、状态转移表、状态转移图和时序图等。
- 3.时序逻辑电路的分析步骤一般为：**逻辑图→时钟方程（异步）、驱动方程、输出方程→状态方程→状态转移表→状态图和时序图→逻辑功能。
- 4.计数器是一种简单而又最常用的时序逻辑器件。**计数器不仅能用于统计输入脉冲的个数，还常用于分频、定时、产生节拍脉冲等。
- 5.用已有的 N 进制集成计数器构成 M (任意)进制的计数器。**

6.5 可编程逻辑器件实现同步时序逻辑电路 (P330-Verilog描述)

6.6 时序逻辑电路中的竞争—冒险现象

因为时序逻辑电路通常都包含组合逻辑电路和存储电路两个部分，所以它的竞争-冒险现象也包含两个方面。

•组合电路部分发生竞争-冒险现象

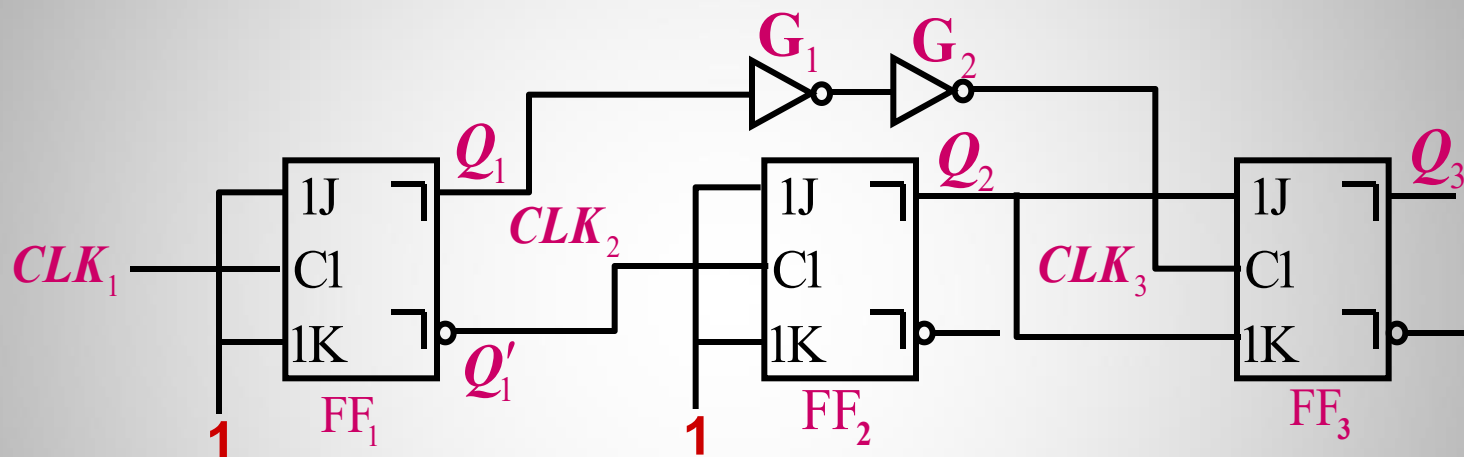
- 竞争产生的尖峰脉冲并不影响组合电路的稳态输出
- 但尖峰脉冲会被触发器接收，而触发器可能会误翻转
- 触发器误翻转会引起整个时序电路误动作

•触发器发生竞争-冒险现象

- 触发器输入需要超前一个时间 T_{setup}
- 如果触发器时钟和输入信号同时改变，则产生竞争

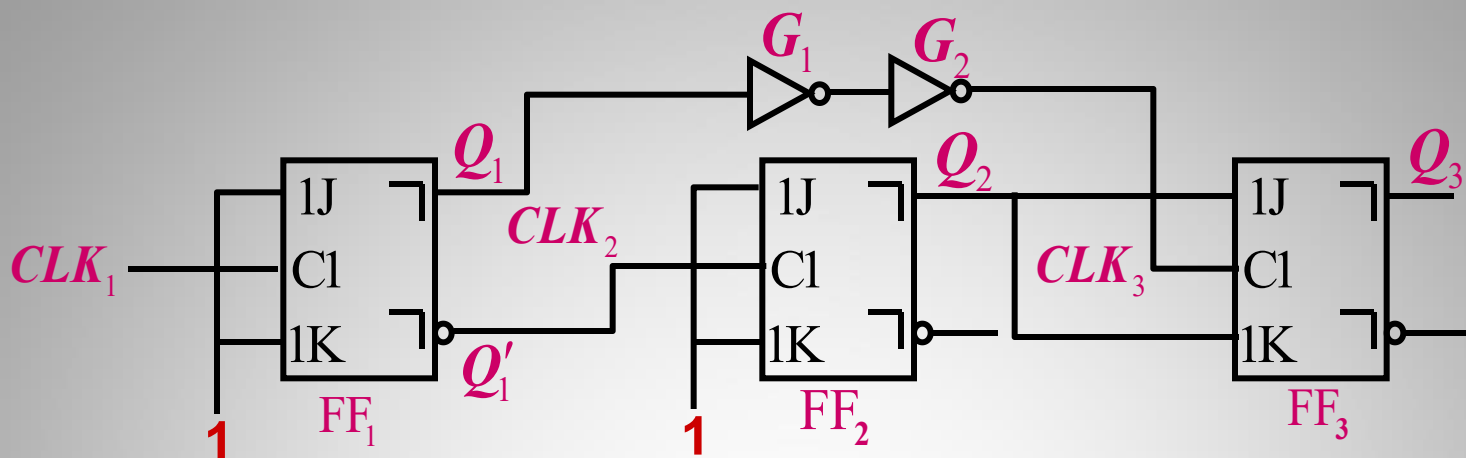
存储电路（或触发器）的竞争-冒险现象

竞争的结果有可能导致触发器误动作，这种现象称为存储电路（或触发器）的竞争-冒险现象。



说明时序电路竞争-冒险现象的例子

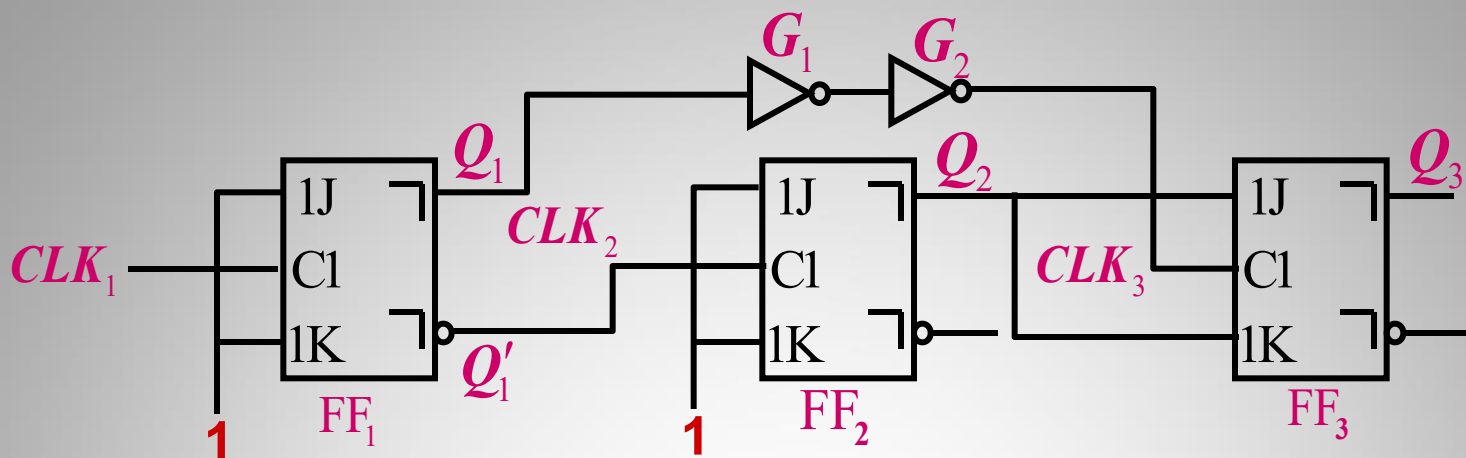
图示八进制异步计数器电路中，就存在着这种存储电路的竞争-冒险现象。



由于 CLK_3 取自 Q_1 ，而 $J_3=K_3=Q_2$ ， FF_2 的时钟信号又取自 Q'_1 ，因而当 FF_1 由0变成1时 FF_3 的输入信号和时钟电平同时改变，导致了竞争-冒险现象的发生。

如果 Q_1 从0变成1时 Q_2 的变化首先完成， CLK_3 的上升沿随后才到，那么在 $CLK_3=1$ 的全部时间里 J_3 和 K_3 的状态将始终不变，可以根据 CLK_3 下降沿到达时 Q_2 的状态决定 FF_3 是否该翻转。

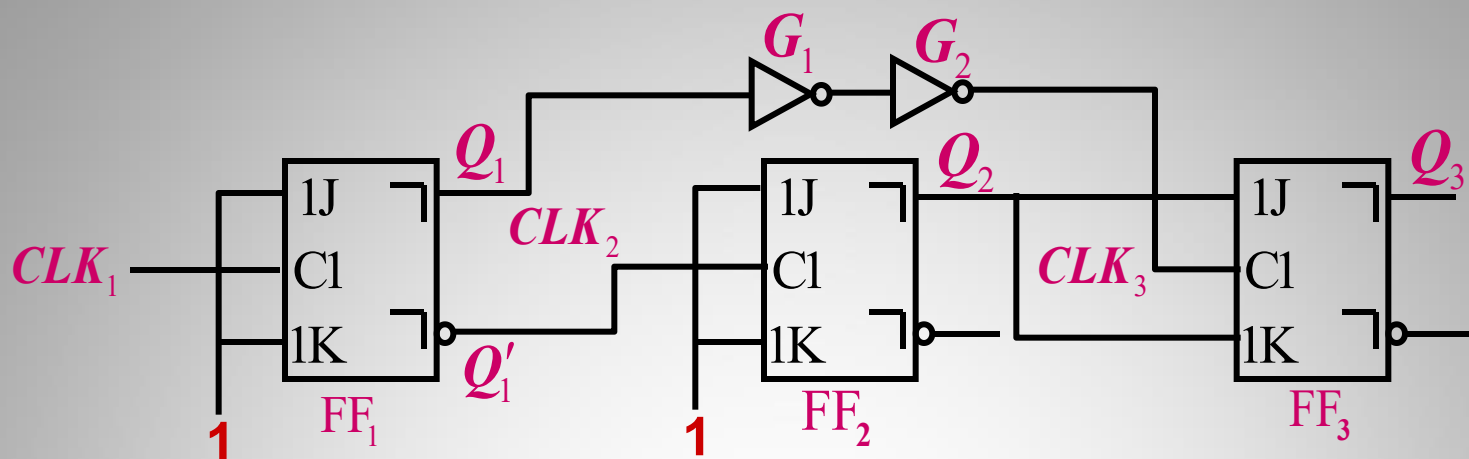
此时电路是一个八进制计数器。



反之，如果 Q_1 从0变成1时 CLK_3 的上升沿首先到达，而 Q_2 的变化在后，则 $CLK_3 = 1$ 的全部时间里 J_3 和 K_3 的状态可能发生变化，这就不能简单地凭 CLK_3 下降沿到达时 Q_2 的状态来决定 Q_3 的次态了。

电路就不按八进制计数循环工作了。

倘若在设计时无法确切知道 CLK_3 和 Q_2 哪一个先改变状态，那么也就不能确定电路状态转换的规律。



为了确保 CLK_3 的上升沿在 Q_2 的新状态稳定建立之后才到达 FF_3 ，可以在 Q_1 到 CLK_3 的传输通道上增加延迟环节， G_1 和 G_2 就是作延迟环节用的。

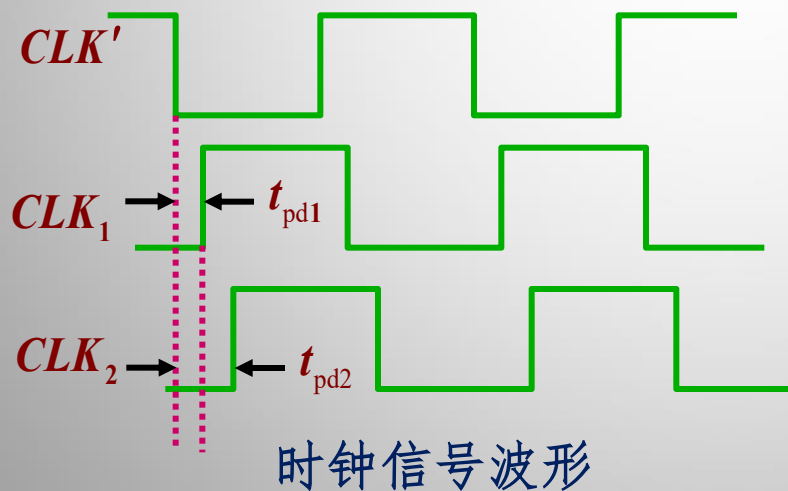
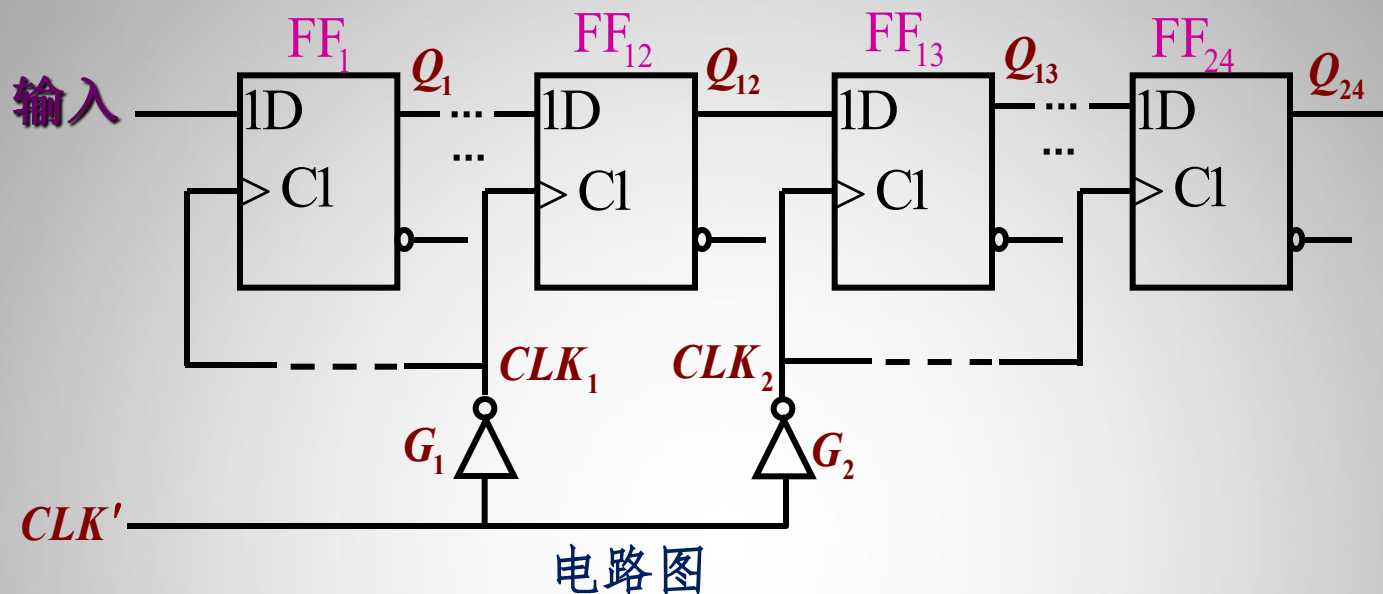
只要 G_1 和 G_2 的传输延迟时间足够长，一定能使 Q_2 的变化先于 CLK_3 的变化，保证电路按八进制计数循环正常工作。

一般认为存储电路的竞争-冒险现象仅发生在异步时序电路中。

在有些规模较大的同步时序电路中，由于每个门的带负载能力有限，所以经常是先用一个时钟信号同时驱动几个门电路，然后再由这几个门电路分别去驱动若干个触发器。

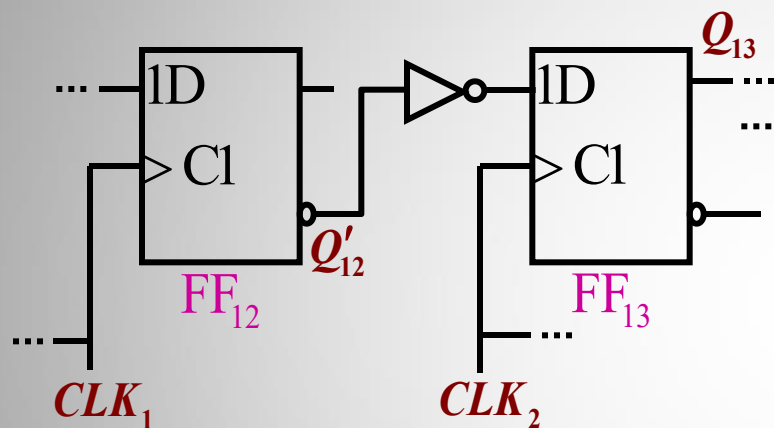
由于每个门的传输时间不同，严格地讲系统已不是真正的同步时序电路了，故仍有可能发生存储器电路的竞争-冒险现象。

移位寄存器中时钟偏移现象

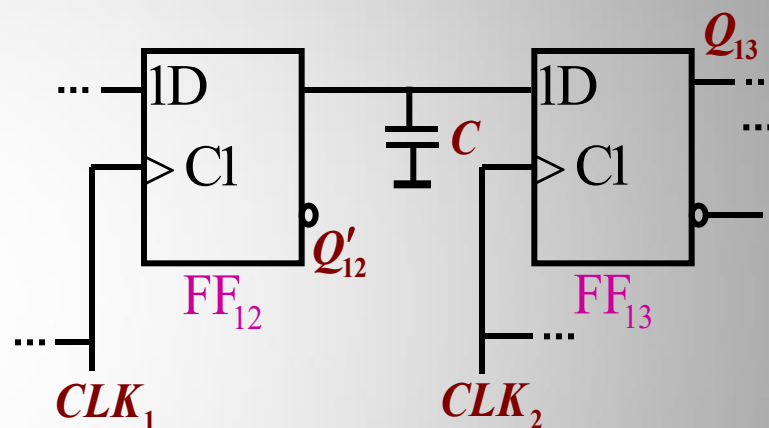


时钟信号偏移有可能造成移位寄存器的误动作。

二、防止移位寄存器错移的方法



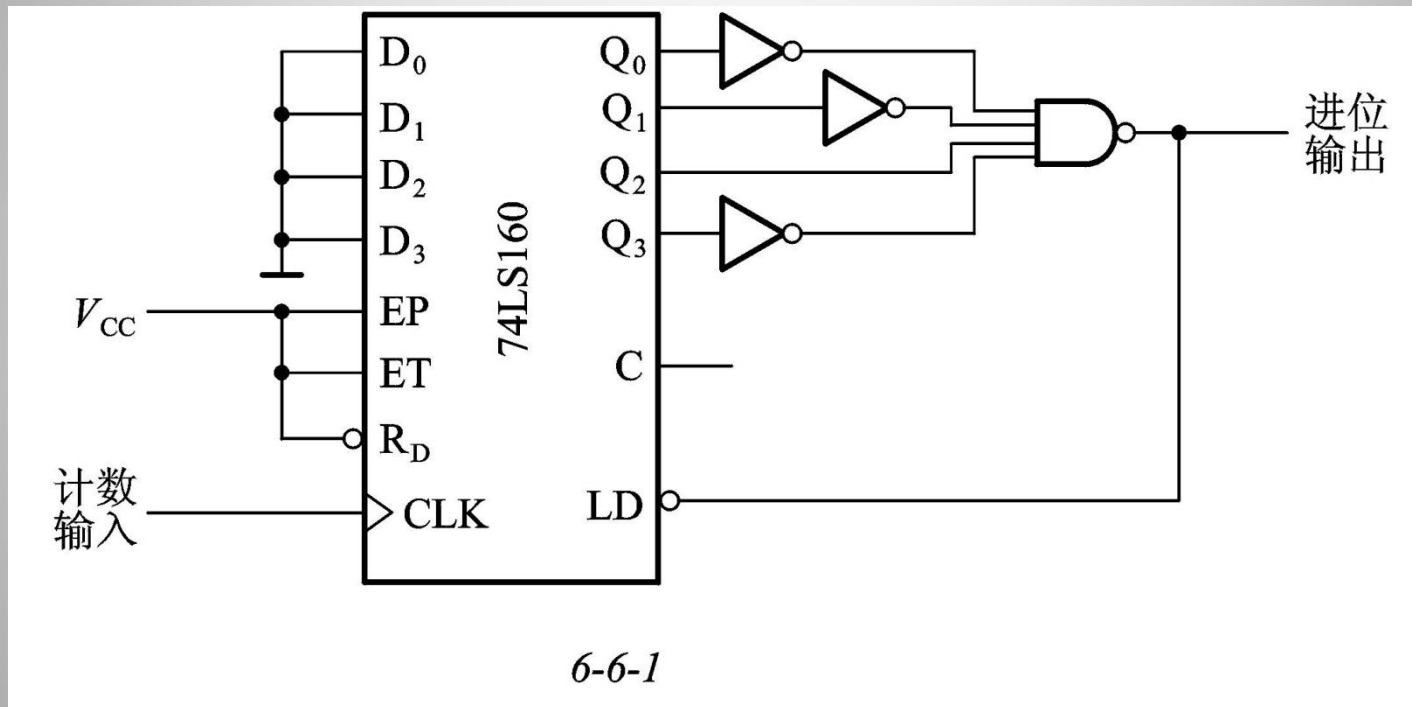
接入反相器作延迟环节

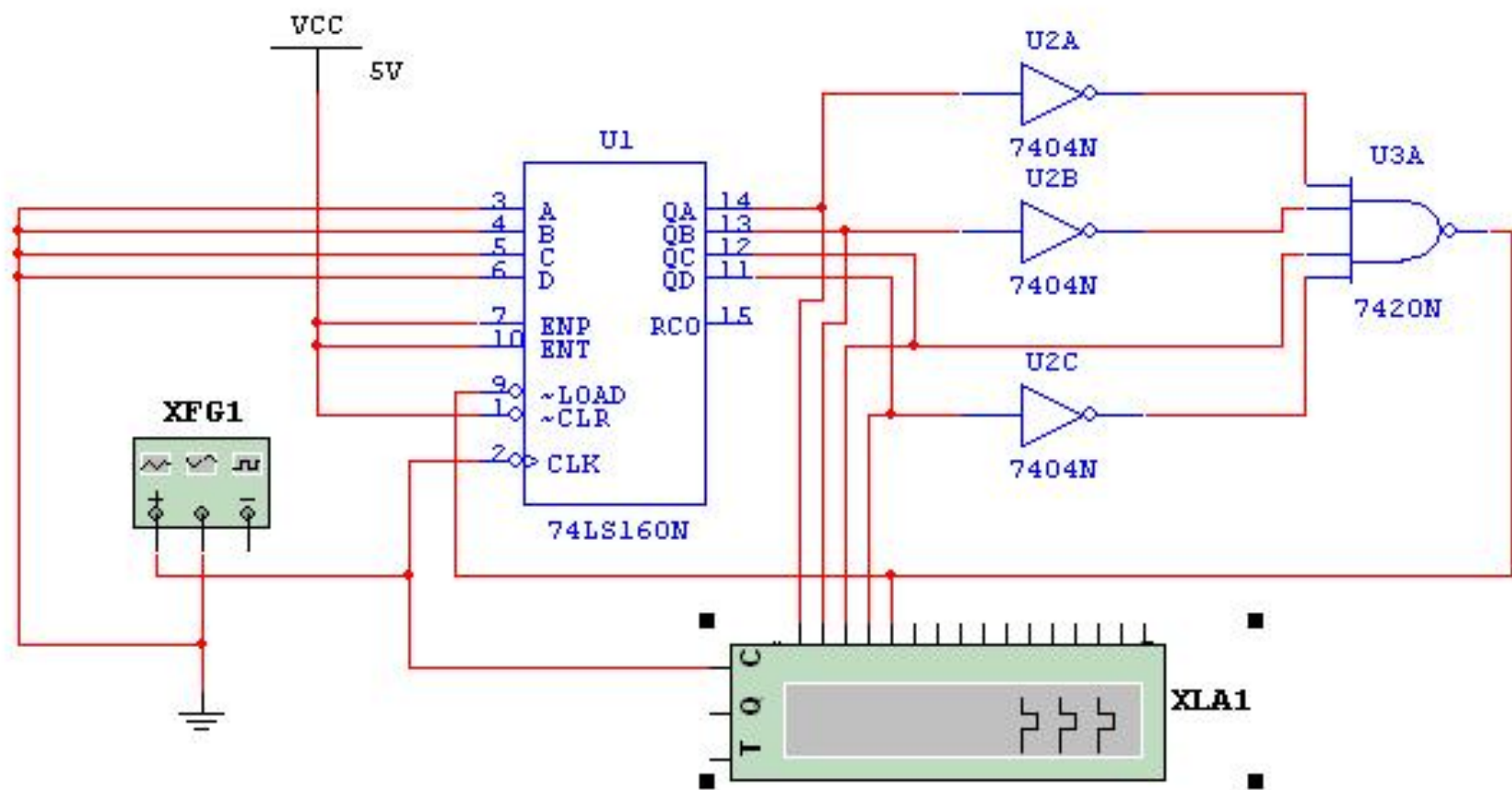


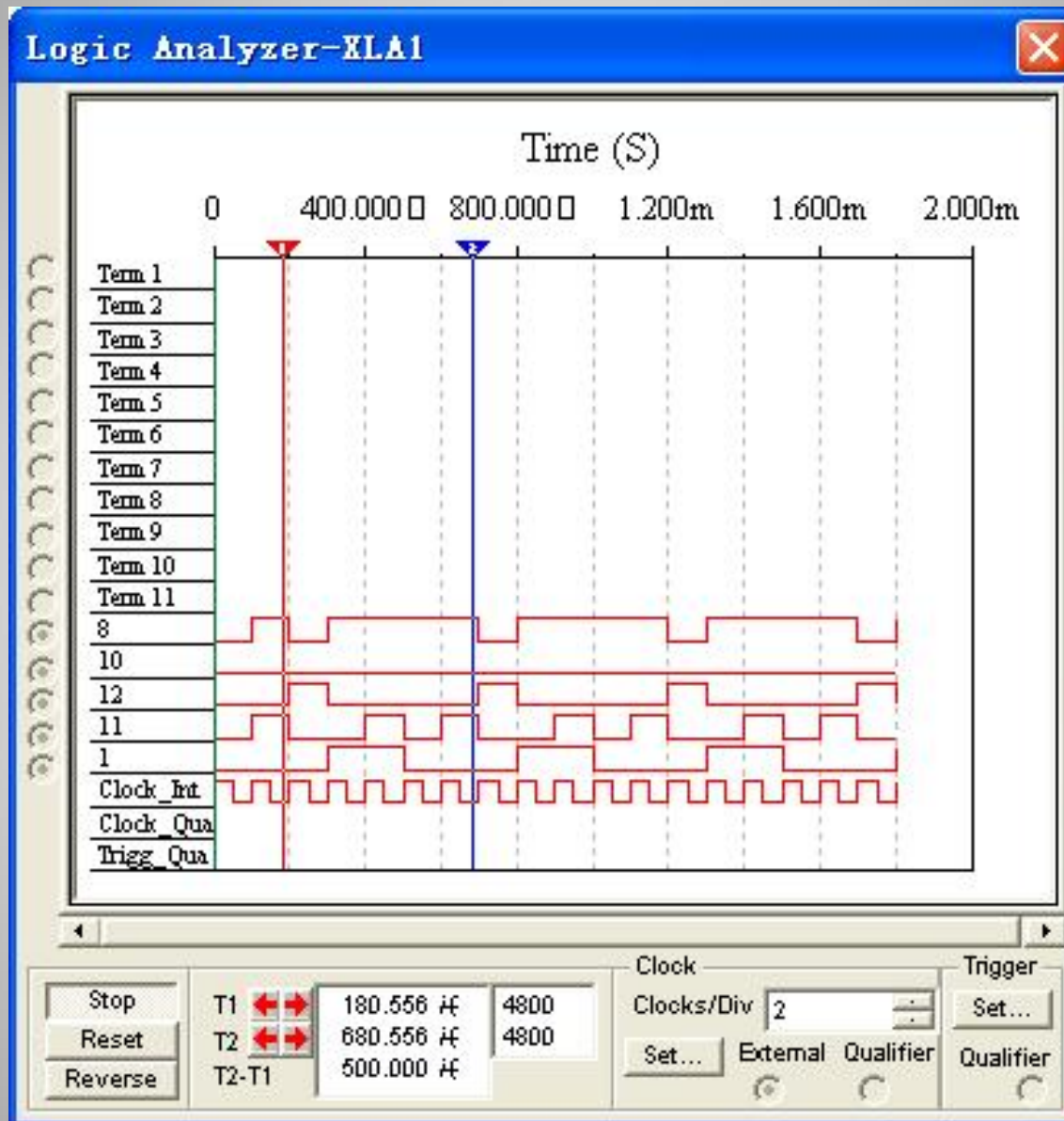
接入延迟电容

6.7用multisim分析时序逻辑电路(自学)

例:分析下图的计数器电路。求电路的时序图.说明这是几进制的计数器。



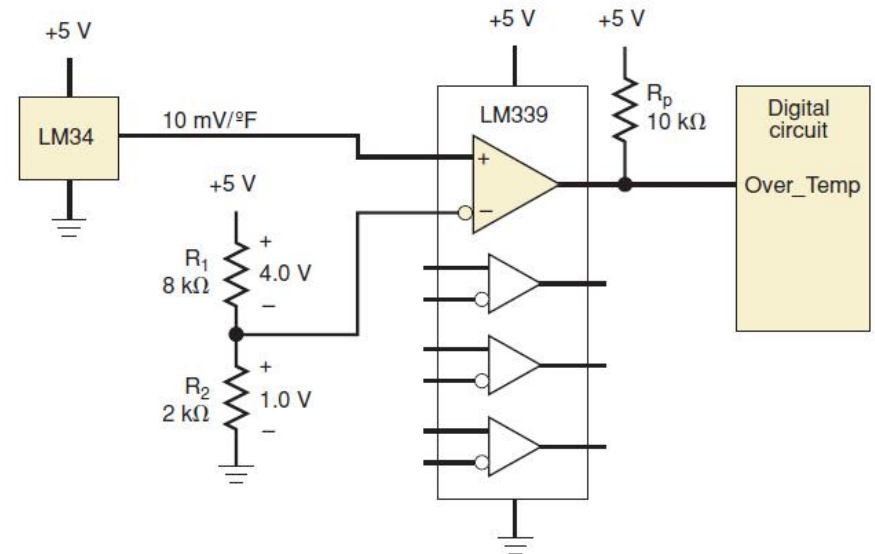




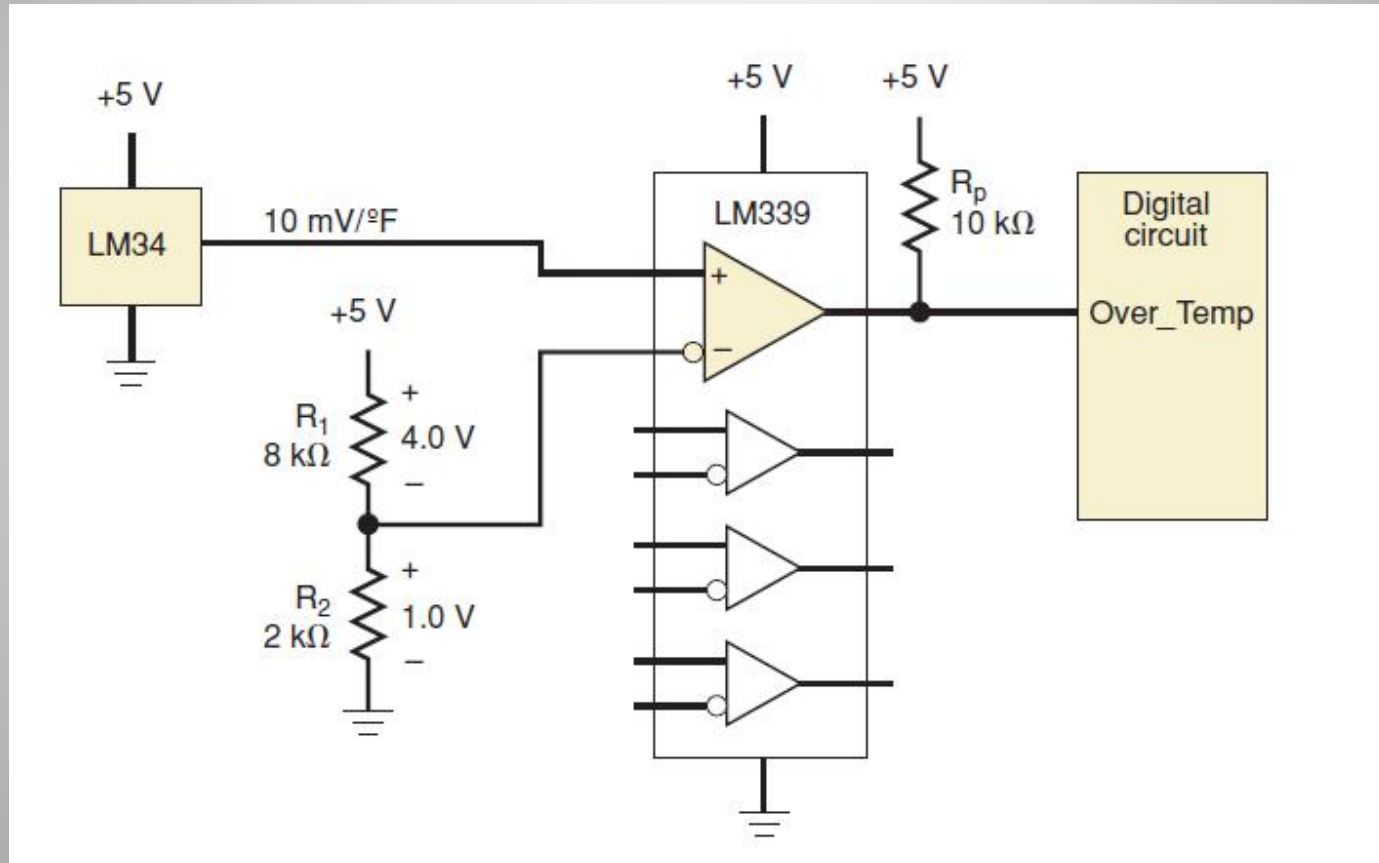
补充: ANALOG VOLTAGE COMPARATORS

■ analog voltage comparator

- Another very useful device interfacing to digital systems
- LM339
- LM34
 - The temperature-measuring device
 - puts out a voltage directly proportional to the temperature
 - $100^{\circ}\text{ F} * 10\text{ mV}/^{\circ}\text{ F} = 1.0\text{ V}$



A temperature-limit detector using an LM339 analog voltage comparator



课后作业

- 查阅：
 - 国际电路公司的
 - » 移位寄存器、计数器芯片的型号、电参数、速度.....
 - 图书馆资源: 电子器件天地, 软件
- 习题、作业：
 - 见学在浙大
- 自学Verilog HDL语言
- 阅读：
 - 推荐英文版电子书“**Digital_Systems_Principles_and_Applications**”,
CHAPTER 6-8
 - » 内容详细、实用性较强;